ΙΟΝΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ



Πτυχιακή Εργασία

Σχεδίαση και Υλοποίηση Συστήματος Ψηφιακών Παραγγελιών και Υπηρεσιών για Καταλύματα Βραχυχρόνιας Μίσθωσης

Design and Implementation of a Digital Ordering and Services System for Short-Term Rental Accommodations

Κωνσταντίνος Βλιώρας

Επιβλέπων: Αλέξανδρος Πανάρετος

12 Σεπτεμβρίου 2026

Επιβλέπων Καθηγητής:

Αλέξανδρος Πανάρετος - ΕΔΙΠ

Ιόνιο Πανεπιστήμιο

Μέλη Επιτροπής:

Μάριος Αναγνώστου - Επίκουρος Καθηγητής

Ιόνιο Πανεπιστήμιο

Ιωσήφ Πολενάκης - Επίκουρος Καθηγητής

Ιόνιο Πανεπιστήμιο

Περίληψη

Η παρούσα εργασία παρουσιάζει τον σχεδιασμό και την υλοποίηση ενός ψηφιακού συστήματος παραγγελιών και υπηρεσιών για καταλύματα βραχυχρόνιας μίσθωσης. Το σύστημα υλοποιείται ως πλατφόρμα multi-tenant SaaS, με κύριο σημείο επαφής για τον επισκέπτη μια εφαρμογή kiosk σε Android tablet εγκατεστημένη εντός του καταλύματος. Οι φιλοξενούμενοι μπορούν να περιηγηθούν στον κατάλογο προϊόντων, να υποβάλουν παραγγελίες και να πληρώσουν από την ίδια συσκευή, μέσω εξωτερικού φορέα πληρωμών, χωρίς να απαιτείται εγκατάσταση εφαρμογής ή δημιουργία λογαριασμού. Οι ιδιοκτήτες καταλυμάτων διαχειρίζονται τα καταλύματα, τον κατάλογο προϊόντων και τις παραγγελίες τους μέσω web panel, ενώ η πλατφόρμα εξυπηρετεί ταυτόχρονα πολλούς ανεξάρτητους ιδιοκτήτες διατηρώντας απομονωμένα τα δεδομένα κάθε ιδιοκτήτη.

Η αρχιτεκτονική του συστήματος βασίζεται στις αρχές της Clean Architecture και του προτύπου Port/Adapter: ο πυρήνας δεν εξαρτάται από frameworks, και η επιχειρησιακή λογική επικοινωνεί με τις εξωτερικές υπηρεσίες μόνο μέσω διεπαφών. Όλος ο κώδικας είναι γραμμένος σε Kotlin, και το Kotlin Multiplatform (KMP) επιτρέπει την κοινή χρήση του domain model και των DTOs μεταξύ backend (JVM), εφαρμογής Android kiosk και web panel (WebAssembly). Το backend υλοποιήθηκε με Ktor και JetBrains Exposed, ενώ η βάση δεδομένων ακολουθεί τη στρατηγική schema-per-tenant σε PostgreSQL: τα δεδομένα κάθε ιδιοκτήτη βρίσκονται σε δικό του schema, το οποίο επιλέγεται σε ένα κεντρικό σημείο του κώδικα για κάθε συναλλαγή. Το web panel αναπτύχθηκε με Compose Multiplatform με στόχο μεταγλώττισης wasmJs.

Τα αποτελέσματα της εργασίας περιλαμβάνουν ένα ολοκληρωμένο πρωτότυπο που λειτουργεί που καλύπτει την ιεραρχία πλατφόρμα → ιδιοκτήτης → κατάλυμα → συσκευή → φιλοξενούμενος. Το σύστημα ενσωματώνει ταυτοποίηση JWT και έλεγχο πρόσβασης βάσει ρόλων (RBAC), αυτοματοποιημένη ενεργοποίηση συσκευών μέσω κωδικού QR, μηχανή καταστάσεων παραγγελιών (order state machine), σύνδεση με πραγματικές εξωτερικές υπηρεσίες (αποστολή email μέσω SMTP, αναζήτηση στοιχείων επιχειρήσεων σε δημόσιο εταιρικό μητρώο και αναζήτηση διευθύνσεων), προσομοίωση των υπηρεσιών διανομής και πληρωμών μέσω emulated adapters, καθώς και κεντρικό audit log. Η υλοποίηση υποδεικνύει ότι η Kotlin, μέσω του KMP, μπορεί να στηρίξει σε μία γλώσσα ένα σύστημα που εκτείνεται σε server, Android και browser.

Λέξεις-κλειδιά: multi-tenant SaaS, kiosk, Android, Kotlin Multiplatform, Clean Architecture, REST API, PostgreSQL

Abstract

This thesis presents the design and implementation of a digital ordering and services system for short-term rental accommodations. The system is built as a multi-tenant SaaS platform whose main point of contact with the guest is a kiosk application on an Android tablet installed in the property. Guests can browse the product catalogue, place orders and pay on the same device, through an external payment provider, without installing an application or creating an account. Owners manage their properties, product catalogue and orders through a web panel, while the platform serves many independent owners at the same time and keeps each owner's data isolated.

The architecture follows the principles of Clean Architecture and the Port/Adapter pattern: the core does not depend on frameworks, and the business logic reaches external services only through interfaces. All the code is written in Kotlin, and Kotlin Multiplatform (KMP) lets the backend (JVM), the Android kiosk application and the web panel (WebAssembly) share the domain model and the DTOs. The backend was built with Ktor and JetBrains Exposed, and the database follows a schema-per-tenant strategy in PostgreSQL: each owner's data lives in its own schema, which a single, central point in the code selects for every transaction. The web panel was developed with Compose Multiplatform, compiled to wasmJs.

The outcome is a working, integrated prototype that covers the hierarchy platform → owner → property → device → guest. The system includes JWT authentication with role-based access control (RBAC), automated device activation through a QR code, an order state machine, connections to real external services (email over SMTP, company lookup in a public company registry and address search), emulated delivery and payment services, and a central audit log. The implementation suggests that Kotlin, through KMP, can carry a system that spans server, Android and browser in a single language.

Keywords: multi-tenant SaaS, kiosk, Android, Kotlin Multiplatform, Clean Architecture, REST API, PostgreSQL

Δήλωση Χρήσης Τεχνητής Νοημοσύνης

Κατά την εκπόνηση της εργασίας χρησιμοποίησα εργαλεία παραγωγικής τεχνητής νοημοσύνης (Claude, της εταιρείας Anthropic) ως βοηθητικά μέσα, με τον τρόπο που περιγράφεται παρακάτω.

Στο κείμενο, τα εργαλεία αυτά τα χρησιμοποίησα στο στάδιο της επιμέλειας: για τον έλεγχο της γραμματικής, της σύνταξης και της διατύπωσης, και για τον έλεγχο της συνέπειας της ορολογίας, της αρίθμησης και των παραπομπών. Τη δομή, το περιεχόμενο και τα συμπεράσματα της εργασίας τα διαμόρφωσα ο ίδιος.

Στην υλοποίηση του συστήματος, τα εργαλεία λειτούργησαν συμβουλευτικά: για τη συζήτηση εναλλακτικών λύσεων, για παράδειγμα στη μοντελοποίηση των οντοτήτων, για την εξήγηση της συμπεριφοράς βιβλιοθηκών και για τον εντοπισμό σφαλμάτων. Η αρχιτεκτονική του συστήματος και οι σχεδιαστικές αποφάσεις είναι δικές μου, και τον κώδικα τον έγραψα και τον δοκίμασα ο ίδιος.

Φέρω την πλήρη ευθύνη για το περιεχόμενο της εργασίας, την ακρίβεια των στοιχείων και των πηγών της και την ορθότητα του κώδικα.

Πίνακας Περιεχομένων

Περίληψη i

Abstract ii

Δήλωση Χρήσης Τεχνητής Νοημοσύνης iii

Κατάλογος Εικόνων ix

Κατάλογος Πινάκων xi

Κεφάλαιο 1. Εισαγωγή 1

1.1 Περιγραφή Προβλήματος 1

1.2 Κίνητρο και Στόχοι 2

1.2.1 Ακαδημαϊκό Κίνητρο 2

1.2.2 Πρακτικοί Στόχοι 2

1.3 Συνεισφορά της Εργασίας 3

1.4 Δομή του Κειμένου 4

Κεφάλαιο 2. Θεωρητικό Υπόβαθρο και Σχετικές Εργασίες 5

2.1 Η Αγορά Βραχυχρόνιας Μίσθωσης και οι Ψηφιακές Υπηρεσίες 5

2.2 Αρχιτεκτονική Multi-Tenant και Μοντέλα Απομόνωσης Δεδομένων 6

2.3 Clean Architecture και το Port/Adapter (Hexagonal) Pattern 9

2.4 Kotlin Multiplatform — Τεχνική Επισκόπηση και Αξιολόγηση 10

2.5 Σχετικά Συστήματα και Ακαδημαϊκές Εργασίες 12

Κεφάλαιο 3. Ανάλυση Απαιτήσεων 15

3.1 Λειτουργικές Απαιτήσεις 15

3.1.1 Διαχειριστής Πλατφόρμας (Platform Admin) 15

3.1.2 Ιδιοκτήτης Καταλύματος (Owner) 16

3.1.3 Φιλοξενούμενος (Guest) 18

3.2 Μη Λειτουργικές Απαιτήσεις 19

3.2.1 Δομική Απομόνωση Δεδομένων (Data Isolation) 19

3.2.2 Ασφάλεια και Ταυτοποίηση 19

3.2.3 Απρόσκοπτη Αλληλεπίδραση Επισκέπτη (Zero-Friction Guest Interaction) 20

3.2.4 Επεκτασιμότητα Εξωτερικών Υπηρεσιών (Port/Adapter Extensibility) 21

3.2.5 Αξιοπιστία και Λειτουργία Kiosk (Kiosk Mode & Availability) 21
3.2.6 Συνέπεια Δεδομένων και Ιστορικότητα (Data Consistency) 21
3.3 Ρόλοι Χρηστών και Περιπτώσεις Χρήσης (Use Cases) 22
3.3.1 Ρόλοι Χρηστών και Σύστημα Ταυτοποίησης 22
3.3.2 Use Case 1: Υποβολή Παραγγελίας από Φιλοξενούμενο 23
3.3.3 Use Case 2: Δημιουργία Νέου Tenant (Tenant Onboarding) 25
3.3.4 Use Case 3: Ενεργοποίηση και Σύνδεση Συσκευής Kiosk 26
3.3.5 Use Case 4: Εποπτεία και Διαχείριση Παραγγελιών από Ιδιοκτήτη 28
3.3.6 Use Case 5: Παραγγελία Φαγητού από Πλατφόρμα Διανομής 29
3.4 Παραδοχές και Όρια του Συστήματος 30
3.4.1 Εξωτερικές Υπηρεσίες — Προσομοίωση μέσω Emulated Adapters 30
3.4.2 Διαχείριση Αποθεμάτων 31
3.4.3 Τοπικοποίηση και Γεωγραφική Προσαρμοστικότητα 32
3.4.4 Εκτός Εύρους — Μελλοντικές Επεκτάσεις 32
Κεφάλαιο 4. Αρχιτεκτονική Συστήματος 33
4.1 Επισκόπηση Αρχιτεκτονικής και Δομή Modules 33
4.2 Clean Architecture ως Δομικός Περιορισμός 34
4.3 Multi-Tenancy — Αρχιτεκτονική Ροής Αιτημάτων 35
4.4 Port/Adapter Pattern — Ενσωμάτωση Εξωτερικών Υπηρεσιών 37
4.5 Αποφάσεις Σχεδίασης και Επιλογή Τεχνολογιών 39
4.5.1 Η Επιλογή Kotlin ως Μοναδικής Γλώσσας 39
4.5.2 Backend Framework: Ktor έναντι Spring Boot και ASP.NET Core 40
4.5.3 Στρατηγική Multi-Tenancy: Schema-Per-Tenant 41
4.5.4 Κοινός Κώδικας: Kotlin Multiplatform 42
4.5.5 Web Panel: Compose Multiplatform (wasmJs) 42
4.5.6 ORM: JetBrains Exposed 43
4.5.7 Ταυτοποίηση: JWT με Refresh Token Rotation 44
4.5.8 Κρυπτογράφηση Μυστικών των Εξωτερικών Υπηρεσιών 46
Κεφάλαιο 5. Σχεδίαση Βάσης Δεδομένων 49

5.1 Δημιουργία και Προετοιμασία των Schemas 49
5.2 Public Schema — Δεδομένα Πλατφόρμας 51
5.2.1 owners — Ιδιοκτήτες Καταλυμάτων 51
5.2.2 platform_admins — Διαχειριστές Πλατφόρμας 51
5.2.3 refresh_tokens — Tokens Ανανέωσης Συνεδρίας 52
5.2.4 audit_log — Αρχείο Καταγραφής Ενεργειών 52
5.2.5 platform_settings — Ρυθμίσεις Πλατφόρμας 52
5.3 Tenant Schema — Δεδομένα Καταλύματος 53
5.3.1 properties — Καταλύματα 53
5.3.2 devices — Συσκευές Kiosk 53
5.3.3 products — Προϊόντα 54
5.3.4 product_property_overrides — Παρακάμψεις Διαθεσιμότητας ανά Κατάλυμα 54
5.3.5 orders — Παραγγελίες 54
5.3.6 order_items — Γραμμές Παραγγελίας 55
5.3.7 payments — Πληρωμές 55
5.3.8 food_platform_connections — Συνδέσεις με Πλατφόρμες Διανομής 56
5.3.9 property_food_platforms — Πλατφόρμες ανά Κατάλυμα 56
5.4 Διάγραμμα Οντοτήτων-Σχέσεων 56
5.5 Αποφάσεις Σχεδίασης 59
5.6 Συναλλαγές και Ταυτόχρονη Πρόσβαση 60
Κεφάλαιο 6. Υλοποίηση Backend 63
6.1 Δομή του REST API και Επίπεδα Δρομολόγησης 63
6.2 Ταυτοποίηση Συσκευής και ο Ρόλος DEVICE 64
6.3 Middleware Pipeline και Διαχείριση Σφαλμάτων 66
6.4 Service Layer και Συναρμολόγηση Εξαρτήσεων 70
6.5 Ο Κύκλος Ζωής της Παραγγελίας 71
6.6 Διασύνδεση με τον Φορέα Πληρωμών 73
6.7 Ενσωμάτωση Πλατφορμών Διανομής Φαγητού 74

6.8 Πραγματικές Εξωτερικές Υπηρεσίες: Ηλεκτρονικό Ταχυδρομείο, Εταιρικό Μητρώο και Γεωκωδικοποίηση 76
6.9 Καταγραφή Συμβάντων και Αρχείο Ενεργειών 77
6.10 Θωράκιση Ασφαλείας 78
6.11 Παράδειγμα Πλήρους Κύκλου Ενός Αιτήματος 79
Κεφάλαιο 7. Owner Web Panel 81
7.1 Build Pipeline και Διανομή του Bundle 81
7.2 Πλοήγηση: Το Μοντέλο Screen 82
7.3 Κύριες Οθόνες ανά Ρόλο 83
7.4 Πρόσβαση βάσει Ρόλων και Πλευρικό Μενού 86
7.5 ApiClient: Επικοινωνία και Ανανέωση Token 86
7.6 Διαχείριση Καταλυμάτων: Αναζήτηση Διεύθυνσης και Πληροφορίες Επισκέπτη 87
7.7 Πλατφόρμες Διανομής, Εισαγωγή Προϊόντων και Παραγγελίες 89
7.8 Ένταξη Ιδιοκτητών και Ρυθμίσεις Πλατφόρμας 91
7.9 Ενημέρωση Δεδομένων, Σφάλματα και Ταυτόχρονες Ενέργειες 93
7.10 Επαναχρησιμοποίηση Οπτικών Στοιχείων 93
Κεφάλαιο 8. Εφαρμογή Guest Kiosk 95
8.1 Ρόλος και Θέση στην Αρχιτεκτονική 95
8.2 Ροή Ενεργοποίησης Συσκευής 95
8.3 Kiosk Mode μέσω Android LockTask API 97
8.4 Ροή Παραγγελίας και Πλοήγηση 98
8.5 Ανάγνωση Κάρτας και Πληρωμή 102
8.6 Παρακολούθηση Κατάστασης Παραγγελίας και Αδράνεια 104
8.7 Σχεδίαση για Λειτουργία Kiosk 107
8.8 Τοπική Αποθήκευση: DataStore και Κρυπτογραφημένα Preferences 108
8.9 Ανεξαρτησία από τον Φορέα Πληρωμών 108
Κεφάλαιο 9. Ολοκλήρωση Συστήματος και Επίδειξη 109
9.1 Πλήρης Ροή Λειτουργίας 109
9.2 Περιβάλλον Ανάπτυξης και Επίδειξης 111

9.3 Επαλήθευση της Ολοκληρωμένης Ροής 113
9.4 Γνωστοί Περιορισμοί 114
9.5 Τεχνολογίες, Εκδόσεις και Μέγεθος του Κώδικα 115
9.6 Διαδικασία Ανάπτυξης 117
Κεφάλαιο 10. Συμπεράσματα και Μελλοντική Εργασία 119
10.1 Αποτίμηση Ακαδημαϊκών Στόχων 119
10.2 Απαιτήσεις έναντι Αποτελεσμάτων ανά Ρόλο 120
10.3 Συμπεράσματα για το Kotlin Multiplatform σε Πρακτική Εφαρμογή 121
10.4 Διδάγματα από την Υλοποίηση 122
10.5 Όρια της Αξιολόγησης 123
10.6 Προτάσεις Μελλοντικής Επέκτασης 124
10.7 Επίλογος 124
Βιβλιογραφία 127
Παράρτημα Α: Αναφορά API 131
Α.1 Ταυτοποίηση Χρηστών (/auth) 131
Α.2 Διαχείριση Πλατφόρμας και Ιδιοκτητών (/owners, /admin, /company-lookup, /settings) 132
Α.3 Διαχείριση Καταλυμάτων (/properties, /geocode) 134
Α.4 Διαχείριση Συσκευών Kiosk (/properties/{propertyId}/devices, /devices) 135
Α.5 Κατάλογος Προϊόντων και Διαθεσιμότητα (/products, /food-platforms) 137
Α.6 Διαχείριση Παραγγελιών (/orders, /device/orders, /food) 139
Α.7 Πληρωμές και Βοηθητικές Υπηρεσίες (/payments, /dashboard, /audit-logs, /health) 141
Παράρτημα Β: Γλωσσάρι Ξενικών Όρων και Ακρωνυμίων 143
Β.1 Ξενικοί όροι 143
Β.2 Ακρωνύμια 151

Κατάλογος Εικόνων

Εικόνα 2.1: Τα τρία μοντέλα απομόνωσης σε multi-tenant σύστημα — shared schema, schema-per-tenant, database-per-tenant

Εικόνα 4.1: Δομή modules του monorepo και κατεύθυνση εξαρτήσεων

Εικόνα 4.2: Εφαρμογή Clean Architecture — κατεύθυνση εξαρτήσεων προς τον πυρήνα

Εικόνα 4.3: Ιεραρχία οντοτήτων και αντιστοίχιση σε PostgreSQL schemas

Εικόνα 4.4: Δρομολόγηση αιτήματος στο σωστό tenant schema μέσω TenantContext

Εικόνα 4.5: Port/Adapter pattern — πραγματικοί (πράσινο) και προσομοιωμένοι (πορτοκαλί) προσαρμογείς πίσω από τα έξι port interfaces

Εικόνα 4.6: Ροή ταυτοποίησης — JWT access token και refresh token rotation

Εικόνα 5.1: Ροή δημιουργίας και ενημέρωσης tenant schema — η provisionTenant() εκτελείται κατά τη δημιουργία ιδιοκτήτη και σε κάθε εκκίνηση της εφαρμογής

Εικόνα 5.2: Οντότητες του public schema — δεδομένα πλατφόρμας

Εικόνα 5.3: Οντότητες του tenant schema — δεδομένα καταλύματος

Εικόνα 6.1: Middleware pipeline — σειρά επεξεργασίας αιτήματος από το Ktor engine έως το επίπεδο υπηρεσιών

Εικόνα 6.2: Μηχανή καταστάσεων της παραγγελίας — επιτρεπτές μεταβάσεις και ποιος κάνει την καθεμία (πληρωμή, ιδιοκτήτης, πλατφόρμα)

Εικόνα 7.1: Οθόνες του owner-web και ροή πλοήγησης ανά ρόλο

Εικόνα 7.2: Ο πίνακας ελέγχου του διαχειριστή της πλατφόρμας

Εικόνα 7.3: Ο πίνακας ελέγχου του ιδιοκτήτη

Εικόνα 7.4: Η φόρμα καταλύματος με την αναζήτηση διεύθυνσης

Εικόνα 7.5: Σύνδεση των πλατφορμών διανομής

Εικόνα 7.6: Μια παραγγελία με τις γραμμές και την πληρωμή της

Εικόνα 7.7: Η φόρμα δημιουργίας ιδιοκτήτη, με αφετηρία τον ΑΦΜ

Εικόνα 7.8: Οι ρυθμίσεις των εξωτερικών υπηρεσιών

Εικόνα 8.1: Ροή ενεργοποίησης συσκευής — σάρωση QR ή χειροκίνητη εισαγωγή, resolve, activate, έκδοση και αποθήκευση token

Εικόνα 8.2: Η οθόνη ρύθμισης της εφαρμογής kiosk

Εικόνα 8.3: Ροή οθονών του επισκέπτη — από την αρχική οθόνη έως την πληρωμή και την παρακολούθηση της παραγγελίας

Εικόνα 8.4: Επιλογή καταστήματος

Εικόνα 8.5: Το κατάστημα του καταλύματος με καλάθι

Εικόνα 8.6: Μενού εστιατορίου με το κόστος αποστολής στο καλάθι

Εικόνα 8.7: Η οθόνη πληρωμής σε αναμονή κάρτας

Εικόνα 8.8: Η οθόνη πληρωμής σε συσκευή χωρίς την υπηρεσία αναγνώστη καρτών

Εικόνα 8.9: Επιβεβαίωση παραγγελίας

Εικόνα 8.10: Παρακολούθηση παραγγελίας, με το κουμπί ακύρωσης πριν ξεκινήσει η προετοιμασία

Εικόνα 8.11: Παρακολούθηση παραγγελίας αφού ο ιδιοκτήτης ξεκίνησε την προετοιμασία

Εικόνα 8.12: Η οθόνη πληροφοριών του καταλύματος

Εικόνα 9.1: Διάγραμμα ακολουθίας (sequence diagram) της πλήρους ροής από άκρο σε άκρο, από τη δημιουργία ιδιοκτήτη έως την παράδοση μιας παραγγελίας φαγητού

Εικόνα 9.2: Τοπολογία Docker Compose — τρεις υπηρεσίες και οι μεταξύ τους εξαρτήσεις εκκίνησης

Κατάλογος Πινάκων

Πίνακας 2.1: Σύγκριση των τριών μοντέλων απομόνωσης δεδομένων

Πίνακας 3.1: Ρόλοι συστήματος, εύρος δεδομένων και μηχανισμός ταυτοποίησης

Πίνακας 4.1: Σύγκριση των υποψήφιων backend frameworks

Πίνακας 4.2: Αξιολόγηση των στρατηγικών multi-tenancy

Πίνακας 4.3: Σύγκριση των εργαλείων ORM

Πίνακας 4.4: Stateless JWT έναντι stateful server sessions

Πίνακας 5.1: Βασικές αποφάσεις σχεδίασης της βάσης δεδομένων

Πίνακας 6.1: Endpoints του API ανά κατηγορία, απαίτηση JWT και ρόλο

Πίνακας 6.2: Αντιστοίχιση εξαιρέσεων σε απαντήσεις HTTP

Πίνακας 9.1: Ομάδες ελέγχων της επαλήθευσης της ολοκληρωμένης ροής

Πίνακας 9.2: Βασικές τεχνολογίες, βιβλιοθήκες και εκδόσεις

Πίνακας 9.3: Μέγεθος του πηγαίου κώδικα ανά module

Πίνακας 10.1: Λειτουργικές απαιτήσεις έναντι αποτελεσμάτων ανά ρόλο

Πίνακας Β.1: Ξενικοί όροι, ελληνική απόδοση και επεξήγηση

Πίνακας Β.2: Ακρωνύμια, πλήρης μορφή και επεξήγηση

Κεφάλαιο 1. Εισαγωγή

Η εργασία αφορά τη σχεδίαση και την υλοποίηση του Guest Desk, ενός ολοκληρωμένου multi-tenant συστήματος για καταλύματα βραχυχρόνιας μίσθωσης. Το σύστημα συνδυάζει την κεντρική διαχείριση της πλατφόρμας και των καταλυμάτων, μέσω μιας διαδικτυακής εφαρμογής (web panel), με την παροχή ψηφιακών υπηρεσιών αυτοεξυπηρέτησης στον επισκέπτη, μέσω ενός tablet (kiosk) μέσα στο κατάλυμα. Οι αρχιτεκτονικές του επιλογές, από την οργάνωση των δεδομένων κάθε ιδιοκτήτη στη βάση έως την ανάπτυξη όλων των υποσυστημάτων στην ίδια γλώσσα προγραμματισμού, απορρέουν από συγκεκριμένες λειτουργικές και μη λειτουργικές απαιτήσεις. Το κεφάλαιο παρουσιάζει το πρόβλημα, το ακαδημαϊκό κίνητρο και τους πρακτικούς στόχους, τη συνεισφορά της εργασίας και τη διάρθρωση των επόμενων κεφαλαίων.

1.1 Περιγραφή Προβλήματος

Η βραχυχρόνια μίσθωση κατοικιών έχει μετασχηματίσει τον κλάδο της φιλοξενίας. Πλατφόρμες κρατήσεων όπως το Airbnb, με περισσότερες από 7,7 εκατομμύρια ενεργές καταχωρήσεις παγκοσμίως στα τέλη του 2023 [1], βασίζουν τη λειτουργία τους στις τεχνολογίες του διαδικτύου [2]: ο επισκέπτης ελέγχει τη διαθεσιμότητα σε πραγματικό χρόνο, ολοκληρώνει την κράτηση ηλεκτρονικά και συχνά εισέρχεται στο κατάλυμα αυτόνομα (self check-in). Μέχρι την άφιξη, η εμπειρία του είναι σε μεγάλο βαθμό ψηφιακή. Μετά την είσοδο στο κατάλυμα, ωστόσο, η ψηφιακή αυτή συνέχεια διακόπτεται.

Όταν ο φιλοξενούμενος χρειαστεί κατά τη διαμονή του ένα είδος πρώτης ανάγκης, ένα γεύμα ή μια τοπική υπηρεσία, συνήθως δεν βρίσκει μέσα στο κατάλυμα ένα ενιαίο σημείο εξυπηρέτησης. Οι εφαρμογές διανομής φαγητού λειτουργούν ανεξάρτητα από το κατάλυμα: δεν δείχνουν τα προϊόντα που διαθέτει ο ίδιος ο ιδιοκτήτης ούτε τις πληροφορίες της διαμονής. Η απευθείας επικοινωνία με τον ιδιοκτήτη, τηλεφωνικά ή με μηνύματα, εξαρτάται από τη διαθεσιμότητά του και κλιμακώνεται δύσκολα όταν διαχειρίζεται πολλά καταλύματα, ενώ η αναζήτηση καταστημάτων στην περιοχή μεταφέρει το βάρος στον επισκέπτη. Προκύπτει έτσι η ανάγκη για ένα σημείο αυτοεξυπηρέτησης (kiosk) μέσα στο ίδιο το κατάλυμα.

Αντίστοιχο κατακερματισμό αντιμετωπίζει και ο ιδιοκτήτης. Τα ψηφιακά εργαλεία του τουρισμού προέρχονται από πολλούς, ασύνδετους μεταξύ τους παρόχους [3], με αποτέλεσμα όποιος διαχειρίζεται πολλά καταλύματα να μην έχει ενιαία εικόνα των παραγγελιών σε καθένα. Αν θέλει να διαθέτει στους επισκέπτες δικά του προϊόντα, πρέπει να καταγράφει ο ίδιος τις παραγγελίες και τις πληρωμές.

Σε επίπεδο πλατφόρμας, η εξυπηρέτηση πολλών ανεξάρτητων ιδιοκτητών από την ίδια υποδομή (multi-tenancy) θέτει ως βασική απαίτηση τον διαχωρισμό των δεδομένων: οι παραγγελίες, οι πληρωμές και τα στοιχεία των επισκεπτών κάθε ιδιοκτήτη δεν πρέπει να είναι προσβάσιμα από άλλους. Ο

διαχωρισμός είναι προτιμότερο να αποτυπώνεται στην ίδια την οργάνωση της βάσης δεδομένων και όχι μόνο σε φίλτρα που πρέπει να θυμάται κάθε ερώτημα (§2.2).

Η αυτοεξυπηρέτηση του φιλοξενούμενου, η συγκεντρωτική διαχείριση από τον ιδιοκτήτη και ο διαχωρισμός των δεδομένων στην πλατφόρμα είναι οι τρεις όψεις του προβλήματος που καλείται να καλύψει το σύστημα.

1.2 Κίνητρο και Στόχοι

1.2.1 Ακαδημαϊκό Κίνητρο

Κύριο ακαδημαϊκό κίνητρο για την ανάπτυξη του συστήματος αποτέλεσε η αρχιτεκτονική του πολυπλοκότητα. Το σύστημα συνδυάζει σε ένα έργο τρεις απαιτητικές περιοχές της μηχανικής λογισμικού: την οργάνωση των δεδομένων σε περιβάλλον multi-tenancy, την αποσύζευξη της επιχειρησιακής λογικής από τις εξωτερικές υπηρεσίες και τον διαμοιρασμό κώδικα ανάμεσα σε ετερογενή περιβάλλοντα εκτέλεσης (server, Android, browser), χωρίς να υλοποιείται ο ίδιος κώδικας πολλές φορές.

Σε επίπεδο τεχνολογίας, η εργασία χρησιμοποιεί την Kotlin ως ενιαία γλώσσα σε όλα τα υποσυστήματα, μέσω του Kotlin Multiplatform (KMP) [4]. Η Kotlin είναι καθιερωμένη στον server και στο Android· το πιο νέο στοιχείο είναι ο browser, όπου το web panel μεταγλωττίζεται σε WebAssembly. Η υλοποίηση δείχνει στην πράξη τόσο τα οφέλη του κοινού κώδικα όσο και τους τρέχοντες περιορισμούς του οικοσυστήματος (§10.3).

Η σχεδίαση βασίζεται στις αρχές της Clean Architecture [5] και στο πρότυπο Port/Adapter, ώστε ο πυρήνας της επιχειρησιακής λογικής να παραμένει ανεξάρτητος από τις λεπτομέρειες των εξωτερικών παρόχων (§4.4).

1.2.2 Πρακτικοί Στόχοι

Σε πρακτικό επίπεδο, στόχος της εργασίας ήταν ένα ολοκληρωμένο σύστημα που λειτουργεί στην πράξη. Οι στόχοι διαρθρώνονται ανά ρόλο χρήστη:

Για τον **φιλοξενούμενο**: ένα σημείο παραγγελίας και πληροφόρησης μέσα στο κατάλυμα, που λειτουργεί χωρίς εγκατάσταση εφαρμογής, χωρίς λογαριασμό και χωρίς να χρειάζεται γνώση της τοπικής αγοράς. Από την ίδια οθόνη ο επισκέπτης παραγγέλνει προϊόντα του καταλύματος ή φαγητό από τις πλατφόρμες διανομής που έχει ενεργοποιήσει ο ιδιοκτήτης, πληρώνει με κάρτα στη συσκευή μέσω εξωτερικού φορέα πληρωμών και βλέπει πληροφορίες της διαμονής του, όπως το δίκτυο Wi-Fi και την ώρα αναχώρησης.

Για τον **ιδιοκτήτη καταλύματος**: ένα ενιαίο web panel για τη διαχείριση καταλυμάτων, συσκευών, προϊόντων και παραγγελιών (Κεφάλαιο 7). Ένα νέο tablet ενεργοποιείται με σάρωση κωδικού QR, χωρίς ειδικές τεχνικές γνώσεις (§8.2). Ο ιδιοκτήτης επιλέγει ποιες πλατφόρμες διανομής προσφέρονται σε κάθε κατάλυμα και καταχωρεί τη διεύθυνση κάθε καταλύματος με αναζήτηση.

Για τον **διαχειριστή πλατφόρμας**: κεντρική εποπτεία των ιδιοκτητών, καταγραφή των σημαντικών ενεργειών (audit log) και ένταξη νέων ιδιοκτητών χωρίς χειροκίνητες επεμβάσεις στη βάση δεδομένων. Τα εταιρικά στοιχεία ενός νέου ιδιοκτήτη συμπληρώνονται από δημόσιο εταιρικό μητρώο με βάση τον ΑΦΜ, και οι ρυθμίσεις των εξωτερικών υπηρεσιών ορίζονται από το ίδιο web panel.

1.3 Συνεισφορά της Εργασίας

Η συνεισφορά της εργασίας συνοψίζεται σε τέσσερα σημεία:

Μία γλώσσα σε όλο το σύστημα (mono-language full-stack). Το backend (Ktor), η εφαρμογή Android kiosk και το web panel (Compose Multiplatform) είναι γραμμένα σε Kotlin, σε ένα ενιαίο αποθετήριο (monorepo). Τα κοινά modules :domain και :data μεταγλωττίζονται για δύο στόχους: τη JVM, την οποία χρησιμοποιούν το backend και η εφαρμογή Android, και το wasmJs για τον browser (§4.5.4). Έτσι, μια ασυμφωνία στη δομή των δεδομένων ή στη σειριοποίηση εντοπίζεται κατά τη μεταγλώττιση (compile-time safety) και όχι κατά την εκτέλεση.

Διαχωρισμός δεδομένων με schema-per-tenant. Κάθε ιδιοκτήτης έχει δικό του PostgreSQL schema (tenant_{id}) [6], ώστε τα δεδομένα του να βρίσκονται σε δικούς του πίνακες και τα ερωτήματα να μη χρειάζονται φίλτρο ανά ιδιοκτήτη (§4.5.3). Το backend χρησιμοποιεί έναν κοινό λογαριασμό βάσης και επιλέγει το schema κεντρικά σε κάθε αίτημα (§4.3)· έτσι τα repositories εκτελούν απλά ερωτήματα, χωρίς λογική multi-tenancy, και περιορίζεται ο κίνδυνος να αναμειχθούν κατά λάθος δεδομένα διαφορετικών ιδιοκτητών.

Ενεργοποίηση συσκευών kiosk με κωδικό QR. Ένα νέο tablet συνδέεται με το κατάλυμα σαρώνοντας έναν κωδικό QR από το web panel· ο κωδικός ενεργοποίησης μίας χρήσης που περιέχει αρκεί για να λάβει η εφαρμογή το διαπιστευτήριο της συσκευής, χωρίς χειροκίνητες ρυθμίσεις (§8.2).

Εξωτερικές υπηρεσίες μέσω του προτύπου Port/Adapter. Κάθε εξωτερική υπηρεσία ορίζεται ως διεπαφή (port) στο module :domain και υλοποιείται από έναν adapter στο επίπεδο υποδομής [7] (§4.4). Τρεις διασυνδέσεις επικοινωνούν με πραγματικές υπηρεσίες: η αποστολή ηλεκτρονικού ταχυδρομείου μέσω SMTP [8], η αναζήτηση στοιχείων επιχειρήσεων στη διεπαφή ανοικτών δεδομένων ενός δημόσιου εταιρικού μητρώου [9] και η αναζήτηση διευθύνσεων σε μια δημόσια υπηρεσία γεωκωδικοποίησης [10], [11]. Οι πλατφόρμες διανομής φαγητού και ο φορέας πληρωμών καρτών προσομοιώνονται, επειδή η σύνδεση με αυτούς προϋποθέτει εμπορική συμφωνία (§3.4.1). Έτσι, οι ιδιαιτερότητες κάθε παρόχου μένουν στον adapter του και δεν επιβαρύνουν την επιχειρησιακή λογική.

1.4 Δομή του Κειμένου

Κεφάλαιο 2 — Θεωρητικό Υπόβαθρο και Σχετικές Εργασίες. Παρουσιάζει το πλαίσιο της βραχυχρόνιας μίσθωσης, τα μοντέλα multi-tenancy, τις αρχές της Clean Architecture και του προτύπου Port/Adapter, το Kotlin Multiplatform και συναφείς λύσεις ψηφιακής φιλοξενίας.

Κεφάλαιο 3 — Ανάλυση Απαιτήσεων. Καταγράφει τις λειτουργικές απαιτήσεις ανά ρόλο, τις μη λειτουργικές απαιτήσεις (διαχωρισμός δεδομένων, ασφάλεια, απρόσκοπτη εμπειρία επισκέπτη) και την οριοθέτηση των πραγματικών και των προσομοιωμένων υπηρεσιών.

Κεφάλαιο 4 — Αρχιτεκτονική Συστήματος. Αναλύει τις αρχιτεκτονικές και τεχνολογικές αποφάσεις και τις αντισταθμίσεις τους (trade-offs): τη δομή των modules, την εφαρμογή της Clean Architecture, τη δρομολόγηση των αιτημάτων ανά tenant, την επιλογή του Ktor, τη χρήση του KMP και την ταυτοποίηση με JWT.

Κεφάλαιο 5 — Σχεδίαση Βάσης Δεδομένων. Περιγράφει τα schemas public και tenant_{id}, τις σχεδιαστικές επιλογές των πινάκων, τους περιορισμούς ακεραιότητας και τις σχέσεις των οντοτήτων.

Κεφάλαιο 6 — Υλοποίηση Backend. Εξετάζει την υλοποίηση του REST API σε Ktor, τα επίπεδα middleware και υπηρεσιών, τη μηχανή καταστάσεων των παραγγελιών και τους adapters των εξωτερικών υπηρεσιών.

Κεφάλαιο 7 — Owner Web Panel. Παρουσιάζει την εφαρμογή διαχείρισης σε Compose Multiplatform (wasmJs), τη διαχείριση κατάστασης, την πλοήγηση και τον διαχωρισμό αρμοδιοτήτων μεταξύ διαχειριστή πλατφόρμας και ιδιοκτήτη.

Κεφάλαιο 8 — Εφαρμογή Guest Kiosk. Περιγράφει την εφαρμογή Android του tablet: την ενεργοποίηση με κωδικό QR, το κλείδωμα σε λειτουργία kiosk μέσω του Android LockTask API, την παραγγελία χωρίς λογαριασμό χρήστη και την πληρωμή με κάρτα.

Κεφάλαιο 9 — Ολοκλήρωση Συστήματος και Επίδειξη. Παρουσιάζει τη λειτουργία των υποσυστημάτων ως συνόλου σε περιβάλλον Docker Compose, την επαλήθευση της ολοκληρωμένης ροής και τους γνωστούς περιορισμούς του πρωτοτύπου.

Κεφάλαιο 10 — Συμπεράσματα και Μελλοντική Εργασία. Αποτιμά την επίτευξη των στόχων, συνοψίζει τα συμπεράσματα από τη χρήση του Kotlin Multiplatform και προτείνει κατευθύνσεις μελλοντικής επέκτασης.

Κεφάλαιο 2. Θεωρητικό Υπόβαθρο και Σχετικές Εργασίες

Το κεφάλαιο τοποθετεί το σύστημα στο εννοιολογικό και τεχνολογικό του πλαίσιο. Ξεκινά από την αγορά της βραχυχρόνιας μίσθωσης και τις προσδοκίες των επισκεπτών και συνεχίζει με τα τέσσερα τεχνικά πεδία στα οποία στηρίζεται η σχεδίαση: τα μοντέλα απομόνωσης δεδομένων στο multi-tenancy, τις αρχές της Clean Architecture και του προτύπου Port/Adapter, το Kotlin Multiplatform (KMP) για την κοινή χρήση κώδικα και τη μεταγλώττιση σε WebAssembly για τον browser. Στο τέλος παρουσιάζονται σχετικές εργασίες και υπάρχοντα συστήματα στην ψηφιακή φιλοξενία. Η παρουσίαση μένει σε επίπεδο εννοιών και συγκρίσεων· οι συγκεκριμένες επιλογές του συστήματος και οι αντισταθμίσεις (trade-offs) τους εξετάζονται στο Κεφάλαιο 4.

2.1 Η Αγορά Βραχυχρόνιας Μίσθωσης και οι Ψηφιακές Υπηρεσίες

Μέσα σε μία δεκαετία, η βραχυχρόνια μίσθωση κατοικιών εξελίχθηκε από μια περιθωριακή μορφή της οικονομίας του διαμοιρασμού σε σημαντικό τμήμα του παγκόσμιου τουρισμού. Ο Guttentag [2] εξετάζει την άνοδο πλατφορμών όπως το Airbnb με βάση τη θεωρία της ανατρεπτικής καινοτομίας (disruptive innovation): μια υπηρεσία που απευθύνθηκε αρχικά σε τμήματα της αγοράς που δεν εξυπηρετούνταν καλά (ταξιδιώτες με μικρό προϋπολογισμό, επισκέπτες που ήθελαν μεγαλύτερους χώρους, ιδιώτες με ακίνητα που δεν χρησιμοποιούσαν) και βελτίωσε σταδιακά την αξιοπιστία και την ποιότητά της· εξετάζει επίσης κατά πόσο το Airbnb μπορεί να εκτοπίσει τα παραδοσιακά τουριστικά καταλύματα.

Μαζί με την αγορά άλλαξαν και οι προσδοκίες των ταξιδιωτών. Η εμπειρική μελέτη των Guttentag κ.ά. για τους λόγους επιλογής του Airbnb διακρίνει πέντε ομάδες επισκεπτών με βάση τα κίνητρά τους (από όσους θέλουν να εξοικονομήσουν χρήματα έως όσους αναζητούν οικιακές ανέσεις ή αυθεντικές τοπικές εμπειρίες), με ισχυρότερο κίνητρο τα πρακτικά πλεονεκτήματα του Airbnb [12]. Οι επισκέπτες που έχουν συνηθίσει τις παροχές ενός ξενοδοχείου (υπηρεσία δωματίου, άμεση εξυπηρέτηση, mini-bar) αναζητούν συχνά αντίστοιχες ευκολίες και σε ένα αυτόνομο κατάλυμα. Η βραχυχρόνια μίσθωση όμως λειτουργεί χωρίς μόνιμο προσωπικό στον χώρο, οπότε τέτοιες υπηρεσίες δύσκολα προσφέρονται με τον παραδοσιακό τρόπο.

Έτσι προκύπτει μια ασυμμετρία στην ψηφιακή εμπειρία του επισκέπτη: τα στάδια μέχρι την άφιξη (αναζήτηση, κράτηση, πληρωμή, αυτόνομη είσοδος) είναι σε μεγάλο βαθμό ψηφιακά και απρόσκοπτα (frictionless), ενώ μέσα στο κατάλυμα συνήθως δεν υπάρχει ανάλογη ψηφιακή υποδομή (§1.1).

Το πρόβλημα επιδεινώνεται από τον κατακερματισμό των τεχνολογικών λύσεων στη φιλοξενία. Οι Aldebert κ.ά. [3], εξετάζοντας καινοτόμα έργα που παρουσιάστηκαν στην έκθεση Tourism@, καταγράφουν έντονη καινοτομία με επίκεντρο τις τεχνολογίες πληροφορικής και επικοινωνιών (ΤΠΕ),

από πολύ διαφορετικούς φορείς: νεοφυείς επιχειρήσεις, μικρομεσαίες εταιρείες υψηλής τεχνολογίας και μεγάλες πολυεθνικές. Επειδή τα εργαλεία προέρχονται από τόσο διαφορετικούς φορείς, τα εργαλεία που έχουν στη διάθεσή τους οι ιδιοκτήτες λειτουργούν συνήθως αυτόνομα, χωρίς να επικοινωνούν μεταξύ τους. Στην πράξη, ένας ιδιοκτήτης με πολλά καταλύματα συνδυάζει ένα σύστημα διαχείρισης καταλυμάτων (Property Management System — PMS) για τα ημερολόγια και τις κρατήσεις, εφαρμογές μηνυμάτων για την επικοινωνία και εξωτερικές πλατφόρμες παραγγελιών. Τα εργαλεία αυτά σπάνια καλύπτουν τη διάθεση προϊόντων και υπηρεσιών στον επισκέπτη κατά τη διάρκεια της διαμονής, δηλαδή προϊόντα και υπηρεσίες που προσφέρονται από το ίδιο το κατάλυμα, με κεντρικό έλεγχο τιμών και άμεση παρακολούθηση των παραγγελιών.

Επειδή η ίδια ανάγκη εμφανίζεται σε πολλούς ανεξάρτητους ιδιοκτήτες, μια κατάλληλη προσέγγιση είναι μια πλατφόρμα multi-tenant SaaS: ένας κοινός πυρήνας λειτουργιών, με τα δεδομένα κάθε ιδιοκτήτη χωριστά από τα δεδομένα των άλλων.

2.2 Αρχιτεκτονική Multi-Tenant και Μοντέλα Απομόνωσης Δεδομένων

Ένα σύστημα multi-tenant εξυπηρετεί πολλούς ανεξάρτητους οργανισμούς ή πελάτες (tenants) από μία εγκατάσταση (instance) της εφαρμογής και κοινή υποδομή, κρατώντας χωριστά τα δεδομένα και τις ρυθμίσεις του καθενός. Στην προσέγγιση single-tenant, αντίθετα, κάθε πελάτης έχει τη δική του εγκατάσταση λογισμικού και υποδομής. Το multi-tenancy επωφελείται από την κοινή χρήση υπολογιστικών, δικτυακών και αποθηκευτικών πόρων και απλοποιεί τη συντήρηση και τις αναβαθμίσεις. Η βασική τεχνική δυσκολία του είναι η απομόνωση (isolation): τα δεδομένα ενός tenant δεν πρέπει να είναι προσβάσιμα από άλλον, ούτε από σφάλμα στον κώδικα ούτε από κακόβουλη ενέργεια.

Στο σύστημα της εργασίας, οι tenants είναι οι ιδιοκτήτες των καταλυμάτων. Η πλατφόρμα διαχειρίζεται οικονομικά στοιχεία, καταλόγους προϊόντων, προσωπικά δεδομένα διαμονής και ιστορικό παραγγελιών, δεδομένα με σημαντική εμπορική αξία· η αξιόπιστη απομόνωσή τους είναι επομένως βασική απαίτηση.

Η βιβλιογραφία (Krebs κ.ά. [6], Chong κ.ά. [13]) περιγράφει τρία βασικά μοντέλα οργάνωσης των δεδομένων σε multi-tenancy. Κάθε μοντέλο επιτυγχάνει διαφορετική ισορροπία ανάμεσα στον βαθμό απομόνωσης, στο κόστος λειτουργίας και στη διαχειριστική πολυπλοκότητα:

1. **Shared Schema (Κοινό Schema):** Όλοι οι tenants μοιράζονται την ίδια βάση δεδομένων και τους ίδιους πίνακες. Κάθε πίνακας έχει μια στήλη (tenant_id) που δείχνει σε ποιον ανήκει η εγγραφή, και κάθε ερώτημα πρέπει να φιλτράρει τις εγγραφές με βάση αυτή τη στήλη. Είναι η πιο οικονομική λύση, και η ένταξη ενός νέου tenant απαιτεί απλώς μια νέα εγγραφή. Η απομόνωση όμως εξαρτάται εξ ολοκλήρου από την ορθότητα της εφαρμογής: αν ένα ερώτημα

παραλείψει το φίλτρο tenant_id, μπορεί να εμφανίσει δεδομένα άλλου πελάτη. Δύσκολη γίνεται επίσης η λήψη αντιγράφων ασφαλείας ή η επαναφορά δεδομένων για έναν μόνο tenant.

2. **Schema-per-Tenant:** Όλοι οι tenants μοιράζονται την ίδια βάση δεδομένων και το ίδιο connection pool, αλλά κάθε tenant έχει το δικό του schema, δηλαδή έναν ξεχωριστό χώρο ονομάτων (namespace) με την ίδια δομή πινάκων. Ένα ερώτημα που δεν αναφέρει ρητά schema εκτελείται στους πίνακες του schema που ορίζει το search_path της σύνδεσης, οπότε η παράλειψη ενός φίλτρου δεν αρκεί για να εκτεθούν εγγραφές άλλου tenant. Το πόσο ισχυρή είναι η απομόνωση εξαρτάται όμως από τον τρόπο σύνδεσης: αν κάθε tenant συνδέεται με δικό του ρόλο της βάσης, το σύστημα διαχείρισης βάσεων δεδομένων απαγορεύει την πρόσβαση στα schemas των άλλων· αν όλοι μοιράζονται έναν ρόλο, ένα ερώτημα μπορεί να διαβάσει τους πίνακες άλλου schema αν αναφέρει ρητά το όνομά του, και η επιλογή του σωστού schema αποτελεί ευθύνη της εφαρμογής. Ένας νέος tenant προστίθεται με αυτόματη εκτέλεση εντολών DDL που δημιουργούν το schema του. Το μοντέλο προσφέρει ισχυρότερο διαχωρισμό από το κοινό schema, με χαμηλό κόστος υποδομής, αφού ένα κοινό connection pool εξυπηρετεί όλο το σύστημα.
3. **Database-per-Tenant:** Κάθε tenant έχει τη δική του φυσική ή λογική βάση δεδομένων, ίσως και σε ξεχωριστό διακομιστή. Το μοντέλο αυτό δίνει τη μεγαλύτερη απομόνωση, επιτρέπει ρύθμιση της απόδοσης ανά πελάτη και διευκολύνει τη συμμόρφωση με αυστηρούς κανονισμούς απορρήτου. Το κόστος υποδομής και ο φόρτος λειτουργίας όμως αυξάνονται ανάλογα με τον αριθμό των tenants: χρειάζονται πολλά connection pools, πολλά αντίγραφα ασφαλείας και ενημέρωση της δομής (migrations) σε κάθε βάση χωριστά.

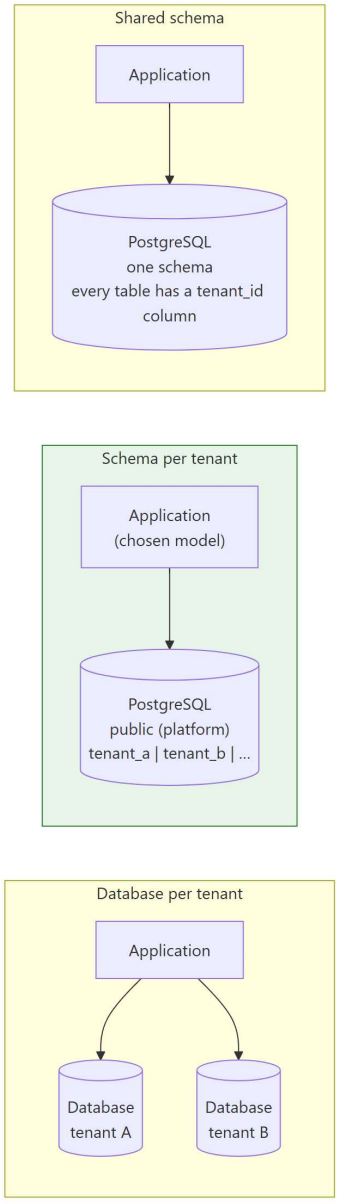
Ο Πίνακας 2.1 συγκρίνει τα τρία μοντέλα:

Πίνακας 2.1: Σύγκριση των τριών μοντέλων απομόνωσης δεδομένων

Μοντέλο Απομόνωσης	**Βαθμός Απομόνωσης**	**Λειτουργικό Κόστος Υποδομής**	**Πολυπλοκότητα Migrations**	**Κίνδυνος Διαρροής Δεδομένων**
Shared Schema	Χαμηλός (λογικός σε επίπεδο κώδικα)	Ελάχιστο (ενιαίοι πόροι)	Χαμηλή (μία εκτέλεση DDL)	Υψηλός (παράλειψη φίλτρου WHERE)
Schema-per-Tenant	Υψηλός (ξεχωριστό namespace, ισχυρότερος με ρόλο βάσης ανά tenant)	Μέτριο (κοινό connection pool)	Μέτρια (εκτέλεση ανά tenant schema)	Χαμηλός (λάθος επιλογή schema, όχι ξεχασμένο φίλτρο)
Database-per-Tenant	Πολύ υψηλός (φυσικός διαχωρισμός)	Υψηλό (ξεχωριστοί πόροι ανά tenant)	Υψηλή (πολλαπλές βάσεις δεδομένων)	Πολύ χαμηλός (ξεχωριστή βάση)

Ανεξάρτητα από το μοντέλο αποθήκευσης, ένα multi-tenant σύστημα χρειάζεται τρόπο να μεταφέρει την ταυτότητα του tenant σε όλη τη διαδρομή του αιτήματος (tenant context propagation), να προετοιμάζει αυτόματα τα δεδομένα κάθε νέου tenant κατά την ένταξη (onboarding) και να διαχειρίζεται τα κοινά δεδομένα της πλατφόρμας.

Η Εικόνα 2.1 δείχνει τα τρία μοντέλα. Η επιλογή του schema-per-tenant για το σύστημα, και η αιτιολόγησή της, παρουσιάζονται στην §4.5.3.



Εικόνα 2.1: Τα τρία μοντέλα απομόνωσης σε multi-tenant σύστημα — shared schema, schema-per-tenant, database-per-tenant.

2.3 Clean Architecture και το Port/Adapter (Hexagonal) Pattern

Στην κλασική πολυεπίπεδη αρχιτεκτονική (n-tier architecture), οι εξαρτήσεις του κώδικα ακολουθούν τη ροή ελέγχου προς τα κάτω: η παρουσίαση εξαρτάται από την επιχειρησιακή λογική, και η επιχειρησιακή λογική από το επίπεδο δεδομένων, που με τη σειρά του είναι δεμένο με τη συγκεκριμένη βάση και τα frameworks. Το μειονέκτημα είναι ότι ο πυρήνας της επιχειρησιακής λογικής, το πιο σταθερό και πολύτιμο μέρος ενός συστήματος, καταλήγει να εξαρτάται από λεπτομέρειες υλοποίησης που αλλάζουν συχνά.

Η Clean Architecture, όπως τη διατύπωσε ο Martin [5], αντιστρέφει αυτή την κατεύθυνση με τον Κανόνα Εξάρτησης (Dependency Rule): οι εξαρτήσεις του κώδικα δείχνουν πάντα προς τα μέσα, δηλαδή προς τον πυρήνα, όπου βρίσκονται οι πιο γενικοί κανόνες του συστήματος. Το σύστημα οργανώνεται σε ομόκεντρα επίπεδα:

Οντότητες (Entities): Περιέχουν τους γενικούς επιχειρησιακούς κανόνες (enterprise-wide business rules).

Περιπτώσεις Χρήσης (Use Cases): Συντονίζουν τη ροή των δεδομένων από και προς τις οντότητες και υλοποιούν τους κανόνες της συγκεκριμένης εφαρμογής.

Προσαρμογείς Διεπαφής (Interface Adapters): Μετατρέπουν τα δεδομένα από τη μορφή της επιχειρησιακής λογικής στη μορφή που χρειάζονται τα εξωτερικά συστήματα (controllers, presenters, gateways).

Πλαίσια και Οδηγοί (Frameworks & Drivers): Το εξωτερικό επίπεδο, με εργαλεία όπως η βάση δεδομένων, το web framework και οι διεπαφές συσκευών.

Τα εσωτερικά επίπεδα δεν γνωρίζουν τίποτα για τα εξωτερικά. Όταν ένα εσωτερικό επίπεδο χρειάζεται δεδομένα ή υπηρεσίες από ένα εξωτερικό, ορίζει το ίδιο μια διεπαφή (interface), και το εξωτερικό επίπεδο την υλοποιεί. Έτσι η ροή ελέγχου μπορεί να κινείται προς τα έξω, ενώ η εξάρτηση του κώδικα μένει στραμμένη προς τον πυρήνα.

Η Hexagonal Architecture, γνωστή και ως πρότυπο Ports and Adapters [7], εκφράζει την ίδια αρχή με έμφαση στα όρια της εφαρμογής. Ο πυρήνας τοποθετείται μέσα σε ένα εξάγωνο και επικοινωνεί με τον έξω κόσμο μόνο μέσω θυρών (ports), που χωρίζονται σε δύο κατηγορίες:

Πρωτεύουσες ή Εισερχόμενες Θύρες (Driving / Inbound Ports): Ορίζουν τις λειτουργίες που προσφέρει ο πυρήνας. Οι αντίστοιχοι προσαρμογείς (REST controllers, CLI handlers ή στοιχεία διεπαφής) καλούν τις μεθόδους των use cases.

Δευτερεύουσες ή Εξερχόμενες Θύρες (Driven / Outbound Ports): Ορίζουν τις υπηρεσίες που χρειάζεται ο πυρήνας από εξωτερικά συστήματα (αποθήκευση, πύλες πληρωμών, ειδοποιήσεις). Εδώ ο πυρήνας καλεί τις διεπαφές που έχει ορίσει ο ίδιος, και τις υλοποιούν οι αντίστοιχοι προσαρμογείς.

Ο συνδυασμός της Clean Architecture και του Port/Adapter έχει τρία βασικά πλεονεκτήματα:

1. **Δυνατότητα Απομονωμένου Ελέγχου (Testability):** Ο πυρήνας μπορεί να ελεγχθεί με unit tests χωρίς βάση δεδομένων, κλήσεις δικτύου ή εξωτερικές υπηρεσίες.
2. **Ανεξαρτησία από Υποδομές (Infrastructure Independence):** Η βάση δεδομένων, το framework δικτύωσης ή η διεπαφή χρήστη μπορούν να αναβαθμιστούν ή να αντικατασταθούν με ελάχιστες αλλαγές στην επιχειρησιακή λογική.
3. **Εναλλαξιμότητα Υλοποιήσεων (Adapter Interchangeability):** Ο adapter μιας πραγματικής εξωτερικής υπηρεσίας και ένας emulated adapter υλοποιούν το ίδιο port, οπότε το σύστημα μπορεί να αναπτυχθεί και να ελεγχθεί πριν από οποιαδήποτε εμπορική συμφωνία με εξωτερικούς παρόχους.

Στο σύστημα της εργασίας, το Port/Adapter χρησιμοποιείται για τις εξωτερικές υπηρεσίες (§4.4): τον φορέα πληρωμών καρτών, τις πλατφόρμες διανομής φαγητού, την αποστολή ηλεκτρονικού ταχυδρομείου, το εταιρικό μητρώο και την υπηρεσία γεωκωδικοποίησης διευθύνσεων. Ο γενικότερος κανόνας της Clean Architecture καθορίζει τις εξαρτήσεις ανάμεσα στα modules (§4.2).

2.4 Kotlin Multiplatform — Τεχνική Επισκόπηση και Αξιολόγηση

Η κοινή χρήση κώδικα (code sharing) ανάμεσα σε διαφορετικές πλατφόρμες είναι παλιό ζητούμενο της μηχανικής λογισμικού. Οι βασικές στρατηγικές που έχουν χρησιμοποιηθεί είναι τέσσερις:

1. **Πλήρως Ανεξάρτητη Ανάπτυξη:** Κάθε πλατφόρμα υλοποιείται χωριστά στη δική της γλώσσα (π.χ. Swift για iOS, Kotlin για Android, TypeScript για Web). Η απόδοση είναι συνήθως η καλύτερη, αλλά το κόστος συντήρησης πολλαπλασιάζεται και οι επιχειρησιακοί κανόνες κινδυνεύουν να αποκλίνουν.
2. **Κοινό Περιβάλλον Εκτέλεσης (Runtime-based / Hybrid):** Ένα κοινό περιβάλλον εκτέλεσης αποκρύπτει τις διαφορές των πλατφορμών, όπως στις υβριδικές εφαρμογές που τρέχουν μέσα σε webview, με γνωστά προβλήματα απόδοσης και διεπαφή που αποκλίνει από την εγγενή (native).
3. **Ενιαίο Γραφικό Περιβάλλον (Cross-Platform UI Frameworks):** Ένα framework με ενιαίο κώδικα διεπαφής για όλες τις πλατφόρμες. Το Flutter σχεδιάζει τα γραφικά στοιχεία με τη δική του μηχανή γραφικών, παρακάμπτοντας τα εγγενή widgets, ενώ το React Native περιγράφει τη διεπαφή σε JavaScript και την αποδίδει με τα εγγενή widgets της πλατφόρμας, μέσω ενός ενδιάμεσου επιπέδου επικοινωνίας με τον εγγενή κώδικα [14].
4. **Κοινή Επιχειρησιακή Λογική με Εγγενή Μεταγλώττιση:** Η λογική και τα μοντέλα δεδομένων μοιράζονται χωρίς κοινό περιβάλλον εκτέλεσης· ο κοινός κώδικας μεταγλωττίζεται απευθείας στην εγγενή μορφή κάθε πλατφόρμας.

Το Kotlin Multiplatform (KMP) ανήκει στην τέταρτη στρατηγική και επιτρέπει, προαιρετικά, να μοιράζεται και η διεπαφή μέσω του Compose Multiplatform. Συγκριτικές μελέτες των cross-platform εργαλείων (Nawrocki κ.ά. [14], Punia κ.ά. [15]) εξετάζουν τις αντισταθμίσεις (trade-offs) κάθε στρατηγικής ανάμεσα στην απόδοση και στο κόστος ανάπτυξης. Σε αντίθεση με προσεγγίσεις που περνούν από ενδιάμεση γέφυρα (όπως το JavaScript bridge του κλασικού React Native), ο κώδικας του KMP εκτελείται απευθείας στην πλατφόρμα-στόχο.

Το KMP στηρίζεται σε τρεις μηχανισμούς [4]:

1. **Σύνολα Πηγαίου Κώδικα (Source Sets):** Ο κώδικας οργανώνεται ιεραρχικά. Το commonMain περιέχει κώδικα Kotlin ανεξάρτητο από πλατφόρμα, ενώ τα source sets ανά πλατφόρμα (jvmMain, androidMain, wasmJsMain) περιέχουν κώδικα για τις ιδιαιτερότητες κάθε περιβάλλοντος.
2. **Μηχανισμός expect / actual:** Όταν ο κοινός κώδικας χρειάζεται λειτουργικότητα που διαφέρει ανά πλατφόρμα, ορίζει μια δήλωση με τη λέξη-κλειδί expect, και κάθε πλατφόρμα δίνει τη δική της υλοποίηση actual. Η αντιστοίχιση γίνεται κατά τη μεταγλώττιση, χωρίς κόστος κατά την εκτέλεση.
3. **Πολλαπλοί Στόχοι Μεταγλώττισης (Compilation Targets):** Ο ίδιος κοινός κώδικας μεταγλωττίζεται από διαφορετικά backends του μεταγλωττιστή Kotlin: από το Kotlin/JVM σε Java bytecode, που εκτελείται στον server και, αφού η αλυσίδα εργαλείων του Android (D8) το μετατρέψει σε DEX bytecode, στο kiosk tablet· και από το Kotlin/Wasm σε WebAssembly για τον browser. Ο μεταγλωττιστής ελέγχει αυστηρά τους τύπους: μια αλλαγή σε κοινό DTO ή domain interface που δεν συμφωνεί με τον κώδικα που τη χρησιμοποιεί προκαλεί σφάλμα μεταγλώττισης.

Στην κοινή χρήση των δεδομένων βοηθά η επίσημη βιβλιοθήκη kotlinx.serialization. Στηρίζεται σε compiler plugin και όχι σε reflection κατά την εκτέλεση: οι κοινές κλάσεις επισημαίνονται με το annotation @Serializable και ο κώδικας σειριοποίησης και αποσειριοποίησης JSON παράγεται αυτόματα για όλους τους στόχους.

Ως προς την ωριμότητα, η JetBrains χαρακτήρισε τον πυρήνα του KMP σταθερό (Stable) τον Νοέμβριο του 2023 [16]. Η ωριμότητα όμως διαφέρει ανά στόχο: οι στόχοι JVM και Android είναι ώριμοι και δοκιμασμένοι σε μεγάλη κλίμακα, ενώ ο στόχος WebAssembly (Kotlin/Wasm) είναι νεότερος.

Το WebAssembly (Wasm) ορίζεται ως μια συμπαγής, φορητή και ασφαλής δυαδική μορφή εντολών για μηχανή στοίβας, με ταχύτητα εκτέλεσης κοντά σε εκείνη του εγγενούς κώδικα, μέσα στο απομονωμένο περιβάλλον (sandbox) του browser [17]. Σχεδιάστηκε αρχικά για γλώσσες χωρίς αυτόματη διαχείριση μνήμης (όπως C++ και Rust)· η επέκταση WasmGC πρόσθεσε υποστήριξη garbage collection απευθείας στη μηχανή του browser [4]. Το Kotlin/Wasm απαιτεί browser που υποστηρίζει το WasmGC και είναι ο στόχος που χρησιμοποιεί το Compose Multiplatform για το web· κατά τη συγγραφή της

εργασίας, η JetBrains το χαρακτηρίζει ως Beta [4]. Το Compose Multiplatform σχεδιάζει τη διεπαφή σε επιφάνεια canvas μέσω της βιβλιοθήκης γραφικών Skia.

Η πρακτική αξιολόγηση της προσέγγισης και η επιλογή του KMP έναντι άλλων λύσεων παρουσιάζονται στις §4.5.1, §4.5.4 και §4.5.5.

2.5 Σχετικά Συστήματα και Ακαδημαϊκές Εργασίες

Το σύστημα βρίσκεται στην τομή τριών περιοχών: οι multi-tenant πλατφόρμες SaaS, τα σταθερά συστήματα αυτοεξυπηρέτησης (kiosk) μέσα στο κατάλυμα και η αυτόματη δρομολόγηση παραγγελιών στη βραχυχρόνια μίσθωση. Στη βιβλιογραφία που εξετάστηκε δεν εντοπίστηκε ένα σύστημα που να καλύπτει και τις τρεις μαζί. Τα σχετικά έργα κατανέμονται σε τρεις χωριστούς τομείς:

1. **Συστήματα Διαδραστικών Συσκευών Δωματίου στη Φιλοξενία:** Στα ξενοδοχεία, τα tablet μέσα στα δωμάτια (όπως οι εμπορικές λύσεις SuitePad και Crave Interactive) είναι καθιερωμένη πρακτική. Σε ακαδημαϊκό επίπεδο, οι Zhu και Morosan [18] μελέτησαν εμπειρικά τους παράγοντες (όπως η γνωστική εμβύθιση, η παιγνιώδης διάθεση και η ασφάλεια) που οδηγούν τους επισκέπτες ξενοδοχείων να χρησιμοποιούν διαδραστικές φορητές συσκευές. Οι Kim και Qu [19] έδειξαν ότι η στάση των ταξιδιωτών απέναντι στα kiosk αυτοεξυπηρέτησης των ξενοδοχείων εξαρτάται από την αντιληπτή χρησιμότητα, την ευκολία χρήσης, τη συμβατότητα και τον αντιληπτό κίνδυνο, κάτι που συμφωνεί με την επιλογή ενός kiosk απλού στη χρήση και χωρίς λογαριασμό. Τα συστήματα αυτά όμως είναι συνήθως σχεδιασμένα για μία μόνο ξενοδοχειακή μονάδα (single-tenant λογική), συνδέονται κυρίως με ξενοδοχειακά PMS και προϋποθέτουν προσωπικό ορόφου και ρεσεψιόν, που συνήθως δεν υπάρχουν στη βραχυχρόνια μίσθωση.
2. **Πλατφόρμες Συγκέντρωσης Παραγγελιών και Logistics:** Η διάδοση των πλατφορμών παραγγελίας φαγητού και ταχυμεταφοράς έχει στρέψει το ενδιαφέρον στη διαχείριση παραγγελιών και στην ανάθεση διαδρομών. Τα συστήματα αυτά όμως απευθύνονται στον τελικό καταναλωτή. Δεν λαμβάνουν υπόψη το κατάλυμα, δεν επιτρέπουν στον ιδιοκτήτη να ορίζει τον δικό του κατάλογο προϊόντων και υπηρεσιών. Γι' αυτό η εργασία αντιμετωπίζει τις πλατφόρμες αυτές ως εξωτερικούς παρόχους που ενσωματώνονται στο σύστημα: ο ιδιοκτήτης επιλέγει ποιες προσφέρονται σε κάθε κατάλυμα, και ο επισκέπτης παραγγέλνει από τα εστιατόριά τους από την ίδια συσκευή με την οποία αγοράζει τα προϊόντα του καταλύματος.
3. **Μηχανική Multi-Tenant Συστημάτων SaaS:** Όπως φάνηκε στην §2.2, η έρευνα στη μηχανική λογισμικού (Krebs κ.ά. [6], Chong κ.ά. [13]) έχει διατυπώσει μοντέλα για την απομόνωση των δεδομένων και τη διαχείριση πόρων σε εφαρμογές cloud. Οι μελέτες αυτές όμως αφορούν γενικές επιχειρησιακές εφαρμογές (enterprise SaaS) και δεν εξετάζουν τη σύνδεση φυσικών συσκευών kiosk με χωριστά schemas δεδομένων.

Πέρα από τους τρεις αυτούς τομείς, ένα σύστημα που απευθύνεται σε επιχειρήσεις μπορεί να αντλεί στοιχεία από δημόσιες πηγές δεδομένων, αντί να ζητά από τον χρήστη να πληκτρολογεί πληροφορίες που υπάρχουν ήδη αλλού. Ορισμένα δημόσια εταιρικά μητρώα, όπως το ελληνικό, διαθέτουν τα δημοσιευμένα στοιχεία των επιχειρήσεων (επωνυμία, νομική μορφή, έδρα, κατάσταση) ως ανοικτά δεδομένα μέσω προγραμματιστικής διεπαφής [9]. Υπάρχουν επίσης παγκόσμια, ελεύθερα διαθέσιμα σύνολα γεωγραφικών δεδομένων, που δημιουργούν οι ίδιοι οι χρήστες τους [11], πάνω στα οποία λειτουργούν υπηρεσίες γεωκωδικοποίησης, δηλαδή μετατροπής μιας διεύθυνσης σε συντεταγμένες και αντίστροφα [10]. Η χρήση τέτοιων πηγών μειώνει τα λάθη καταχώρησης και επιταχύνει την ένταξη ιδιοκτητών και καταλυμάτων, με αντίτιμο την εξάρτηση από τη διαθεσιμότητα και τους όρους χρήσης τρίτων υπηρεσιών.

Συνολικά, η βιβλιογραφία για τη βραχυχρόνια μίσθωση [2], [3], [12] εστιάζει κυρίως στη διοίκηση και στην οικονομία του τουρισμού και περιγράφει την εμπειρία του επισκέπτη, χωρίς να εξετάζει ζητήματα αρχιτεκτονικής λογισμικού.

Η συνεισφορά της εργασίας βρίσκεται επομένως στον συνδυασμό: εφαρμόζει μαζί τις αρχές της Clean Architecture, ένα multi-tenant μοντέλο δεδομένων και το Kotlin Multiplatform σε ένα ολοκληρωμένο σύστημα για τις ανάγκες της βραχυχρόνιας μίσθωσης, όπου τα καταλύματα ανήκουν σε πολλούς ανεξάρτητους ιδιοκτήτες.

Κεφάλαιο 3. Ανάλυση Απαιτήσεων

Η ανάλυση απαιτήσεων αποτελεί τη θεμελιώδη φάση σχεδιασμού κάθε συστήματος λογισμικού: πριν από τη λήψη οποιασδήποτε αρχιτεκτονικής απόφασης και πριν από τη συγγραφή του κώδικα, πρέπει να έχουν αποσαφηνιστεί με ακρίβεια το εύρος των λειτουργιών και οι ανάγκες των χρηστών. Στο πλαίσιο της παρούσας εργασίας, η ανάλυση απαιτήσεων λειτούργησε ως η πρωταρχική βάση για τη διαμόρφωση του αρχιτεκτονικού προτύπου, τον καθορισμό των ορίων απομόνωσης των δεδομένων και τη σχεδίαση των επιμέρους πινάκων στη βάση δεδομένων.

Το σύστημα εξυπηρετεί τρεις κατηγορίες χρηστών με διαφορετικές επιχειρησιακές ανάγκες και επίπεδα πρόσβασης: τον διαχειριστή της υποδομής που εποπτεύει πολλαπλούς tenants, τους ιδιοκτήτες, που επιζητούν πλήρη διαχειριστικό έλεγχο των καταλυμάτων τους χωρίς περιττή τεχνική επιβάρυνση, και τους φιλοξενούμενους που απαιτούν άμεση εξυπηρέτηση από τον χώρο διαμονής τους χωρίς την ανάγκη εγκατάστασης λογισμικού ή δημιουργίας προσωπικού λογαριασμού. Κεντρικός στόχος του σχεδιασμού ήταν η σύνθεση ενός ενιαίου, ασφαλούς και αποδοτικού συστήματος που καλύπτει αυτές τις ετερογενείς απαιτήσεις.

3.1 Λειτουργικές Απαιτήσεις

Οι λειτουργικές απαιτήσεις προσδιορίζουν τις συγκεκριμένες υπηρεσίες και λειτουργίες που παρέχει το σύστημα σε κάθε κατηγορία χρηστών. Η οργάνωσή τους ακολουθεί τον διαχωρισμό των ρόλων, καθώς η πρόσβαση στις λειτουργίες και στα δεδομένα διέπεται από αυστηρό έλεγχο δικαιωμάτων βάσει ρόλων (Role-Based Access Control — RBAC).

3.1.1 Διαχειριστής Πλατφόρμας (Platform Admin)

Ο Platform Admin αποτελεί τον κεντρικό διαχειριστή (operator) της υποδομής SaaS. Είναι υπεύθυνος για την εποπτεία της πλατφόρμας, τη διαχείριση των λογαριασμών των ιδιοκτητών και τις ρυθμίσεις των εξωτερικών υπηρεσιών, χωρίς να εμπλέκεται στην καθημερινή επιχειρησιακή λειτουργία των επιμέρους καταλυμάτων.

Διαχείριση tenants. Ο διαχειριστής δημιουργεί, επεξεργάζεται, απενεργοποιεί και επανενεργοποιεί λογαριασμούς ιδιοκτητών. Ένας ιδιοκτήτης δεν διαγράφεται οριστικά, ώστε να διατηρείται το ιστορικό των παραγγελιών και των πληρωμών του. Κατά την καταχώρηση νέου ιδιοκτήτη, το σύστημα δημιουργεί αυτόματα ένα ξεχωριστό, απομονωμένο schema στη βάση δεδομένων (tenant_{id}). Ο ιδιοκτήτης καταχωρείται ως επιχείρηση: η εγγραφή περιλαμβάνει την επωνυμία και τον διακριτικό τίτλο, τον ΑΦΜ, τον αριθμό μητρώου, τη νομική μορφή, τη δραστηριότητα, τη διεύθυνση της έδρας

και τα στοιχεία του υπεύθυνου επικοινωνίας. Με την εισαγωγή του ΑΦΜ, το σύστημα αναζητά την επιχείρηση στο δημόσιο εταιρικό μητρώο [9] και συμπληρώνει αυτόματα τα δημοσιευμένα στοιχεία της, τα οποία ο διαχειριστής ελέγχει πριν από την αποθήκευση. Κατά την ολοκλήρωση της δημιουργίας, παράγεται αυτόματα ένας προσωρινός κωδικός πρόσβασης, ο οποίος αποστέλλεται στον ιδιοκτήτη με μήνυμα καλωσορίσματος μέσω ηλεκτρονικού ταχυδρομείου και εμφανίζεται μία φορά στον διαχειριστή. Αν ένας ιδιοκτήτης ξεχάσει τον κωδικό του, ο διαχειριστής μπορεί να τον επαναφέρει: παράγεται νέος προσωρινός κωδικός, ο οποίος αποστέλλεται με τον ίδιο τρόπο, και οι ανοιχτές συνεδρίες του ιδιοκτήτη δεν μπορούν πλέον να ανανεωθούν (§4.5.7).

Κεντρική εποπτεία πλατφόρμας. Ο διαχειριστής έχει πρόσβαση σε συγκεντρωτικό πίνακα ελέγχου (dashboard) που απεικονίζει τη συνολική δραστηριότητα: αριθμό ενεργών και ανενεργών ιδιοκτητών, συνολικό πλήθος καταλυμάτων και συσκευών kiosk, καθώς και την κατανομή των παραγγελιών όλης της πλατφόρμας ανά κατάσταση. Από την ίδια οθόνη μπορεί να μεταβεί στα δεδομένα οποιουδήποτε ιδιοκτήτη (καταλύματα, συσκευές, προϊόντα, παραγγελίες) για σκοπούς υποστήριξης.

Αρχείο καταγραφής ενεργειών (Audit Log). Οι σημαντικές διαχειριστικές ενέργειες — όπως η δημιουργία ιδιοκτήτη, η μεταβολή στοιχείων καταλύματος, η απενεργοποίηση συσκευής, η διαγραφή πόρων και οι επιστροφές χρημάτων — καταγράφονται σε κεντρικό πίνακα (audit log) στο schema της πλατφόρμας (public), στον οποίο γίνονται μόνο εισαγωγές. Κάθε εγγραφή περιλαμβάνει το αναγνωριστικό του ενεργούντος χρήστη (actor), τον ρόλο του, το είδος της ενέργειας, την οντότητα που επηρεάστηκε, το σχετικό tenant schema και τη χρονοσφραγίδα. Το εύρος της καταγραφής αιτιολογείται στην §6.9.

Ρυθμίσεις εξωτερικών υπηρεσιών. Ο διαχειριστής ρυθμίζει από το web panel τις υπηρεσίες που αφορούν ολόκληρη την πλατφόρμα: τον διακομιστή εξερχόμενου ηλεκτρονικού ταχυδρομείου (SMTP), με δυνατότητα αποστολής δοκιμαστικού μηνύματος, και το κλειδί πρόσβασης στη διεπαφή ανοικτών δεδομένων του εταιρικού μητρώου. Για τα μυστικά στοιχεία των ρυθμίσεων αυτών (κωδικός SMTP, κλειδί του μητρώου), ο διαχειριστής βλέπει μόνο αν έχουν οριστεί (§3.2.2).

3.1.2 Ιδιοκτήτης Καταλύματος (Owner)

Ο ιδιοκτήτης αποτελεί τον tenant της πλατφόρμας και διαχειρίζεται αποκλειστικά τα δικά του καταλύματα, τις συσκευές και τον κατάλογο των προϊόντων του. Τα δεδομένα του είναι απομονωμένα από εκείνα των άλλων ιδιοκτητών (§3.2.1).

Διαχείριση καταλυμάτων. Ο ιδιοκτήτης καταχωρεί και διαμορφώνει τα καταλύματά του. Κάθε κατάλυμα περιλαμβάνει πλήρη στοιχεία διεύθυνσης (οδός, αριθμός, ταχυδρομικός κώδικας, περιοχή, πόλη, όροφος, όνομα στο κουδούνι), τηλέφωνο επικοινωνίας και προαιρετικές σημειώσεις. Η διεύθυνση εισάγεται μέσω αναζήτησης: ο ιδιοκτήτης πληκτρολογεί μέρος της διεύθυνσης ή

χρησιμοποιεί την τρέχουσα θέση του, και το σύστημα συμπληρώνει τα επιμέρους πεδία και τις γεωγραφικές συντεταγμένες από μια δημόσια υπηρεσία γεωκωδικοποίησης [10]. Για κάθε κατάλυμα ο ιδιοκτήτης ορίζει επίσης τις πληροφορίες που προβάλλονται στον επισκέπτη (όνομα και κωδικό του δικτύου Wi-Fi, ώρα αναχώρησης, κανόνες του καταλύματος). Ο ιδιοκτήτης μπορεί να θέσει ένα κατάλυμα σε προσωρινή αδράνεια χωρίς να χαθούν οι σχετικές ρυθμίσεις και το ιστορικό του.

Διαχείριση συσκευών kiosk. Για κάθε κατάλυμα, ο ιδιοκτήτης μπορεί να δηλώσει και να ενεργοποιήσει μία ή περισσότερες συσκευές Android tablet. Η δημιουργία συσκευής στο web panel παράγει αυτόματα έναν μοναδικό κωδικό ενεργοποίησης και έναν αντίστοιχο κωδικό QR, ώστε η φυσική συσκευή να ενεργοποιείται άμεσα στον χώρο. Για κάθε συσκευή ο ιδιοκτήτης βλέπει την κατάσταση ενεργοποίησής της, την ένδειξη τελευταίας επικοινωνίας (lastSeenAt), την οποία η συσκευή ανανεώνει κάθε λεπτό, και το ιστορικό παραγγελιών της. Έτσι διακρίνει άμεσα μια συσκευή που έχει χάσει τη σύνδεσή της. Μπορεί να απενεργοποιήσει μια συσκευή, οπότε το διαπιστευτήριό της απορρίπτεται από το επόμενο κιόλας αίτημα, αλλά η εγγραφή του kiosk διατηρείται και η επανενεργοποίηση στο web panel αρκεί για να συνεχίσει τη λειτουργία του· ή να την αποσυνδέσει από το tablet, οπότε παράγεται νέος κωδικός ενεργοποίησης και το kiosk επιστρέφει στην οθόνη εγγραφής για επανεγκατάσταση.

Διαχείριση καταλόγου προϊόντων. Ο ιδιοκτήτης καθορίζει τα προϊόντα που διατίθενται στους επισκέπτες. Κάθε προϊόν χαρακτηρίζεται από όνομα, αναλυτική περιγραφή, κατηγορία (π.χ. σνακ, ροφήματα, είδη προσωπικής υγιεινής), προαιρετική εικόνα και τιμή πώλησης. Ο κατάλογος είναι κοινός για όλα τα καταλύματα του ιδιοκτήτη· ο ιδιοκτήτης μπορεί να μεταβάλει άμεσα τη διαθεσιμότητα ενός προϊόντος συνολικά (isAvailable) ή να το απενεργοποιήσει μόνο σε συγκεκριμένα καταλύματα, και η αλλαγή αποτυπώνεται στα αντίστοιχα tablet. Για την ταχύτερη δημιουργία του καταλόγου, ο ιδιοκτήτης μπορεί να εισαγάγει προϊόντα από τον κατάλογο μιας συνδεδεμένης πλατφόρμας διανομής.

Σύνδεση πλατφορμών διανομής φαγητού. Ο ιδιοκτήτης συνδέει τις πλατφόρμες διανομής φαγητού με τις οποίες συνεργάζεται, εισάγοντας το κλειδί πρόσβασης που του έχει δώσει κάθε πλατφόρμα. Το κλειδί επαληθεύεται από την πλατφόρμα κατά τη σύνδεση και αποθηκεύεται κρυπτογραφημένο. Στη συνέχεια, ο ιδιοκτήτης ορίζει για κάθε κατάλυμα ποιες από τις συνδεδεμένες πλατφόρμες προσφέρονται στους επισκέπτες του. Μια πλατφόρμα δεν μπορεί να αποσυνδεθεί όσο υπάρχουν παραγγελίες της σε εξέλιξη.

Εποπτεία και διαχείριση παραγγελιών. Ο ιδιοκτήτης έχει πρόσβαση στις εισερχόμενες παραγγελίες ανά κατάλυμα, με δυνατότητα φιλτραρίσματος βάσει κατάστασης. Μπορεί να δει αναλυτικά τα επιλεγμένα προϊόντα, τις ποσότητες, την τιμή ανά τεμάχιο κατά τον χρόνο παραγγελίας, τις απόπειρες πληρωμής και τα σχόλια του επισκέπτη. Για τις παραγγελίες προϊόντων του καταλύματος, ενημερώνει την κατάσταση προόδου μέσα από μια μηχανή καταστάσεων (CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED)· την επιβεβαίωση την κάνει η εγκεκριμένη πληρωμή. Την

πρόοδο των παραγγελιών φαγητού την ενημερώνει η ίδια η πλατφόρμα διανομής, οπότε ο ιδιοκτήτης μπορεί μόνο να τις ακυρώσει. Η ακύρωση επιτρέπεται από οποιοδήποτε μη τελικό στάδιο και, αν η παραγγελία έχει ήδη πληρωθεί, το σύστημα ζητά αυτόματα από τον φορέα πληρωμών την επιστροφή του ποσού.

Πίνακας επισκόπησης επιχείρησης. Ο ιδιοκτήτης παρακολουθεί συνοπτικά στατιστικά στοιχεία: το πλήθος των ενεργών καταλυμάτων και των συνδεδεμένων συσκευών και την τρέχουσα κατανομή των παραγγελιών ανά στάδιο επεξεργασίας.

3.1.3 Φιλοξενούμενος (Guest)

Ο φιλοξενούμενος αποτελεί τον τελικό χρήστη της εφαρμογής kiosk που βρίσκεται εγκατεστημένη εντός του καταλύματος. Βασική αρχή του σχεδιασμού είναι η απρόσκοπτη χρήση (frictionless experience), χωρίς λήψη εφαρμογής ή λογαριασμό (§3.2.3).

Επιλογή καταστήματος. Η αρχική οθόνη αγορών παρουσιάζει τα «καταστήματα» από τα οποία μπορεί να παραγγείλει ο επισκέπτης: το κατάστημα του καταλύματος, με τα προϊόντα του ιδιοκτήτη, και τα εστιατόρια των πλατφορμών διανομής που είναι ενεργοποιημένες στο συγκεκριμένο κατάλυμα, ομαδοποιημένα ανά πλατφόρμα. Για κάθε εστιατόριο εμφανίζονται το είδος κουζίνας, η βαθμολογία, το κόστος αποστολής, ο εκτιμώμενος χρόνος παράδοσης και, όπου υπάρχει, η ελάχιστη παραγγελία.

Πλοήγηση στον κατάλογο. Ο επισκέπτης περιηγείται στα διαθέσιμα προϊόντα του καταλύματος ή στο μενού ενός εστιατορίου, ταξινομημένα σε κατηγορίες, βλέποντας αναλυτικές περιγραφές και τιμές. Από τον κατάλογο του καταλύματος προβάλλονται αποκλειστικά τα είδη που είναι διαθέσιμα στο συγκεκριμένο κατάλυμα.

Διαχείριση καλαθιού και υποβολή παραγγελίας. Ο επισκέπτης επιλέγει ποσότητες, επισκοπεί το περιεχόμενο του καλαθιού με αυτόματο υπολογισμό του τελικού κόστους (συμπεριλαμβανομένου του κόστους αποστολής για τις παραγγελίες φαγητού) και προχωρά στην υποβολή της παραγγελίας χωρίς διαδικασία σύνδεσης. Κάθε καλάθι αφορά ένα κατάστημα· αν ο επισκέπτης προσθέσει είδος από άλλο κατάστημα, η εφαρμογή ζητά επιβεβαίωση και, αν συμφωνήσει, αδειάζει το καλάθι. Προαιρετικά, ο επισκέπτης μπορεί να δηλώσει διεύθυνση email για την παραλαβή ψηφιακής απόδειξης.

Πληρωμή στη συσκευή. Ο επισκέπτης πληρώνει στη συσκευή, με την κάρτα του στον ενσωματωμένο αναγνώστη καρτών, χωρίς ανακατεύθυνση σε άλλη εφαρμογή. Η χρέωση, η έγκριση ή η απόρριψη της συναλλαγής και οι επιστροφές χρημάτων είναι αρμοδιότητα του εξωτερικού φορέα πληρωμών· το σύστημα του διαβιβάζει το αίτημα και καταγράφει το αποτέλεσμα. Μια απορριφθείσα συναλλαγή δεν ακυρώνει την παραγγελία: ο επισκέπτης ενημερώνεται για την αιτία και μπορεί να δοκιμάσει ξανά.

Ζωντανή παρακολούθηση κατάστασης. Μετά την επιτυχή πληρωμή, η συσκευή μεταβαίνει αυτόματα σε οθόνη επιβεβαίωσης και παρακολούθησης, προβάλλοντας σε σχεδόν πραγματικό χρόνο

τα διαδοχικά στάδια εκτέλεσης των παραγγελιών της τρέχουσας επίσκεψης μέσω περιοδικού ελέγχου (polling). Από την ίδια οθόνη ο επισκέπτης μπορεί να ακυρώσει ο ίδιος μια παραγγελία, εφόσον δεν έχει αρχίσει ακόμη η προετοιμασία της, οπότε ζητείται αυτόματα η επιστροφή του ποσού. Αφού αρχίσει η προετοιμασία, η ακύρωση γίνεται μόνο από τον ιδιοκτήτη (§3.1.2), του οποίου τα στοιχεία επικοινωνίας εμφανίζονται στην ίδια οθόνη.

Πληροφορίες διαμονής. Ο επισκέπτης έχει πρόσβαση σε οθόνη πληροφοριών με τα στοιχεία του καταλύματος, το δίκτυο Wi-Fi, την ώρα αναχώρησης, τους κανόνες του καταλύματος, το τηλέφωνο του ιδιοκτήτη και τους αριθμούς έκτακτης ανάγκης.

3.2 Μη Λειτουργικές Απαιτήσεις

Οι μη λειτουργικές απαιτήσεις καθορίζουν τα ποιοτικά χαρακτηριστικά, τις αρχιτεκτονικές ιδιότητες και τους λειτουργικούς περιορισμούς που διέπουν το σύστημα.

3.2.1 Δομική Απομόνωση Δεδομένων (Data Isolation)

Η απομόνωση των δεδομένων αποτελεί την πλέον κρίσιμη μη λειτουργική απαίτηση του συστήματος. Τα δεδομένα κάθε ιδιοκτήτη (κατάλογοι, παραγγελίες, οικονομικές συναλλαγές, στοιχεία συσκευών) πρέπει να παραμένουν χωριστά από εκείνα των υπολοίπων tenants.

Η απαίτηση αυτή υλοποιείται με την αρχιτεκτονική schema-per-tenant [6], [13]. Απορρίπτεται ο διαχωρισμός μέσω φίλτρου σε κάθε ερώτημα της εφαρμογής (π.χ. φίλτρο WHERE owner_id = ?), για τους λόγους της §2.2. Κάθε ιδιοκτήτης έχει δικό του PostgreSQL schema, και το search_path ορίζεται αυτόματα σε κάθε συναλλαγή από ένα μόνο σημείο του κώδικα (§4.3).

3.2.2 Ασφάλεια και Ταυτοποίηση

Το σύστημα οφείλει να καλύπτει τις ακόλουθες απαιτήσεις ασφάλειας:

1. **Διαπιστευτήρια χρηστών:** Οι κωδικοί πρόσβασης δεν αποθηκεύονται σε απλό κείμενο (plaintext), αλλά μόνο ως hashes με χρήση του αλγορίθμου BCrypt [20]. Η προστασία τους κατά τη μετάδοση προϋποθέτει κρυπτογραφημένη σύνδεση (TLS), την οποία η εγκατάσταση επίδειξης δεν διαθέτει (§9.4).
2. **Διαχείριση συνεδριών (Sessions):** Η ταυτοποίηση βασίζεται σε JSON Web Tokens (JWT) [21] σύντομης διάρκειας ζωής (15 λεπτά για χρήστες), συνοδευόμενα από τυχαία refresh tokens διάρκειας 30 ημερών με εναλλαγή (token rotation): κάθε refresh token γίνεται δεκτό μόνο μία φορά και αντικαθίσταται από νέο [22].

3. **Ταυτότητα συσκευών:** Οι συσκευές kiosk διαθέτουν εξειδικευμένο JWT ταυτότητας συσκευής (role = DEVICE) με ενσωματωμένα το αναγνωριστικό της συσκευής και το όνομα του tenant schema. Το token εκδίδεται αποκλειστικά κατά την επιτυχή ενεργοποίηση της συσκευής και αποθηκεύεται σε ασφαλή, κρυπτογραφημένο χώρο στο Android tablet. Επειδή η διάρκεια ζωής του είναι μεγάλη (ένα έτος), το backend ελέγχει σε κάθε αίτημα ότι η συσκευή παραμένει ενεργή, ώστε η απενεργοποίησή της από τον ιδιοκτήτη να ισχύει αμέσως· ο ίδιος έλεγχος σταματά και τις συσκευές ενός απενεργοποιημένου ιδιοκτήτη (§4.5.7).
4. **Μυστικά εξωτερικών υπηρεσιών:** Τα κλειδιά πρόσβασης των πλατφορμών διανομής, ο κωδικός του διακομιστή SMTP και το κλειδί του εταιρικού μητρώου αποθηκεύονται στη βάση δεδομένων κρυπτογραφημένα με συμμετρικό αλγόριθμο, με κλειδί που ορίζεται στο περιβάλλον εκτέλεσης του backend και δεν αποθηκεύεται στη βάση. Κανένα από αυτά τα στοιχεία δεν επιστρέφεται ποτέ μέσω του API.
5. **Διαχείριση κωδικών και άμεση ανάκληση:** Κάθε διαχειριστής και ιδιοκτήτης μπορεί να αλλάξει τον κωδικό του, επιβεβαιώνοντας τον τρέχοντα· μετά την αλλαγή, οι άλλες συνεδρίες του χρήστη δεν μπορούν πλέον να ανανεωθούν. Η απενεργοποίηση ενός ιδιοκτήτη ισχύει από το επόμενο κιόλας αίτημά του, χωρίς να αναμένεται η λήξη του access token.
6. **Rate limiting:** Στα δημόσια endpoints που λαμβάνουν μυστικά στοιχεία (σύνδεση, ανανέωση συνεδρίας, κωδικός ενεργοποίησης) επιτρέπεται περιορισμένος αριθμός αιτημάτων ανά διεύθυνση IP, ώστε να αποτρέπονται οι μαζικές απόπειρες εύρεσης κωδικών με διαδοχικές δοκιμές (brute force). Αντίστοιχο όριο ανά χρήστη ισχύει για την αναζήτηση διευθύνσεων.
7. **Δεδομένα κάρτας:** Τα ευαίσθητα δεδομένα της κάρτας δεν διαβιβάζονται στο backend· ο χειρισμός τους ανήκει στον αναγνώστη καρτών και στον φορέα πληρωμών. Η συσκευή αποστέλλει μόνο τον τρόπο ανάγνωσης της κάρτας, το αποτέλεσμα της επαλήθευσης του κατόχου και, όπου είναι διαθέσιμα, τα τελευταία τέσσερα ψηφία της κάρτας.

3.2.3 Απρόσκοπτη Αλληλεπίδραση Επισκέπτη (Zero-Friction Guest Interaction)

Ο σχεδιασμός του συστήματος επιβάλλει να μη χρειάζεται ο επισκέπτης καμία ταυτοποίηση: ο φιλοξενούμενος δεν δημιουργεί λογαριασμό χρήστη ούτε εισάγει κωδικούς πρόσβασης. Η εγκυρότητα του αιτήματος παραγγελίας προκύπτει από την ίδια τη φυσική συσκευή kiosk· το tablet, που διαθέτει ενεργό διαπιστευτήριο συσκευής (DEVICE token), πιστοποιεί στο backend ότι η παραγγελία προέρχεται από το συγκεκριμένο κατάλυμα, απαλλάσσοντας τον χρήστη από κάθε διαδικασία εγγραφής.

3.2.4 Επεκτασιμότητα Εξωτερικών Υπηρεσιών (Port/Adapter Extensibility)

Το σύστημα πρέπει να ενσωματώνει εξωτερικές υπηρεσίες: πλατφόρμες παραγγελίας φαγητού, έναν φορέα πληρωμών καρτών (acquirer), υπηρεσίες ηλεκτρονικού ταχυδρομείου, ένα δημόσιο εταιρικό μητρώο και μια υπηρεσία γεωκωδικοποίησης διευθύνσεων.

Η αρχιτεκτονική απαίτηση επιβάλλει την αποσύνδεση του πυρήνα της εφαρμογής από τις υλοποιήσεις των παρόχων μέσω του προτύπου Port/Adapter [7]. Κάθε εξωτερική υπηρεσία ορίζεται ως port interface στο module :domain. Οι προσαρμογείς (adapters) υλοποιούν αυτές τις διεπαφές εξωτερικά, επιτρέποντας την εναλλαγή μεταξύ προσομοιωμένων (emulated) και πραγματικών παρόχων χωρίς μεταβολή στην επιχειρησιακή λογική του συστήματος.

3.2.5 Αξιοπιστία και Λειτουργία Kiosk (Kiosk Mode & Availability)

Η εφαρμογή tablet οφείλει να λειτουργεί ως αποκλειστικό τερματικό εξυπηρέτησης (dedicated kiosk device):

1. **Περιορισμός πρόσβασης (LockTask Mode):** Ο επισκέπτης δεν πρέπει να μπορεί να βγει από την εφαρμογή, να ανοίξει τις ρυθμίσεις του Android, να χρησιμοποιήσει άλλες εφαρμογές ή να πλοηγηθεί στο διαδίκτυο. Το κλείδωμα γίνεται μέσω του Android LockTask API [23] και προϋποθέτει την αντίστοιχη διαμόρφωση της συσκευής (§8.3).
2. **Αυτονομία και επαναφορά:** Αν η συσκευή επανεκκινηθεί (π.χ. έπειτα από απροσδόκητο τερματισμό ή πτώση τάσης), η εφαρμογή πρέπει να εκκινεί αυτόματα (auto-restart) μόλις ολοκληρωθεί η εκκίνηση του λειτουργικού συστήματος· το σύστημα το επιδιώκει μέσω ενός BroadcastReceiver για το system event BOOT_COMPLETED του Android, με την επιφύλαξη των περιορισμών των νεότερων εκδόσεων (§8.3).
3. **Ανθεκτικότητα δικτύου:** Προσωρινές απώλειες δικτυακής σύνδεσης δεν οδηγούν σε κατάρρευση της εφαρμογής, αλλά σε σαφή ενημέρωση του χρήστη με δυνατότητα αυτόματης επανασύνδεσης.

3.2.6 Συνέπεια Δεδομένων και Ιστορικότητα (Data Consistency)

Το σύστημα οφείλει να διατηρεί τη συνέπεια των οικονομικών δεδομένων και του ιστορικού των παραγγελιών:

1. **Αμετάβλητο στιγμιότυπο τιμών (Price Snapshotting):** Τυχόν μελλοντική μεταβολή της τιμής ή της ονομασίας ενός προϊόντος από τον ιδιοκτήτη δεν πρέπει να επηρεάζει αναδρομικά προγενέστερες παραγγελίες. Κατά την καταχώρηση κάθε παραγγελίας, η εφαρμογή αποθηκεύει αποκανονικοποιημένο στιγμιότυπο (snapshot) της ισχύουσας τιμής μονάδας και του ονόματος του προϊόντος στον πίνακα γραμμών παραγγελίας (order_items).

2. **Ακεραιότητα μεταβάσεων:** Η ροή εκτέλεσης της παραγγελίας ελέγχεται από αυστηρή μηχανή καταστάσεων στο backend service layer. Μη έγκυρες μεταβάσεις (π.χ. απευθείας μετάβαση από κατάσταση ακύρωσης σε ολοκλήρωση) απορρίπτονται άμεσα με επιστροφή δομημένου σφάλματος HTTP 409 Conflict. Μια παραγγελία επιβεβαιώνεται αποκλειστικά από μια εγκεκριμένη πληρωμή· ο ιδιοκτήτης δεν μπορεί να επιβεβαιώσει χειροκίνητα μια απλήρωτη παραγγελία.
3. **Συνέπεια πληρωμών:** Μια παραγγελία μπορεί να έχει το πολύ μία εγκεκριμένη πληρωμή, ακόμη και όταν δύο αιτήματα πληρωμής για την ίδια παραγγελία φτάνουν ταυτόχρονα. Οι απορριφθείσες απόπειρες καταγράφονται χωρίς να επηρεάζουν την παραγγελία. Κάθε ακύρωση πληρωμένης παραγγελίας, είτε από τον ιδιοκτήτη, είτε από τον επισκέπτη, είτε επειδή η πλατφόρμα διανομής δεν την αποδέχθηκε, συνοδεύεται από αίτημα επιστροφής προς τον φορέα πληρωμών, το οποίο καταγράφεται στο audit log· μια αποτυχημένη επιστροφή επισημαίνεται εκεί ως περίπτωση που απαιτεί χειροκίνητη ενέργεια.

3.3 Ρόλοι Χρηστών και Περιπτώσεις Χρήσης (Use Cases)

3.3.1 Ρόλοι Χρηστών και Σύστημα Ταυτοποίησης

Το σύστημα ορίζει τρεις διακριτούς ταυτοποιημένους ρόλους λογισμικού και μία κατηγορία τελικού φυσικού χρήστη, όπως συνοψίζει ο Πίνακας 3.1:

Πίνακας 3.1: Ρόλοι συστήματος, εύρος δεδομένων και μηχανισμός ταυτοποίησης

Ρόλος Συστήματος	**Εφαρμογή / Client**	**Εύρος Δεδομένων**	**Μηχανισμός Ταυτοποίησης**
PLATFORM_ADMIN	owner-web	Πλατφόρμα & μεταδεδομένα (`public` schema), καθώς και τα δεδομένα οποιουδήποτε tenant για υποστήριξη	JWT Access Token, BCrypt Password Hash
OWNER	owner-web	Αποκλειστικά το tenant schema (`tenant\_{id}`)	JWT Access Token, BCrypt Password Hash
DEVICE	app (Android kiosk)	Κατάλογος, παραγγελίες & πληρωμές του καταλύματος	Device-scoped JWT (εκδίδεται κατά την ενεργοποίηση)

Ρόλος Συστήματος	**Εφαρμογή / Client**	**Εύρος Δεδομένων**	**Μηχανισμός Ταυτοποίησης**
GUEST (Φυσικός Χρήστης)	app (Android kiosk)	Διεπαφή πλοήγησης & υποβολής παραγγελίας	Χωρίς λογαριασμό (λειτουργεί μέσω της συσκευής `DEVICE`)

Ο φιλοξενούμενος (GUEST) χρησιμοποιεί τη συσκευή που είναι ήδη συνδεδεμένη και ταυτοποιημένη ως DEVICE. Κάθε αίτημα παραγγελίας φέρει την κρυπτογραφική υπογραφή του token της συσκευής, επιτρέποντας στο backend να αναγνωρίζει μονοσήμαντα το κατάλυμα και το αντίστοιχο tenant schema, χωρίς να απαιτείται ταυτοποίηση του επισκέπτη.

3.3.2 Use Case 1: Υποβολή Παραγγελίας από Φιλοξενούμενο

Κύριος Actor: Φιλοξενούμενος (GUEST) μέσω της ταυτοποιημένης συσκευής (DEVICE).

Προϋποθέσεις:

Η συσκευή kiosk είναι πλήρως ενεργοποιημένη και συνδεδεμένη με συγκεκριμένο κατάλυμα.
Η συσκευή διαθέτει ενεργό DEVICE JWT.
Υπάρχουν διαθέσιμα προϊόντα στον κατάλογο του καταλύματος.

Κανονική Ροή (Main Success Scenario):

1. Ο επισκέπτης επιλέγει το κατάστημα του καταλύματος και περιηγείται στον κατάλογο προϊόντων της οθόνης kiosk.
2. Επιλέγει επιθυμητά προϊόντα και ποσότητες, προσθέτοντάς τα στο καλάθι αγορών.
3. Το kiosk υπολογίζει δυναμικά το μερικό σύνολο κάθε είδους και το συνολικό κόστος σε λεπτά του ευρώ (cents).
4. Ο επισκέπτης πατά το κουμπί Checkout και μεταβαίνει στην οθόνη πληρωμής, όπου μπορεί προαιρετικά να δηλώσει διεύθυνση email για λήψη απόδειξης· ο αναγνώστης καρτών αναμένει την κάρτα.
5. Ο επισκέπτης ακουμπά την κάρτα του στον αναγνώστη (ανέπαφη συναλλαγή) ή την εισάγει στην υποδοχή chip και πληκτρολογεί το PIN.
6. Με το πρώτο συμβάν κάρτας, το kiosk αποστέλλει αίτημα δημιουργίας παραγγελίας (POST /orders?propertyId={id}) συνοδευόμενο από το DEVICE JWT. Το αίτημα περιέχει τα αναγνωριστικά των προϊόντων και τις ποσότητες, όχι τιμές.
7. Το backend επαληθεύει την ταυτότητα της συσκευής και ελέγχει ότι το κατάλυμα του αιτήματος είναι εκείνο στο οποίο ανήκει η συσκευή, υπολογίζει το συνολικό ποσό από τις τρέχουσες τιμές του καταλόγου, εισάγει εγγραφή στον πίνακα orders με αρχική κατάσταση

PENDING, αποθηκεύει τα στιγμιότυπα τιμών στον πίνακα order_items και επιστρέφει την παραγγελία με το τελικό ποσό.

8. Το kiosk αποστέλλει αμέσως αίτημα πληρωμής (POST /payments) για την παραγγελία αυτή, με τον τρόπο ανάγνωσης της κάρτας. Το backend ζητά έγκριση από τον φορέα πληρωμών, καταγράφει την εγκεκριμένη συναλλαγή (status = COMPLETED) και μεταβάλλει την κατάσταση της παραγγελίας από PENDING σε CONFIRMED. Αν έχει δοθεί email, αποστέλλεται απόδειξη.
9. Η συσκευή εμφανίζει οθόνη επιτυχούς υποβολής και ξεκινά περιοδικό έλεγχο (polling) για την παρακολούθηση της κατάστασης (§8.6).
10. Ο ιδιοκτήτης βλέπει την παραγγελία στο web panel και διαδοχικά ενημερώνει την κατάσταση (PREPARING → OUT_FOR_DELIVERY → DELIVERED).
11. Το tablet του επισκέπτη εμφανίζει κάθε αλλαγή κατάστασης στον επόμενο περιοδικό έλεγχο.

Εναλλακτική Ροή 1α (Απόρριψη Πληρωμής):

Στο βήμα 8, ο φορέας πληρωμών απορρίπτει τη συναλλαγή.

Η απόπειρα καταγράφεται ως FAILED μαζί με την αιτία απόρριψης, ενώ η παραγγελία παραμένει σε κατάσταση PENDING.

Το kiosk προβάλλει την αιτία της απόρριψης και ο αναγνώστης καρτών αναμένει εκ νέου κάρτα, ώστε ο επισκέπτης να δοκιμάσει ξανά χωρίς να δημιουργηθεί νέα παραγγελία.

Εναλλακτική Ροή 1β (Ακύρωση από τον Επισκέπτη):

Μετά το βήμα 9, και πριν ο ιδιοκτήτης αρχίσει την προετοιμασία, ο επισκέπτης επιλέγει ακύρωση της παραγγελίας από την οθόνη παρακολούθησης και την επιβεβαιώνει.

Το backend μεταβάλλει την κατάσταση σε CANCELLED, ζητά από τον φορέα πληρωμών την επιστροφή του ποσού της εγκεκριμένης πληρωμής και σημειώνει την πληρωμή ως REFUNDED.

Εναλλακτική Ροή 1γ (Προσωρινή Απώλεια Σύνδεσης κατά το Polling):

Κατά τη διάρκεια του βήματος 9, η συσκευή χάνει προσωρινά τη σύνδεση Wi-Fi.

Η εφαρμογή διατηρεί στη μνήμη τις παραγγελίες της τρέχουσας επίσκεψης και συνεχίζει τον περιοδικό έλεγχο, ανακτώντας την τρέχουσα κατάσταση αμέσως μόλις αποκατασταθεί το δίκτυο.

Μετα-συνθήκες (Post-conditions):

Η παραγγελία και η πληρωμή έχουν καταγραφεί με ασφάλεια στο tenant schema.

Τα στιγμιότυπα τιμών έχουν παγιωθεί.

Η παραγγελία εμφανίζεται στη λίστα παραγγελιών του ιδιοκτήτη στο web panel.

3.3.3 Use Case 2: Δημιουργία Νέου Tenant (Tenant Onboarding)

Κύριος Actor: Διαχειριστής Πλατφόρμας (PLATFORM_ADMIN).

Προϋποθέσεις:

Ο διαχειριστής είναι συνδεδεμένος στο σύστημα με έγκυρα διαπιστευτήρια διαχειριστή.

Κανονική Ροή:

1. Ο διαχειριστής μεταβαίνει στην οθόνη διαχείρισης ιδιοκτητών, ανοίγει τη φόρμα καταχώρησης και πληκτρολογεί τον ΑΦΜ της επιχείρησης.
2. Το web panel καλεί την υπηρεσία αναζήτησης εταιρικών στοιχείων του backend, η οποία υποβάλλει ερώτημα στη διεπαφή ανοικτών δεδομένων του εταιρικού μητρώου [9] και επιστρέφει την επωνυμία, τον διακριτικό τίτλο, τον αριθμό μητρώου, τη νομική μορφή, τη δραστηριότητα και τη διεύθυνση της έδρας. Τα στοιχεία αυτά συμπληρώνονται αυτόματα στη φόρμα.
3. Ο διαχειριστής ελέγχει τα στοιχεία και συμπληρώνει όσα αφορούν τον υπεύθυνο επικοινωνίας (ονοματεπώνυμο, email, τηλέφωνο).
4. Το σύστημα επικυρώνει τα υποχρεωτικά πεδία (ο ΑΦΜ πρέπει να αποτελείται από εννέα ψηφία) και ελέγχει ότι το email επικοινωνίας, που χρησιμοποιείται για τη σύνδεση του ιδιοκτήτη, δεν ανήκει ήδη σε άλλον ιδιοκτήτη.
5. Το σύστημα παράγει ένα μοναδικό αναγνωριστικό UUID και ένα ασφαλές όνομα schema (tenant_ ακολουθούμενο από δώδεκα δεκαεξαδικούς χαρακτήρες του αναγνωριστικού).
6. Το backend:
   – Δημιουργεί την εγγραφή του ιδιοκτήτη στον πίνακα owners του public schema, μαζί με το BCrypt hash ενός τυχαίου κωδικού πρόσβασης 12 χαρακτήρων.
   – Δημιουργεί δυναμικά το απομονωμένο PostgreSQL schema (CREATE SCHEMA tenant_{id}).
   – Εκτελεί τα DDL scripts για τη δημιουργία όλων των tenant πινάκων (properties, devices, products, orders, κ.λπ.).
7. Η ενέργεια καταγράφεται αναλυτικά στον πίνακα audit_log.
8. Το backend αποστέλλει στον ιδιοκτήτη μήνυμα καλωσορίσματος με τη διεύθυνση σύνδεσης και τον προσωρινό κωδικό, ενώ το web panel εμφανίζει τον κωδικό μία φορά και στον διαχειριστή. Ο κωδικός δεν αποθηκεύεται σε απλό κείμενο και δεν εμφανίζεται ξανά.

Εναλλακτική Ροή 2α (Η Επιχείρηση δεν Βρέθηκε στο Μητρώο):

Στο βήμα 2, το μητρώο δεν επιστρέφει επιχείρηση για τον συγκεκριμένο ΑΦΜ, η υπηρεσία δεν είναι διαθέσιμη ή δεν έχει οριστεί κλειδί πρόσβασης.

Το web panel ενημερώνει τον διαχειριστή, ο οποίος συμπληρώνει τα εταιρικά στοιχεία χειροκίνητα· η ροή συνεχίζεται από το βήμα 3.

Εναλλακτική Ροή 2β (Ήδη Υπάρχον Email):

Στο βήμα 4, διαπιστώνεται ότι το email επικοινωνίας χρησιμοποιείται ήδη από άλλον ιδιοκτήτη.

Το αίτημα απορρίπτεται άμεσα με σφάλμα HTTP 409 Conflict.

Δεν δημιουργείται κανένα νέο schema στη βάση δεδομένων και ενημερώνεται ο διαχειριστής.

Εναλλακτική Ροή 2γ (Αποτυχία Αποστολής Email):

Στο βήμα 8, ο διακομιστής ηλεκτρονικού ταχυδρομείου δεν είναι ρυθμισμένος ή δεν είναι διαθέσιμος.

Η δημιουργία του ιδιοκτήτη δεν επηρεάζεται· ο διαχειριστής παραδίδει τον προσωρινό κωδικό που εμφανίστηκε στο web panel μέσω άλλου καναλιού.

Μετα-συνθήκες:

Ο νέος tenant διαθέτει διαμορφωμένο και απομονωμένο χώρο δεδομένων, έτοιμο για χρήση.

3.3.4 Use Case 3: Ενεργοποίηση και Σύνδεση Συσκευής Kiosk

Κύριος Actor: Ιδιοκτήτης Καταλύματος (OWNER).

Προϋποθέσεις:

Ο ιδιοκτήτης έχει δημιουργήσει τουλάχιστον ένα κατάλυμα στο σύστημα.

Η εφαρμογή Android έχει εγκατασταθεί στο φυσικό tablet και εκτελείται για πρώτη φορά.

Κανονική Ροή (Σάρωση Κωδικού QR):

1. Ο ιδιοκτήτης επιλέγει ένα κατάλυμα στο web panel και ζητά τη δημιουργία νέας συσκευής, ορίζοντας ένα περιγραφικό όνομα (π.χ. «Tablet Σαλονιού»).
2. Το backend παράγει έναν μοναδικό, κρυπτογραφικά τυχαίο κωδικό ενεργοποίησης οκτώ χαρακτήρων (κεφαλαία λατινικά γράμματα και ψηφία) και συνθέτει ένα JSON payload που περιλαμβάνει και τη δημόσια διεύθυνση του API: {"deviceId": "...", "activationCode": "...", "schemaName": "...", "endpoint": "..."}.
3. Το backend παράγει τον αντίστοιχο κωδικό QR ως εικόνα PNG (κωδικοποιημένη σε Base64), την οποία το web panel προβάλλει στην οθόνη.
4. Στο tablet, η εφαρμογή kiosk εκκινεί στην οθόνη εγγραφής (Registration Screen) και ζητά σάρωση του κωδικού QR.

5. Το tablet σαρώνει τον κωδικό μέσω της κάμερας και αποκωδικοποιεί τα πεδία deviceId, activationCode, schemaName και endpoint· έτσι η συσκευή γνωρίζει και τη διεύθυνση του server χωρίς χειροκίνητη ρύθμιση.
6. Το tablet αποστέλλει αίτημα ενεργοποίησης POST /devices/{id}/activate?schemaName={schemaName} χωρίς JWT, διαβιβάζοντας τον κωδικό ενεργοποίησης και το αναγνωριστικό υλικού της συσκευής (androidDeviceId).
7. Το backend επαληθεύει ότι η συσκευή δεν έχει ενεργοποιηθεί στο παρελθόν, ότι ο κωδικός ταυτίζεται και ότι τόσο το κατάλυμα όσο και ο ιδιοκτήτης είναι ενεργοί.
8. Το backend ενεργοποιεί τη συσκευή με μία ατομική ενημέρωση, καταγράφει το androidDeviceId και διαγράφει τον κωδικό ενεργοποίησης, ώστε να μην μπορεί να ξαναχρησιμοποιηθεί (§6.2).
9. Το backend εκδίδει ένα υπογεγραμμένο DEVICE JWT και το επιστρέφει στην απάντηση.
10. Η εφαρμογή αποθηκεύει με ασφάλεια το token, το αναγνωριστικό συσκευής και το schema στο κρυπτογραφημένο DevicePreferences της συσκευής.
11. Το tablet προφορτώνει τον κατάλογο και τα στοιχεία του καταλύματος, εισέρχεται σε λειτουργία LockTask (kiosk mode) και μεταβαίνει στην οθόνη αδράνειας, έτοιμο για χρήση.

Εναλλακτική Ροή 3α (Χειροκίνητη Εισαγωγή Κωδικού Ενεργοποίησης):

Εάν η κάμερα του tablet δεν είναι διαθέσιμη, ο χρήστης δεν σαρώνει τον κωδικό QR αλλά συμπληρώνει τα πεδία της ίδιας οθόνης.

Πληκτρολογεί τη διεύθυνση του server και τον κωδικό ενεργοποίησης οκτώ χαρακτήρων που εμφανίζεται στο web panel κάτω από τον κωδικό QR, και πατά Activate Device.

Το tablet καλεί το δημόσιο endpoint GET /devices/resolve?code={code}.

Το backend εντοπίζει σε ποιο tenant schema ανήκει ο κωδικός και επιστρέφει το deviceId και το schemaName.

Η ροή συνεχίζεται κανονικά από το βήμα 6.

Εναλλακτική Ροή 3β (Μη Έγκυρος ή Ήδη Χρησιμοποιημένος Κωδικός):

Εάν ο κωδικός δεν αντιστοιχεί σε καμία συσκευή, το backend επιστρέφει HTTP 404 Not Found· εάν δεν ταιριάζει με τη συσκευή, HTTP 403 Forbidden· εάν η συσκευή έχει ήδη ενεργοποιηθεί ή εάν το κατάλυμα ή ο ιδιοκτήτης της έχουν απενεργοποιηθεί, HTTP 409 Conflict. Επειδή η ενεργοποίηση εφαρμόζεται ατομικά, από δύο ταυτόχρονες ενεργοποιήσεις με τον ίδιο κωδικό επιτυγχάνει μόνο η μία και η άλλη λαμβάνει επίσης 409.

Η εφαρμογή απορρίπτει την ενεργοποίηση, εμφανίζει στον χρήστη μήνυμα με την αιτία και παραμένει στην οθόνη εγγραφής.

Μετα-συνθήκες:

Το Android tablet είναι συνδεδεμένο με το συγκεκριμένο κατάλυμα και διαθέτει κρυπτογραφικά επικυρωμένη ταυτότητα συσκευής (DEVICE role).

3.3.5 Use Case 4: Εποπτεία και Διαχείριση Παραγγελιών από Ιδιοκτήτη

Κύριος Actor: Ιδιοκτήτης Καταλύματος (OWNER).

Προϋποθέσεις:

Ο ιδιοκτήτης είναι συνδεδεμένος στο web panel.
Υπάρχουν καταγεγραμμένες παραγγελίες στο κατάλυμα.

Κανονική Ροή:

1. Ο ιδιοκτήτης επιλέγει ένα κατάλυμα και ανοίγει τις παραγγελίες του (Orders).
2. Το web panel ανακτά τη λίστα παραγγελιών καλώντας GET /orders?propertyId={id} με το JWT του ιδιοκτήτη.
3. Ο ιδιοκτήτης επιλέγει μια παραγγελία και επισκοπεί τα πλήρη στοιχεία: τα προϊόντα της παραγγελίας, τις ποσότητες, τις παγιωμένες τιμές μονάδας, την κατάσταση πληρωμής και τα στοιχεία επικοινωνίας του επισκέπτη (εφόσον δόθηκαν).
4. Για μια παραγγελία προϊόντων του καταλύματος, ο ιδιοκτήτης επιλέγει τη μετάβαση της κατάστασης στο επόμενο στάδιο (π.χ. από CONFIRMED σε PREPARING). Οι μεταβάσεις γίνονται μία προς μία, χωρίς παράλειψη σταδίων.
5. Το web panel αποστέλλει αίτημα ενημέρωσης PATCH /orders/{id}/status.
6. Το backend επικυρώνει ότι η μετάβαση είναι επιτρεπτή σύμφωνα με τη μηχανή καταστάσεων των παραγγελιών, ενημερώνει τη βάση δεδομένων και επιστρέφει την ενημερωμένη κατάσταση.
7. Στην επόμενη περιοδική κλήση polling της συσκευής kiosk, το tablet του επισκέπτη απεικονίζει τη νέα κατάσταση.

Εναλλακτική Ροή 4α (Ακύρωση Παραγγελίας):

Ο ιδιοκτήτης διαπιστώνει αδυναμία εξυπηρέτησης και επιλέγει την κατάσταση CANCELLED στον διάλογο ενημέρωσης της κατάστασης.
Το backend επιτρέπει τη μετάβαση σε CANCELLED εφόσον η παραγγελία δεν βρίσκεται σε τελική κατάσταση (DELIVERED ή CANCELLED). Για μια παραγγελία φαγητού, η ακύρωση ζητείται και από την πλατφόρμα διανομής· αν η πλατφόρμα την αρνηθεί, η παραγγελία δεν ακυρώνεται και ο ιδιοκτήτης ενημερώνεται (§6.5).
Αν η παραγγελία έχει πληρωθεί, μόλις ολοκληρωθεί η ακύρωση το σύστημα ζητά αυτόματα, μία μόνο φορά, την επιστροφή του ποσού από τον φορέα πληρωμών, και η επιστροφή καταγράφεται στο audit log.

Η κατάσταση ενημερώνεται και το tablet του επισκέπτη εμφανίζει την ακύρωση στον επόμενο περιοδικό έλεγχο.

Εναλλακτική Ροή 4β (Παραγγελία Φαγητού):

Στο βήμα 4, η παραγγελία είναι παραγγελία φαγητού από πλατφόρμα διανομής.
Το web panel προβάλλει την πλατφόρμα και το εστιατόριο, αλλά δεν επιτρέπει χειροκίνητη προώθηση της κατάστασης, καθώς αυτή ενημερώνεται από την πλατφόρμα (Use Case 5)· η μόνη διαθέσιμη ενέργεια είναι η ακύρωση.

3.3.6 Use Case 5: Παραγγελία Φαγητού από Πλατφόρμα Διανομής

Κύριος Actor: Φιλοξενούμενος (GUEST) μέσω της ταυτοποιημένης συσκευής (DEVICE).

Δευτερεύων Actor: Πλατφόρμα διανομής φαγητού.

Προϋποθέσεις:

Ο ιδιοκτήτης έχει συνδέσει την πλατφόρμα με το κλειδί πρόσβασής του και την έχει ενεργοποιήσει για το συγκεκριμένο κατάλυμα.
Η συσκευή kiosk είναι ενεργοποιημένη και διαθέτει ενεργό DEVICE JWT.

Κανονική Ροή:

1. Ο επισκέπτης ανοίγει την οθόνη καταστημάτων, όπου εμφανίζονται τα εστιατόρια των πλατφορμών που προσφέρονται στο κατάλυμα, και επιλέγει ένα εστιατόριο.
2. Το kiosk ανακτά το μενού του εστιατορίου μέσω του backend, το οποίο το λαμβάνει από την πλατφόρμα χρησιμοποιώντας το κλειδί πρόσβασης του ιδιοκτήτη.
3. Ο επισκέπτης προσθέτει πιάτα στο καλάθι. Το kiosk προβάλλει το κόστος αποστολής του εστιατορίου και ειδοποιεί τον επισκέπτη αν δεν έχει καλυφθεί η ελάχιστη παραγγελία.
4. Ο επισκέπτης ολοκληρώνει την παραγγελία και, στην οθόνη πληρωμής, χρησιμοποιεί την κάρτα του. Το kiosk αποστέλλει τότε POST /orders με τύπο FOOD_DELIVERY, την πλατφόρμα, το εστιατόριο και τα αναγνωριστικά των πιάτων με τις ποσότητες.
5. Το backend ελέγχει ότι η πλατφόρμα είναι διαθέσιμη στο κατάλυμα της συσκευής, τιμολογεί κάθε πιάτο με την τρέχουσα τιμή του μενού της πλατφόρμας, προσθέτει το κόστος αποστολής, επαληθεύει την ελάχιστη παραγγελία και αποθηκεύει την παραγγελία σε κατάσταση PENDING.
6. Το kiosk αποστέλλει το αίτημα πληρωμής, όπως στο βήμα 8 του Use Case 1, και η παραγγελία μεταβαίνει σε CONFIRMED.
7. Μόνο μετά την εγκεκριμένη πληρωμή, το backend υποβάλλει την παραγγελία στην πλατφόρμα, μαζί με τη διεύθυνση παράδοσης του καταλύματος (οδός, όροφος, όνομα στο κουδούνι, τηλέφωνο), και καταγράφει το αναγνωριστικό που επιστρέφει η πλατφόρμα.

8. Η πλατφόρμα ενημερώνει το backend για την πρόοδο της παραγγελίας (PREPARING, OUT_FOR_DELIVERY, DELIVERED) μαζί με τον εκτιμώμενο χρόνο παράδοσης. Το backend εφαρμόζει μόνο τις ενημερώσεις που οδηγούν την παραγγελία σε επόμενο στάδιο του κύκλου ζωής της, αγνοώντας καθυστερημένα ή διπλά μηνύματα.
9. Το tablet του επισκέπτη προβάλλει κάθε νέα κατάσταση και τον εκτιμώμενο χρόνο παράδοσης μέσω του περιοδικού ελέγχου.

Εναλλακτική Ροή 5α (Μη Καλυμμένη Ελάχιστη Παραγγελία):

Στο βήμα 5, η αξία των πιάτων υπολείπεται της ελάχιστης παραγγελίας του εστιατορίου.
Το backend απορρίπτει το αίτημα με HTTP 400 Bad Request και δεν δημιουργείται παραγγελία.

Εναλλακτική Ροή 5β (Η Πλατφόρμα δεν Αποδέχεται την Παραγγελία):

Στο βήμα 7, η πλατφόρμα αρνείται ή δεν μπορεί να καταχωρήσει την ήδη πληρωμένη παραγγελία.
Το backend ακυρώνει την παραγγελία, ζητά από τον φορέα πληρωμών την επιστροφή του ποσού και καταγράφει την επιστροφή στο audit log.
Το kiosk ενημερώνει τον επισκέπτη ότι το εστιατόριο δεν μπόρεσε να δεχθεί την παραγγελία και ότι το ποσό θα επιστραφεί.

Μετα-συνθήκες:

Η παραγγελία είναι καταχωρημένη τόσο στο tenant schema του ιδιοκτήτη όσο και στην πλατφόρμα διανομής, με κοινό σημείο αναφοράς το αναγνωριστικό της πλατφόρμας.
Καμία παραγγελία δεν υποβάλλεται στην πλατφόρμα πριν από την εγκεκριμένη πληρωμή της.

3.4 Παραδοχές και Όρια του Συστήματος

Η σαφής καταγραφή των ορίων και των τεχνικών παραδοχών είναι απαραίτητη για την ακριβή οριοθέτηση του πρωτοτύπου και την αποτροπή λανθασμένων συμπερασμάτων κατά την αξιολόγηση.

3.4.1 Εξωτερικές Υπηρεσίες — Προσομοίωση μέσω Emulated Adapters

Οι εξωτερικές διασυνδέσεις του πρωτοτύπου χωρίζονται σε δύο κατηγορίες. Όπου ήταν δυνατή η ελεύθερη ή ανοικτή πρόσβαση, το σύστημα συνδέεται με την πραγματική υπηρεσία· όπου η πρόσβαση προϋποθέτει εμπορική συμφωνία, η υπηρεσία υλοποιείται με emulated adapter που προσομοιώνει τη συμπεριφορά του πραγματικού παρόχου.

Πραγματικές είναι τρεις διασυνδέσεις:

Ηλεκτρονικό ταχυδρομείο: Τα μηνύματα καλωσορίσματος των ιδιοκτητών και οι αποδείξεις των επισκεπτών αποστέλλονται μέσω του πρωτοκόλλου SMTP [8] σε οποιονδήποτε διακομιστή ορίσει ο διαχειριστής. Αν δεν έχει οριστεί διακομιστής, η αποστολή παραλείπεται χωρίς να επηρεάζεται η υπόλοιπη λειτουργία.

Εταιρικό μητρώο: Η αναζήτηση εταιρικών στοιχείων γίνεται στη δημόσια διεπαφή ανοικτών δεδομένων του μητρώου [9], με κλειδί πρόσβασης που εκδίδεται μετά από εγγραφή και έγκριση.

Γεωκωδικοποίηση διευθύνσεων: Η αναζήτηση διευθύνσεων και η μετατροπή συντεταγμένων σε διεύθυνση γίνονται μέσω μιας δημόσιας υπηρεσίας γεωκωδικοποίησης [10], [11], τηρώντας τις απαιτήσεις της πολιτικής χρήσης της ως προς το rate limit και την ταυτοποίηση της εφαρμογής [24] (ένα αίτημα ανά δευτερόλεπτο και αναγνωριστικό εφαρμογής σε κάθε αίτημα).

Προσομοιωμένες είναι δύο διασυνδέσεις:

Πλατφόρμες διανομής φαγητού: Οι εμπορικές πλατφόρμες διανομής συνήθως δεν διαθέτουν δημόσια προγραμματιστική διεπαφή για τρίτους. Κάθε πλατφόρμα προσομοιώνεται από ξεχωριστό adapter με δικό της σύνολο εστιατορίων, μενού και καταλόγου προϊόντων. Ο adapter δέχεται κάθε κλειδί πρόσβασης με επαρκές μήκος, αποδέχεται τις παραγγελίες και, σε τακτά χρονικά διαστήματα, αναφέρει την πρόοδό τους μέσω του ίδιου μηχανισμού ειδοποιήσεων που θα χρησιμοποιούσε μια πραγματική πλατφόρμα.

Φορέας πληρωμών καρτών: Η σύνδεση με πραγματικό φορέα προϋποθέτει εμπορική σύμβαση και πιστοποίηση της συσκευής. Ο emulated adapter απαντά στη θέση του: εγκρίνει ή απορρίπτει μια συναλλαγή ανάλογα με τον τρόπο ανάγνωσης της κάρτας και ολοκληρώνει τις επιστροφές. Τα συμβάντα της κάρτας προέρχονται από τον πραγματικό αναγνώστη καρτών της συσκευής kiosk.

Ο τρόπος με τον οποίο ένας προσομοιωμένος adapter μπορεί να αντικατασταθεί από πραγματικό περιγράφεται στην §4.4.

3.4.2 Διαχείριση Αποθεμάτων

Η παρακολούθηση της διαθεσιμότητας των προϊόντων στο παρόν πρωτότυπο υλοποιείται μέσω του boolean πεδίου Product.isAvailable, το οποίο ο ιδιοκτήτης ενεργοποιεί ή απενεργοποιεί χειροκίνητα, συνολικά ή ανά κατάλυμα. Το σύστημα δεν τηρεί ποσότητες αποθέματος· η αυτοματοποιημένη μείωση αποθεμάτων ανά παραγγελία αποτελεί μελλοντική επέκταση και δεν εφαρμόζεται στη ροή του τρέχοντος πρωτοτύπου.

3.4.3 Τοπικοποίηση και Γεωγραφική Προσαρμοστικότητα

Η δομή των δεδομένων του συστήματος δεν δεσμεύεται σε συγκεκριμένη γεωγραφική τοποθεσία: κάθε κατάλυμα αποθηκεύει τη δική του διεύθυνση, χώρα και ζώνη ώρας (timezone), ενώ οι χρονοσφραγίδες αποθηκεύονται σε UTC. Το πρωτότυπο απευθύνεται ωστόσο στην ελληνική αγορά: η καταχώρηση ιδιοκτητών βασίζεται στον ελληνικό ΑΦΜ και στο ελληνικό εταιρικό μητρώο, τα νέα καταλύματα λαμβάνουν τις τιμές «Greece» και Europe/Athens, η αναζήτηση διευθύνσεων περιορίζεται σε ελληνικές διευθύνσεις, και τα δεδομένα επίδειξης (seed data) αφορούν ελληνικές πόλεις. Η επέκταση σε άλλη χώρα θα απαιτούσε έναν adapter για το αντίστοιχο εθνικό εταιρικό μητρώο, τη δυνατότητα επιλογής χώρας και ζώνης ώρας κατά την καταχώρηση καταλύματος και την αντίστοιχη ρύθμιση της αναζήτησης διευθύνσεων, χωρίς αλλαγή στη δομή της βάσης δεδομένων.

3.4.4 Εκτός Εύρους — Μελλοντικές Επεκτάσεις

Οι ακόλουθες λειτουργίες έχουν αξιολογηθεί και ορίζονται ρητά ως εκτός του εύρους της παρούσας εργασίας, αποτελώντας αντικείμενο μελλοντικής ανάπτυξης:

Εφαρμογή Διανομέα και Συνεργαζόμενων Καταστημάτων (Rider & Store Partner application): Υλοποίηση εξειδικευμένης εφαρμογής κινητών συσκευών για τους διανομείς και τα συνεργαζόμενα τοπικά καταστήματα, η οποία θα απαιτούσε νέες οντότητες και πίνακες στο tenant schema.

Έξυπνο Minibar / Αυτοματισμοί IoT (Smart Cabinet): Διασύνδεση του kiosk tablet με φυσικές ηλεκτρονικές κλειδαριές ή αισθητήρες ραφιών για αυτόματη καταγραφή κατανάλωσης προϊόντων.

Δυναμική Τιμολόγηση (Dynamic Pricing): Αυτόματη αναπροσαρμογή τιμών καταλόγου βάσει πληρότητας, εποχικότητας ή ωραρίου.

Μηχανή Συστάσεων με Τεχνητή Νοημοσύνη (AI Recommendation Engine): Εξατομικευμένες προτάσεις προϊόντων και τοπικών υπηρεσιών βάσει προτιμήσεων του επισκέπτη.

Ψηφιακές Υπηρεσίες Concierge (Digital Concierge Services): Αυτοματοποιημένη κράτηση εισιτηρίων, περιηγήσεων και υπηρεσιών μετακίνησης.

Προηγμένη Ασφάλεια Συσκευών (Mobile Device Security): Μηχανισμοί ανίχνευσης παραβίασης λειτουργικού συστήματος (rooting detection) και απομακρυσμένης εκκαθάρισης δεδομένων (remote wipe).

Κεφάλαιο 4. Αρχιτεκτονική Συστήματος

Το κεφάλαιο περιγράφει την αρχιτεκτονική του συστήματος και τις τεχνολογικές επιλογές στις οποίες στηρίζεται: τη δομή των modules (§4.1), την εφαρμογή της Clean Architecture (§4.2), τη δρομολόγηση κάθε αιτήματος στον σωστό tenant (§4.3), τη σύνδεση με τις εξωτερικές υπηρεσίες (§4.4) και, τέλος, τις αποφάσεις σχεδίασης μαζί με τις εναλλακτικές που εξετάστηκαν (§4.5).

4.1 Επισκόπηση Αρχιτεκτονικής και Δομή Modules

Το σύστημα βρίσκεται σε ένα ενιαίο αποθετήριο κώδικα (monorepo) με πέντε modules, που περιέχουν μαζί τη λογική, τα μοντέλα δεδομένων και τις εφαρμογές της πλατφόρμας. Επειδή όλα βρίσκονται στο ίδιο αποθετήριο, τα κοινά modules (:domain και :data) έχουν πάντα την ίδια έκδοση για όλα τα modules που τα χρησιμοποιούν, και ο μεταγλωττιστής εντοπίζει αμέσως μια ασυμφωνία τύπων ή μια αλλαγή σε ένα DTO ως σφάλμα μεταγλώττισης (compile-time error) σε όλες τις πλατφόρμες μαζί, αντί να εμφανιστεί κατά την εκτέλεση (runtime).

Τα πέντε modules χωρίζονται σε δύο επίπεδα: στα **κοινά modules** (:domain, :data) και στα **modules εφαρμογής** (:backend, :owner-web, :app).

Το module :domain είναι ο πυρήνας του συστήματος. Περιέχει τις βασικές επιχειρησιακές οντότητες (Order, Product, Device, Property), τα interfaces των repositories και τα port interfaces για τις εξωτερικές υπηρεσίες. Δεν εξαρτάται από κανένα framework, βιβλιοθήκη υποδομής ή άλλο module. Είναι γραμμένο αποκλειστικά σε Kotlin και μεταγλωττίζεται για δύο στόχους: σε Java bytecode (JVM jar), που το χρησιμοποιούν το backend και η εφαρμογή kiosk του Android (η μεταγλώττιση του Android το μετατρέπει σε DEX bytecode), και σε WebAssembly (wasmJs bundle) για τον browser. Ο ίδιος κώδικας εκτελείται έτσι σε τρία διαφορετικά περιβάλλοντα.

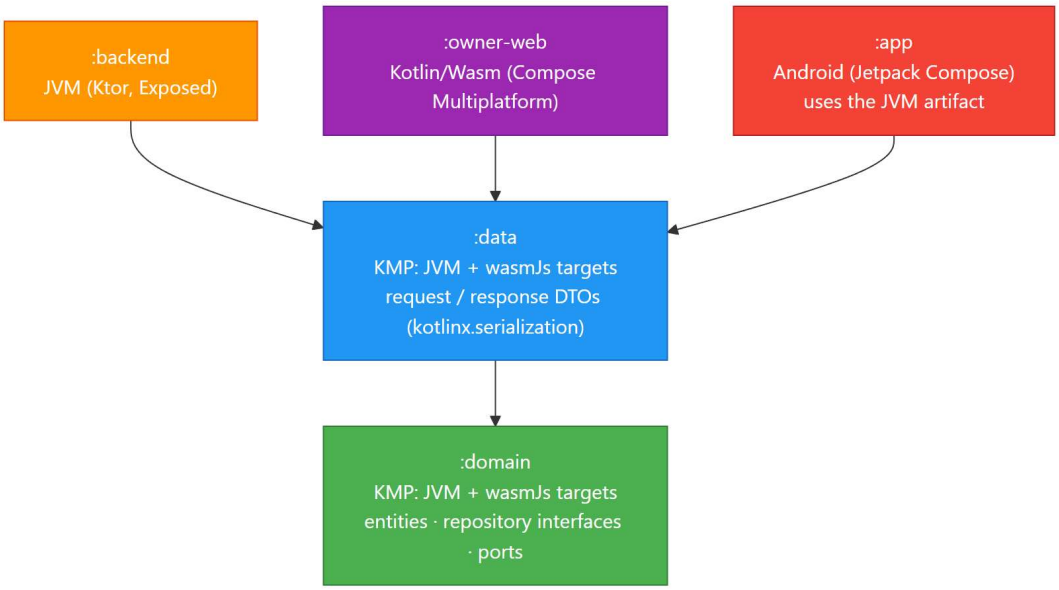
Το module :data εξαρτάται μόνο από το :domain και τη βιβλιοθήκη kotlinx.serialization. Ορίζει τα αντικείμενα μεταφοράς δεδομένων (Data Transfer Objects — DTOs), δηλαδή τα αιτήματα και τις αποκρίσεις του REST API [25], ως κλάσεις Kotlin με το annotation @Serializable. Μεταγλωττίζεται για τους ίδιους δύο στόχους (JVM, wasmJs)· το JVM artifact το χρησιμοποιεί και η εφαρμογή kiosk, και το module αποτελεί το κοινό συμβόλαιο επικοινωνίας ανάμεσα στον server και στους clients.

Τα modules εφαρμογής εξαρτώνται από το :data και το :domain, και ποτέ το αντίστροφο [26]:

> Το :backend είναι εφαρμογή JVM που υλοποιεί το REST API με το Ktor και επικοινωνεί με την PostgreSQL μέσω του JetBrains Exposed.
>
> Το :owner-web είναι η εφαρμογή διαχείρισης για τον browser, γραμμένη με Compose Multiplatform και μεταγλωττισμένη σε WebAssembly (wasmJs).

Το :app είναι η εφαρμογή kiosk του καταλύματος για Android, γραμμένη με Jetpack Compose.

Η φορά των εξαρτήσεων προκύπτει από όσα δηλώνει κάθε module στο multi-project build του Gradle [26]: επειδή το :domain δεν δηλώνει εξάρτηση από το :data ή από κάποιο module εφαρμογής, κάθε αναφορά σε αυτά αποτυγχάνει κατά τη μεταγλώττιση. Η Εικόνα 4.1 δείχνει τα modules και την κατεύθυνση των εξαρτήσεων.



Εικόνα 4.1: Δομή modules του monorepo και κατεύθυνση εξαρτήσεων.

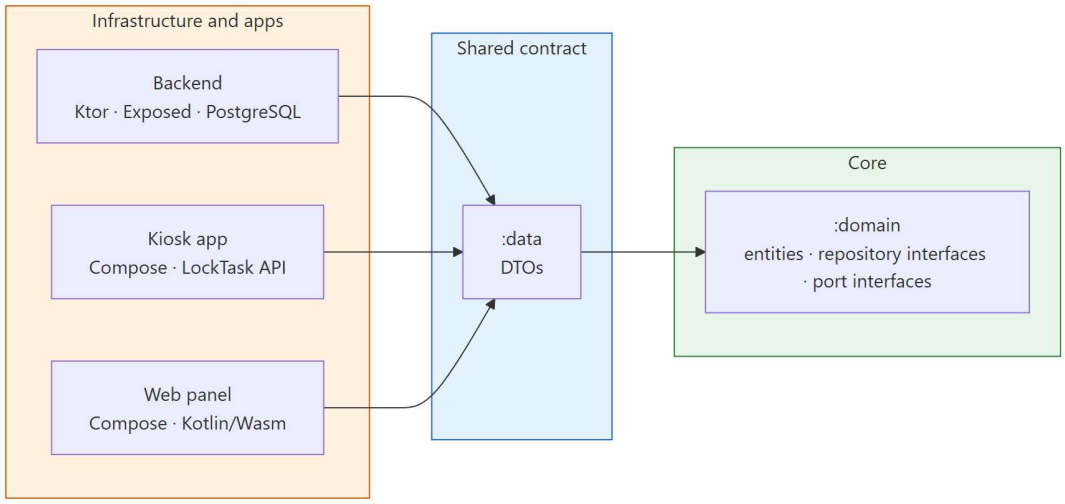
4.2 Clean Architecture ως Δομικός Περιορισμός

Η αρχιτεκτονική του συστήματος ακολουθεί τις αρχές της Clean Architecture [5], όπως τις διατύπωσε ο Robert C. Martin. Ο Κανόνας Εξάρτησης (§2.3) σημαίνει εδώ ότι τα εξωτερικά επίπεδα (υποδομή, βιβλιοθήκες δικτύου, διεπαφή χρήστη, βάση δεδομένων) εξαρτώνται από τον πυρήνα, ενώ ο πυρήνας δεν εξαρτάται από αυτά.

Στο σύστημα, ο κανόνας αυτός επιβάλλεται από το ίδιο το build system. Το :domain δεν έχει καμία εξωτερική εξάρτηση: δεν χρησιμοποιεί το Ktor, το Exposed ή άλλη βιβλιοθήκη εξαρτημένη από συγκεκριμένη πλατφόρμα. Μια οντότητα Order του :domain έχει ακριβώς την ίδια μορφή και σημασία στον server, στο Android tablet και στον browser του ιδιοκτήτη.

Χαρακτηριστικό παράδειγμα είναι τα repository interfaces. Το OrderRepository ορίζεται στο :domain και περιγράφει τις υπογραφές των μεθόδων ανάκτησης και αποθήκευσης (method signatures) που χρειάζεται η επιχειρησιακή λογική, ενώ η υλοποίησή του (ExposedOrderRepository) βρίσκεται στο :backend και χρησιμοποιεί το DSL του Exposed και την PostgreSQL. Αν η PostgreSQL αντικαθιστόταν από άλλη τεχνολογία αποθήκευσης, οι οντότητες και τα interfaces του :domain δεν θα άλλαζαν· οι

αλλαγές θα έμεναν στο :backend. Εκεί θα άλλαζαν κυρίως οι υλοποιήσεις των repositories, αλλά και ορισμένες υπηρεσίες: όσες ορίζουν οι ίδιες τα όρια μιας συναλλαγής χρησιμοποιούν απευθείας τη βοηθητική συνάρτηση tenantQuery, που στηρίζεται στο Exposed (§5.6). Το επίπεδο υπηρεσιών βρίσκεται δηλαδή στο :backend και όχι στον πυρήνα, επιλογή που διατηρεί τον κώδικα απλό αλλά συνδέει στενά τις υπηρεσίες με τον μηχανισμό συναλλαγών. Αντίστοιχα, η επιλογή του HTTP framework (Ktor) αφορά μόνο τα routes και δεν επηρεάζει τον πυρήνα. Η Εικόνα 4.2 δείχνει τα επίπεδα του συστήματος και την κατεύθυνση των εξαρτήσεών τους.

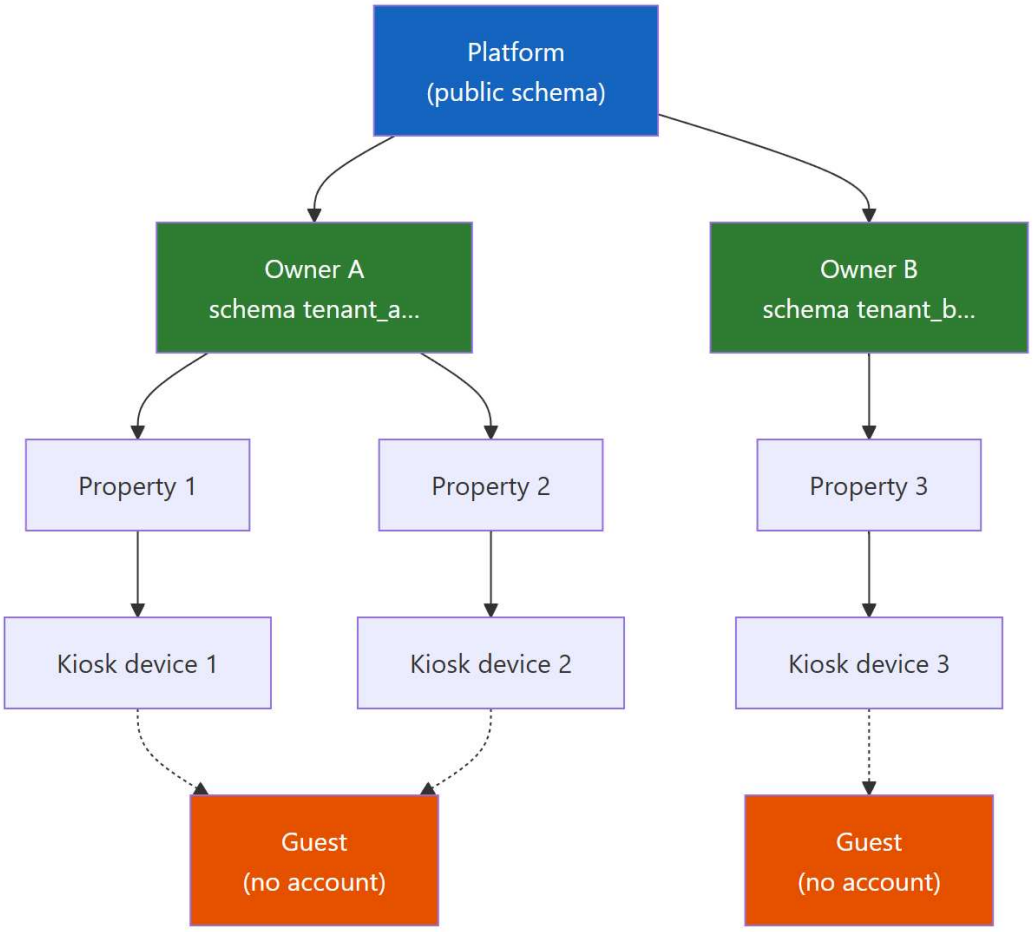


Εικόνα 4.2: Εφαρμογή Clean Architecture — κατεύθυνση εξαρτήσεων προς τον πυρήνα.

4.3 Multi-Tenancy — Αρχιτεκτονική Ροής Αιτημάτων

Τα δεδομένα και τα δικαιώματα πρόσβασης οργανώνονται σε μια ιεραρχία πέντε επιπέδων: **Πλατφόρμα → Ιδιοκτήτης → Κατάλυμα → Συσκευή → Φιλοξενούμενος**.

Ο Platform Admin ανήκει στο schema της πλατφόρμας (public) και εποπτεύει όλο το σύστημα. Κάθε ιδιοκτήτης (Owner) έχει το δικό του schema στη βάση (tenant_{id}), όπου βρίσκονται τα καταλύματα, οι συσκευές, τα προϊόντα και οι παραγγελίες του. Οι συσκευές kiosk δεν έχουν δικό τους schema· η καθεμία ανήκει σε ένα συγκεκριμένο κατάλυμα του ιδιοκτήτη. Ο φιλοξενούμενος δεν έχει λογαριασμό ή διαπιστευτήρια· ο tenant στον οποίο ανήκουν οι παραγγελίες του προκύπτει από την ταυτοποιημένη συσκευή kiosk που χρησιμοποιεί. Η Εικόνα 4.3 δείχνει την ιεραρχία και την αντιστοίχισή της στα schemas της PostgreSQL.



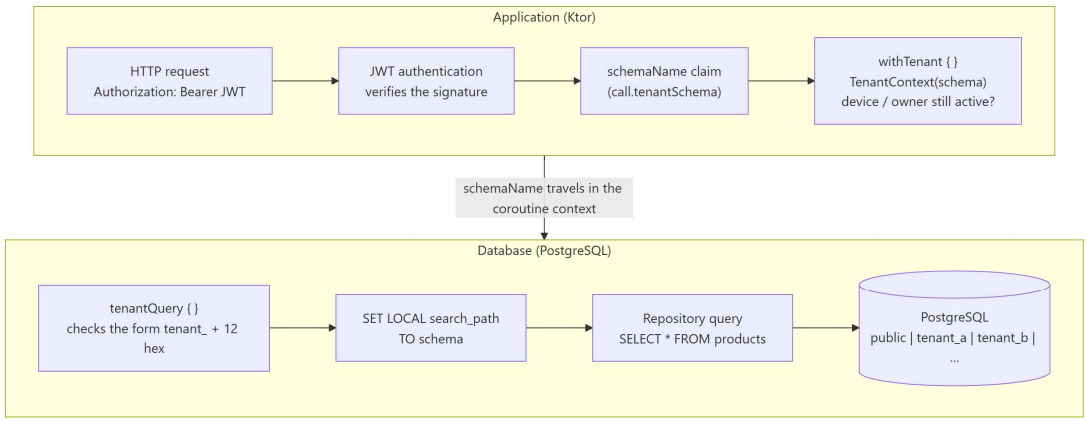
Εικόνα 4.3: Ιεραρχία οντοτήτων και αντιστοίχιση σε PostgreSQL schemas.

Κάθε HTTP αίτημα φτάνει στο σωστό schema μέσα από τέσσερα βήματα:

1. **Ο tenant μέσα στο token:** Όταν ταυτοποιείται ένας χρήστης ή μια συσκευή, το JWT access token που εκδίδεται περιέχει το πεδίο (claim) schemaName (π.χ. tenant_a1b2c3d4e5f6 για έναν ιδιοκτήτη ή για μια ενεργοποιημένη συσκευή kiosk, και null για τον Platform Admin). Το claim είναι υπογεγραμμένο, ώστε ο client να μην μπορεί να το αλλάξει.
2. **Επαλήθευση στο middleware:** Το plugin ταυτοποίησης JWT του Ktor επαληθεύει την υπογραφή του token σε κάθε αίτημα και διαβάζει το schemaName.
3. **Μεταφορά με το TenantContext:** Το schema αποθηκεύεται σε ένα TenantContext (υλοποίηση του CoroutineContext.Element), το οποίο ακολουθεί αυτόματα το coroutine του αιτήματος σε όλη την αλυσίδα κλήσεων, χωρίς να περνά ως παράμετρος από μέθοδο σε μέθοδο.
4. **Επιλογή schema στη βάση:** Η βοηθητική συνάρτηση tenantQuery {} διαβάζει το TenantContext, ελέγχει ότι το όνομα του schema έχει την αναμενόμενη μορφή (tenant_ και δώδεκα δεκαεξαδικοί χαρακτήρες) και εκτελεί ως πρώτη εντολή της συναλλαγής του Exposed

το SET LOCAL search_path TO {schemaName}. Έτσι όλα τα ερωτήματα της συναλλαγής εκτελούνται στο schema του tenant. Το SET LOCAL ισχύει μόνο για τη συναλλαγή, ώστε μια σύνδεση που επιστρέφει στο κοινό connection pool να μη «μεταφέρει» το schema ενός tenant στο επόμενο αίτημα, ενώ ο έλεγχος της μορφής εμποδίζει την εισαγωγή αυθαίρετου κειμένου στην εντολή SQL.

Έτσι τα repositories δεν χρειάζεται να γνωρίζουν ότι το σύστημα είναι multi-tenant: εκτελούν απλά ερωτήματα της μορφής SELECT * FROM products, και ο πίνακας products είναι αυτός του tenant του αιτήματος. Αν ένα αίτημα επιχειρήσει να φτάσει στη βάση χωρίς TenantContext (π.χ. από σφάλμα στη ροή του αιτήματος), η εκτέλεση σταματά με σφάλμα, ώστε να μη διαβαστούν ή αλλοιωθούν δεδομένα άλλου tenant. Η Εικόνα 4.4 συνοψίζει τη δρομολόγηση ενός αιτήματος στο σωστό tenant schema.



Εικόνα 4.4: Δρομολόγηση αιτήματος στο σωστό tenant schema μέσω TenantContext.

4.4 Port/Adapter Pattern — Ενσωμάτωση Εξωτερικών Υπηρεσιών

Οι εξωτερικές υπηρεσίες συνδέονται χωρίς να επηρεάζουν την επιχειρησιακή λογική μέσω του προτύπου Port/Adapter (Hexagonal Architecture) [7]. Ο πυρήνας επικοινωνεί με τον έξω κόσμο μόνο μέσω θυρών (port interfaces, §2.3) που ορίζονται στο :domain. Κάθε εξωτερική υπηρεσία αντιστοιχεί σε μία διεπαφή, ενώ οι υλοποιήσεις (adapters) βρίσκονται στο :backend και δίνονται στις υπηρεσίες μέσω των constructors τους κατά την εκκίνηση (χειροκίνητη σύνδεση στη συνάρτηση configureRouting(), χωρίς framework έγχυσης εξαρτήσεων).

Το σύστημα ορίζει έξι port interfaces:

PaymentGateway: Η σύνδεση με τον φορέα πληρωμών καρτών, ο οποίος εγκρίνει μια συναλλαγή ή εκτελεί επιστροφή χρημάτων.

FoodDeliveryProvider: Η σύνδεση με τις πλατφόρμες παραγγελίας φαγητού: επαλήθευση του κλειδιού πρόσβασης, εστιατόρια, μενού και κατάλογος προϊόντων, υποβολή και ακύρωση παραγγελίας.

FoodOrderEventListener: Εισερχόμενη θύρα (driving port, §2.3), μέσω του οποίου μια πλατφόρμα ενημερώνει το σύστημα για την πρόοδο μιας παραγγελίας. Αντιστοιχεί στη διεύθυνση ειδοποιήσεων (webhook) που θα καλούσε μια πραγματική πλατφόρμα.

NotificationService: Η αποστολή μηνυμάτων καλωσορίσματος, επαναφοράς κωδικού και αποδείξεων με ηλεκτρονικό ταχυδρομείο.

CompanyRegistry: Η αναζήτηση των δημοσιευμένων στοιχείων μιας επιχείρησης με βάση τον ΑΦΜ.

Geocoder: Η αναζήτηση διευθύνσεων και η μετατροπή συντεταγμένων σε διεύθυνση.

Οι επιχειρησιακές υπηρεσίες (OrderService, PaymentService, OwnerService κ.ά.) καλούν μόνο αυτές τις διεπαφές και δεν γνωρίζουν τα πρωτόκολλα, τις βιβλιοθήκες ή τις μορφές δεδομένων κάθε παρόχου.

Στο πρωτότυπο, τρία ports υλοποιούνται από adapters που συνδέονται με πραγματικές υπηρεσίες και δύο από emulated adapters (§3.4.1)· το έκτο, το FoodOrderEventListener, το υλοποιεί μια υπηρεσία του backend:

Ο SmtpEmailAdapter στέλνει πραγματικά μηνύματα μέσω SMTP [8] και διαβάζει τις ρυθμίσεις του διακομιστή από τη βάση σε κάθε αποστολή.

Ο adapter του CompanyRegistry καλεί τη διεπαφή ανοικτών δεδομένων του εταιρικού μητρώου [9].

Ο adapter του Geocoder καλεί τη δημόσια υπηρεσία γεωκωδικοποίησης [10] και τηρεί, για όλη τη διεργασία, το όριο ενός αιτήματος ανά δευτερόλεπτο που ορίζει η πολιτική χρήσης της [24].

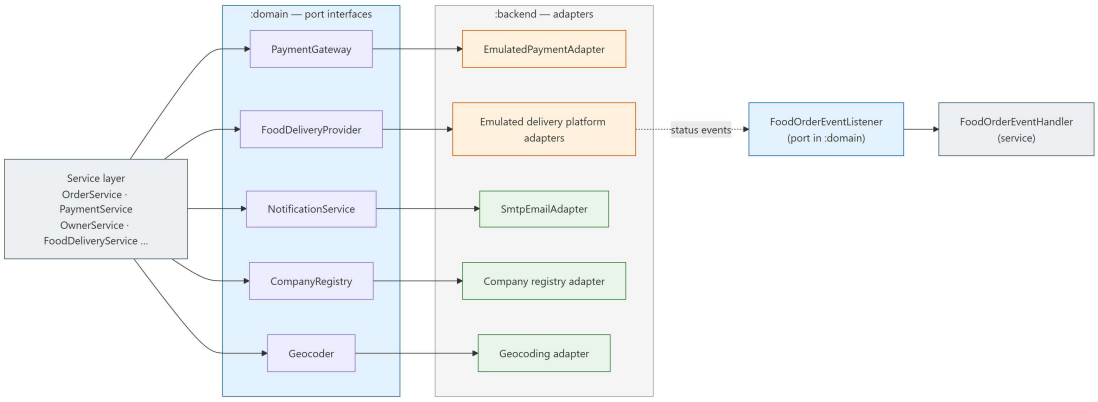
Ο EmulatedPaymentAdapter απαντά στη θέση ενός πραγματικού φορέα και εγκρίνει ή απορρίπτει μια συναλλαγή με βάση τον τρόπο ανάγνωσης της κάρτας.

Οι τρεις προσομοιωμένοι adapters των πλατφορμών διανομής, με κοινή βάση την κλάση EmulatedFoodPlatformAdapter, διαβάζουν τα εστιατόρια, τα μενού και τους καταλόγους τους από αρχεία δεδομένων και αναφέρουν την πρόοδο κάθε παραγγελίας που δέχτηκαν μέσω του FoodOrderEventListener, τον οποίο υλοποιεί η υπηρεσία FoodOrderEventHandler.

Το όφελος του προτύπου φαίνεται στη μετάβαση σε παραγωγή: για να αντικατασταθεί ο EmulatedPaymentAdapter από τον adapter ενός πραγματικού φορέα πληρωμών, εφόσον η διεπαφή του φορέα αντιστοιχεί στο port, αρκεί να γραφτεί η νέα κλάση στο :backend και να αλλάξει μία γραμμή στη συνάρτηση configureRouting(), όπου συναρμολογούνται οι εξαρτήσεις. Οι domain entities, οι υπηρεσίες και τα routes δεν αλλάζουν. Το ότι πραγματικοί και emulated adapters συνυπάρχουν ήδη πίσω από διεπαφές της ίδιας μορφής δείχνει στην πράξη αυτή την ιδιότητα μέσα στο ίδιο το πρωτότυπο.

Ο τύπος κάθε παραγγελίας (type) καθορίζει ποια ports χρησιμοποιούνται. Μια παραγγελία προϊόντων του καταλύματος (IN_ROOM_STORE) τιμολογείται από τον κατάλογο του ιδιοκτήτη και την εξυπηρετεί ο ίδιος. Μια παραγγελία φαγητού (FOOD_DELIVERY) τιμολογείται από το μενού της πλατφόρμας μέσω του FoodDeliveryProvider, υποβάλλεται στην πλατφόρμα μετά την εγκεκριμένη

πληρωμή και η πρόοδός της ενημερώνεται στη συνέχεια από τα συμβάντα του FoodOrderEventListener. Η απόφαση λαμβάνεται στο service layer, χωρίς άμεση εξάρτηση από συγκεκριμένες υλοποιήσεις εξωτερικών παρόχων. Η Εικόνα 4.5 δείχνει τα έξι port interfaces μαζί με τους πραγματικούς και τους προσομοιωμένους adapters τους.



Εικόνα 4.5: Port/Adapter pattern — πραγματικοί (πράσινο) και προσομοιωμένοι (πορτοκαλί) προσαρμογείς πίσω από τα έξι port interfaces.

4.5 Αποφάσεις Σχεδίασης και Επιλογή Τεχνολογιών

Πριν από την υλοποίηση συγκρίθηκαν οι διαθέσιμες τεχνολογίες ως προς τα λειτουργικά και μη λειτουργικά τους χαρακτηριστικά και τις αντισταθμίσεις (trade-offs) τους. Οι ενότητες που ακολουθούν παρουσιάζουν τις βασικές αποφάσεις.

4.5.1 Η Επιλογή Kotlin ως Μοναδικής Γλώσσας

Η βασική απόφαση της εργασίας ήταν να χρησιμοποιηθεί **μία γλώσσα προγραμματισμού σε όλο το λογισμικό**: στον server, στην εφαρμογή kiosk για Android και στο web panel. Ο λόγος ήταν η συνέπεια των τύπων σε ένα σύστημα με πολλά υποσυστήματα που επικοινωνούν μεταξύ τους.

Σε μια πολυγλωσσική προσέγγιση (polyglot architecture), το backend γράφεται συνήθως σε Java ή Go, η εφαρμογή Android σε Kotlin και το web frontend σε TypeScript. Κάθε υποσύστημα χρησιμοποιεί τα καταλληλότερα εργαλεία της πλατφόρμας του, αλλά ο κώδικας κατακερματίζεται: τα μοντέλα, τα DTOs και οι κανόνες επικύρωσης γράφονται πολλές φορές σε διαφορετικές γλώσσες, με μόνιμο κίνδυνο να αποκλίνουν και με σφάλματα αποσειριοποίησης που φαίνονται μόνο κατά την εκτέλεση.

Το **Kotlin Multiplatform (KMP)** [4] περιορίζει ουσιαστικά αυτό το πρόβλημα. Ο κοινός κώδικας μεταγλωττίζεται για όλους τους στόχους του συστήματος (§2.4, §4.1). Οι οντότητες, τα DTOs και τα port interfaces γράφονται μία φορά και χρησιμοποιούνται ως έχουν. Κάθε αλλαγή σε ένα πεδίο την ελέγχει αμέσως ο μεταγλωττιστής σε όλο το σύστημα (end-to-end type safety).

Η Kotlin είναι επιπλέον μια σύγχρονη γλώσσα με στατικούς τύπους, έλεγχο για null (null safety) και ενσωματωμένη υποστήριξη ασύγχρονου προγραμματισμού με coroutines. Εκτελείται στη JVM δίπλα στη Java και είναι η γλώσσα που προτιμά η Google για το Android (Kotlin-first) [27], [28]. Στην πράξη της ανάπτυξης, μία γλώσσα σημαίνει ένα σύστημα δόμησης (Gradle), ένα περιβάλλον ανάπτυξης (IDE) και ενιαίο τρόπο προσέγγισης για όλο το έργο.

4.5.2 Backend Framework: Ktor έναντι Spring Boot και ASP.NET Core

Για το backend εξετάστηκαν τρεις διαδεδομένες λύσεις: το **ASP.NET Core**, το **Spring Boot** και το **Ktor**.

Το **ASP.NET Core** αποκλείστηκε από την αρχή, επειδή στηρίζεται στη C#. Θα ακύρωνε τη στρατηγική του KMP, αφού τα :domain και :data δεν θα μπορούσαν να χρησιμοποιηθούν στον server.

Το **Spring Boot** είναι το πιο διαδεδομένο framework της JVM και υποστηρίζει την Kotlin. Σχεδιάστηκε όμως αρχικά για τη Java και χρησιμοποιεί εκτεταμένα reflection και annotations (@Autowired, @Component, @RestController). Στηρίζεται επίσης στην αυτόματη διαμόρφωση (auto-configuration), που ρυθμίζει την εφαρμογή ανάλογα με τις βιβλιοθήκες που βρίσκει στο classpath [29]· κατά την εκτίμηση του συγγραφέα, αυτή η άρρητη συμπεριφορά (implicit behavior) κάνει τη ροή εκτέλεσης λιγότερο ορατή.

Το **Ktor** [30], που αναπτύσσει η JetBrains ειδικά για την Kotlin, ακολουθεί διαφορετική φιλοσοφία. Στηρίζεται στα Kotlin Coroutines, ώστε, όσο ένα αίτημα αναμένει μια εξωτερική υπηρεσία, να μη δεσμεύεται νήμα του server (non-blocking I/O). Δεν χρησιμοποιεί reflection για τη δρομολόγηση: τα routes και οι παράμετροί τους δηλώνονται ρητά σε ένα Kotlin DSL. Είναι επίσης αρθρωτό (modular): κάθε λειτουργία (ταυτοποίηση, CORS, σειριοποίηση JSON, status pages) εγκαθίσταται ρητά ως plugin. Αυτή η ρητή δήλωση ταιριάζει με τις αρχές της Clean Architecture, αφού οι εξαρτήσεις και η ροή ελέγχου φαίνονται στον κώδικα.

Ο Πίνακας 4.1 συγκρίνει τα τρία frameworks:

Πίνακας 4.1: Σύγκριση των υποψήφιων backend frameworks

Χαρακτηριστικό Σύγκρισης	**Ktor**	**Spring Boot**	**ASP.NET Core**
Γλώσσα & KMP Συμβατότητα	Kotlin (Πλήρης κοινή χρήση μέσω KMP)	Java / Kotlin (Μερική αξιοποίηση KMP)	C# (.NET — Ασύμβατο με KMP)
Μοντέλο Εκτέλεσης I/O	Non-blocking (Kotlin Coroutines)	Thread pool / Project Reactor (WebFlux)	Non-blocking (async / await)

Χαρακτηριστικό Σύγκρισης	**Ktor**	**Spring Boot**	**ASP.NET Core**
Μηχανισμός Reflection	Ελάχιστος (Ρητή δήλωση κώδικα)	Εκτεταμένος (Runtime annotations)	Μέτριος (Dependency injection reflection)
Φιλοσοφία Παραμετροποίησης	Ρητά modular plugins μέσω DSL	Convention over configuration (auto-config)	Καταχώρηση στο middleware pipeline

4.5.3 Στρατηγική Multi-Tenancy: Schema-Per-Tenant

Ο τρόπος απομόνωσης των δεδομένων στη βάση είναι από τις σημαντικότερες αποφάσεις της αρχιτεκτονικής [6], [13]. Από τα τρία μοντέλα του Κεφαλαίου 2:

1. **Shared Schema:** Έχει το χαμηλότερο κόστος λειτουργίας, αλλά η ασφάλεια των δεδομένων εξαρτάται από το φίλτρο κάθε ερωτήματος (§2.2)· για ένα σύστημα με εμπορικές συναλλαγές, ο κίνδυνος αυτός κρίθηκε απαράδεκτος.
2. **Database-per-Tenant:** Δίνει τη μεγαλύτερη φυσική απομόνωση, αλλά με δυσανάλογο διαχειριστικό κόστος (ξεχωριστά connection pools, αντίγραφα ασφαλείας και migrations ανά βάση), που δεν δικαιολογείται για την κλίμακα της εφαρμογής.
3. **Schema-per-Tenant:** Κρίθηκε η καλύτερη ισορροπία μεταξύ απομόνωσης και λειτουργικής αποδοτικότητας για το σύστημα. Κάθε tenant διαθέτει δικό του namespace στην PostgreSQL, οπότε τα δεδομένα του βρίσκονται σε δικούς του πίνακες και τα ερωτήματα δεν χρειάζονται φίλτρο ανά tenant. Όλοι οι tenants συνδέονται με τον ίδιο ρόλο της βάσης, επομένως η απομόνωση στηρίζεται στη σωστή επιλογή του schema, που γίνεται σε ένα μόνο σημείο του κώδικα (§4.3). Όλη η πλατφόρμα εξυπηρετείται από ένα κοινό connection pool (HikariCP), και ένας νέος tenant δημιουργείται με εκτέλεση εντολών DDL μέσα στην ίδια βάση.

Ο Πίνακας 4.2 συγκρίνει τις τρεις στρατηγικές:

Πίνακας 4.2: Αξιολόγηση των στρατηγικών multi-tenancy

Παράμετρος Αξιολόγησης	**Shared Schema**	**Schema-per-Tenant (Επιλογή)**	**Database-per-Tenant**
Επίπεδο Επιβολής Απομόνωσης	Επίπεδο Εφαρμογής (WHERE φίλτρα)	Δομή της βάσης (PostgreSQL Schema), με επιλογή του schema από την εφαρμογή	Φυσικό Επίπεδο (Ανεξάρτητη Βάση)
Αξιοποίηση Connection Pool	Βέλτιστη (κοινό pool)	Βέλτιστη (κοινό pool μέσω HikariCP)	Κατακερματισμένη (Pool ανά βάση)

Παράμετρος Αξιολόγησης	**Shared Schema**	**Schema-per-Tenant (Επιλογή)**	**Database-per-Tenant**
Κίνδυνος Διαρροής Δεδομένων	Υπαρκτός (σε περίπτωση σφάλματος στον κώδικα)	Περιορισμένος (αρκεί η σωστή επιλογή του schema σε ένα σημείο)	Ελάχιστος (φυσική απομόνωση)
Διαχείριση Backup ανά Tenant	Εξαιρετικά σύνθετη	Απλή (pg_dump -n tenant_x)	Απλή (Backup όλης της βάσης)
Κόστος Υποδομής	Ελάχιστο	Χαμηλό / Προβλέψιμο	Υψηλό (αύξηση πόρων ανάλογη με τους tenants)

4.5.4 Κοινός Κώδικας: Kotlin Multiplatform

Η κοινή χρήση κώδικα μέσω του KMP αντιμετωπίζει το πρόβλημα του συγχρονισμού ανάμεσα στον server και στους clients. Ο κώδικας του :data (DTOs, αιτήματα και απαντήσεις) γράφεται μία φορά και μεταγλωττίζεται για κάθε στόχο.

Ο server σειριοποιεί σε JSON ένα αντικείμενο OrderResponse, και ο client (Android kiosk ή web panel) το αποσειριοποιεί με την ίδια ακριβώς κλάση OrderResponse. Επειδή η κλάση είναι η ίδια παντού, τα σφάλματα από ασυμφωνία πεδίων (type mismatches) ή ονομάτων περιορίζονται από την ίδια τη δομή του κώδικα, και δεν χρειάζονται εργαλεία παραγωγής κώδικα (code generators) ή schemas Swagger/OpenAPI για τον εσωτερικό συγχρονισμό του έργου.

4.5.5 Web Panel: Compose Multiplatform (wasmJs)

Για το web panel των ιδιοκτητών, η συνηθισμένη επιλογή θα ήταν ένα framework JavaScript/TypeScript (όπως React ή Vue). Επιλέχθηκε το **Compose Multiplatform με στόχο μεταγλώττισης WebAssembly (wasmJs)** [31], ώστε το επίπεδο διεπαφής να ακολουθεί το ίδιο μοντέλο σε όλο το σύστημα.

Τα τεχνικά χαρακτηριστικά του WebAssembly [17], του στόχου Kotlin/Wasm και της σχεδίασης σε canvas περιγράφηκαν στην §2.4.

Τα βασικά πλεονεκτήματα της επιλογής είναι τα εξής:

1. **Ενιαίο Μοντέλο Προγραμματισμού UI:** Οι ίδιες declarative συναρτήσεις Composable, ο ίδιος τρόπος διαχείρισης κατάστασης (State) και τα ίδια αρχιτεκτονικά patterns χρησιμοποιούνται στο web και στο Android.
2. **Απευθείας Επαναχρησιμοποίηση Κώδικα:** Το web panel χρησιμοποιεί απευθείας τα Kotlin DTOs του :data, χωρίς ενδιάμεσες μετατροπές.

3. **Ανεξαρτησία από το DOM:** Η σχεδίαση (rendering) σε canvas περιορίζει τις διαφορές εμφάνισης ανάμεσα στους browsers και δεν απαιτεί CSS frameworks.

Το κύριο μειονέκτημα είναι ότι το οικοσύστημα του Kotlin/Wasm είναι νεότερο από τα καθιερωμένα εργαλεία της JavaScript· στο πλαίσιο της εργασίας, το αντισταθμίζουν η συνοχή και η κοινή χρήση κώδικα που προσφέρει.

4.5.6 ORM: JetBrains Exposed

Για την επικοινωνία με τη σχεσιακή βάση εξετάστηκαν τρεις λύσεις της JVM: το **Hibernate/JPA**, το **JOOQ** και το **JetBrains Exposed**.

Το **Hibernate/JPA** είναι το καθιερωμένο ORM. Στηρίζεται όμως σε reflection, σε annotations και σε έναν σύνθετο κύκλο ζωής οντοτήτων (managed, detached, transient states). Επιπλέον, δεν προσφέρει εγγενή ενσωμάτωση με τις coroutines της Kotlin.

Το **JOOQ** παράγει type-safe κώδικα από ένα υπάρχον schema βάσης (database-first). Χρειάζεται όμως ένα επιπλέον βήμα παραγωγής κώδικα (code generation step) στο build, συνήθως με ενεργή βάση δεδομένων.

Το **Exposed** [32] ακολουθεί την αντίστροφη, code-first προσέγγιση, με ένα Kotlin DSL:

> Οι πίνακες ορίζονται στον κώδικα ως αντικείμενα Kotlin (Table objects), οπότε ο κώδικας είναι η μοναδική πηγή για τη δομή της βάσης.
>
> Τα ερωτήματα γράφονται με type-safe DSL, και ο μεταγλωττιστής ελέγχει τα ονόματα των στηλών και τους τύπους τους ήδη στο build.
>
> Προσφέρει τη συνάρτηση suspendTransaction, που εντάσσει τις συναλλαγές στο μοντέλο των Kotlin Coroutines. Οι κλήσεις προς τη βάση μέσω JDBC παραμένουν ανασταλτικές (blocking), όπως σε κάθε εργαλείο που στηρίζεται στο JDBC.
>
> Δεν βασίζεται σε reflection και runtime annotations για τον ορισμό των πινάκων και των ερωτημάτων, γεγονός που διατηρεί τη ροή του κώδικα σαφή.

Ο Πίνακας 4.3 συνοψίζει τη σύγκριση:

Πίνακας 4.3: Σύγκριση των εργαλείων ORM

Κριτήριο Αξιολόγησης	**JetBrains Exposed (Επιλογή)**	**Hibernate / JPA**	**JOOQ**
Ορισμός Schema (Source of Truth)	Code-first μέσω Kotlin Table DSL	Code-first μέσω Java Annotations	Database-first μέσω Code Generation

Κριτήριο Αξιολόγησης	**JetBrains Exposed (Επιλογή)**	**Hibernate / JPA**	**JOOQ**
Ενσωμάτωση με Coroutines	Ενσωματωμένη (suspendTransaction)	Απαιτεί εξωτερικό thread bridging	Απαιτεί Reactive Extensions / R2DBC
Μηχανισμός Reflection	Ελάχιστος	Εκτεταμένος κατά το runtime	Μηδενικός (παραγόμενος κώδικας)
Εξάρτηση από Live DB στο Build	Καμία	Καμία	Συνήθως απαιτείται για το generation
Ευκολία Multi-Schema Routing	Άμεση υποστήριξη SET search_path	Σύνθετη παραμετροποίηση Multi-tenancy	Υποστηρίζεται μέσω configuration

4.5.7 Ταυτοποίηση: JWT με Refresh Token Rotation

Η ταυτοποίηση στηρίζεται σε **stateless JSON Web Tokens (JWT)** [21] με **ανανέωση και εναλλαγή του refresh token (Refresh Token Rotation)** [22].

Ανεξάρτητα από τα tokens, οι κωδικοί πρόσβασης των χρηστών αποθηκεύονται στη βάση μόνο ως hashes με τον αλγόριθμο **BCrypt** και cost factor 12 [20]. Ο συντελεστής επιλέχθηκε ώστε ο υπολογισμός να διαρκεί περίπου 250 ms (μέτρηση στον υπολογιστή ανάπτυξης): αρκετά για να γίνονται ασύμφορες οι επιθέσεις λεξικού ή brute force, χωρίς να καθυστερεί αισθητά τη σύνδεση του χρήστη.

Η επιλογή συγκρίθηκε με την κλασική ταυτοποίηση μέσω συνεδριών που διατηρεί ο server (stateful server sessions), όπως δείχνει ο Πίνακας 4.4:

Πίνακας 4.4: Stateless JWT έναντι stateful server sessions

Παράμετρος Σύγκρισης	**Stateless JWT με Token Rotation (Επιλογή)**	**Stateful Server Sessions (Cookies)**
Οριζόντια Κλιμάκωση (Scalability)	Απλούστερη (Κανένας διαμοιρασμός μνήμης μεταξύ servers)	Απαιτεί distributed session cache (π.χ. Redis)
Επιβάρυνση Βάσης ανά Αίτημα	Μικρή (Επαλήθευση της υπογραφής και ένα ή δύο σύντομα ερωτήματα ελέγχου του λογαριασμού ή της συσκευής)	Υποχρεωτικό ερώτημα ανάκτησης του session σε κάθε κλήση
Διάρκεια Έκθεσης Access Token	Μικρή (Σύντομος χρόνος ζωής 15 λεπτών)	Εξαρτάται από το session timeout

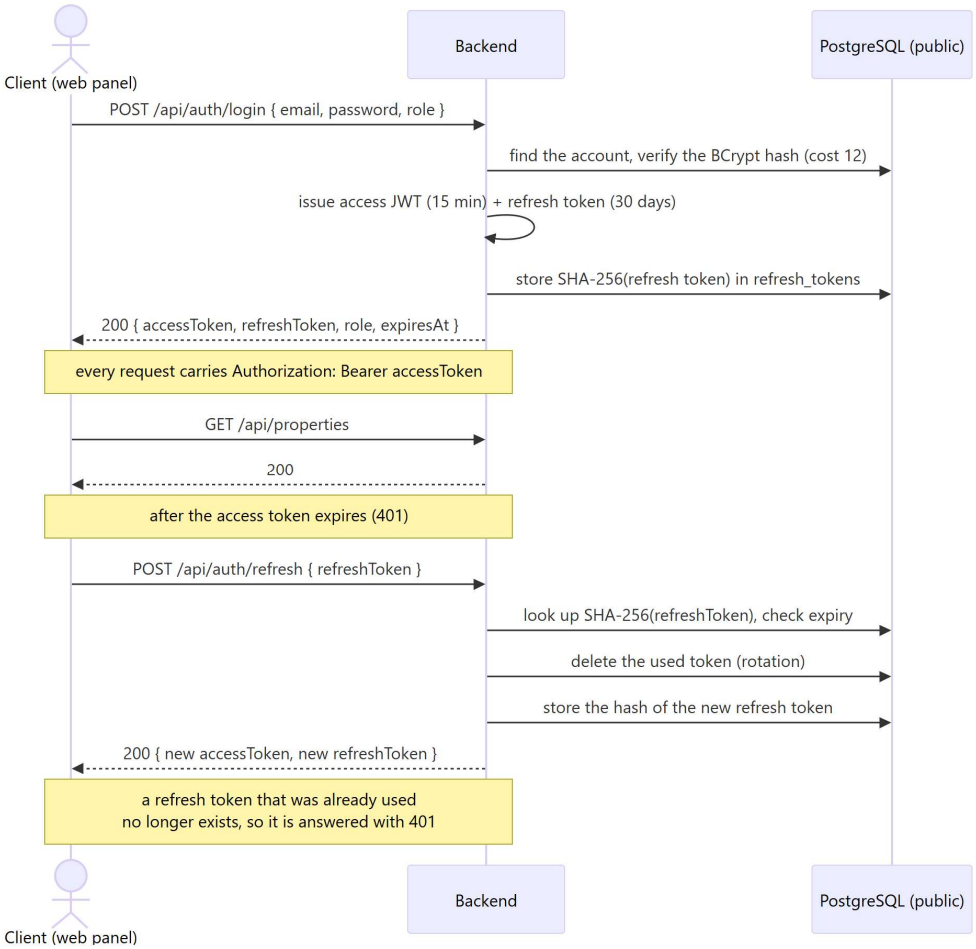
Παράμετρος Σύγκρισης	**Stateless JWT με Token Rotation (Επιλογή)**	**Stateful Server Sessions (Cookies)**
Ανάκληση Πρόσβασης	Άμεση για ιδιοκτήτες και συσκευές, μέσω του ελέγχου της κατάστασης του λογαριασμού και της διαγραφής των refresh tokens	Άμεση μέσω διαγραφής του session
Καταλληλότητα για Διαφορετικούς Clients	Καλή (Κοινό HTTP Authorization header)	Περιορισμοί με CORS και WebAssembly

Το access token έχει σκόπιμα σύντομη διάρκεια (15 λεπτά για χρήστες και διαχειριστές), ώστε ένα token που υποκλάπηκε να μπορεί να χρησιμοποιηθεί μόνο για σύντομο διάστημα. Το refresh token ισχύει 30 ημέρες και χρησιμοποιείται μόνο για την έκδοση νέου access token χωρίς νέα εισαγωγή κωδικού.

Με την εναλλαγή (**Refresh Token Rotation**), κάθε refresh token που χρησιμοποιείται για ανανέωση διαγράφεται από τη βάση πριν εκδοθεί το νέο ζεύγος tokens. Ένα ήδη χρησιμοποιημένο refresh token απορρίπτεται επομένως αν σταλεί ξανά (replay attack), καθώς δεν αντιστοιχεί πλέον σε καμία εγγραφή. Η επαναχρησιμοποίηση δεν αντιμετωπίζεται όμως ως ένδειξη υποκλοπής: η απόπειρα απλώς απορρίπτεται, χωρίς να ακυρώνονται τα tokens που εκδόθηκαν στο μεταξύ. Η ανανέωση απορρίπτεται επίσης αν ο λογαριασμός του ιδιοκτήτη έχει απενεργοποιηθεί στο μεταξύ, και σε κάθε αίτημα ιδιοκτήτη προς τα δεδομένα του tenant ελέγχεται ότι ο λογαριασμός είναι ακόμη ενεργός, ώστε η απενεργοποίηση να ισχύει από το επόμενο αίτημα και όχι από τη λήξη του access token. Ο χρήστης μπορεί να αλλάξει ο ίδιος τον κωδικό του· η αλλαγή διαγράφει όλα τα refresh tokens του, ώστε οι υπόλοιπες συνεδρίες του να μην μπορούν να ανανεωθούν και να λήγουν το αργότερο με τη λήξη του access token τους, και επιστρέφει νέο ζεύγος tokens για την τρέχουσα.

Οι συσκευές kiosk αποτελούν ειδική περίπτωση. Μια συσκευή δεν έχει κωδικό και δεν χρησιμοποιεί refresh token· κατά την ενεργοποίησή της λαμβάνει ένα access token με ρόλο DEVICE και διάρκεια ενός έτους. Για να μην εξαρτάται η ανάκληση από τη λήξη του token, το backend ελέγχει σε κάθε αίτημα συσκευής ότι η εγγραφή της υπάρχει και είναι ενεργή· η απενεργοποίηση ή η αποσύνδεση μιας συσκευής από τον ιδιοκτήτη ισχύει έτσι από το επόμενο κιόλας αίτημά της. Μια συσκευή που έχει αποσυνδεθεί ή είναι άγνωστη απορρίπτεται με 401, ώστε το kiosk της να επιστρέψει στην οθόνη εγγραφής, ενώ μια απενεργοποιημένη συσκευή ή μια συσκευή απενεργοποιημένου ιδιοκτήτη απορρίπτεται με 403, ώστε το kiosk να διατηρήσει την ενεργοποίησή του και να συνεχίσει μόλις επανενεργοποιηθεί η συσκευή ή ο ιδιοκτήτης. Οι έλεγχοι αυτοί κοστίζουν δύο επιπλέον ερωτήματα ανά αίτημα συσκευής, τίμημα που θεωρήθηκε αποδεκτό για την άμεση ανάκληση.

Τέλος, τα refresh tokens δεν αποθηκεύονται σε απλό κείμενο, αλλά μόνο ως hashes με τη συνάρτηση **SHA-256** [33]. Αν διαρρεύσει ο πίνακας refresh_tokens, τα hashes δεν αρκούν σε έναν επιτιθέμενο, αφού για την ανανέωση χρειάζεται το ίδιο το token και όχι το hash του. Η Εικόνα 4.6 συνοψίζει τη ροή ταυτοποίησης και την εναλλαγή του refresh token.



Εικόνα 4.6: Ροή ταυτοποίησης — JWT access token και refresh token rotation.

4.5.8 Κρυπτογράφηση Μυστικών των Εξωτερικών Υπηρεσιών

Η σύνδεση με εξωτερικές υπηρεσίες απαιτεί την αποθήκευση μυστικών: του κωδικού του διακομιστή SMTP, του κλειδιού πρόσβασης του εταιρικού μητρώου και των κλειδιών πρόσβασης των πλατφορμών διανομής κάθε ιδιοκτήτη. Αντίθετα με τους κωδικούς των χρηστών, αυτά δεν μπορούν να αποθηκευτούν ως hashes, επειδή το backend πρέπει να τα ανακτά αυτούσια για να τα χρησιμοποιήσει. Εξετάστηκαν δύο λύσεις: μεταβλητές περιβάλλοντος του server ή κρυπτογραφημένη αποθήκευση στη βάση.

Οι μεταβλητές περιβάλλοντος ταιριάζουν σε ρυθμίσεις που ορίζονται μία φορά κατά την εγκατάσταση, αλλά κάθε αλλαγή τους απαιτεί επανεκκίνηση του server, και δεν μπορούν να αποθηκεύσουν στοιχεία που διαφέρουν ανά tenant, όπως τα κλειδιά των πλατφορμών κάθε ιδιοκτήτη. Επιλέχθηκε επομένως η

αποθήκευση στη βάση με συμμετρική κρυπτογράφηση **AES-256 σε λειτουργία GCM** [34], που προστατεύει τόσο το περιεχόμενο όσο και την ακεραιότητα του κρυπτογραφημένου κειμένου. Κάθε κρυπτογράφηση χρησιμοποιεί νέο τυχαίο διάνυσμα αρχικοποίησης (IV) 12 bytes, που αποθηκεύεται μαζί με το κρυπτογραφημένο κείμενο. Το κλειδί προκύπτει από μια μυστική τιμή που ορίζεται ως μεταβλητή περιβάλλοντος (SETTINGS_SECRET) και δεν αποθηκεύεται στη βάση, ώστε ένα αντίγραφο της βάσης να μην αρκεί για να ανακτηθούν τα μυστικά. Οι ρυθμίσεις διαβάζονται από τη βάση σε κάθε χρήση, οπότε μια αλλαγή από το web panel ισχύει αμέσως, χωρίς επανεκκίνηση. Κανένα μυστικό δεν επιστρέφεται μέσω του API· η διεπαφή λαμβάνει μόνο την ένδειξη ότι έχει οριστεί τιμή.

Κεφάλαιο 5. Σχεδίαση Βάσης Δεδομένων

Η σχεδίαση της βάσης δεδομένων μετατρέπει τις οντότητες του επιπέδου domain (Κεφάλαια 3 και 4) σε συγκεκριμένη δομή αποθήκευσης στην PostgreSQL. Κάθε πίνακας αντιστοιχεί σε μια επιχειρησιακή οντότητα, κάθε στήλη, κατά κανόνα, σε ένα πεδίο των οντοτήτων του :domain, και κάθε constraint ή τύπος δεδομένων εκφράζει μια συγκεκριμένη απόφαση με την αντιστάθμιση (trade-off) που αυτή συνεπάγεται. Το κεφάλαιο τεκμηριώνει τη δομή αυτή σε επίπεδο schema, πίνακα και στήλης και συμπληρώνει την αρχιτεκτονική περιγραφή των §4.3 και §4.5.3 με το τελικό αποτέλεσμα της υλοποίησης.

Η βάση δεδομένων οργανώνεται σε δύο επίπεδα schemas: ένα ενιαίο public schema για τα δεδομένα της πλατφόρμας και ένα tenant schema (tenant_{id}) για κάθε ιδιοκτήτη καταλύματος, που περιέχει τα επιχειρησιακά δεδομένα του. Η απόφαση αυτή, που αιτιολογείται ως στρατηγική επιλογή στην §4.5.3, καθορίζει τη συνολική δομή που ακολουθεί.

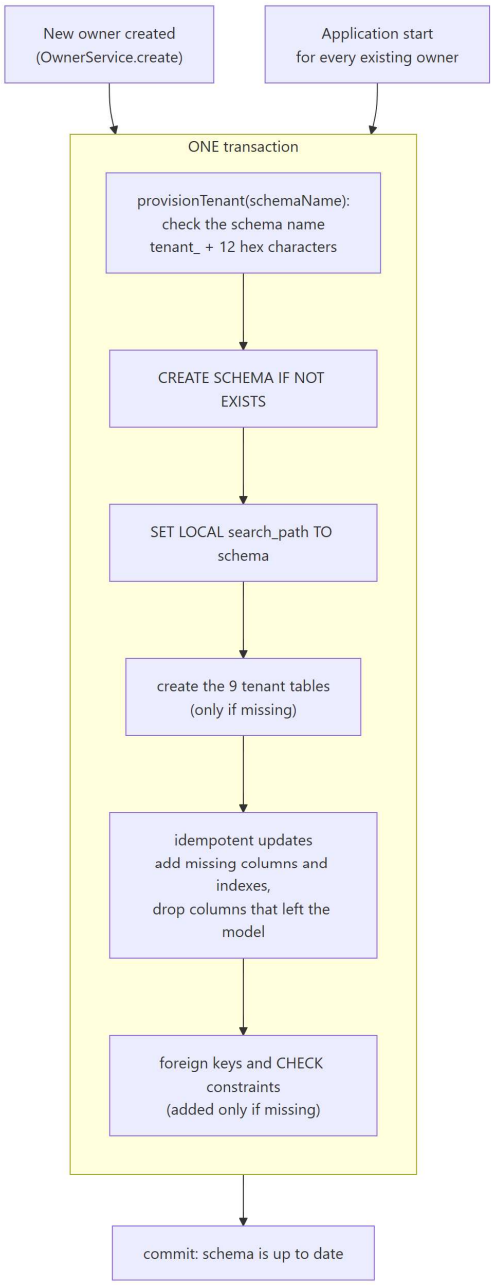
5.1 Δημιουργία και Προετοιμασία των Schemas

Ο μηχανισμός με τον οποίο ένα ταυτοποιημένο αίτημα δρομολογείται από το claim schemaName του JWT μέχρι την εκτέλεση του SET LOCAL search_path περιγράφεται στην §4.3. Εδώ καλύπτονται όσα δεν εξετάζονται εκεί: πώς δημιουργείται, προετοιμάζεται και διατηρείται ενημερωμένο το schema ενός tenant.

Τη δημιουργία των schemas αναλαμβάνει το αντικείμενο SchemaManager του module :backend, το οποίο εκτελεί αποκλειστικά εντολές DDL (Data Definition Language). Η μέθοδος createPlatformTables() δημιουργεί τους πέντε πίνακες του public schema κατά την εκκίνηση της εφαρμογής. Η μέθοδος provisionTenant(schemaName) εκτελεί όλα τα βήματα που χρειάζεται ένα tenant schema μέσα σε μία μόνο συναλλαγή: ελέγχει τη μορφή του ονόματος, εκτελεί CREATE SCHEMA IF NOT EXISTS, ορίζει το schema ως ενεργό με SET LOCAL search_path και δημιουργεί τους εννέα πίνακες του tenant μαζί με τους δείκτες, τα foreign keys και τους ελέγχους (CHECK constraints) τους. Η εκτέλεση σε μία συναλλαγή είναι ουσιώδης: η εντολή SET LOCAL ισχύει μόνο μέσα στη συναλλαγή, επομένως αν τα βήματα διασπώνταν σε περισσότερες συναλλαγές, μέρος των εντολών DDL θα μπορούσε να εκτελεστεί σε άλλη σύνδεση του pool και, κατά συνέπεια, σε λάθος schema.

Η provisionTenant() εκτελείται κατά τη δημιουργία κάθε νέου ιδιοκτήτη, αλλά και ξανά για κάθε υπάρχοντα tenant σε κάθε εκκίνηση της εφαρμογής (Εικόνα 5.1). Όλες οι εντολές της είναι idempotent (IF NOT EXISTS, έλεγχος ύπαρξης πριν από την προσθήκη ενός constraint), οπότε σε ένα ήδη ενημερωμένο schema δεν αλλάζουν τίποτα. Αντίθετα, σε schema που δημιουργήθηκε από παλαιότερη

έκδοση του κώδικα προσθέτουν τους πίνακες, τις στήλες, τους δείκτες και τα constraints που προστέθηκαν αργότερα και αφαιρούν τις στήλες που δεν υπάρχουν πια στο μοντέλο. Ο μηχανισμός λειτουργεί έτσι ως απλή, ενσωματωμένη διαδικασία migration, χωρίς εξωτερικό εργαλείο, με δύο περιορισμούς: κάθε αλλαγή πρέπει να διατυπώνεται ως idempotent εντολή και δεν τηρείται πίνακας εκδόσεων του schema. Το όνομα κάθε δείκτη ταυτίζεται σκόπιμα με το όνομα που θα έδινε αυτόματα το ίδιο το Exposed σε ένα νέο schema, ώστε να μη δημιουργούνται διπλοί δείκτες.



Εικόνα 5.1: Ροή δημιουργίας και ενημέρωσης tenant schema — η provisionTenant() εκτελείται κατά τη δημιουργία ιδιοκτήτη και σε κάθε εκκίνηση της εφαρμογής.

5.2 Public Schema — Δεδομένα Πλατφόρμας

Το public schema περιέχει τα δεδομένα που ανήκουν στην πλατφόρμα συνολικά και δεν εντάσσονται σε κάποιον συγκεκριμένο tenant. Αποτελείται από πέντε πίνακες.

5.2.1 owners — Ιδιοκτήτες Καταλυμάτων

Ο πίνακας owners είναι το σημείο εισόδου στην ιεραρχία του συστήματος: κάθε εγγραφή αντιστοιχεί σε έναν tenant και συνδέει τα διοικητικά στοιχεία του με το απομονωμένο schema του. Κάθε ιδιοκτήτης καταχωρείται ως επιχείρηση, με πεδία που αντιστοιχούν στα δημοσιευμένα στοιχεία του εταιρικού μητρώου: company_name και distinctive_title (επωνυμία και διακριτικός τίτλος), vat_id (ΑΦΜ), ο αριθμός μητρώου, legal_form (νομική μορφή), company_activity (κύρια δραστηριότητα) και η διεύθυνση της έδρας (street_name, street_no, postal_code, city). Τα πεδία που προέρχονται από το μητρώο αλλά δεν είναι απαραίτητα για τη λειτουργία της πλατφόρμας είναι προαιρετικά, ώστε η καταχώρηση να μπορεί να γίνει και χειροκίνητα όταν το μητρώο δεν είναι διαθέσιμο.

Τα contact_first_name, contact_last_name, contact_email και contact_phone αφορούν τον υπεύθυνο επικοινωνίας. Το contact_email χρησιμεύει ως αναγνωριστικό σύνδεσης (login) του ιδιοκτήτη· αποθηκεύεται με πεζά γράμματα και διαθέτει μοναδικό δείκτη στην έκφραση lower(contact_email). Έτσι η βάση απορρίπτει ένα διπλότυπο που διαφέρει μόνο στα κεφαλαία, ακόμη κι αν κάποιο σημείο του κώδικα παραλείψει την κανονικοποίηση. Τα secondary_contact_name, secondary_contact_phone και secondary_contact_email προορίζονται για ένα δεύτερο πρόσωπο επικοινωνίας, όπως ο λογιστής ή ο διαχειριστής ακινήτων, ενώ τα website και notes είναι προαιρετικά περιγραφικά πεδία.

Το schema_name συνδέει την εγγραφή με το αντίστοιχο schema στη βάση (tenant_{id}): ορίζεται κατά τη δημιουργία του λογαριασμού, δεν αλλάζει ποτέ μετά, και περνά ως claim σε κάθε JWT που εκδίδεται για τον ιδιοκτήτη ή για συσκευή που ανήκει σε κατάλυμά του. Το password_hash αποθηκεύει το BCrypt hash του κωδικού πρόσβασης (αιτιολόγηση στην §4.5.7) και ποτέ τον ίδιο τον κωδικό. Ο προσωρινός κωδικός (§3.1.1) δεν αποθηκεύεται πουθενά σε αναγνώσιμη μορφή. Το is_active επιτρέπει την απενεργοποίηση ενός tenant χωρίς διαγραφή δεδομένων (§4.5.7)· κατά την επανενεργοποίηση, το schema και το ιστορικό παραγγελιών παραμένουν ανέπαφα.

5.2.2 platform_admins — Διαχειριστές Πλατφόρμας

Ο πίνακας platform_admins αποθηκεύει τους λογαριασμούς των διαχειριστών της πλατφόρμας, χωριστά από τους ιδιοκτήτες. Η δομή του είναι σκόπιμα ελάχιστη: id, email, password_hash, created_at, με μοναδικό δείκτη στο lower(email). Ο διαχωρισμός από τον πίνακα owners, αντί για έναν ενιαίο πίνακα χρηστών με πεδίο ρόλου, αποτρέπει την ανάμειξη δύο ρόλων με εντελώς διαφορετικό εύρος πρόσβασης: ο Platform Admin δεν έχει tenant schema και το claim schemaName στο JWT του

παραμένει null. Όταν εξετάζει τα δεδομένα ενός ιδιοκτήτη για λόγους υποστήριξης, ορίζει ρητά το schema του σε κάθε αίτημα (§6.3).

5.2.3 refresh_tokens — Tokens Ανανέωσης Συνεδρίας

Ο πίνακας refresh_tokens βρίσκεται στο public schema επειδή εξυπηρετεί ταυτόχρονα χρήστες όλων των tenants: ένας ιδιοκτήτης του tenant_a και ένας του tenant_b έχουν εδώ τα δικά τους refresh tokens. Το user_id αναφέρεται στο αναγνωριστικό του χρήστη ανεξαρτήτως ρόλου, ενώ το role αντιγράφεται στο νέο access token κατά την ανανέωση και, για τους ιδιοκτήτες, ενεργοποιεί τον έλεγχο ότι ο λογαριασμός παραμένει ενεργός. Το token_hash, που διαθέτει δείκτη για γρήγορη αναζήτηση, αποθηκεύει το SHA-256 hash του token αντί για το ίδιο το token (αιτιολόγηση στην §4.5.7). Το schema_name (nullable) αντιγράφεται από τον ιδιοκτήτη ώστε να περιληφθεί στο νέο access token κατά την ανανέωση, και παραμένει null για τον Platform Admin. Το expires_at επιβάλλει τη λήξη του token (30 ημέρες). Οι συσκευές kiosk δεν έχουν εγγραφές σε αυτόν τον πίνακα, γιατί δεν χρησιμοποιούν refresh tokens.

5.2.4 audit_log — Αρχείο Καταγραφής Ενεργειών

Ο πίνακας audit_log καταγράφει τις σημαντικές διαχειριστικές ενέργειες και τις επιστροφές χρημάτων (§6.9). Τα actor_id και actor_role δείχνουν ποιος εκτέλεσε την ενέργεια, το action περιγράφει το είδος της (π.χ. CREATE_PROPERTY, PAYMENT_REFUNDED_ORDER_CANCELLED), και τα target_type και target_id (nullable) τον τύπο και το αντικείμενο που επηρεάστηκε. Το schema_name (nullable) επιτρέπει στον Platform Admin να φιλτράρει τις εγγραφές ανά tenant, ενώ το details (nullable) περιέχει ελεύθερο κείμενο με πρόσθετο πλαίσιο, για παράδειγμα το ποσό και το αναγνωριστικό συναλλαγής μιας επιστροφής χρημάτων. Η εφαρμογή εκτελεί στον πίνακα μόνο εισαγωγές, χωρίς ενημερώσεις ή διαγραφές· κάθε εγγραφή περιγράφει ένα συμβάν που έχει ήδη συντελεστεί και όχι μια κατάσταση που αλλάζει.

5.2.5 platform_settings — Ρυθμίσεις Πλατφόρμας

Ο πίνακας platform_settings αποθηκεύει τις ρυθμίσεις των εξωτερικών υπηρεσιών που αφορούν ολόκληρη την πλατφόρμα, δηλαδή τον διακομιστή SMTP και το κλειδί πρόσβασης του εταιρικού μητρώου. Έχει τη μορφή απλού ζεύγους κλειδιού-τιμής (key, value, updated_at), με το key ως πρωτεύον κλειδί (π.χ. smtp.host, smtp.port). Η μορφή αυτή επιλέχθηκε επειδή οι ρυθμίσεις είναι λίγες, διαβάζονται πάντα μαζί και η προσθήκη μιας νέας ρύθμισης δεν απαιτεί αλλαγή στη δομή του πίνακα. Οι μυστικές τιμές (ο κωδικός SMTP και το κλειδί του μητρώου) αποθηκεύονται κρυπτογραφημένες με AES-256-GCM (§4.5.8).

5.3 Tenant Schema — Δεδομένα Καταλύματος

Κάθε ιδιοκτήτης έχει δικό του schema με εννέα πίνακες, που δημιουργούνται αυτόματα από τον SchemaManager (§5.1): properties, devices, products, product_property_overrides, orders, order_items, payments, food_platform_connections και property_food_platforms. Οι σχέσεις μεταξύ των πινάκων επιβάλλονται από foreign key constraints μέσα στο schema, με κανόνες διαγραφής που ανταποκρίνονται στη σημασία κάθε σχέσης (§5.5). Επειδή όλοι οι πίνακες ενός tenant βρίσκονται στο ίδιο schema, κανένα foreign key δεν υπερβαίνει τα όρια ενός schema.

5.3.1 properties — Καταλύματα

Ο πίνακας properties αποθηκεύει τα φυσικά καταλύματα ενός ιδιοκτήτη, δηλαδή το δεύτερο επίπεδο της ιεραρχίας Ιδιοκτήτης → Κατάλυμα → Συσκευή → Φιλοξενούμενος. Κάθε κατάλυμα ορίζεται με πλήρη στοιχεία διεύθυνσης (street_name, street_no, postal_code, area (προαιρετικό), city, level, name_on_doorbell, contact_phone). Τα στοιχεία αυτά χρησιμοποιούνται και ως διεύθυνση παράδοσης όταν μια παραγγελία φαγητού υποβάλλεται σε πλατφόρμα διανομής, γι' αυτό το όνομα στο κουδούνι είναι υποχρεωτικό και ο όροφος αποθηκεύεται πάντα (0 όταν μένει κενός). Τα latitude/longitude (προαιρετικά) συμπληρώνονται αυτόματα όταν η διεύθυνση εισάγεται μέσω της αναζήτησης διευθύνσεων (§3.1.2). Τα country και timezone αποθηκεύονται ανά κατάλυμα (§3.4.3), ενώ οι χρονοσφραγίδες τηρούνται σε UTC epoch milliseconds.

Τα πεδία wifi_name, wifi_password, check_out_time (μορφή HH:mm) και house_rules είναι προαιρετικά και περιέχουν τις πληροφορίες που η εφαρμογή kiosk εμφανίζει στον επισκέπτη στην οθόνη πληροφοριών διαμονής. Βρίσκονται στον ίδιο πίνακα επειδή σχετίζονται 1:1 με το κατάλυμα. Το is_active επιτρέπει την απενεργοποίηση χωρίς απώλεια ιστορικών δεδομένων· όταν απενεργοποιείται ένα κατάλυμα, απενεργοποιούνται και οι συσκευές του.

5.3.2 devices — Συσκευές Kiosk

Ο πίνακας devices αποθηκεύει τις συσκευές Android που εκτελούν την εφαρμογή kiosk σε κάθε κατάλυμα. Η σχεδίασή του ανταποκρίνεται σε μια πρακτική ανάγκη: ο ιδιοκτήτης καταχωρεί τη συσκευή στο web panel πριν αυτή εγκατασταθεί στο κατάλυμα, οπότε αρκετά από τα πεδία της είναι άγνωστα τη στιγμή της δημιουργίας. Το property_id είναι foreign key προς το κατάλυμα, και το name είναι το όνομα που δίνει ο ιδιοκτήτης στη συσκευή, ώστε να την αναγνωρίζει στο web panel. Το android_device_id είναι nullable, επειδή λαμβάνει τιμή μόνο όταν ενεργοποιηθεί η εφαρμογή στη συσκευή. Το activation_code (οκτώ χαρακτήρες) παράγεται κατά τη δημιουργία και διαγράφεται κατά την ενεργοποίηση, ώστε να μην μπορεί να χρησιμοποιηθεί δεύτερη φορά· το activated_at (nullable) καταγράφει τη στιγμή που ολοκληρώθηκε η ενεργοποίηση. Όταν μια συσκευή αποσυνδεθεί, τα

android_device_id και activated_at λαμβάνουν τιμή null, η συσκευή απενεργοποιείται και παράγεται νέο activation_code. Η πλήρης ροή ενεργοποίησης περιγράφεται στην §3.3.4 και στην §8.2. Το last_seen_at (nullable) καταγράφει την τελευταία επικοινωνία της συσκευής: η εφαρμογή kiosk στέλνει κάθε λεπτό ένα αίτημα heartbeat (PATCH /devices/{id}/ping) με το δικό της token, ώστε ο ιδιοκτήτης να ξεχωρίζει μια συσκευή που δεν έχει ενεργοποιηθεί ποτέ από μια συσκευή που είναι απλώς εκτός σύνδεσης. Το is_active ελέγχεται σε κάθε αίτημα της συσκευής (§4.5.7). Το is_kiosk_enabled προβλέφθηκε για να δηλώνει αν η συσκευή λειτουργεί σε κατάσταση kiosk (§8.3)· στο τρέχον πρωτότυπο λαμβάνει την τιμή false κατά τη δημιουργία της συσκευής και δεν χρησιμοποιείται.

5.3.3 products — Προϊόντα

Ο πίνακας products αποθηκεύει τον κατάλογο προϊόντων ενός ιδιοκτήτη, ο οποίος είναι κοινός για όλα τα καταλύματά του. Το category δηλώνει τον τύπο του προϊόντος (π.χ. BEVERAGE, SNACK, PHARMACY) και χρησιμοποιείται για την ομαδοποίηση στην εφαρμογή kiosk. Το base_price αποθηκεύεται σε λεπτά (cents) και υπόκειται σε έλεγχο (CHECK) ώστε να μην είναι αρνητικό, το image_url είναι προαιρετικό, και το is_available απενεργοποιεί το προϊόν σε όλα τα καταλύματα. Τα προϊόντα που εισάγονται από τον κατάλογο μιας πλατφόρμας διανομής (§3.1.2) αποθηκεύονται ως απλές εγγραφές του πίνακα, χωρίς σύνδεση με την πλατφόρμα προέλευσης, ώστε ο ιδιοκτήτης να μπορεί να αλλάξει ελεύθερα την τιμή και την περιγραφή τους.

5.3.4 product_property_overrides — Παρακάμψεις Διαθεσιμότητας ανά Κατάλυμα

Ο πίνακας product_property_overrides επιτρέπει σε έναν ιδιοκτήτη με περισσότερα καταλύματα να απενεργοποιεί συγκεκριμένα προϊόντα σε συγκεκριμένα καταλύματα, χωρίς να επηρεάζεται ο υπόλοιπος κατάλογος. Ο πίνακας περιλαμβάνει μόνο το ζεύγος product_id και property_id, που αποτελεί και το σύνθετο πρωτεύον κλειδί του: η ύπαρξη μιας γραμμής σημαίνει ότι το προϊόν είναι απενεργοποιημένο στο συγκεκριμένο κατάλυμα, ενώ η απουσία της ότι είναι διαθέσιμο. Και οι δύο στήλες είναι foreign keys με αλυσιδωτή διαγραφή (CASCADE), ώστε όταν διαγράφεται ένα προϊόν ή ένα κατάλυμα να αφαιρούνται αυτόματα και οι παρακάμψεις του. Ο πίνακας δεν έχει στήλη τιμής: η τιμή ενός προϊόντος είναι ενιαία σε όλα τα καταλύματα του ίδιου ιδιοκτήτη.

5.3.5 orders — Παραγγελίες

Ο πίνακας orders είναι ο κεντρικός πίνακας της επιχειρησιακής λειτουργίας του συστήματος. Το device_id συνδέει την παραγγελία με τη συσκευή που την υπέβαλε: ο φιλοξενούμενος δεν ταυτοποιείται μέσω λογαριασμού, αλλά το κατάλυμα προκύπτει από τη συσκευή που υπέβαλε την παραγγελία. Το property_id αποθηκεύεται ρητά, παρότι προκύπτει και έμμεσα από τη συσκευή, ώστε

να αποφεύγεται ένα επιπλέον join σε κάθε αναζήτηση παραγγελιών ανά κατάλυμα. Και τα δύο είναι foreign keys με κανόνα RESTRICT, επομένως ένα κατάλυμα ή μια συσκευή με ιστορικό παραγγελιών δεν μπορεί να διαγραφεί οριστικά, παρά μόνο να απενεργοποιηθεί. Το guest_email (nullable) καταγράφεται μόνο αν το δώσει ο φιλοξενούμενος οικειοθελώς για να λάβει απόδειξη· σε καμία περίπτωση δεν δημιουργείται λογαριασμός.

Το status αποτυπώνει την κατάσταση της παραγγελίας στον κύκλο ζωής της (PENDING, CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED), ενώ οι επιτρεπτές μεταβάσεις επιβάλλονται στο επίπεδο υπηρεσιών (§6.5). Το type ξεχωρίζει τις παραγγελίες προϊόντων του καταλύματος (IN_ROOM_STORE) από τις παραγγελίες φαγητού (FOOD_DELIVERY). Για τις δεύτερες, τα external_platform, restaurant_id και restaurant_name καταγράφουν την πλατφόρμα και το εστιατόριο, το external_order_id το αναγνωριστικό που επέστρεψε η πλατφόρμα κατά την υποβολή, το delivery_fee το κόστος αποστολής και το estimated_delivery_at τον εκτιμώμενο χρόνο παράδοσης όπως τον ανακοινώνει η πλατφόρμα. Το total_amount υπολογίζεται από το backend κατά τη δημιουργία και αποθηκεύεται, ενώ το updated_at ενημερώνεται σε κάθε αλλαγή κατάστασης· το backend το χρησιμοποιεί για να κρίνει πότε μια υποβολή σε πλατφόρμα διανομής θεωρείται αποτυχημένη (§6.5). Δείκτες στα device_id, property_id και status εξυπηρετούν τις συχνότερες αναζητήσεις.

5.3.6 order_items — Γραμμές Παραγγελίας

Ο πίνακας order_items αποθηκεύει τα επιμέρους προϊόντα κάθε παραγγελίας και λειτουργεί ως στιγμιότυπο (snapshot): αντί να αποθηκεύει μόνο το product_id και να αναζητά κάθε φορά τα στοιχεία του προϊόντος, αντιγράφει τα product_name και unit_price τη στιγμή που δημιουργείται η παραγγελία. Έτσι το ιστορικό παραμένει ακριβές ακόμη κι αν ο ιδιοκτήτης αλλάξει αργότερα το όνομα ή την τιμή ενός προϊόντος, ή αν μια πλατφόρμα αλλάξει το μενού της. Επιπλέον, το product_id σκόπιμα δεν είναι foreign key: σε μια παραγγελία φαγητού αναφέρεται σε πιάτο του μενού μιας πλατφόρμας, το οποίο δεν υπάρχει στη βάση δεδομένων του συστήματος. Το order_id είναι foreign key με αλυσιδωτή διαγραφή. Το subtotal αποθηκεύεται απευθείας αντί να υπολογίζεται ξανά σε κάθε ερώτημα, και CHECK constraints εξασφαλίζουν θετική ποσότητα και μη αρνητικά ποσά.

5.3.7 payments — Πληρωμές

Ο πίνακας payments καταγράφει τις απόπειρες πληρωμής με κάρτα για μια παραγγελία. Η σχέση με την παραγγελία είναι 1:N, επειδή ο επισκέπτης μπορεί να ξαναδοκιμάσει μετά από μια απόρριψη. Το entry_mode δείχνει με ποιον τρόπο διαβάστηκε η κάρτα (CARD_CONTACTLESS, CARD_CHIP, CARD_MAGSTRIPE), το card_last4 (nullable) τα τέσσερα τελευταία ψηφία της κάρτας όταν είναι διαθέσιμα, και το status το αποτέλεσμα (COMPLETED, FAILED ή, μετά από επιστροφή χρημάτων,

REFUNDED), ενώ το decline_reason εξηγεί την αιτία μιας απόρριψης. Το provider_transaction_id αποθηκεύει το αναγνωριστικό συναλλαγής που επέστρεψε ο φορέας πληρωμών (acquirer) και χρησιμοποιείται στο αίτημα επιστροφής προς τον φορέα. Ένας μερικός μοναδικός δείκτης (partial unique index) στο order_id με συνθήκη status = 'COMPLETED' επιτρέπει πολλές απορριφθείσες απόπειρες αλλά το πολύ μία εγκεκριμένη πληρωμή ανά παραγγελία, και αποτελεί την τελευταία δικλίδα της βάσης απέναντι σε διπλή καταγραφή εγκεκριμένης πληρωμής.

5.3.8 food_platform_connections — Συνδέσεις με Πλατφόρμες Διανομής

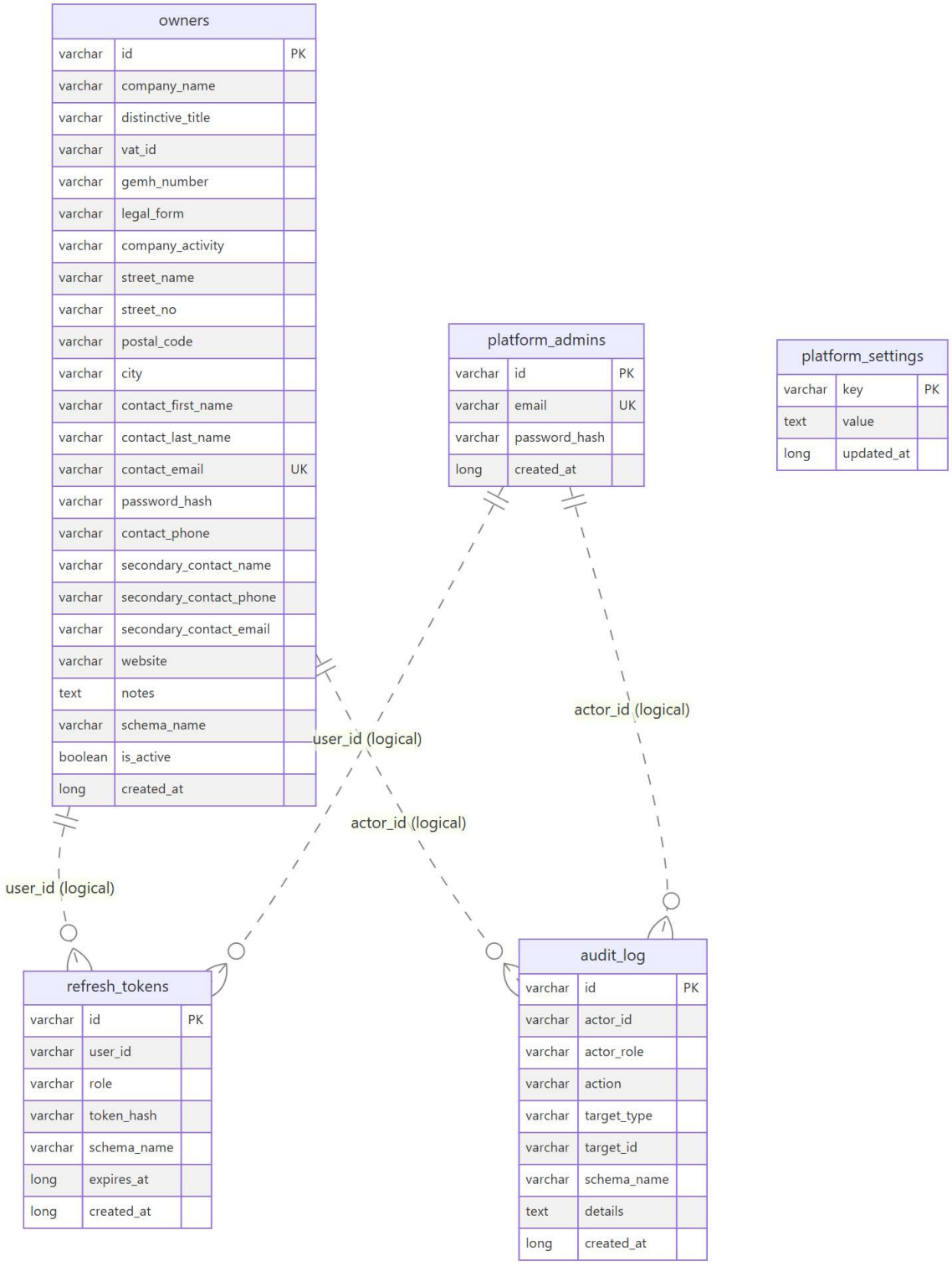
Ο πίνακας food_platform_connections καταγράφει τις πλατφόρμες διανομής φαγητού που έχει συνδέσει ο ιδιοκτήτης. Πρωτεύον κλειδί είναι η ίδια η πλατφόρμα (platform), αφού κάθε ιδιοκτήτης συνδέεται το πολύ μία φορά με κάθε πλατφόρμα. Το api_key αποθηκεύει το κλειδί πρόσβασης του ιδιοκτήτη κρυπτογραφημένο με AES-256-GCM (§4.5.8) και το connected_at τη στιγμή της σύνδεσης. Επειδή ο πίνακας βρίσκεται στο tenant schema και όχι στο public, τα κλειδιά κάθε ιδιοκτήτη απομονώνονται με τον ίδιο μηχανισμό που προστατεύει και τα υπόλοιπα δεδομένα του.

5.3.9 property_food_platforms — Πλατφόρμες ανά Κατάλυμα

Ο πίνακας property_food_platforms ορίζει ποιες από τις συνδεδεμένες πλατφόρμες προσφέρονται σε κάθε κατάλυμα. Ακολουθεί την ίδια ελάχιστη σχεδίαση με τον product_property_overrides: σύνθετο πρωτεύον κλειδί (property_id, platform), όπου η ύπαρξη μιας γραμμής σημαίνει ότι η πλατφόρμα είναι διαθέσιμη στο κατάλυμα. Το property_id είναι foreign key με αλυσιδωτή διαγραφή. Όταν ο ιδιοκτήτης αποσυνδέει μια πλατφόρμα, αφαιρούνται και οι αντίστοιχες γραμμές· η αποσύνδεση δεν επιτρέπεται όσο υπάρχουν παραγγελίες της πλατφόρμας σε εξέλιξη.

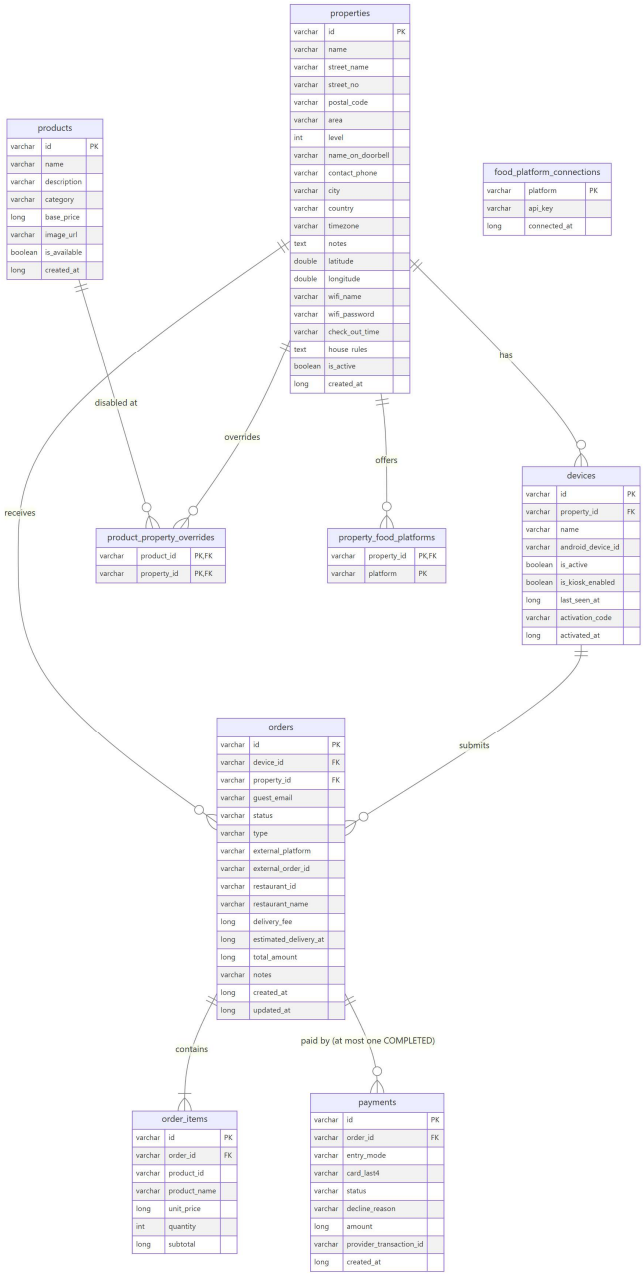
5.4 Διάγραμμα Οντοτήτων-Σχέσεων

Το public schema (Εικόνα 5.2) περιλαμβάνει τους πίνακες owners, platform_admins και platform_settings, που δεν σχετίζονται άμεσα μεταξύ τους. Οι πίνακες refresh_tokens και audit_log συνδέονται λογικά με τους χρήστες μέσω των πεδίων user_id/role και actor_id/actor_role αντίστοιχα, χωρίς foreign key constraint, γιατί ο χρήστης στον οποίο αναφέρονται μπορεί να βρίσκεται σε οποιονδήποτε από τους δύο πίνακες χρηστών ή, στην περίπτωση του audit_log, να είναι συσκευή που ανήκει σε tenant schema.



Εικόνα 5.2: Οντότητες του public schema — δεδομένα πλατφόρμας.

Το tenant schema (Εικόνα 5.3) περιλαμβάνει τους εννέα πίνακες της §5.3, με τις εξής σχέσεις: properties ↔ devices (1:N), properties ↔ orders (1:N), devices ↔ orders (1:N), orders ↔ order_items (1:N), orders ↔ payments (1:N, με το πολύ μία εγκεκριμένη πληρωμή), products ↔ product_property_overrides (1:N), properties ↔ product_property_overrides (1:N) και properties ↔ property_food_platforms (1:N). Ο πίνακας food_platform_connections δεν συνδέεται με foreign key με τους υπόλοιπους· οι γραμμές του property_food_platforms παραπέμπουν στις συνδεδεμένες πλατφόρμες μέσω της τιμής της στήλης platform, και η συνέπεια ελέγχεται στο επίπεδο υπηρεσιών.



Εικόνα 5.3: Οντότητες του tenant schema — δεδομένα καταλύματος.

5.5 Αποφάσεις Σχεδίασης

Ο Πίνακας 5.1 συνοψίζει τις βασικές αποφάσεις σχεδίασης, οι οποίες εφαρμόζονται ομοιόμορφα σε όλους τους πίνακες.

Πίνακας 5.1: Βασικές αποφάσεις σχεδίασης της βάσης δεδομένων

Απόφαση	**Επιλογή**	**Αιτιολόγηση**
Πρωτεύοντα κλειδιά	UUID v4 (varchar(36))	Τα αναγνωριστικά είναι μοναδικά σε όλα τα tenant schemas. Το API δεν εκθέτει προβλέψιμα αναγνωριστικά. Παράγονται στο επίπεδο υπηρεσιών, χωρίς επιπλέον round-trip προς τη βάση.
Χρηματικά ποσά	Long (λεπτά / cents)	Αποφεύγονται τα σφάλματα στρογγυλοποίησης των αριθμών κινητής υποδιαστολής (floating point). Η μετατροπή σε ευρώ γίνεται μόνο στο επίπεδο παρουσίασης.
Χρονοσφραγίδες	Long (epoch milliseconds, UTC)	Η αποθήκευση δεν εξαρτάται από ζώνη ώρας· η μετατροπή σε τοπική ώρα γίνεται στον client, ώστε να αποφεύγονται προβλήματα με τη θερινή ώρα.
Τιμές απαρίθμησης (enum)	varchar	Αναγνώσιμες τιμές στη βάση, ανεξάρτητες από τη σειρά με την οποία ορίζονται στο enum της Kotlin.
Αποθήκευση κωδικού πρόσβασης	BCrypt hash, cost 12	Πλήρης αιτιολόγηση στην §4.5.7.
Αποθήκευση refresh token	SHA-256 hash	Πλήρης αιτιολόγηση στην §4.5.7.
Αποθήκευση μυστικών εξωτερικών υπηρεσιών	AES-256-GCM	Οι τιμές πρέπει να ανακτώνται αυτούσιες· πλήρης αιτιολόγηση στην §4.5.8.
Foreign key constraints	Μέσα σε κάθε tenant schema	RESTRICT όπου η διαγραφή θα κατέστρεφε ιστορικό (συσκευές, παραγγελίες, πληρωμές), CASCADE για τους πίνακες

Απόφαση	**Επιλογή**	**Αιτιολόγηση**
		συσχέτισης και τις γραμμές παραγγελίας. Κανένα foreign key δεν υπερβαίνει τα όρια ενός schema.
Έλεγχοι τιμών	CHECK constraints	Θετική ποσότητα, μη αρνητικές τιμές και ποσά: η βάση απορρίπτει μη έγκυρα δεδομένα ακόμη κι αν κάποιο σφάλμα του κώδικα τα παράγει.
Ενημέρωση δομής	Idempotent DDL σε κάθε εκκίνηση	Τα υπάρχοντα tenant schemas ενσωματώνουν αυτόματα τις αλλαγές του μοντέλου (§5.1), χωρίς εξωτερικό εργαλείο migration.
Ονομασία στηλών	snake_case	Σύμβαση της PostgreSQL· τα ονόματα με πεζά γράμματα δεν χρειάζονται εισαγωγικά (quoted identifiers).
Λογική διαγραφή (soft delete)	Πεδίο is_active	Αποτρέπει τη μόνιμη απώλεια δεδομένων. Οι ιδιοκτήτες δεν διαγράφονται ποτέ οριστικά· τα καταλύματα και οι συσκευές διαγράφονται οριστικά μόνο όταν δεν έχουν παραγγελίες.

5.6 Συναλλαγές και Ταυτόχρονη Πρόσβαση

Η δομή των πινάκων εξασφαλίζει την ορθότητα των δεδομένων μόνο αν συνοδεύεται από σωστή διαχείριση των συναλλαγών, αφού πολλά αιτήματα εξυπηρετούνται ταυτόχρονα: ένα kiosk υποβάλλει πληρωμή, ο ιδιοκτήτης αλλάζει την κατάσταση της ίδιας παραγγελίας και μια πλατφόρμα διανομής στέλνει συμβάν προόδου. Οι αποφάσεις του συστήματος σε αυτό το επίπεδο συνοψίζονται σε πέντε σημεία.

Πρώτον, όλες οι συνδέσεις προς την PostgreSQL προέρχονται από ένα κοινό connection pool του HikariCP με το πολύ δέκα συνδέσεις, το οποίο εξυπηρετεί όλους τους tenants (§4.5.3). Το schema επιλέγεται σε κάθε συναλλαγή όπως περιγράφεται στην §4.3. Επιπλέον, κάθε ερώτημα στο public

schema ορίζει ρητά το search_path σε public, ώστε ένα ερώτημα πλατφόρμας να εκτελείται σωστά ακόμη κι αν κληθεί εμφωλευμένο μέσα σε συναλλαγή tenant.

Δεύτερον, οι εμφωλευμένες κλήσεις συμμετέχουν στην ίδια συναλλαγή. Κάθε μέθοδος repository εκτελείται μέσα σε δικό της tenantQuery, αλλά όταν μια υπηρεσία καλεί περισσότερα repositories μέσα σε ένα εξωτερικό tenantQuery, το Exposed επαναχρησιμοποιεί τη συναλλαγή που είναι ήδη ανοιχτή αντί να ανοίξει νέα. Έτσι η δημιουργία μιας παραγγελίας μαζί με τις γραμμές της (§6.5), ή η καταγραφή μιας πληρωμής μαζί με την αλλαγή κατάστασης της παραγγελίας (§6.6), είναι ατομική: ολοκληρώνονται είτε όλες οι εγγραφές είτε καμία.

Τρίτον, το επίπεδο απομόνωσης των συναλλαγών είναι το READ COMMITTED, το προεπιλεγμένο της PostgreSQL. Ένα αυστηρότερο επίπεδο, όπως το REPEATABLE READ, θα απέρριπτε με σφάλμα σειριοποίησης κάθε συναλλαγή που ενημερώνει γραμμή την οποία άλλη συναλλαγή άλλαξε στο μεταξύ, και θα απαιτούσε μηχανισμό επανάληψης σε κάθε σημείο του κώδικα. Γι' αυτό οι λίγες λειτουργίες ανάγνωσης-τροποποίησης-εγγραφής του συστήματος προστατεύονται ρητά με τους δύο μηχανισμούς που ακολουθούν.

Τέταρτον, οι αλλαγές κατάστασης μιας παραγγελίας γίνονται με compare-and-set: η εντολή UPDATE περιέχει ως συνθήκη την κατάσταση που διαβάστηκε (WHERE id = ? AND status = ?). Αν στο μεταξύ μια άλλη ενέργεια έχει αλλάξει την παραγγελία, η εντολή δεν επηρεάζει καμία γραμμή και η υπηρεσία το αντιμετωπίζει ως σύγκρουση (§6.5). Ο μηχανισμός δεν διατηρεί κλείδωμα ανάμεσα στην ανάγνωση και την εγγραφή και ταιριάζει σε ενέργειες που μπορούν απλώς να αποτύχουν και να επαναληφθούν από τον χρήστη.

Πέμπτον, κατά την πληρωμή, όπου μια διπλή εκτέλεση θα οδηγούσε σε διπλή χρέωση, η γραμμή της παραγγελίας κλειδώνεται με SELECT … FOR UPDATE στην αρχή της συναλλαγής (§6.6). Δύο ταυτόχρονες απόπειρες πληρωμής για την ίδια παραγγελία εκτελούνται έτσι η μία μετά την άλλη, και η δεύτερη βρίσκει την παραγγελία ήδη πληρωμένη. Ο μερικός μοναδικός δείκτης του πίνακα payments (§5.3.7) και η αντιστοίχιση των παραβιάσεων μοναδικότητας σε HTTP 409 (§6.3) αποτελούν τη δεύτερη, ανεξάρτητη δικλίδα. Στην επαλήθευση της §9.3, μια δεύτερη απόπειρα πληρωμής για ήδη πληρωμένη παραγγελία απορρίφθηκε με 409 χωρίς να φτάσει στον φορέα πληρωμών.

Η δομή που περιγράφηκε σε αυτό το κεφάλαιο αντιστοιχεί ακριβώς στο τρέχον στάδιο υλοποίησης του συστήματος: εννέα πίνακες ανά tenant schema και πέντε στο schema της πλατφόρμας, χωρίς καμία οντότητα που προϋποθέτει λειτουργικότητα εκτός του εύρους της εργασίας. Η υλοποίηση της επιχειρησιακής λογικής που χρησιμοποιεί αυτή τη δομή αναλύεται στο Κεφάλαιο 6.

Κεφάλαιο 6. Υλοποίηση Backend

Το module :backend μετατρέπει τις αρχιτεκτονικές αποφάσεις των Κεφαλαίων 4 και 5 σε λειτουργικό κώδικα: ένα REST API σε Ktor, οργανωμένο σε σαφή επίπεδα (routing, επίπεδο υπηρεσιών, repositories, adapters), στο οποίο ο μηχανισμός TenantContext (§4.3) διατρέχει την αλυσίδα κλήσεων κάθε αιτήματος. Το κεφάλαιο περιγράφει πώς συνεργάζονται στην πράξη αυτά τα επίπεδα: πώς δρομολογείται ένα αίτημα, πώς επαληθεύεται η ταυτότητα της συσκευής kiosk και του χρήστη, πώς η επιχειρησιακή λογική υλοποιεί τον κύκλο ζωής μιας παραγγελίας και μιας πληρωμής, πώς ενσωματώνονται οι πλατφόρμες διανομής φαγητού και οι πραγματικές εξωτερικές υπηρεσίες, και πώς τα σφάλματα γίνονται απαντήσεις HTTP.

6.1 Δομή του REST API και Επίπεδα Δρομολόγησης

Το API εκτίθεται αποκλειστικά κάτω από το πρόθεμα /api και οργανώνεται σε δεκαπέντε αρχεία routing, καθένα για μία οντότητα ή λειτουργία: HealthRoute, AuthRoutes, OwnerRoutes, PropertyRoutes, DeviceRoutes, ProductRoutes, OrderRoutes, PaymentRoutes, FoodPlatformRoutes, DashboardRoutes, AdminRoutes, AuditLogRoutes, SettingsRoutes, CompanyLookupRoutes και GeocodeRoutes. Κάθε ομάδα routes ορίζεται ως extension function πάνω στο Ktor Route, σύμβαση που διατηρεί το routing declarative και ευανάγνωστο, χωρίς annotations (§4.5.2). Ορισμένα αρχεία δηλώνουν περισσότερες από μία ομάδες: το OrderRoutes.kt ξεχωρίζει τις παραγγελίες όπως τις βλέπει ο ιδιοκτήτης (/orders) από αυτές που βλέπει η συσκευή που τις υπέβαλε (/device/orders), ενώ το FoodPlatformRoutes.kt ξεχωρίζει τις διαδρομές με τις οποίες ο ιδιοκτήτης διαχειρίζεται τις συνδέσεις (/food-platforms, /properties/{id}/food-platforms) από τον κατάλογο εστιατορίων που διαβάζει η συσκευή (/food/restaurants).

Μόνο πέντε endpoints δεν απαιτούν ταυτοποίηση: το GET /health, τα POST /auth/login και POST /auth/refresh, και δύο endpoints που εξυπηρετούν την εκκίνηση (bootstrap) μιας συσκευής kiosk πριν αποκτήσει ταυτότητα, δηλαδή η επίλυση ενός κωδικού ενεργοποίησης που πληκτρολογήθηκε χειροκίνητα (GET /devices/resolve) και η ίδια η ενεργοποίηση (POST /devices/{id}/activate). Τέσσερα από αυτά δέχονται μυστικά (κωδικό πρόσβασης, refresh token ή κωδικό ενεργοποίησης) και προστατεύονται επιπλέον με rate limiting (§6.3). Το heartbeat μιας συσκευής (PATCH /devices/{id}/ping) απαιτεί το token της ίδιας της συσκευής, ώστε μόνο εκείνη να μπορεί να ενημερώσει την τελευταία επικοινωνία της. Όλα τα υπόλοιπα endpoints βρίσκονται μέσα σε μπλοκ authenticate("jwt") { } και απαιτούν έγκυρο JWT. Επομένως, η δημιουργία παραγγελίας και η πληρωμή, που εκτελούνται από το kiosk εκ μέρους του φιλοξενούμενου, απαιτούν ταυτοποιημένη συσκευή· κανένα endpoint παραγγελίας ή πληρωμής δεν δέχεται ανώνυμα αιτήματα.

Μια λεπτομέρεια του Ktor επηρέασε την οργάνωση των routes των συσκευών. Τα δημόσια endpoints της συσκευής (/devices/resolve, /devices/{id}/activate) και τα endpoints του ιδιοκτήτη που απαιτούν ταυτοποίηση (/devices/{id}, /devices/{id}/deactivate κ.ά.) έχουν κοινό πρόθεμα. Όταν δηλώνονταν ως δύο ξεχωριστά δέντρα routes, το Ktor ταίριαζε το resolve με την παράμετρο {id} του δέντρου που απαιτεί ταυτοποίηση και απαντούσε 401. Γι' αυτό το DeviceRoutes.kt δηλώνει ένα ενιαίο δέντρο /devices, όπου μόνο οι διαδρομές του ιδιοκτήτη περικλείονται σε authenticate("jwt").

Ο Πίνακας 6.1 ομαδοποιεί τα endpoints ανά κατηγορία, μαζί με το αν η κάθε κατηγορία απαιτεί JWT και ποιους ρόλους δέχεται.

Πίνακας 6.1: Endpoints του API ανά κατηγορία, απαίτηση JWT και ρόλο

Κατηγορία	**Endpoints (ενδεικτικά)**	**Απαίτηση JWT**	**Ρόλοι**
Bootstrap πλατφόρμας και συσκευής	/health, /auth/login, /auth/refresh, /devices/resolve, /devices/{id}/activate	Όχι	—
Λειτουργίες kiosk	POST /orders, /device/orders/*, POST /payments, GET /products, /food/restaurants/*, GET /properties/{id}, /devices/{id}/ping	Ναι	DEVICE
Λογαριασμός χρήστη	POST /auth/change-password	Ναι	OWNER, PLATFORM_ADMIN
Διαχείριση tenant	/properties, /devices, /products, /orders, /food-platforms, /geocode/*, /dashboard	Ναι	OWNER, PLATFORM_ADMIN (το /dashboard μόνο OWNER)
Διαχείριση πλατφόρμας	/owners, /admin/*, /audit-logs, /settings/*, /company-lookup	Ναι	PLATFORM_ADMIN

6.2 Ταυτοποίηση Συσκευής και ο Ρόλος DEVICE

Η συσκευή kiosk δεν έχει κωδικό πρόσβασης ούτε λογαριασμό χρήστη· αποκτά ταυτότητα αποκλειστικά μέσω της ενεργοποίησης. Όταν ο ιδιοκτήτης δημιουργεί μια νέα συσκευή

(DeviceService.create()), παράγονται ένας κωδικός ενεργοποίησης οκτώ χαρακτήρων, από κρυπτογραφικά ασφαλή γεννήτρια τυχαίων αριθμών (SecureRandom), και ένας κωδικός QR που κωδικοποιεί σε μορφή JSON το αναγνωριστικό της συσκευής, τον κωδικό ενεργοποίησης, το όνομα του tenant schema και τη δημόσια διεύθυνση του API (μεταβλητή περιβάλλοντος PUBLIC_API_URL). Κατά την ενεργοποίηση (DeviceService.activate()), η υπηρεσία επαληθεύει τον κωδικό και ελέγχει ότι το κατάλυμα και ο ιδιοκτήτης της συσκευής είναι ενεργοί, αλλιώς απαντά με 409, όπως και για μια συσκευή που έχει ήδη ενεργοποιηθεί. Στη συνέχεια η συσκευή συνδέεται με το αναγνωριστικό υλικού της και ο κωδικός διαγράφεται με μία ενημέρωση compare-and-set, με συνθήκη activated_at IS NULL και κωδικό που εξακολουθεί να ταυτίζεται, ώστε από δύο ταυτόχρονες ενεργοποιήσεις με τον ίδιο κωδικό να επιτυγχάνει μόνο η μία και η άλλη να λαμβάνει 409. Τέλος εκδίδεται ένα JWT access token με claim ρόλου UserRole.DEVICE και διάρκεια ζωής ενός έτους (§4.5.7). Το token επιστρέφεται μόνο μία φορά, στο πεδίο token του DeviceResponse· καμία μεταγενέστερη ανάγνωση της συσκευής δεν το εμφανίζει ξανά.

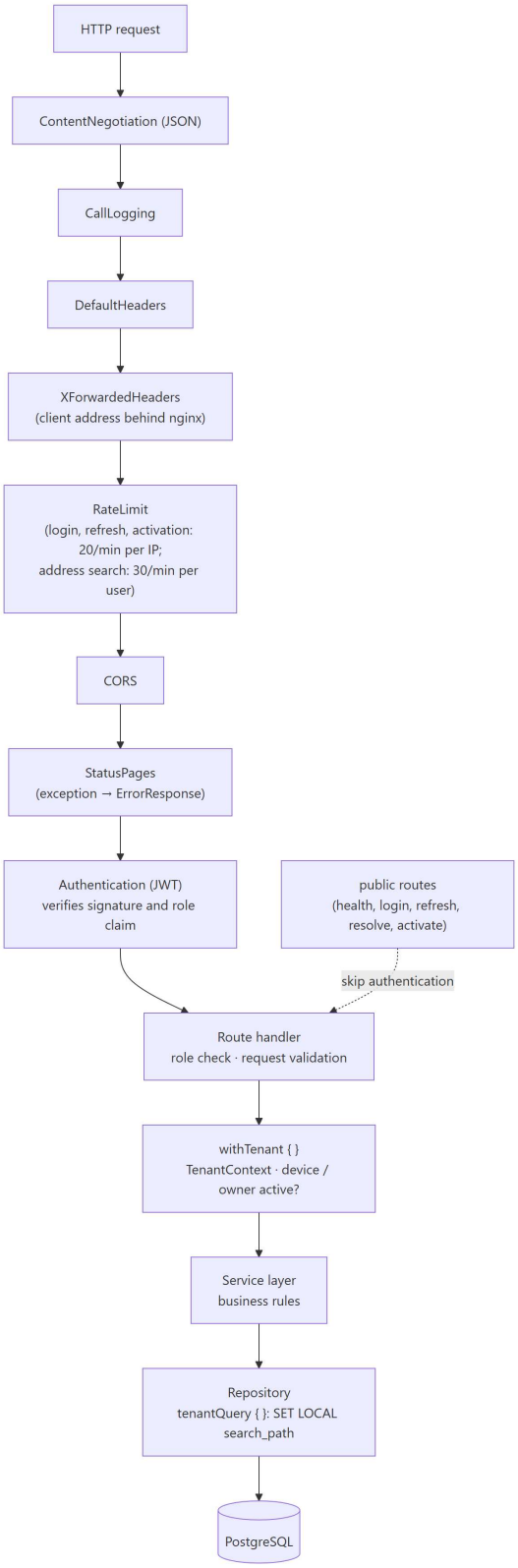
Όταν δεν είναι δυνατή η σάρωση του κωδικού QR, η ενεργοποίηση γίνεται εναλλακτικά μέσω του GET /devices/resolve: ο χρήστης πληκτρολογεί τον κωδικό και το DeviceService.resolveByCode() τον αναζητά διαδοχικά στο schema κάθε καταχωρημένου ιδιοκτήτη, επιστρέφοντας το αναγνωριστικό της συσκευής και το όνομα του σωστού schema, ώστε η εφαρμογή να προχωρήσει στην κανονική ενεργοποίηση. Η γραμμική αναζήτηση σε όλα τα schemas είναι αποδεκτή για μια σπάνια, χειροκίνητη ενέργεια, αλλά το κόστος της αυξάνεται με τον αριθμό των tenants.

Επειδή το token μιας συσκευής ισχύει για ένα έτος, δεν αρκεί να είναι έγκυρη η υπογραφή του. Η συνάρτηση withTenant, την οποία καλεί κάθε route που χρειάζεται πρόσβαση σε δεδομένα tenant, εφαρμόζει σε κάθε αίτημα με ρόλο DEVICE τους ελέγχους της §4.5.7: απάντηση 401 για άγνωστη ή αποσυνδεδεμένη συσκευή και 403 για απενεργοποιημένη συσκευή ή ιδιοκτήτη. Η συσκευή που ανακτάται κατά τον έλεγχο τοποθετείται σε ένα στοιχείο του coroutine context, ώστε οι route handlers που χρειάζονται το κατάλυμά της (π.χ. ο κατάλογος προϊόντων ή εστιατορίων) να μην την αναζητούν δεύτερη φορά. Το ίδιο στοιχείο επιτρέπει στο POST /orders να ελέγξει ότι το κατάλυμα του αιτήματος είναι αυτό της συσκευής, ώστε μια συσκευή να μην μπορεί να παραγγείλει για άλλο κατάλυμα.

Ο ρόλος DEVICE δεν θεωρείται ισοδύναμος με τον OWNER: τα routes ελέγχουν ρητά τον ρόλο του principal ή την ταυτότητα της συσκευής πριν εκτελέσουν την ενέργεια. Το GET /products συμπεριφέρεται διαφορετικά ανάλογα με τον καλούντα: όταν το αίτημα προέρχεται από συσκευή, ο ProductService επιστρέφει μόνο τα προϊόντα που είναι διαθέσιμα στο κατάλυμα της συσκευής (λαμβάνοντας υπόψη τον πίνακα product_property_overrides, §5.3.4), ενώ όταν προέρχεται από ιδιοκτήτη επιστρέφει ολόκληρο τον κατάλογο, μαζί με τα καταλύματα όπου κάθε προϊόν είναι απενεργοποιημένο. Ο έλεγχος ρόλου γίνεται μέσα στους routing handlers, χωρίς ενδιάμεσο middleware, επιλογή σκόπιμη για λόγους απλότητας, αφού οι ρόλοι είναι λίγοι.

6.3 Middleware Pipeline και Διαχείριση Σφαλμάτων

Η σειρά με την οποία εγκαθίστανται τα Ktor plugins στο Application.module() καθορίζει τη σειρά επεξεργασίας κάθε αιτήματος (Εικόνα 6.1): ContentNegotiation (σειριοποίηση και αποσειριοποίηση JSON με το kotlinx.serialization), CallLogging, DefaultHeaders, XForwardedHeaders, RateLimit, CORS (ανοιχτό σε οποιοδήποτε origin, με ρητά επιτρεπόμενες μεθόδους PUT/DELETE/PATCH/OPTIONS και κεφαλίδες Authorization/Content-Type), StatusPages για τη διαχείριση σφαλμάτων, και τέλος Authentication με το JWT plugin. Το RateLimit ορίζει δύο όρια με τη μέθοδο token bucket: τα δημόσια endpoints που δέχονται μυστικά επιτρέπουν είκοσι αιτήματα το λεπτό ανά διεύθυνση IP, ώστε να αποτρέπονται οι μαζικές δοκιμές κωδικών, και η αναζήτηση διευθύνσεων τριάντα αιτήματα το λεπτό ανά χρήστη. Επειδή το web panel επικοινωνεί με το backend μέσω του nginx (§7.1), όλα τα αιτήματα των browsers θα φαίνονταν να προέρχονται από τη διεύθυνση του proxy· το XForwardedHeaders ανακτά την πραγματική διεύθυνση του client από την κεφαλίδα X-Forwarded-For, στην οποία το nginx γράφει τη διεύθυνση της σύνδεσης, αντικαθιστώντας κάθε τιμή που έστειλε ο client, ώστε το όριο να ισχύει ανά χρήστη και όχι συνολικά. Πριν εγκατασταθούν τα plugins, κατά την εκκίνηση της εφαρμογής δημιουργούνται οι πίνακες του public schema και ενημερώνεται το schema κάθε υπάρχοντος tenant (§5.1).



Εικόνα 6.1: Middleware pipeline — σειρά επεξεργασίας αιτήματος από το Ktor engine έως το επίπεδο υπηρεσιών.

Το StatusPages plugin μετατρέπει κάθε εξαίρεση σε δομημένο ErrorResponse, με σταθερό κωδικό σφάλματος και το αντίστοιχο HTTP status (Πίνακας 6.2).

Πίνακας 6.2: Αντιστοίχιση εξαιρέσεων σε απαντήσεις HTTP

Εξαίρεση	**HTTP**	**Κωδικός**	**Χαρακτηριστική περίπτωση**
AuthenticationException	401	UNAUTHORIZED	Λάθος κωδικός, ληγμένο refresh token, ανενεργός λογαριασμός
ForbiddenException	403	FORBIDDEN	Λάθος ρόλος, παραγγελία άλλης συσκευής, λάθος κωδικός ενεργοποίησης, απενεργοποιημένη συσκευή, συσκευή απενεργοποιημένου ιδιοκτήτη
NotFoundException	404	NOT_FOUND	Ανύπαρκτη οντότητα
ConflictException	409	CONFLICT	Μη έγκυρη μετάβαση κατάστασης, ήδη πληρωμένη παραγγελία, διπλό email
IllegalArgumentException	400	BAD_REQUEST	Αποτυχία επικύρωσης (π.χ. δεν καλύπτεται η ελάχιστη παραγγελία)
Σφάλμα αποσειριοποίησης	400	BAD_REQUEST	Μη έγκυρο σώμα αιτήματος· σταθερό μήνυμα χωρίς εσωτερικές λεπτομέρειες
ExternalServiceException	502	EXTERNAL_SERVICE_ERROR	Το εταιρικό μητρώο, η υπηρεσία γεωκωδικοποίησης ή μια πλατφόρμα δεν

Εξαίρεση	**HTTP**	**Κωδικός**	**Χαρακτηριστική περίπτωση**
			απάντησε ή απέρριψε το κλειδί
Υπέρβαση του rate limit	429	TOO_MANY_REQUESTS	Πάρα πολλές απόπειρες σύνδεσης ή αναζητήσεις διευθύνσεων μέσα σε ένα λεπτό
ExposedSQLException (παραβίαση μοναδικότητας ή foreign key)	409	CONFLICT	Ταυτόχρονο διπλότυπο αίτημα ή διαγραφή εγγραφής που αναφέρεται αλλού
Κάθε άλλη εξαίρεση	500	INTERNAL_SERVER_ERROR	Μη αναμενόμενο σφάλμα

Στην τελευταία περίπτωση, το μήνυμα της εξαίρεσης δεν φτάνει ποτέ στον client· καταγράφεται πλήρως στο log του server, ενώ ο client λαμβάνει ένα σταθερό, γενικό μήνυμα, ώστε να μη διαρρέουν λεπτομέρειες SQL ή του driver της βάσης. Επειδή οι παραβιάσεις constraints της βάσης αντιστοιχίζονται σε 409, τα constraints της §5.5 λειτουργούν ως τελευταία δικλίδα, με την κατάλληλη απάντηση, ακόμη και όταν δύο αιτήματα παρακάμπτουν ταυτόχρονα έναν έλεγχο του επιπέδου υπηρεσιών.

Το TenantContext δεν εγκαθίσταται ως plugin στο pipeline. Εφαρμόζεται ρητά σε κάθε route μέσω δύο extension functions του ApplicationCall: το call.tenantSchema() ανακτά το claim schemaName από το JWT (ή, για τον Platform Admin, που δεν ανήκει σε κανέναν tenant, από μια παράμετρο του αιτήματος), και το call.withTenant { } εκτελεί το σώμα του handler σε coroutine scope εμπλουτισμένο με TenantContext(schema), ελέγχοντας παράλληλα αν η συσκευή ή ο ιδιοκτήτης έχουν απενεργοποιηθεί (§6.2). Τα δημόσια endpoints της συσκευής δημιουργούν το TenantContext από την τιμή schema του αιτήματος ή από το αποτέλεσμα της αναζήτησης, αφού δεν υπάρχει ακόμη JWT· η μορφή του ονόματος του schema ελέγχεται πάντα πριν χρησιμοποιηθεί (§4.3). Η ρητή κλήση ταιριάζει με τη φιλοσοφία του Ktor (§4.5.2): κάθε route δηλώνει ότι χρειάζεται tenant context, χωρίς κρυφή συμπεριφορά.

6.4 Service Layer και Συναρμολόγηση Εξαρτήσεων

Κάθε route παραμένει απλό: επικυρώνει την είσοδο, λαμβάνει τις παραμέτρους και τις παραδίδει αμέσως στον αντίστοιχο service. Η επιχειρησιακή λογική βρίσκεται στο επίπεδο υπηρεσιών: AuthService, OwnerService, PropertyService, DeviceService, ProductService, OrderService, PaymentService, RefundService, FoodDeliveryService, FoodOrderEventHandler, DashboardService, PlatformSettingsService και AuditLogService. Κάθε service λαμβάνει μέσω του constructor τα repository interfaces, τα port interfaces (§4.4) και τις άλλες υπηρεσίες που χρειάζεται· ορισμένα χρησιμοποιούν επιπλέον απευθείας βοηθητικά στοιχεία του backend, όπως το tenantQuery για τα όρια των συναλλαγών (§4.2). Η συνάρτηση configureRouting() είναι το μοναδικό σημείο του κώδικα όπου δημιουργούνται οι συγκεκριμένες υλοποιήσεις (ExposedOrderRepository, EmulatedPaymentAdapter, SmtpEmailAdapter κ.ο.κ.) και συνδέονται μεταξύ τους μέσω των constructors τους (χειροκίνητη συναρμολόγηση), χωρίς framework έγχυσης εξαρτήσεων (dependency injection). Η ίδια συνάρτηση εγγράφεται στο συμβάν τερματισμού της εφαρμογής, ώστε να κλείνουν οι HTTP clients των adapters και να ακυρώνονται οι εργασίες παρασκηνίου.

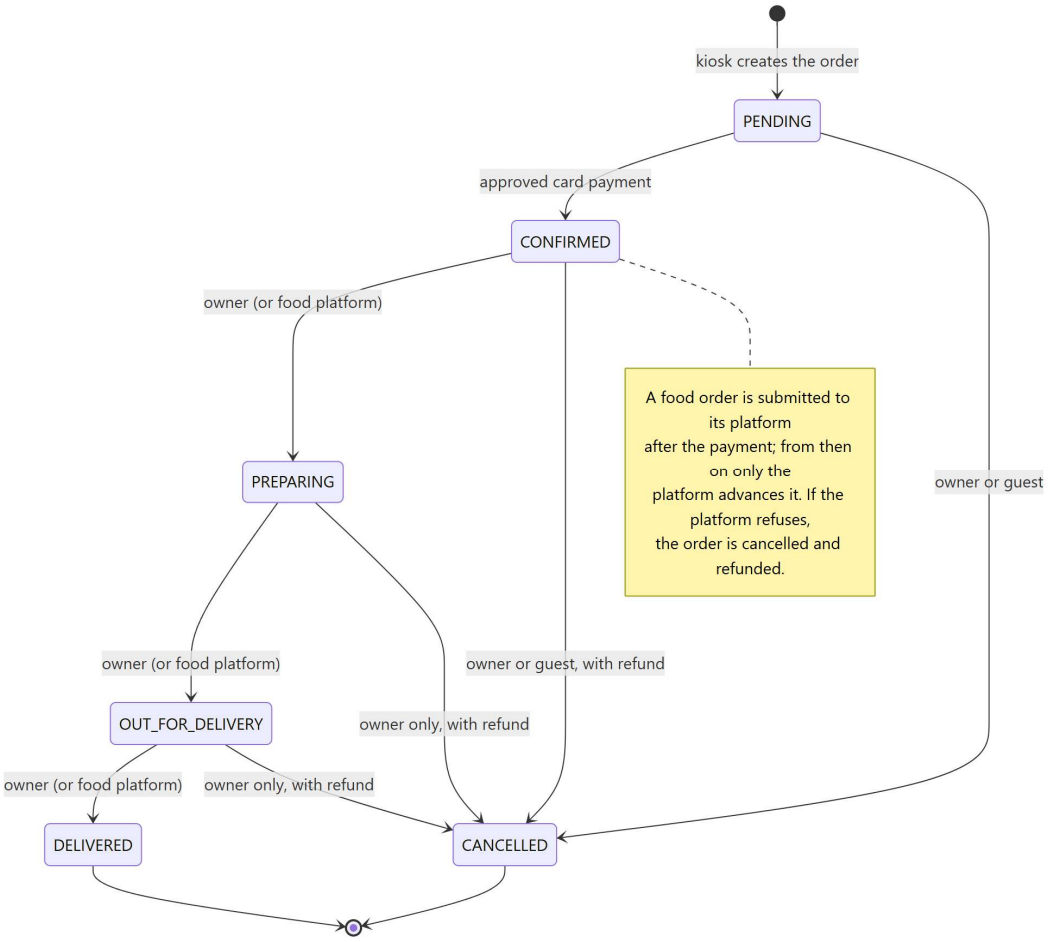
Ο AuthService υλοποιεί την ταυτοποίηση που αιτιολογήθηκε στην §4.5.7. Η login() δέχεται έναν δηλωμένο ρόλο (PLATFORM_ADMIN ή OWNER), βρίσκει τον αντίστοιχο λογαριασμό από το email, χωρίς διάκριση πεζών-κεφαλαίων, επαληθεύει τον κωδικό με BCrypt και εκδίδει ζεύγος access/refresh token. Ένας απενεργοποιημένος ιδιοκτήτης απορρίπτεται· ο έλεγχος γίνεται μετά την επαλήθευση του κωδικού, ώστε η κατάσταση του λογαριασμού να αποκαλύπτεται μόνο σε όποιον γνωρίζει τον κωδικό. Ο ρόλος DEVICE απορρίπτεται ρητά από τη διαδικασία login. Η refresh() υπολογίζει το SHA-256 hash του token που έστειλε ο client, βρίσκει την αντίστοιχη εγγραφή, ελέγχει τη λήξη της και διαγράφει το παλιό token πριν εκδώσει νέο ζεύγος, ελέγχοντας ξανά ότι ο ιδιοκτήτης παραμένει ενεργός. Η changePassword() επιτρέπει στον συνδεδεμένο χρήστη να αλλάξει τον κωδικό του: επαληθεύει τον τρέχοντα κωδικό, απαιτεί τουλάχιστον οκτώ χαρακτήρες για τον νέο, διαγράφει όλα τα refresh tokens του χρήστη ώστε οι υπόλοιπες συνεδρίες του να μην μπορούν να ανανεωθούν, και επιστρέφει νέο ζεύγος tokens για την τρέχουσα. Αν ο τρέχων κωδικός είναι λάθος, η απάντηση είναι 400 και όχι 401, επειδή το web panel θεωρεί κάθε 401 λήξη συνεδρίας και θα αποσύνδεε τον χρήστη (§7.5).

Ο OwnerService συντονίζει την ένταξη ενός νέου ιδιοκτήτη: επικυρώνει τα στοιχεία (ο ΑΦΜ πρέπει να έχει εννέα ψηφία), κανονικοποιεί το email, δημιουργεί την εγγραφή με το BCrypt hash ενός τυχαίου κωδικού δώδεκα χαρακτήρων, δημιουργεί το tenant schema και στο τέλος στέλνει το μήνυμα καλωσορίσματος. Αν η δημιουργία του schema αποτύχει αφού έχει ήδη αποθηκευτεί η εγγραφή, ο ιδιοκτήτης απενεργοποιείται αυτόματα, ώστε να μη μείνει ενεργός λογαριασμός χωρίς έγκυρο schema. Η χειροκίνητη απενεργοποίηση ενός ιδιοκτήτη διαγράφει επίσης όλα τα refresh tokens του, οπότε η πρόσβασή του τερματίζεται από το επόμενο αίτημα (§6.2). Τέλος, η resetPassword() εξυπηρετεί τον ιδιοκτήτη που ξέχασε τον κωδικό του: παράγει νέο προσωρινό κωδικό, διαγράφει τα refresh tokens του,

τον στέλνει στον ιδιοκτήτη μέσω ηλεκτρονικού ταχυδρομείου και τον επιστρέφει μία φορά στον διαχειριστή.

6.5 Ο Κύκλος Ζωής της Παραγγελίας

Ο κύκλος ζωής μιας παραγγελίας επιβάλλεται από τη μέθοδο isValidTransition() του OrderService, που υλοποιεί τη μηχανή καταστάσεων (state machine) της §3.2.6 (Εικόνα 6.2): PENDING → CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED. Από τις τελικές καταστάσεις (DELIVERED, CANCELLED) δεν επιτρέπεται καμία μετάβαση, η μετάβαση σε CANCELLED επιτρέπεται από οποιαδήποτε μη τελική κατάσταση, και κάθε άλλη μετάβαση πρέπει να ακολουθεί ακριβώς τη σειρά του κύκλου. Τρεις κανόνες περιορίζουν περαιτέρω τις μεταβάσεις. Μια παραγγελία σε κατάσταση PENDING είναι απλήρωτη και μπορεί μόνο να ακυρωθεί· τη μετάβαση σε CONFIRMED την κάνει αποκλειστικά ο PaymentService μετά από εγκεκριμένη πληρωμή (§6.6). Την πρόοδο μιας παραγγελίας φαγητού την ενημερώνει η πλατφόρμα διανομής (§6.7), οπότε η μόνη χειροκίνητη ενέργεια που επιτρέπεται είναι η ακύρωση. Τέλος, ο επισκέπτης μπορεί να ακυρώσει μια παραγγελία από τη συσκευή μόνο όσο είναι PENDING ή CONFIRMED, δηλαδή πριν αρχίσει η προετοιμασία της· από εκεί και πέρα, η ακύρωση και η επιστροφή χρημάτων είναι απόφαση του ιδιοκτήτη. Κάθε μη επιτρεπτή μετάβαση απορρίπτεται με ConflictException, που αντιστοιχίζεται σε HTTP 409.



Εικόνα 6.2: Μηχανή καταστάσεων της παραγγελίας — επιτρεπτές μεταβάσεις και ποιος κάνει την καθεμία (πληρωμή, ιδιοκτήτης, πλατφόρμα).

Η κατάσταση ενημερώνεται με compare-and-set (§5.6): αν η παραγγελία άλλαξε στο μεταξύ (από πληρωμή, από συμβάν της πλατφόρμας ή από δεύτερη ακύρωση), η ενημέρωση δεν εφαρμόζεται και ο καλών λαμβάνει απάντηση 409, ώστε να ανανεώσει τα δεδομένα του. Όταν μια παραγγελία ακυρώνεται, ζητείται, μέσω του RefundService, η επιστροφή του ποσού κάθε εγκεκριμένης πληρωμής της (§6.6). Για μια παραγγελία φαγητού η ακύρωση εκτελείται ολόκληρη σε context που δεν ακυρώνεται (NonCancellable) και ακολουθεί σταθερή σειρά: η κατάσταση γίνεται πρώτα CANCELLED με compare-and-set, έπειτα ζητείται από την πλατφόρμα διανομής η ακύρωση της παραγγελίας και, αν η πλατφόρμα την αρνηθεί, η κατάσταση επιστρέφει, πάλι με compare-and-set, στην προηγούμενη και το αίτημα απαντάται με 409 χωρίς επιστροφή χρημάτων· μόνο αφού η πλατφόρμα αποδεχθεί την ακύρωση ζητείται η επιστροφή των ποσών των εγκεκριμένων πληρωμών. Μια πληρωμένη παραγγελία φαγητού που δεν έχει ακόμη αναγνωριστικό της πλατφόρμας, επειδή η υποβολή της βρίσκεται σε εξέλιξη (§6.6), δεν μπορεί να ακυρωθεί: τόσο το route του ιδιοκτήτη όσο και το route της συσκευής απαντούν με 409 («The order is being placed with the delivery platform, try

again in a moment»). Πέντε λεπτά μετά την τελευταία ενημέρωση της παραγγελίας (PLACEMENT_GRACE_MS) η υποβολή θεωρείται αποτυχημένη, για παράδειγμα λόγω επανεκκίνησης κατά τη διάρκεια της κλήσης, και η ακύρωση επιτρέπεται ξανά.

Η δημιουργία μιας παραγγελίας (OrderService.create()) ξεκινά με επικύρωση: η παραγγελία πρέπει να έχει τουλάχιστον ένα είδος και κάθε ποσότητα να είναι από 1 έως 99, ενώ το email του επισκέπτη, αν δοθεί, πρέπει να είναι έγκυρη διεύθυνση έως 255 χαρακτήρων και οι σημειώσεις να μην ξεπερνούν τους 1.024 χαρακτήρες. Ο client στέλνει μόνο αναγνωριστικά και ποσότητες· οι τιμές υπολογίζονται πάντα στον server, από τον κατάλογο του ιδιοκτήτη για μια παραγγελία IN_ROOM_STORE (όπου ελέγχεται επίσης ότι κάθε προϊόν είναι διαθέσιμο τόσο συνολικά όσο και στο συγκεκριμένο κατάλυμα, §5.3.4) ή από το μενού της πλατφόρμας για μια παραγγελία FOOD_DELIVERY, στην οποία προστίθεται και το κόστος αποστολής του εστιατορίου. Η εγγραφή της παραγγελίας και των γραμμών της εκτελείται σε μία συναλλαγή (§5.6), ώστε μια αποτυχία σε οποιοδήποτε σημείο να αναιρεί όλες τις εγγραφές. Η παραγγελία αποθηκεύεται σε κατάσταση PENDING και επιστρέφεται στη συσκευή με το τελικό ποσό, ώστε η πληρωμή να αφορά ακριβώς το ποσό που υπολόγισε ο server.

Οι μέθοδοι ανάγνωσης του OrderService εξυπηρετούν τη λίστα παραγγελιών ενός καταλύματος για τον ιδιοκτήτη, το ιστορικό μιας συγκεκριμένης συσκευής και το φιλτράρισμα ανά κατάσταση. Η διαδρομή /device/orders/{id} περιορίζει την ανάγνωση και την ακύρωση μιας παραγγελίας στη συσκευή που τη δημιούργησε, μέσω της παραμέτρου requireDeviceId: αν το αναγνωριστικό συσκευής του JWT δεν ταυτίζεται με το deviceId της παραγγελίας, η κλήση απορρίπτεται με ForbiddenException.

6.6 Διασύνδεση με τον Φορέα Πληρωμών

Η χρέωση, η έγκριση και η επιστροφή χρημάτων είναι αρμοδιότητα του φορέα πληρωμών, με τον οποίο το backend επικοινωνεί μόνο μέσω του port PaymentGateway (§4.4). Ευθύνη του συστήματος είναι να ζητά κάθε ενέργεια μία μόνο φορά και να καταγράφει το αποτέλεσμα.

Η πληρωμή είναι ξεχωριστό βήμα από τη δημιουργία της παραγγελίας: αφού δημιουργηθεί η παραγγελία, το kiosk καλεί το POST /payments με το αναγνωριστικό της, τον τρόπο ανάγνωσης της κάρτας, το αποτέλεσμα της επαλήθευσης του κατόχου και, όπου υπάρχουν, τα τέσσερα τελευταία ψηφία της κάρτας. Η PaymentService.initiate() εκτελεί τον έλεγχο, την έγκριση και την καταγραφή μέσα σε μία συναλλαγή, που ξεκινά κλειδώνοντας τη γραμμή της παραγγελίας (SELECT … FOR UPDATE, §5.6). Έτσι, μια ταυτόχρονη δεύτερη απόπειρα πληρωμής, για παράδειγμα η επανάληψη ενός αιτήματος μετά από καθυστέρηση του δικτύου, βρίσκει την παραγγελία ήδη πληρωμένη και απορρίπτεται με 409 πριν φτάσει στον φορέα πληρωμών.

Μέσα στη συναλλαγή, η υπηρεσία επαληθεύει ότι η παραγγελία ανήκει στη συσκευή που πληρώνει, ότι δεν έχει ήδη πληρωθεί και ότι είναι σε κατάσταση PENDING, και ζητά έγκριση από το

PaymentGateway. Η απάντηση καταγράφεται ως νέα εγγραφή πληρωμής. Η απόρριψη θεωρείται κανονικό αποτέλεσμα της πληρωμής: η εγγραφή αποθηκεύεται ως FAILED με την αιτία της απόρριψης, η παραγγελία μένει PENDING και η απάντηση επιστρέφεται στο kiosk, που ενημερώνει τον επισκέπτη και περιμένει νέα κάρτα. Η έγκριση αποθηκεύεται ως COMPLETED μαζί με το αναγνωριστικό της συναλλαγής και θέτει την παραγγελία σε CONFIRMED.

Μετά την έγκριση, ο φορέας έχει ήδη χρεώσει την κάρτα, γι' αυτό η υπόλοιπη επεξεργασία εκτελείται σε coroutine context που δεν ακυρώνεται (NonCancellable), ώστε η διακοπή του αιτήματος από τον client να μην αφήσει τη διαδικασία ημιτελή. Για μια παραγγελία φαγητού, η υπηρεσία υποβάλλει τώρα την παραγγελία στην πλατφόρμα (§6.7). Αν η πλατφόρμα δεν την αποδεχθεί, η υπηρεσία ακυρώνει την παραγγελία με compare-and-set από CONFIRMED σε CANCELLED και ζητά την επιστροφή του ποσού μόνο αν η ενημέρωση αυτή επιτύχει, ώστε να μη γίνει δεύτερη επιστροφή για μια παραγγελία που ακυρώθηκε στο μεταξύ· η απάντηση προς το kiosk περιέχει, μαζί με την εγκεκριμένη πληρωμή, την κατάσταση της παραγγελίας και ένα μήνυμα για τον επισκέπτη. Αν η παραγγελία έχει email, η απόδειξη στέλνεται σε ξεχωριστό coroutine scope, ώστε ένας αργός ή μη διαθέσιμος διακομιστής SMTP να μην καθυστερεί την απάντηση της πληρωμής.

Ο RefundService χειρίζεται κεντρικά τα αιτήματα επιστροφής χρημάτων σε όλες τις περιπτώσεις ακύρωσης μιας πληρωμένης παραγγελίας: από τον ιδιοκτήτη, από τον επισκέπτη ή λόγω άρνησης της πλατφόρμας. Πρώτα δεσμεύει την πληρωμή με compare-and-set από COMPLETED σε REFUNDED, και μόνο ο καλών που κερδίζει τη δέσμευση ζητά την επιστροφή από το PaymentGateway με το αναγνωριστικό της αρχικής συναλλαγής· μια δεύτερη, ταυτόχρονη απόπειρα για την ίδια πληρωμή χάνει τη δέσμευση και δεν στέλνει αίτημα, οπότε δεν ζητείται δεύτερη επιστροφή για την ίδια πληρωμή. Αν αποτύχει ο φορέας πληρωμών, η πληρωμή επανέρχεται σε COMPLETED. Η ενέργεια καταγράφεται στο audit log με το ποσό και το αναγνωριστικό της συναλλαγής. Ο RefundService δεν διακόπτει την ακύρωση με εξαίρεση: μια αποτυχημένη επιστροφή καταγράφεται στο audit log ως ενέργεια που απαιτεί χειροκίνητο χειρισμό, ενώ η ακύρωση της παραγγελίας ολοκληρώνεται κανονικά.

Ο EmulatedPaymentAdapter απαντά στη θέση ενός πραγματικού φορέα: εγκρίνει ή απορρίπτει μια συναλλαγή ανάλογα με τον τρόπο ανάγνωσης της κάρτας και ολοκληρώνει με επιτυχία κάθε επιστροφή (§3.4.1). Επειδή ο adapter παράγει και εγκρίσεις και απορρίψεις, και οι δύο διαδρομές της ροής πληρωμής, καθώς και η επιστροφή χρημάτων, μπορούν να παρατηρηθούν στο πρωτότυπο.

6.7 Ενσωμάτωση Πλατφορμών Διανομής Φαγητού

Ο FoodDeliveryService συντονίζει όλες τις επαφές με τις πλατφόρμες διανομής, μέσω ενός adapter ανά πλατφόρμα που υλοποιεί το port FoodDeliveryProvider. Όταν ο ιδιοκτήτης συνδέει μια πλατφόρμα, ζητείται από τον adapter να επαληθεύσει το κλειδί πρόσβασης και, αν η επαλήθευση επιτύχει, το κλειδί

αποθηκεύεται κρυπτογραφημένο στον πίνακα food_platform_connections (§5.3.8). Στη συνέχεια ο ιδιοκτήτης ορίζει ποιες συνδεδεμένες πλατφόρμες προσφέρονται σε κάθε κατάλυμα· η υπηρεσία απορρίπτει πλατφόρμα που δεν έχει συνδεθεί. Η αποσύνδεση μιας πλατφόρμας απορρίπτεται όσο υπάρχουν παραγγελίες της σε εξέλιξη, επειδή το κλειδί της χρειάζεται για μια ενδεχόμενη ακύρωση.

Για τη συσκευή, η υπηρεσία επιστρέφει τα εστιατόρια όλων των πλατφορμών που είναι ενεργοποιημένες στο κατάλυμά της και το μενού ενός εστιατορίου, χρησιμοποιώντας κάθε φορά το αποκρυπτογραφημένο κλειδί του ιδιοκτήτη. Κατά τη δημιουργία μιας παραγγελίας φαγητού, η priceOrder() ελέγχει ότι η πλατφόρμα είναι διαθέσιμη στο κατάλυμα, τιμολογεί κάθε πιάτο από το τρέχον μενού και επαληθεύει την ελάχιστη παραγγελία του εστιατορίου. Μετά την εγκεκριμένη πληρωμή, η placeOrder() υποβάλλει την παραγγελία στην πλατφόρμα μαζί με τη διεύθυνση παράδοσης, που συντίθεται από τα στοιχεία του καταλύματος (οδός, όροφος, όνομα στο κουδούνι, τηλέφωνο), και καταγράφει το αναγνωριστικό που επιστρέφει η πλατφόρμα, εφόσον η τοπική παραγγελία δεν έχει ακυρωθεί στο μεταξύ. Αν η καταγραφή αποτύχει ή αν η παραγγελία ακυρώθηκε όσο εκτελούνταν η κλήση προς την πλατφόρμα, η υπηρεσία ακυρώνει την παραγγελία στην πλατφόρμα (best effort) πριν αναφέρει το σφάλμα, ώστε να μη μείνει στην πλατφόρμα παραγγελία που το σύστημα δεν μπορεί να παρακολουθήσει ή να ακυρώσει.

Η πρόοδος μιας αποδεκτής παραγγελίας φτάνει στο σύστημα μέσω του port FoodOrderEventListener, που υλοποιεί ο FoodOrderEventHandler. Επειδή τα συμβάντα της πλατφόρμας δεν συνδέονται με κάποιο αίτημα HTTP, κάθε συμβάν φέρει το όνομα του tenant schema, από το οποίο ο handler ανακτά το TenantContext. Ο handler εφαρμόζει μια νέα κατάσταση μόνο αν έπεται της τρέχουσας στον κύκλο ζωής και μόνο με compare-and-set, οπότε ένα καθυστερημένο ή διπλό συμβάν, ή συμβάν για παραγγελία που ακυρώθηκε στο μεταξύ, αγνοείται. Μαζί με την κατάσταση αποθηκεύεται και ο εκτιμώμενος χρόνος παράδοσης που αναφέρει η πλατφόρμα.

Οι τρεις emulated adapters των πλατφορμών μοιράζονται τη λογική τους στην κλάση EmulatedFoodPlatformAdapter και διαφέρουν μόνο στο όνομα της πλατφόρμας και στα αρχεία δεδομένων τους, που περιέχουν τα εστιατόρια, τα μενού και τον κατάλογο προϊόντων της πλατφόρμας. Για κάθε αποδεκτή παραγγελία, ο adapter ξεκινά ένα coroutine που, ανά τακτά διαστήματα (μεταβλητή περιβάλλοντος FOOD_PLATFORM_STEP_SECONDS, προεπιλογή 45 δευτερόλεπτα), αναφέρει διαδοχικά τις καταστάσεις PREPARING, OUT_FOR_DELIVERY και DELIVERED μέσω του FoodOrderEventListener, όπως θα έκαναν οι ειδοποιήσεις (webhooks) μιας πραγματικής πλατφόρμας. Επειδή οι χρονομετρητές αυτοί υπάρχουν μόνο στη μνήμη, η κλάση EmulatedFoodOrderRecovery εντοπίζει κατά την εκκίνηση της εφαρμογής, σε κάθε tenant, τις παραγγελίες φαγητού που ήταν σε εξέλιξη και προγραμματίζει ξανά τα υπόλοιπα βήματά τους· ένα σφάλμα σε έναν tenant δεν εμποδίζει την ανάκτηση των υπολοίπων. Μια πραγματική πλατφόρμα διατηρεί η ίδια την κατάσταση και δεν θα χρειαζόταν τέτοιο μηχανισμό.

Ο κατάλογος προϊόντων κάθε πλατφόρμας εξυπηρετεί και μια δεύτερη λειτουργία: ο ιδιοκτήτης μπορεί να εισαγάγει επιλεγμένα είδη του ως προϊόντα του δικού του καταστήματος. Ο ProductService λαμβάνει τις τιμές πάντα από την πλατφόρμα και όχι από τον client, παραλείπει τα είδη που υπάρχουν ήδη στον κατάλογο με το ίδιο όνομα και επιστρέφει ποια εισήχθησαν και ποια παραλείφθηκαν.

6.8 Πραγματικές Εξωτερικές Υπηρεσίες: Ηλεκτρονικό Ταχυδρομείο, Εταιρικό Μητρώο και Γεωκωδικοποίηση

Οι τρεις adapters που συνδέονται με πραγματικές υπηρεσίες ακολουθούν κοινές αρχές. Κάθε κλήση δικτύου έχει χρονικό όριο, ώστε μια μη διαθέσιμη υπηρεσία να μη δεσμεύει επ' αόριστον ένα αίτημα, και κάθε αποτυχία γίνεται είτε ExternalServiceException (HTTP 502 με κατανοητό μήνυμα) είτε, όταν η υπηρεσία δεν είναι απαραίτητη για την ενέργεια του χρήστη, απλή καταγραφή στο log χωρίς διακοπή της ροής. Οι ρυθμίσεις τους (διακομιστής SMTP, κλειδί του μητρώου) διαβάζονται από τον PlatformSettingsService σε κάθε χρήση, οπότε κάθε αλλαγή από τη σελίδα ρυθμίσεων του web panel ισχύει αμέσως.

Ο SmtpEmailAdapter υλοποιεί το port NotificationService με τη βιβλιοθήκη Jakarta Mail, συντάσσοντας και στέλνοντας μηνύματα μέσω SMTP [8] με την ασφάλεια σύνδεσης που έχει επιλέξει ο διαχειριστής (STARTTLS, SSL ή καμία). Υποστηρίζει τρεις ειδοποιήσεις: την απόδειξη μιας παραγγελίας, το μήνυμα καλωσορίσματος ενός νέου ιδιοκτήτη και το μήνυμα επαναφοράς κωδικού. Καμία από αυτές δεν προκαλεί εξαίρεση: αν δεν έχει οριστεί διακομιστής ή η αποστολή αποτύχει, η αποτυχία καταγράφεται στο log και η ενέργεια του χρήστη (πληρωμή ή δημιουργία ιδιοκτήτη) ολοκληρώνεται κανονικά. Εξαίρεση αποτελεί η αποστολή δοκιμαστικού μηνύματος από τη σελίδα ρυθμίσεων, όπου το σφάλμα επιστρέφεται στον διαχειριστή ώστε να διορθώσει τις ρυθμίσεις. Στα logs, οι διευθύνσεις email εμφανίζονται μερικώς κρυμμένες, ώστε να μπορούν να συσχετιστούν χωρίς να εκτίθενται προσωπικά δεδομένα του επισκέπτη.

Ο adapter του port CompanyRegistry αναζητά μια επιχείρηση με βάση τον ΑΦΜ στη διεπαφή ανοικτών δεδομένων του εταιρικού μητρώου [9]. Επειδή ένας ΑΦΜ μπορεί να αντιστοιχεί τόσο στην έδρα όσο και σε υποκαταστήματα, ο adapter προτιμά την εγγραφή της έδρας και, από τις δραστηριότητες της επιχείρησης, επιλέγει την κύρια. Για κλειδί που λείπει ή απορρίπτεται, για υπέρβαση του ορίου αιτημάτων και για μη διαθεσιμότητα της υπηρεσίας, ο διαχειριστής βλέπει διαφορετικό, κατανοητό μήνυμα.

Ο adapter του port Geocoder προσφέρει δύο λειτουργίες: την αναζήτηση διεύθυνσης από ελεύθερο κείμενο και τη μετατροπή συντεταγμένων σε διεύθυνση (reverse geocoding), που χρησιμοποιείται όταν ο ιδιοκτήτης επιλέγει την τρέχουσα θέση του. Η πολιτική χρήσης της δημόσιας υπηρεσίας γεωκωδικοποίησης [10], [24] απαιτεί αναγνωριστικό εφαρμογής σε κάθε αίτημα, το πολύ ένα αίτημα

ανά δευτερόλεπτο και αποφυγή της αναζήτησης την ώρα που πληκτρολογεί ο χρήστης. Ο adapter τηρεί το όριο σε επίπεδο διεργασίας με ένα κοινό Mutex, που παραμένει δεσμευμένο για όλη τη διάρκεια κάθε αιτήματος, ώστε οι κλήσεις να εκτελούνται αυστηρά η μία μετά την άλλη, με απόσταση τουλάχιστον ενός δευτερολέπτου. Η αναμονή για το Mutex περιορίζεται σε δέκα δευτερόλεπτα: όταν φτάνουν πολλές αναζητήσεις ταυτόχρονα, όσες θα περίμεναν περισσότερο απορρίπτονται αμέσως με κατανοητό μήνυμα αντί να συσσωρεύονται. Σε συνδυασμό με το όριο ανά χρήστη (§6.3), ένας μόνο χρήστης δεν μπορεί να αποκλείσει την υπηρεσία για τους υπόλοιπους, ενώ το web panel κάνει αναζήτηση μόνο όταν τη ζητήσει ρητά ο χρήστης. Τα αποτελέσματα μετατρέπονται στα επιμέρους πεδία της διεύθυνσης (οδός, αριθμός, ταχυδρομικός κώδικας, περιοχή, πόλη) και στις συντεταγμένες, με τα οποία συμπληρώνεται η φόρμα του καταλύματος.

6.9 Καταγραφή Συμβάντων και Αρχείο Ενεργειών

Το backend έχει δύο διακριτά επίπεδα καταγραφής, με διαφορετικό σκοπό και διαφορετικό κοινό. Το πρώτο είναι η τεχνική καταγραφή (logging) μέσω της βιβλιοθήκης Logback πάνω από τη διεπαφή SLF4J, που απευθύνεται σε όποιον λειτουργεί τον server. Το plugin CallLogging του Ktor καταγράφει κάθε αίτημα, ενώ κάθε service και adapter χρησιμοποιεί δικό του logger για τα σημαντικά συμβάντα: την έγκριση ή απόρριψη μιας πληρωμής, την αποτυχία μιας αποστολής email, την αποτυχία υποβολής μιας παραγγελίας σε πλατφόρμα, ή ένα συμβάν πλατφόρμας που αγνοήθηκε επειδή η παραγγελία είχε ήδη αλλάξει. Τα μη αναμενόμενα σφάλματα καταγράφονται με πλήρες stack trace, μαζί με τη μέθοδο και τη διαδρομή του αιτήματος (§6.3). Τα προσωπικά δεδομένα περιορίζονται σκόπιμα: οι διευθύνσεις email των επισκεπτών εμφανίζονται μερικώς κρυμμένες, το email δεν καταγράφεται κατά τη σύνδεση, και τα ευαίσθητα δεδομένα της κάρτας δεν φτάνουν στον server.

Το δεύτερο επίπεδο είναι το αρχείο ενεργειών (audit log, §5.2.4), το οποίο απευθύνεται στον Platform Admin και καταγράφει τον εκτελέσαντα, την ενέργεια, τον tenant και τον χρόνο. Καταγράφονται 27 τύποι ενεργειών, που καλύπτουν τον κύκλο ζωής των ιδιοκτητών (δημιουργία, ενημέρωση, απενεργοποίηση, επανενεργοποίηση, επαναφορά κωδικού), την αλλαγή κωδικού από τον ίδιο τον χρήστη, τα καταλύματα (δημιουργία, ενημέρωση, απενεργοποίηση, ενεργοποίηση, διαγραφή), τις συσκευές (δημιουργία, απενεργοποίηση, επανενεργοποίηση, αποσύνδεση, διαγραφή, μαζική απενεργοποίηση μαζί με το κατάλυμα), τη διαγραφή και την εισαγωγή προϊόντων, τη διαθεσιμότητα ανά κατάλυμα, τη σύνδεση και αποσύνδεση πλατφορμών διανομής, την επιλογή πλατφορμών ανά κατάλυμα, τις αλλαγές στις ρυθμίσεις SMTP και εταιρικού μητρώου, και τις επιστροφές χρημάτων. Για τις ρυθμίσεις, το audit log καταγράφει ότι άλλαξε κάποιο μυστικό, χωρίς να καταγράφει την τιμή του. Για τις επιστροφές, καταγράφει το ποσό, το αναγνωριστικό της συναλλαγής και το αποτέλεσμα, ώστε μια αποτυχημένη επιστροφή να εντοπίζεται και να ολοκληρώνεται χειροκίνητα. Ο Platform Admin διαβάζει το αρχείο μέσω του GET /audit-logs, σε σελίδες των πενήντα εγγραφών.

Το τι καταγράφεται είναι σκόπιμη επιλογή: το audit log καλύπτει τις διαχειριστικές ενέργειες και τις επιστροφές χρημάτων, όχι την καθημερινή ροή των παραγγελιών. Οι αλλαγές κατάστασης των παραγγελιών φαίνονται ήδη στον ίδιο τον πίνακα orders (κατάσταση και χρόνος τελευταίας ενημέρωσης), ενώ μια παραγγελία φαγητού αλλάζει κατάσταση αυτόματα πολλές φορές, οπότε η καταγραφή κάθε μετάβασης θα γέμιζε το αρχείο με εγγραφές χαμηλής αξίας. Το τίμημα της επιλογής είναι ότι το σύστημα δεν τηρεί πλήρες ιστορικό για το ποιος άλλαξε την κατάσταση μια παραγγελία προϊόντων σε κάθε στάδιο.

6.10 Θωράκιση Ασφαλείας

Πέρα από τους μηχανισμούς ταυτοποίησης της §4.5.7, το backend εφαρμόζει μια σειρά μέτρων στα σημεία όπου δεδομένα από τον client φτάνουν στη βάση ή στη λογική των πληρωμών. Πολλά από αυτά προέκυψαν από συστηματικό έλεγχο ποιότητας και ασφάλειας του κώδικα κατά την ανάπτυξη, ο οποίος εντόπισε συγκεκριμένα ευρήματα που διορθώθηκαν πριν από την ολοκλήρωση.

Το σοβαρότερο εύρημα αφορούσε την επιλογή του tenant schema. Τα δημόσια endpoints ενεργοποίησης της συσκευής δέχονται το όνομα του schema ως παράμετρο του αιτήματος, και μια αρχική υλοποίηση το ενσωμάτωνε χωρίς έλεγχο στην εντολή SET search_path, επιτρέποντας σε έναν ανώνυμο χρήστη να εισαγάγει αυθαίρετο SQL. Η διόρθωση έγινε σε ένα μόνο σημείο, από το οποίο περνά κάθε πρόσβαση σε tenant schema: πριν από οποιαδήποτε χρήση, το όνομα ελέγχεται με βάση τη μοναδική μορφή που παράγει το σύστημα (tenant_ και δώδεκα δεκαεξαδικοί χαρακτήρες), και κάθε άλλη τιμή απορρίπτεται με 400 (§4.3). Η επαλήθευση της §9.3 περιλαμβάνει ακριβώς μια τέτοια απόπειρα.

Τα υπόλοιπα μέτρα είναι τα εξής. Οι τιμές που πρέπει να είναι απρόβλεπτες (προσωρινοί κωδικοί, κωδικοί ενεργοποίησης) παράγονται με κρυπτογραφικά ασφαλή γεννήτρια (SecureRandom). Το μυστικό υπογραφής των JWT και το μυστικό κρυπτογράφησης των ρυθμίσεων είναι υποχρεωτικές μεταβλητές περιβάλλοντος, χωρίς προεπιλεγμένη τιμή στον κώδικα, ώστε αν παραλειφθούν να μην ξεκινά η εφαρμογή, αντί να χρησιμοποιηθεί σιωπηλά μια γνωστή, δημόσια τιμή. Δεν υπάρχει διαδρομή που να διαβάζει τα claims ενός token χωρίς έλεγχο υπογραφής. Τα routes που αλλάζουν δεδομένα ελέγχουν ρητά τον ρόλο του καλούντος, και τα endpoints της συσκευής ελέγχουν ότι η παραγγελία ή το κατάλυμα ανήκει στη συσκευή που κάνει το αίτημα. Οι τιμές υπολογίζονται πάντα στον server από τον κατάλογο ή το μενού, ενώ ο client στέλνει μόνο αναγνωριστικά και ποσότητες από 1 έως 99· έτσι ένα τροποποιημένο kiosk δεν μπορεί να ορίσει δική του τιμή. Τα σφάλματα δεν αποκαλύπτουν εσωτερικές λεπτομέρειες στον client (§6.3), οι παράμετροι που ενσωματώνονται στις διευθύνσεις (URL) εξωτερικών υπηρεσιών κωδικοποιούνται, και οι διευθύνσεις των υπηρεσιών αυτών είναι σταθερές στον κώδικα, ώστε κανείς να μην μπορεί να κατευθύνει τον server σε αυθαίρετο προορισμό.

Ένας τελικός έλεγχος ασφάλειας του κώδικα, μετά την προσθήκη των πλατφορμών διανομής, των επιστροφών χρημάτων και των πραγματικών εξωτερικών υπηρεσιών, δεν εντόπισε κρίσιμα ευρήματα στα σημεία που εξετάστηκαν: η απομόνωση ανάμεσα στους tenants, οι έλεγχοι ρόλων, ο υπολογισμός των τιμών, η κρυπτογράφηση των μυστικών και οι κλήσεις προς το εταιρικό μητρώο και την υπηρεσία γεωκωδικοποίησης δεν παρουσίασαν προβλήματα. Τα ευρήματα χαμηλότερης σοβαρότητας (η δυνατότητα ακύρωσης από τη συσκευή μιας παραγγελίας που ήδη ετοιμαζόταν, η παράλειψη του ελέγχου διαθεσιμότητας ανά κατάλυμα κατά τη δημιουργία παραγγελίας, και η απουσία rate limit στη σύνδεση και στην αναζήτηση διευθύνσεων) διορθώθηκαν και επαληθεύτηκαν (§6.5, §6.3, §9.3).

6.11 Παράδειγμα Πλήρους Κύκλου Ενός Αιτήματος

Για να φανεί συγκεκριμένα πώς συνεργάζονται τα επίπεδα που περιγράφηκαν, η ενότητα αυτή παρακολουθεί ένα αίτημα POST /api/orders από τη στιγμή που φτάνει στον server μέχρι την απάντηση προς το kiosk. Το αίτημα περνά πρώτα από το Authentication plugin, που επαληθεύει την υπογραφή του JWT και ανακτά το JWTPrincipal· αν το token είναι άκυρο ή έχει λήξει, η αλυσίδα διακόπτεται αμέσως με απάντηση 401, χωρίς να φτάσει ποτέ στο route. Στη συνέχεια, ο handler ελέγχει ότι ο ρόλος του principal είναι DEVICE, διαβάζει την παράμετρο propertyId από το query string και το αναγνωριστικό της συσκευής από το claim subject του JWT, μετατρέπει το σώμα του αιτήματος σε CreateOrderRequest και καλεί το call.withTenant { }. Η withTenant εμπλουτίζει το coroutine scope με το TenantContext που αντιστοιχεί στο claim schemaName του token, επαληθεύει ότι η συσκευή είναι ακόμη ενεργή και επιτρέπει στον handler να ελέγξει ότι το propertyId είναι το κατάλυμα της συσκευής.

Μέσα σε αυτό το scope, ο OrderService.create() εκτελεί τα βήματα της §6.5: επικύρωση των ειδών, τιμολόγηση από τον κατάλογο του ιδιοκτήτη μέσω του ExposedProductRepository (που εκτελεί το ερώτημά του στο σωστό tenant schema χάρη στο ενεργό TenantContext, χωρίς η κλάση repository να γνωρίζει τίποτα για το multi-tenancy) ή από το μενού της πλατφόρμας μέσω του FoodDeliveryService, υπολογισμός του συνολικού ποσού και εισαγωγή της παραγγελίας και των γραμμών της σε μία συναλλαγή. Αν οποιοδήποτε βήμα αποτύχει, για παράδειγμα αν ένα προϊόν δεν είναι πλέον διαθέσιμο ή δεν καλύπτεται η ελάχιστη παραγγελία, προκαλείται η αντίστοιχη εξαίρεση, δεν μένει καμία μερική εγγραφή στη βάση, και η εξαίρεση φτάνει μέχρι το StatusPages, που τη μετατρέπει σε δομημένο ErrorResponse. Αν όλα τα βήματα ολοκληρωθούν κανονικά, ο handler απαντά με 201 Created και την παραγγελία σε κατάσταση PENDING, έτοιμη για το επόμενο αίτημα της συσκευής, το POST /payments. Όλη αυτή η αλυσίδα (ταυτοποίηση, έλεγχος δικαιωμάτων βάσει ρόλου, έλεγχος απενεργοποίησης, επιλογή tenant schema, εκτέλεση της επιχειρησιακής λογικής και μετατροπή των σφαλμάτων) περνά από τέσσερα διαφορετικά επίπεδα κώδικα, καθένα με τη δική του αρμοδιότητα, σύμφωνα με τη δομή του Κεφαλαίου 4.

Κεφάλαιο 7. Owner Web Panel

Το module :owner-web υλοποιεί το panel διαχείρισης που χρησιμοποιούν οι δύο ανθρώπινοι ρόλοι της πλατφόρμας, ο Platform Admin και ο Owner (§3.1.1, §3.1.2). Η επιλογή του Compose Multiplatform με στόχο μεταγλώττισης wasmJs, αντί για κάποιο καθιερωμένο JavaScript framework, αιτιολογήθηκε στην §4.5.5, μαζί με τη σύγκριση με εναλλακτικές όπως το React ή το Vue. Το κεφάλαιο περιγράφει πώς οργανώνεται στην πράξη μια single-page εφαρμογή γραμμένη εξ ολοκλήρου σε Kotlin: από το build μέχρι την παράδοση του τελικού bundle στον browser, την πλοήγηση ανάμεσα στις οθόνες, τη διαχείριση της συνεδρίας, τον καταμερισμό αρμοδιοτήτων μεταξύ των δύο ρόλων και τις οθόνες μέσω των οποίων ο χρήστης χρησιμοποιεί τις εξωτερικές υπηρεσίες του συστήματος.

7.1 Build Pipeline και Διανομή του Bundle

Το module μεταγλωττίζεται μόνο για τον στόχο wasmJs, με τη δήλωση kotlin { wasmJs { binaries.executable() } } στο αρχείο Gradle. Εξαρτάται από τα :domain και :data, τα ίδια modules που χρησιμοποιεί και το backend (§4.1), οπότε τα DTOs που στέλνει και λαμβάνει ο client (OwnerResponse, PropertyResponse, OrderResponse, CompanyLookupResponse κ.ο.κ.) είναι ακριβώς οι ίδιες κλάσεις Kotlin που σειριοποιεί το backend, χωρίς χειροκίνητη αντιστοίχιση τύπων ή εργαλεία παραγωγής κώδικα (§4.5.4).

Η μεταγλώττιση γίνεται με το Gradle task wasmJsBrowserDistribution, που παράγει το βελτιστοποιημένο (production) σύνολο στατικών αρχείων (HTML, JavaScript loader και το δυαδικό αρχείο .wasm). Στο Docker Compose deployment (§9.2), το βήμα αυτό εκτελείται στο πρώτο στάδιο ενός image πολλαπλών σταδίων (multi-stage): ένα container με JDK 21 μεταγλωττίζει τόσο το backend (:backend:installDist) όσο και το bundle του owner-web, και το αποτέλεσμα αντιγράφεται σε ένα ξεχωριστό, ελαφρύ container nginx:alpine που λειτουργεί μόνο ως web server. Το αρχείο nginx.conf ορίζει δύο κανόνες δρομολόγησης: κάθε αίτημα προς τη ρίζα (/) εξυπηρετείται από τα στατικά αρχεία του bundle, ενώ κάθε αίτημα προς το path /api προωθείται (proxy_pass) στο container του backend. Έτσι το nginx λειτουργεί ταυτόχρονα ως στατικός web server και ως reverse proxy, και ο browser επικοινωνεί με ένα μόνο origin, τόσο για τα στατικά αρχεία όσο και για το REST API, χωρίς ζητήματα CORS.

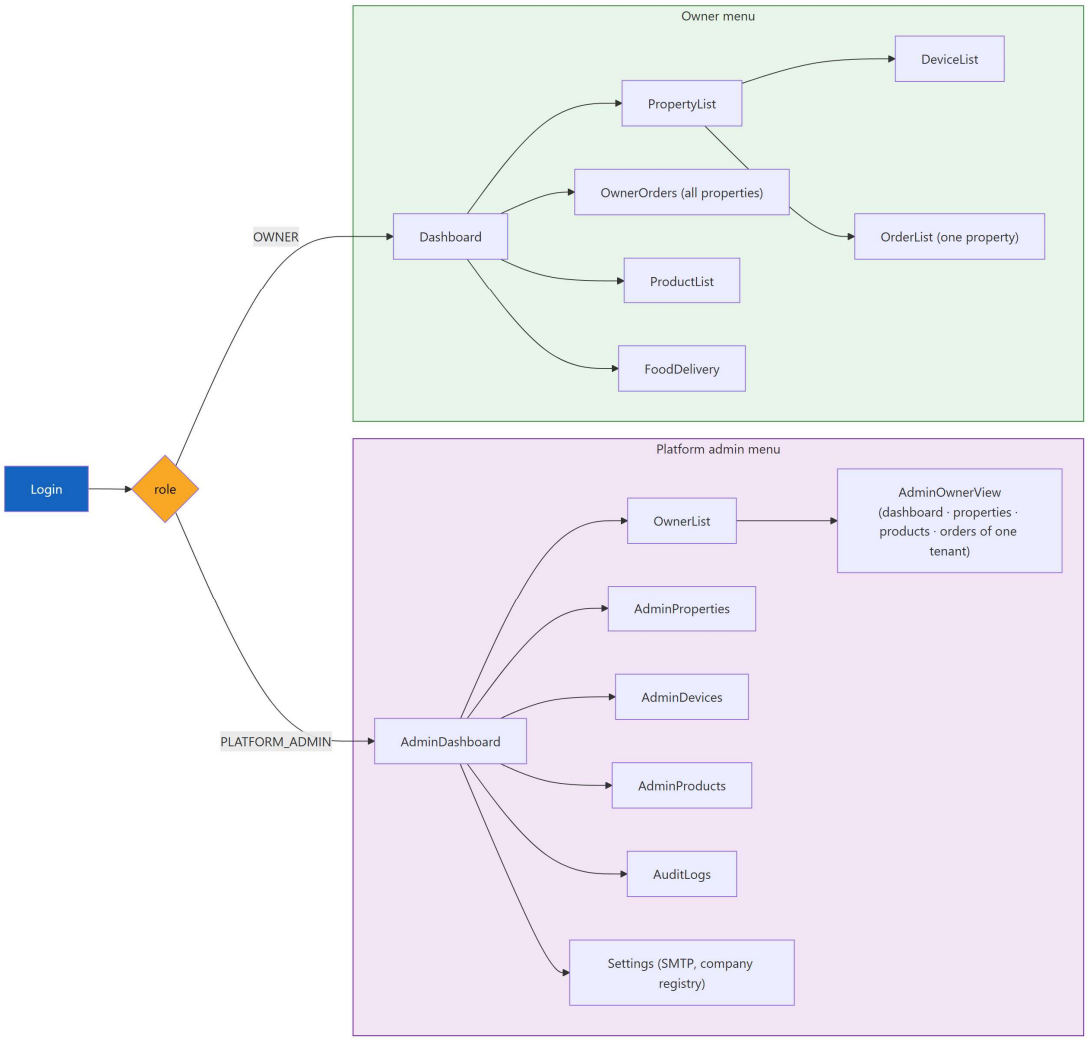
Δεν υπάρχει server-side rendering ούτε πολλαπλές σελίδες HTML: ολόκληρη η εφαρμογή σχεδιάζεται μέσα σε ένα μόνο στοιχείο <canvas> από τον renderer του Compose, και το σημείο εισόδου Main.kt απλώς αρχικοποιεί το runtime και καλεί το ριζικό composable App(). Ο πηγαίος κώδικας είναι οργανωμένος ανά αρμοδιότητα: ο κατάλογος screen/ περιέχει τις οθόνες της εφαρμογής, το ui/ τα

επαναχρησιμοποιήσιμα οπτικά στοιχεία, το auth/ τη σύνδεση και τη διαχείριση συνεδρίας, το api/ την επικοινωνία με το backend και το util/ βοηθητικές συναρτήσεις μορφοποίησης.

7.2 Πλοήγηση: Το Μοντέλο Screen

Η πλοήγηση δεν στηρίζεται σε εξωτερική βιβλιοθήκη routing ούτε σε διευθύνσεις URL με ιστορικό του browser, αλλά σε ένα απλό, κλειστό μοντέλο καταστάσεων: η sealed class Screen απαριθμεί όλες τις δυνατές οθόνες της εφαρμογής (Εικόνα 7.1). Οι περισσότερες οθόνες δεν έχουν παραμέτρους και δηλώνονται ως object (π.χ. Login, Dashboard, PropertyList, FoodDelivery, AdminDashboard, Settings), ενώ όσες χρειάζονται πλαίσιο πλοήγησης δηλώνονται ως data class: οι DeviceList(propertyId, propertyName) και OrderList(propertyId, propertyName) μεταφέρουν το κατάλυμα για το οποίο εμφανίζονται συσκευές ή παραγγελίες, ενώ η AdminOwnerView(ownerId, schemaName, displayName) μεταφέρει την ταυτότητα του tenant του οποίου τα στοιχεία επέλεξε να δει ο Platform Admin.

Το ριζικό composable App() διατηρεί την τρέχουσα οθόνη σε ένα mutableStateOf<Screen>, και ένα ενιαίο μπλοκ when αποφασίζει, με βάση την τιμή αυτή, ποιο composable θα εμφανιστεί. Η προσέγγιση ταιριάζει με το declarative μοντέλο του Compose: κάθε αλλαγή του state προκαλεί αυτόματα recomposition του δέντρου της διεπαφής, χωρίς χειροκίνητη εναλλαγή όψεων. Η συνάρτηση navigate(screen: Screen), που δίνεται ως παράμετρος σε κάθε οθόνη, απλώς αναθέτει νέα τιμή σε αυτό το state. Δεν υπάρχει back stack, ιστορικό διαδρομών ή deep linking, γιατί η εφαρμογή αποτελείται από ένα κλειστό σύνολο διαχειριστικών οθονών και όχι από δημόσιο ιστότοπο· η επιστροφή στο προηγούμενο επίπεδο γίνεται από τη γραμμή πλοήγησης (breadcrumb) στο πάνω μέρος των οθονών δεύτερου επιπέδου.



Εικόνα 7.1: Οθόνες του owner-web και ροή πλοήγησης ανά ρόλο.

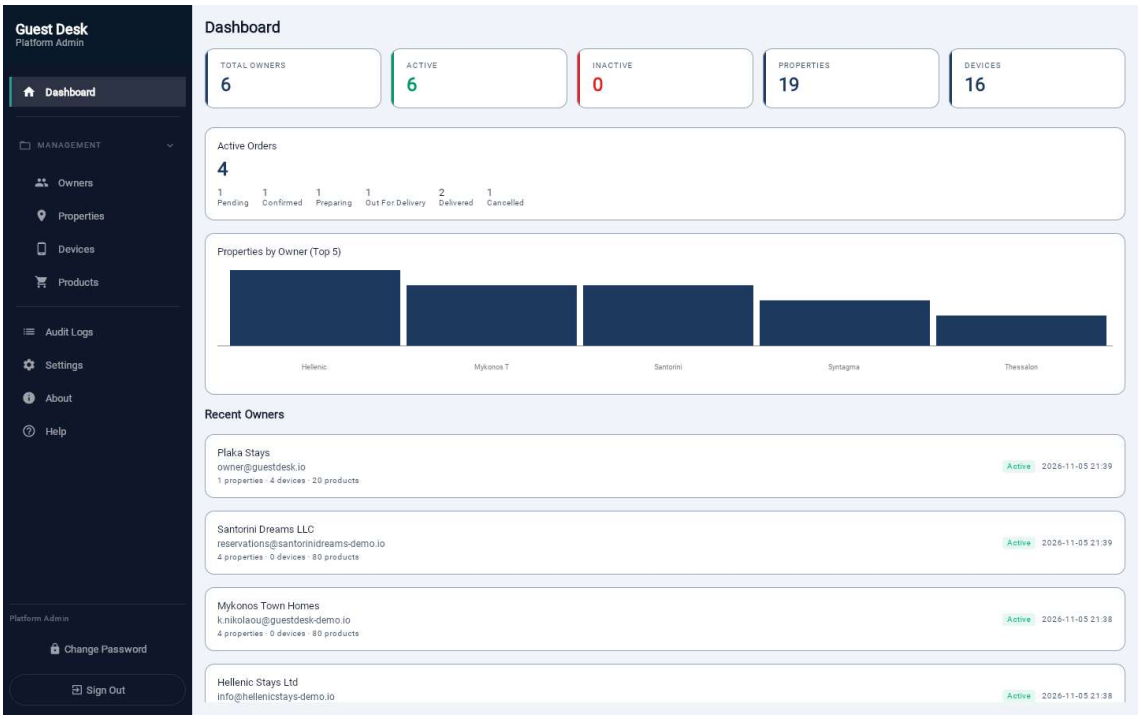
Η αρχική οθόνη μετά την επιτυχή σύνδεση εξαρτάται από τον ρόλο του χρήστη: AdminDashboard για τον PLATFORM_ADMIN και Dashboard για τον OWNER. Η ίδια λογική ισχύει και όταν ο χρήστης ανανεώνει τη σελίδα με αποθηκευμένη συνεδρία (§7.5): δεν υπάρχει κοινή αρχική σελίδα, αλλά διαφορετικό σημείο εισόδου ανά ρόλο.

7.3 Κύριες Οθόνες ανά Ρόλο

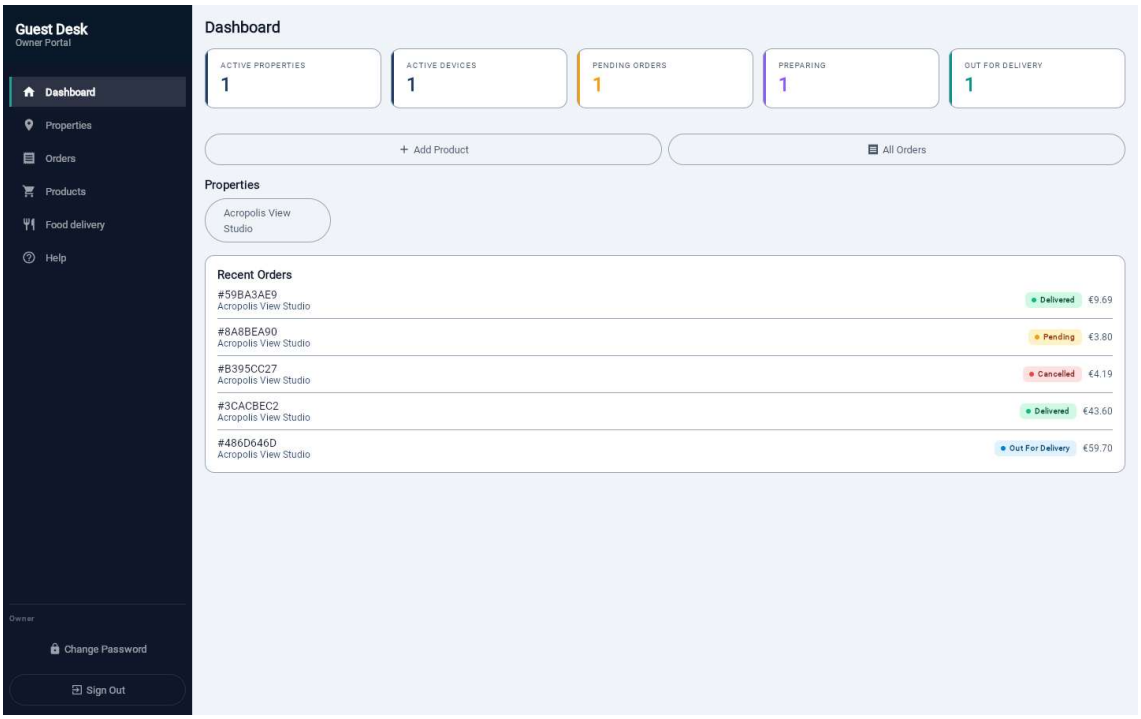
Ο Owner έχει ένα σύνολο οθονών που αφορούν μόνο τον δικό του tenant. Το Dashboard παρουσιάζει συνοπτικά στατιστικά για τα καταλύματα, τις συσκευές και τις παραγγελίες ανά στάδιο (§3.1.2). Το PropertyList επιτρέπει τη δημιουργία, επεξεργασία, απενεργοποίηση και διαγραφή καταλυμάτων, με πλοήγηση από κάθε εγγραφή προς τις συσκευές ή τις παραγγελίες του συγκεκριμένου καταλύματος (§7.6). Το DeviceList, ανά κατάλυμα, καλύπτει τη δημιουργία συσκευής, την εμφάνιση του κωδικού

QR ενεργοποίησης (§3.3.4), την απενεργοποίηση, την αποσύνδεση και το ιστορικό παραγγελιών κάθε συσκευής. Το ProductList διαχειρίζεται τον κατάλογο προϊόντων του tenant, μαζί με τη διαθεσιμότητα ανά κατάλυμα (§5.3.4) και την εισαγωγή προϊόντων από τον κατάλογο μιας πλατφόρμας (§7.7). Το FoodDelivery διαχειρίζεται τις συνδέσεις με τις πλατφόρμες διανομής (§7.7). Τέλος, τα OwnerOrders και OrderList εμφανίζουν τις παραγγελίες συνολικά ή ανά κατάλυμα (§7.9) και επιτρέπουν την αλλαγή κατάστασης σύμφωνα με τη μηχανή καταστάσεων (state machine) της §6.5.

Ο Platform Admin έχει ένα ευρύτερο σύνολο οθονών, προσαρμοσμένο στον εποπτικό του ρόλο. Το AdminDashboard συγκεντρώνει στατιστικά για όλους τους tenants: πλήθος ενεργών και ανενεργών ιδιοκτητών, καταλυμάτων και συσκευών, καθώς και κατανομή των παραγγελιών ανά κατάσταση. Το OwnerList καλύπτει τη διαχείριση των ιδιοκτητών, δηλαδή τη δημιουργία νέου ιδιοκτήτη με αναζήτηση στο εταιρικό μητρώο (§7.8), την επεξεργασία των στοιχείων του και την απενεργοποίηση ή επανενεργοποίηση του λογαριασμού. Επιλέγοντας έναν ιδιοκτήτη, ο διαχειριστής μεταβαίνει στο AdminOwnerView (§7.2), που παρουσιάζει τα στοιχεία του ιδιοκτήτη αυτού σε τέσσερις καρτέλες (Dashboard, Properties, Products, Orders), μόνο για ανάγνωση. Τη διαχείριση των δεδομένων την αναλαμβάνουν οι οθόνες AdminProperties, AdminDevices και AdminProducts, στις οποίες ο διαχειριστής μεταβαίνει απευθείας από το μενού: καθεμία εμφανίζει τις εγγραφές όλων των ιδιοκτητών, ομαδοποιημένες ανά ιδιοκτήτη, και επιτρέπει τη διαχείρισή τους χωρίς τα διαπιστευτήρια των ιδιοκτητών. Το AuditLogs εμφανίζει το αρχείο καταγραφής ενεργειών της πλατφόρμας (§5.2.4), και το Settings τις ρυθμίσεις των εξωτερικών υπηρεσιών (§7.8). Οι οθόνες About και Help περιέχουν στατικές πληροφορίες. Οι Εικόνες 7.2 και 7.3 δείχνουν τον πίνακα ελέγχου του διαχειριστή και του ιδιοκτήτη αντίστοιχα.



Εικόνα 7.2: Ο πίνακας ελέγχου του διαχειριστή της πλατφόρμας.



Εικόνα 7.3: Ο πίνακας ελέγχου του ιδιοκτήτη.

7.4 Πρόσβαση βάσει Ρόλων και Πλευρικό Μενού

Ο διαχωρισμός ανάμεσα στον Platform Admin και τον Owner στο web panel λειτουργεί σε δύο ανεξάρτητα επίπεδα, με σαφή διάκριση μεταξύ της εμπειρίας χρήστη και της ασφάλειας. Στη διεπαφή, το κοινό στοιχείο AppScaffold εμφανίζει διαφορετικό πλευρικό μενού ανάλογα με τον ρόλο της τρέχουσας συνεδρίας: ο Platform Admin βλέπει τις επιλογές Dashboard, Owners, Properties, Devices, Products, Audit Logs, Settings, About και Help, ενώ ο Owner τις επιλογές Dashboard, Properties, Orders, Products, Food delivery και Help. Στο κάτω μέρος του μενού, κοινές και για τους δύο ρόλους, βρίσκονται η αλλαγή κωδικού και η αποσύνδεση. Ο διαχωρισμός αυτός βελτιώνει την εμπειρία χρήστη, αφού αποφεύγει τη σύγχυση από επιλογές που δεν αφορούν τον ρόλο, αλλά δεν αποτελεί μηχανισμό ασφάλειας.

Ο πραγματικός έλεγχος πρόσβασης γίνεται αποκλειστικά στο backend, όπου κάθε route ελέγχει ρητά τον ρόλο που φέρει το JWT του αιτήματος πριν εκτελέσει οποιαδήποτε ενέργεια (§6.1, §6.2). Ακόμη κι αν κάποιος παρακάμψει τη διεπαφή, για παράδειγμα καλώντας απευθείας το API, ο server θα απορρίψει το αίτημα με σφάλμα εξουσιοδότησης, αφού η απόφαση πρόσβασης δεν εξαρτάται καθόλου από το τι εμφανίζει η διεπαφή.

Στις λειτουργίες που αφορούν άλλον tenant (cross-tenant), ο client στέλνει ρητά το schemaName του επιλεγμένου tenant ως query parameter σε κάθε κλήση προς το backend· μέθοδοι όπως οι listPropertiesAdmin, createProductAdmin ή deactivateDeviceAdmin δέχονται το schemaName ως παράμετρο και το προσθέτουν στο αίτημα HTTP. Αυτό αντιστοιχεί στον μηχανισμό επιλογής tenant για τον Platform Admin (§4.3, §6.3).

7.5 ApiClient: Επικοινωνία και Ανανέωση Token

Η επικοινωνία με το backend γίνεται μέσω μιας ενιαίας κλάσης ApiClient, που ενσωματώνει έναν Ktor HttpClient με μηχανή εκτέλεσης Js και προσφέρει μία μέθοδο για κάθε endpoint, για παράδειγμα getOwnerStats(), lookupCompany(), searchAddress(), connectFoodPlatform(), updateOrderStatus(). Η κλάση εγκαθιστά το plugin ContentNegotiation για αυτόματη σειριοποίηση και αποσειριοποίηση JSON, και έναν προσαρμοσμένο HttpResponseValidator, που μετατρέπει τις αποτυχημένες αποκρίσεις σε εξαιρέσεις με το μήνυμα σφάλματος του backend. Εξαιρείται ο κωδικός 401, ώστε να τον χειρίζεται ξεχωριστά ο μηχανισμός ανανέωσης του token.

Κάθε κλήση που απαιτεί ταυτοποίηση περνά από μια εσωτερική συνάρτηση withTokenRefresh: αν το backend απαντήσει με 401, ο client επιχειρεί αυτόματα να ανανεώσει τη συνεδρία καλώντας το /api/auth/refresh με το αποθηκευμένο refresh token, και μετά επαναλαμβάνει το αρχικό αίτημα μία μόνο φορά με το νέο access token. Επειδή τα refresh tokens αντικαθίστανται σε κάθε χρήση (§4.5.7), δύο αιτήματα που λαμβάνουν ταυτόχρονα 401 δεν πρέπει να ανανεώσουν ανεξάρτητα τη συνεδρία· το

δεύτερο θα χρησιμοποιούσε ένα token που έχει ήδη καταναλωθεί και θα τερμάτιζε λανθασμένα τη συνεδρία. Γι' αυτό η ανανέωση προστατεύεται με Mutex: το πρώτο αίτημα εκτελεί την ανανέωση και τα υπόλοιπα χρησιμοποιούν το αποτέλεσμά της. Αν η ανανέωση αποτύχει, ή δεν υπάρχει αποθηκευμένη συνεδρία, ο client δηλώνει ότι η συνεδρία έληξε. Έτσι υλοποιείται διαφανώς, από την πλευρά του client, ο μηχανισμός refresh token rotation: ο χρήστης δεν αντιλαμβάνεται την ανανέωση, εκτός αν αποτύχει.

Η συνεδρία, που αναπαρίσταται από την κλάση AuthSession (access token, refresh token, ρόλος, schemaName), αποθηκεύεται στο localStorage του browser ώστε να διατηρείται μετά την ανανέωση της σελίδας. Τη λήξη της συνεδρίας τη διαχειρίζεται ο SessionController, ένα απλό reactive singleton που εκθέτει μια τιμή mutableStateOf<Boolean> η οποία δείχνει αν η συνεδρία έχει λήξει. Το ριζικό composable App() παρακολουθεί την τιμή αυτή με LaunchedEffect: μόλις γίνει true, καθαρίζει την τοπική συνεδρία, εμφανίζει ενημερωτικό μήνυμα, επιστρέφει τον χρήστη στην οθόνη σύνδεσης και επαναφέρει την τιμή σε false. Καμία οθόνη δεν χρειάζεται να ελέγχει η ίδια αν έληξε η συνεδρία· η ευθύνη αυτή βρίσκεται σε ένα μόνο σημείο.

Η αποθήκευση των tokens στο localStorage είναι απλή, αλλά τα κάνει προσβάσιμα σε οποιονδήποτε κώδικα JavaScript εκτελείται στην ίδια σελίδα. Ο κίνδυνος μειώνεται από τη σύντομη διάρκεια ζωής του access token και από την απουσία κώδικα τρίτων στη σελίδα, παραμένει όμως ένας από τους γνωστούς περιορισμούς του πρωτοτύπου (§9.4).

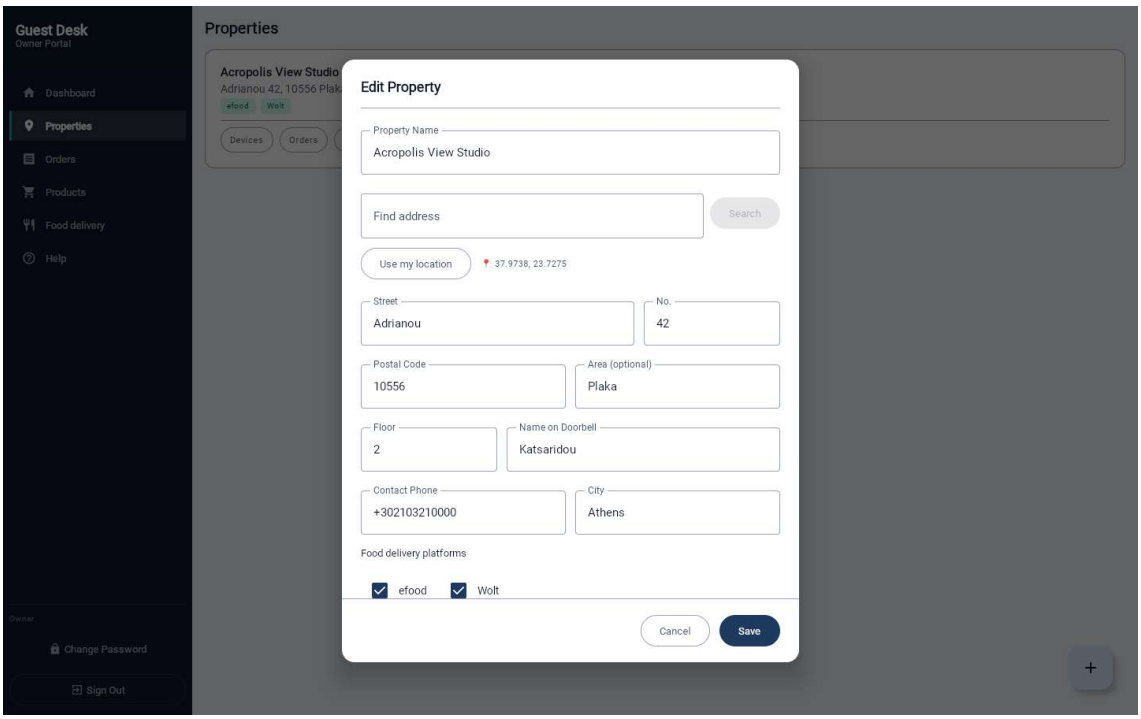
7.6 Διαχείριση Καταλυμάτων: Αναζήτηση Διεύθυνσης και Πληροφορίες Επισκέπτη

Η οθόνη PropertyList, τόσο στην έκδοση του Owner όσο και στην αντίστοιχη cross-tenant έκδοση του Platform Admin (AdminProperties), είναι αντιπροσωπευτικό παράδειγμα του πώς οργανώνεται μια φόρμα διαχείρισης στο panel. Η φόρμα δημιουργίας ή επεξεργασίας καταλύματος συλλέγει το όνομα, τη διεύθυνση (οδός, αριθμός, ταχυδρομικός κώδικας, πόλη και, προαιρετικά, περιοχή και όροφος), το όνομα στο κουδούνι, το τηλέφωνο επικοινωνίας και προαιρετικές σημειώσεις. Κάθε πεδίο αντιστοιχεί σε μεταβλητό state (remember { mutableStateOf(...) }) μέσα στο composable της φόρμας, και το κουμπί επιβεβαίωσης ενεργοποιείται μόνο όταν όλα τα υποχρεωτικά πεδία έχουν τιμή. Κατά την επιβεβαίωση, τα δεδομένα τοποθετούνται σε ένα CreatePropertyRequest ή UpdatePropertyRequest, τα ίδια DTOs του module :data που χρησιμοποιεί και το backend.

Η διεύθυνση δεν χρειάζεται να πληκτρολογηθεί πεδίο προς πεδίο. Το στοιχείο AddressSearch (Εικόνα 7.4) προσφέρει δύο τρόπους συμπλήρωσης: ο χρήστης πληκτρολογεί μια διεύθυνση και πατά το κουμπί αναζήτησης, ή επιλέγει την τρέχουσα θέση του, οπότε η εφαρμογή ζητά τις συντεταγμένες από το Geolocation API του browser. Επειδή το API αυτό δεν είναι άμεσα διαθέσιμο στον κώδικα Kotlin/Wasm, καλείται μέσω μιας μικρής συνάρτησης JavaScript που δηλώνεται με το annotation @JsFun, και ενσωματώνεται σε suspend συνάρτηση της Kotlin. Η ίδια τεχνική χρησιμοποιείται και σε

λίγα ακόμη σημεία όπου το web panel χρειάζεται δυνατότητες του browser που δεν καλύπτονται από τις βιβλιοθήκες της Kotlin. Και στις δύο περιπτώσεις το αίτημα διεκπεραιώνεται από το backend (§6.8), το οποίο υποβάλλει ερώτημα στην υπηρεσία γεωκωδικοποίησης και επιστρέφει μία ή περισσότερες διευθύνσεις· όταν ο χρήστης επιλέξει μία, συμπληρώνονται τα επιμέρους πεδία και οι συντεταγμένες, τα οποία μπορεί να διορθώσει. Η αναζήτηση γίνεται μόνο όταν τη ζητήσει ρητά ο χρήστης και όχι καθώς πληκτρολογεί, σύμφωνα με την πολιτική χρήσης της υπηρεσίας [24], ενώ το component εμφανίζει την αναφορά προέλευσης των δεδομένων που απαιτεί η πολιτική. Επειδή οι browsers επιτρέπουν το Geolocation API μόνο σε ασφαλείς συνδέσεις (HTTPS) ή στο localhost, αν αποτύχει ο εντοπισμός θέσης, ο χρήστης ενημερώνεται και μπορεί πάντα να καταφύγει στην αναζήτηση με κείμενο.

Η ίδια φόρμα περιλαμβάνει δύο ακόμη ενότητες. Το στοιχείο GuestInfoFields συλλέγει τις προαιρετικές πληροφορίες που εμφανίζει η εφαρμογή kiosk στον επισκέπτη (δίκτυο και κωδικός Wi-Fi, ώρα αναχώρησης, κανόνες του καταλύματος), και το FoodPlatformControls εμφανίζει ένα πλαίσιο επιλογής για κάθε πλατφόρμα διανομής που έχει συνδέσει ο ιδιοκτήτης, ώστε να οριστεί ποιες προσφέρονται στο κατάλυμα. Η οθόνη AdminProperties επαναλαμβάνει την ίδια φόρμα, με τη διαφορά ότι οι κλήσεις προς το backend συνοδεύονται από το schemaName του ιδιοκτήτη στον οποίο ανήκει το κατάλυμα (§7.4).



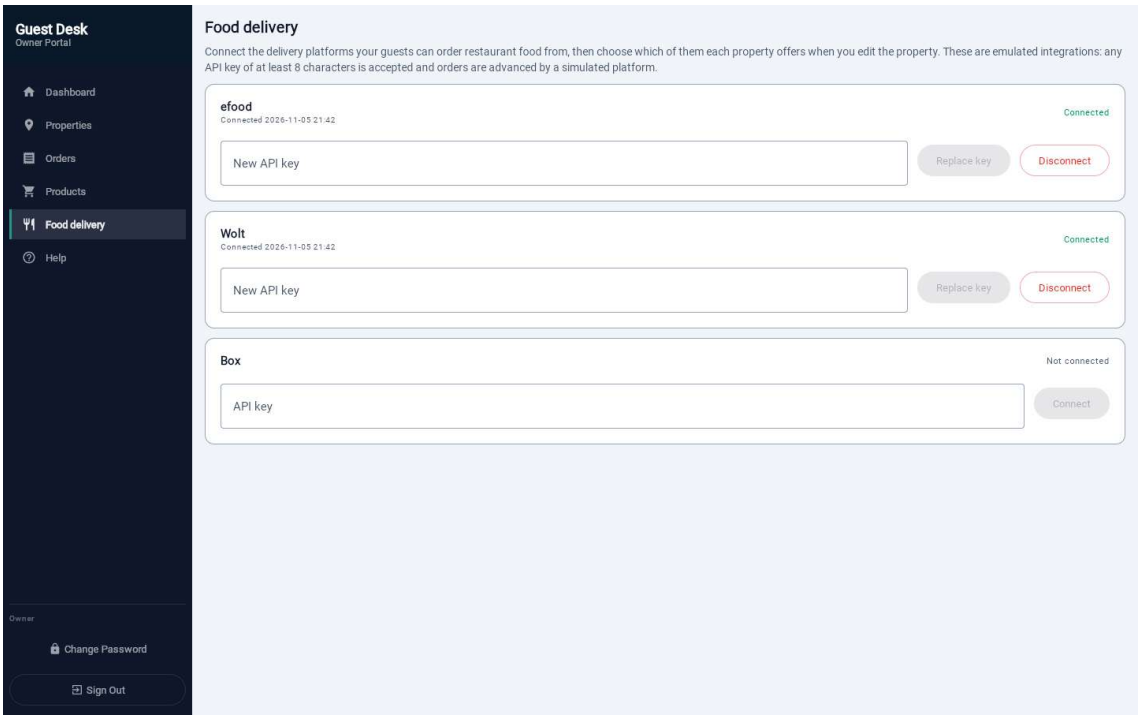
Εικόνα 7.4: Η φόρμα καταλύματος με την αναζήτηση διεύθυνσης.

7.7 Πλατφόρμες Διανομής, Εισαγωγή Προϊόντων και Παραγγελίες

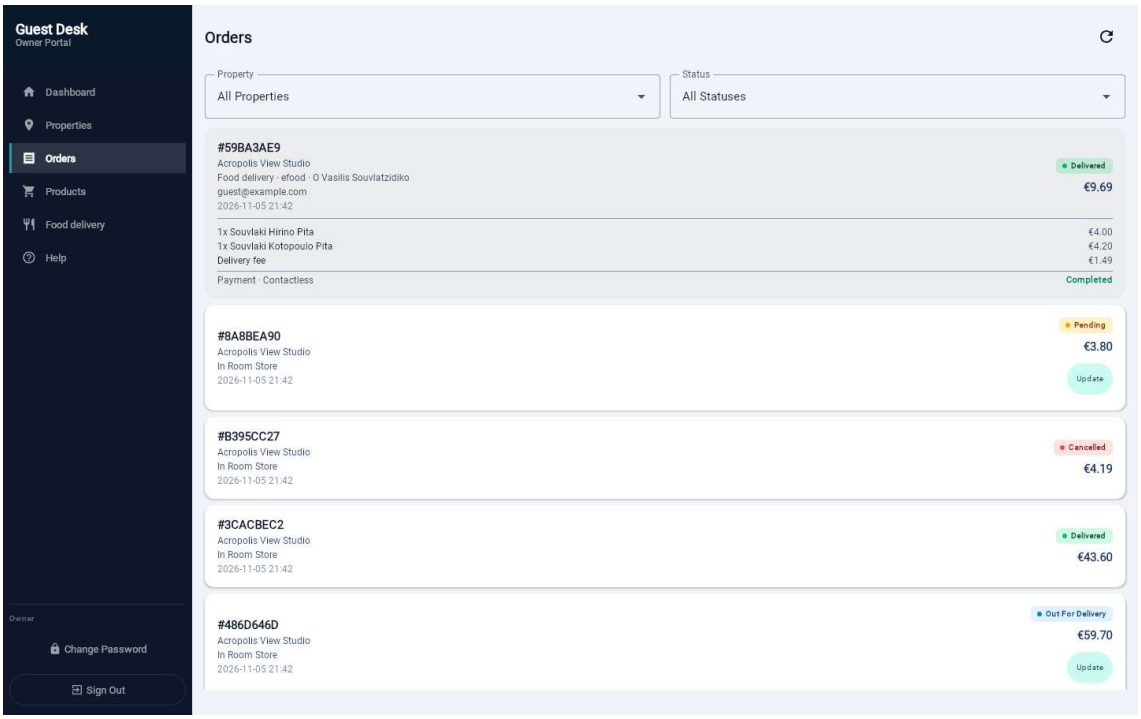
Η οθόνη FoodDelivery (Εικόνα 7.5) εμφανίζει μία κάρτα για κάθε υποστηριζόμενη πλατφόρμα, με την κατάσταση της σύνδεσης. Για να συνδέσει μια πλατφόρμα, ο ιδιοκτήτης πληκτρολογεί το κλειδί πρόσβασης που έχει λάβει από την πλατφόρμα· το backend το επαληθεύει με την πλατφόρμα πριν το αποθηκεύσει κρυπτογραφημένο (§6.7), οπότε ένα λάθος κλειδί απορρίπτεται αμέσως. Η αποσύνδεση ζητά επιβεβαίωση και απορρίπτεται από το backend όσο υπάρχουν παραγγελίες της πλατφόρμας σε εξέλιξη (§6.7).

Από την οθόνη προϊόντων, ο διάλογος ImportProductsDialog επιτρέπει στον ιδιοκτήτη να επιλέξει μια συνδεδεμένη πλατφόρμα, να περιηγηθεί στον κατάλογο προϊόντων της και να επιλέξει τα είδη που θέλει να προσθέσει στο κατάστημα του καταλύματός του. Μετά την εισαγωγή, ο διάλογος αναφέρει ποια είδη προστέθηκαν και ποια παραλείφθηκαν επειδή υπήρχαν ήδη στον κατάλογο. Οι τιμές των εισαγόμενων ειδών προέρχονται πάντα από την πλατφόρμα και μπορούν στη συνέχεια να αλλάξουν όπως σε κάθε άλλο προϊόν.

Στις οθόνες παραγγελιών, κάθε παραγγελία φαγητού εμφανίζεται με την πλατφόρμα και το εστιατόριο από τα οποία προέρχεται. Ο διάλογος αλλαγής κατάστασης προσφέρει για αυτές μόνο την ακύρωση, με μήνυμα που εξηγεί ότι την πρόοδο ενημερώνει η πλατφόρμα, ενώ για τις παραγγελίες από το κατάστημα του καταλύματος προσφέρει την επόμενη επιτρεπτή κατάσταση. Οι επιλογές υπολογίζονται στον client για ευκολία του χρήστη· ο δεσμευτικός έλεγχος γίνεται στο backend (§6.5). Το στοιχείο PaymentSection εμφανίζει για κάθε παραγγελία τις απόπειρες πληρωμής με τη σειρά που έγιναν: τον τρόπο ανάγνωσης της κάρτας, τα τελευταία ψηφία της όπου είναι διαθέσιμα, το αποτέλεσμα, την αιτία μιας απόρριψης και την επιστροφή χρημάτων μετά από ακύρωση. Η Εικόνα 7.6 δείχνει μια παραγγελία με τις γραμμές και την πληρωμή της.



Εικόνα 7.5: Σύνδεση των πλατφορμών διανομής.



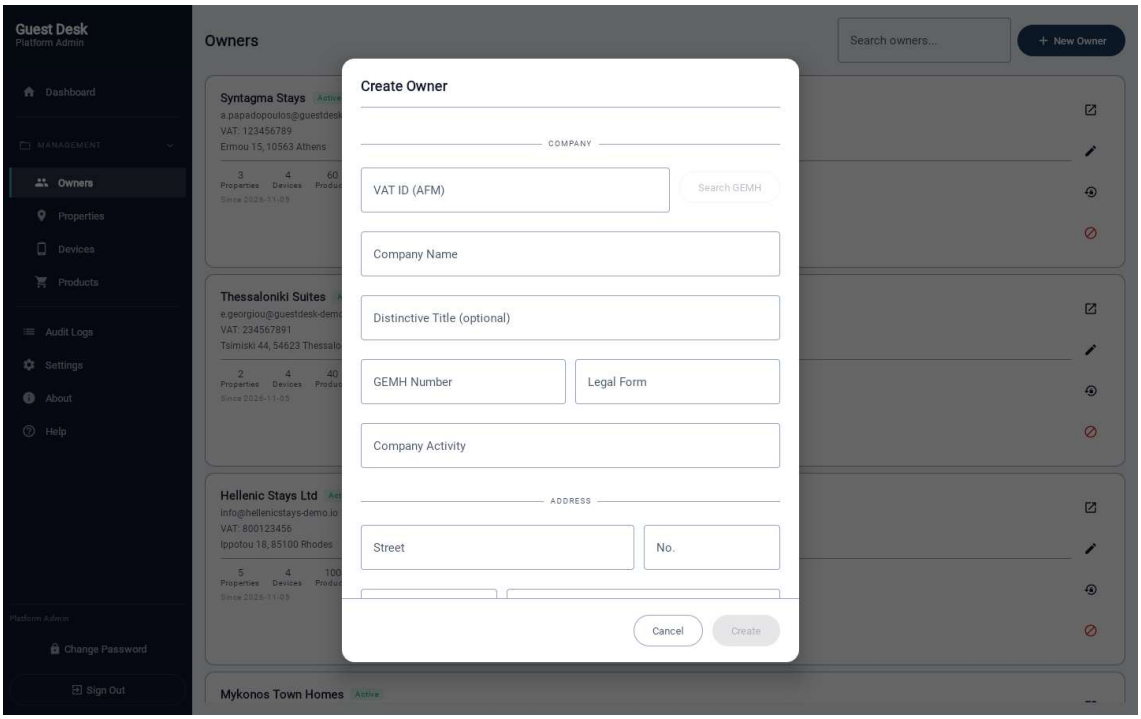
Εικόνα 7.6: Μια παραγγελία με τις γραμμές και την πληρωμή της.

7.8 Ένταξη Ιδιοκτητών και Ρυθμίσεις Πλατφόρμας

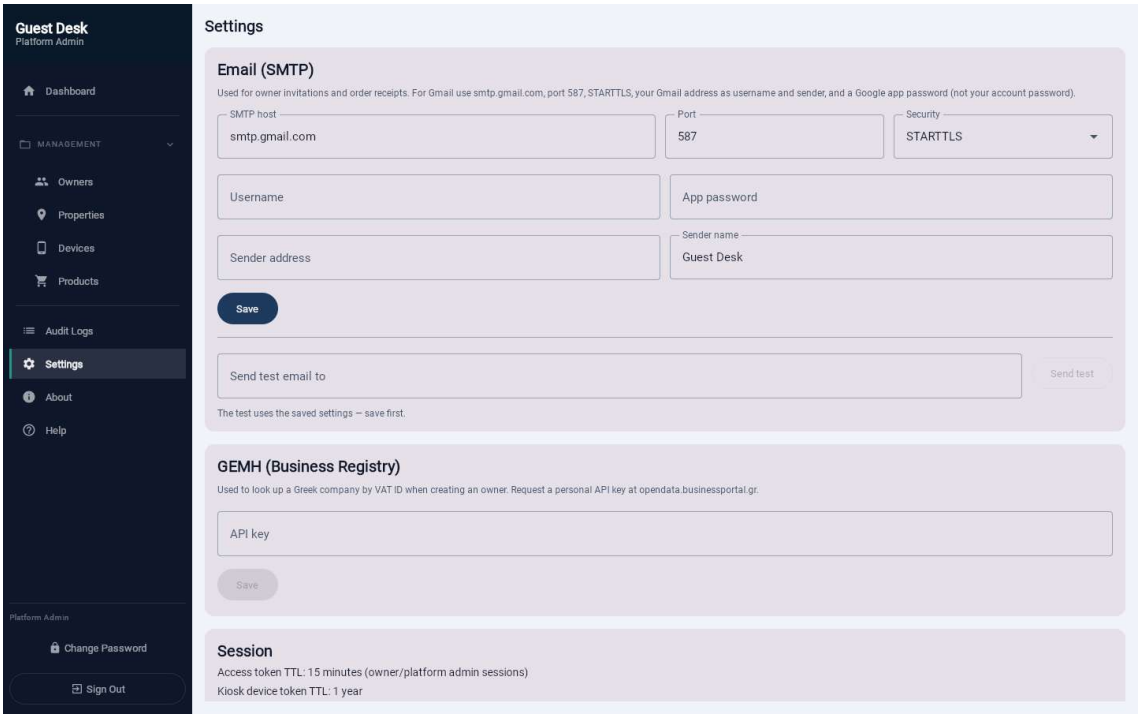
Η φόρμα δημιουργίας ιδιοκτήτη στην οθόνη OwnerList (Εικόνα 7.7) ξεκινά από τον ΑΦΜ. Ο διαχειριστής πληκτρολογεί τον ΑΦΜ και πατά το κουμπί αναζήτησης· ο client καλεί το lookupCompany(), το backend υποβάλλει ερώτημα στο εταιρικό μητρώο (§6.8), και τα στοιχεία που επιστρέφονται (επωνυμία, διακριτικός τίτλος, αριθμός μητρώου, νομική μορφή, δραστηριότητα, διεύθυνση έδρας) συμπληρώνονται στα αντίστοιχα πεδία. Ο διαχειριστής τα ελέγχει, συμπληρώνει τα στοιχεία του υπεύθυνου επικοινωνίας και αποθηκεύει. Αν η αναζήτηση αποτύχει, το μήνυμα του backend εμφανίζεται στη φόρμα και τα πεδία μπορούν να συμπληρωθούν χειροκίνητα. Μετά την αποθήκευση, ένας διάλογος εμφανίζει μία φορά τον προσωρινό κωδικό του νέου ιδιοκτήτη, με δυνατότητα αντιγραφής, για την περίπτωση που το μήνυμα καλωσορίσματος δεν έχει φτάσει.

Η λίστα ιδιοκτητών προσφέρει επίσης επαναφορά του κωδικού για έναν ιδιοκτήτη που τον έχει ξεχάσει. Μετά από επιβεβαίωση, το backend παράγει νέο προσωρινό κωδικό και τον στέλνει στον ιδιοκτήτη (§6.4), ενώ ο ίδιος διάλογος που εμφανίζεται κατά τη δημιουργία δείχνει τον κωδικό μία φορά στον διαχειριστή. Ο ιδιοκτήτης, με τη σειρά του, αντικαθιστά τον προσωρινό κωδικό με δικό του από τον διάλογο αλλαγής κωδικού του πλευρικού μενού, που ζητά τον τρέχοντα κωδικό και δύο φορές τον νέο, και ενεργοποιεί το κουμπί αποθήκευσης μόνο όταν ο νέος κωδικός έχει τουλάχιστον οκτώ χαρακτήρες και οι δύο τιμές ταυτίζονται. Μετά την επιτυχή αλλαγή, ο client αποθηκεύει το νέο ζεύγος tokens που επιστρέφει ο server, ώστε η τρέχουσα συνεδρία να συνεχιστεί ενώ οι υπόλοιπες δεν μπορούν πλέον να ανανεωθούν.

Η οθόνη Settings (Εικόνα 7.8), διαθέσιμη μόνο στον Platform Admin, συγκεντρώνει τις ρυθμίσεις των εξωτερικών υπηρεσιών της πλατφόρμας. Η ενότητα του ηλεκτρονικού ταχυδρομείου περιέχει τον διακομιστή SMTP, τη θύρα, τον τύπο ασφάλειας σύνδεσης, τα διαπιστευτήρια και τη διεύθυνση και το όνομα αποστολέα, με προσυμπληρωμένες τιμές για μια συνηθισμένη δημόσια υπηρεσία ηλεκτρονικού ταχυδρομείου, και επιτρέπει την αποστολή δοκιμαστικού μηνύματος σε οποιαδήποτε διεύθυνση. Η ενότητα του εταιρικού μητρώου περιέχει το κλειδί πρόσβασης. Για τα μυστικά στοιχεία (κωδικός SMTP, κλειδί του μητρώου), η οθόνη δείχνει μόνο αν έχουν οριστεί· αν το πεδίο του κωδικού SMTP μείνει κενό κατά την αποθήκευση, διατηρείται ο υπάρχων κωδικός, ώστε ο διαχειριστής να αλλάζει τις υπόλοιπες ρυθμίσεις χωρίς να τον πληκτρολογεί ξανά. Η ίδια οθόνη αναφέρει τη διάρκεια ζωής των tokens (§4.5.7).



Εικόνα 7.7: Η φόρμα δημιουργίας ιδιοκτήτη, με αφετηρία τον ΑΦΜ.



Εικόνα 7.8: Οι ρυθμίσεις των εξωτερικών υπηρεσιών.

7.9 Ενημέρωση Δεδομένων, Σφάλματα και Ταυτόχρονες Ενέργειες

Μια διαχειριστική εφαρμογή που εκτελείται στον browser οφείλει να αντιμετωπίσει τρία ζητήματα που δεν φαίνονται από μια στατική περιγραφή των οθονών: πώς διατηρούνται τα δεδομένα επίκαιρα, πώς εμφανίζονται τα σφάλματα και πώς αποτρέπεται η διπλή εκτέλεση μιας ενέργειας.

Οι οθόνες παραγγελιών του ιδιοκτήτη και η καρτέλα παραγγελιών του διαχειριστή ανανεώνουν τα δεδομένα τους κάθε 15 δευτερόλεπτα. Η ανανέωση υλοποιείται ως βρόχος μέσα σε ένα LaunchedEffect, επομένως ο βρόχος διαρκεί όσο και η οθόνη: όταν ο χρήστης μεταβεί σε άλλη οθόνη, το Compose ακυρώνει το coroutine και ο βρόχος σταματά, χωρίς να στέλνει αιτήματα στο παρασκήνιο. Για να λειτουργεί αυτή η ακύρωση, κάθε σημείο που συλλαμβάνει (catch) εξαιρέσεις κατά τη φόρτωση δεδομένων προωθεί ξανά ρητά (rethrow) την CancellationException, αντί να τη θεωρεί σφάλμα δικτύου. Αν παραλειπόταν αυτό, ο βρόχος θα έμενε ενεργός και μετά την έξοδο από την οθόνη· το σφάλμα εντοπίστηκε σε έλεγχο του κώδικα και διορθώθηκε συστηματικά σε όλες τις οθόνες (το μοτίβο εμφανίζεται πλέον σε περισσότερα από σαράντα σημεία του module).

Τα σφάλματα του backend φτάνουν στη διεπαφή ως εξαιρέσεις με το μήνυμα του ErrorResponse (§6.3) και εμφανίζονται με ενιαίο τρόπο: τα σφάλματα φόρτωσης σε ένα πλαίσιο ErrorBanner στο πάνω μέρος της οθόνης, που ο χρήστης μπορεί να κλείσει, και τα αποτελέσματα των ενεργειών ως προσωρινά μηνύματα του SnackbarController. Επειδή τα μηνύματα του backend είναι σύντομα και κατανοητά (για παράδειγμα «Cannot delete a device with existing orders — deactivate it instead»), η διεπαφή μπορεί να τα εμφανίζει αυτούσια, χωρίς δικό της κατάλογο μηνυμάτων ανά κωδικό σφάλματος.

Η διπλή εκτέλεση μιας ενέργειας αποτρέπεται σε δύο επίπεδα. Στη διεπαφή, ο διάλογος επιβεβαίωσης απενεργοποιεί τα κουμπιά του μετά το πρώτο πάτημα, και οι διάλογοι κλείνουν τόσο όταν η ενέργεια πετύχει όσο και όταν αποτύχει, ώστε κανένας διάλογος να μην παραμένει ανοιχτός με απενεργοποιημένα κουμπιά. Στο backend, οι κρίσιμες ενέργειες προστατεύονται ούτως ή άλλως από τους μηχανισμούς της §5.6, οπότε ακόμη και δύο ταυτόχρονα αιτήματα από δύο ανοιχτές καρτέλες του browser δεν μπορούν να αλλάξουν δύο φορές την κατάσταση της ίδιας παραγγελίας.

Για τις ανάγκες της επίδειξης, η οθόνη σύνδεσης περιλαμβάνει μια ενότητα γρήγορης σύνδεσης με δύο κουμπιά, για τον λογαριασμό του διαχειριστή και για τον δοκιμαστικό ιδιοκτήτη που δημιουργούν τα scripts της §9.2. Οι λογαριασμοί αυτοί προορίζονται για επίδειξη και οι κωδικοί τους είναι δημόσιοι· σε πραγματική εγκατάσταση η ενότητα θα αφαιρούνταν.

7.10 Επαναχρησιμοποίηση Οπτικών Στοιχείων

Πέρα από τις οθόνες, το module :owner-web οργανώνει στον κατάλογο ui/ ένα σύνολο κοινών οπτικών στοιχείων, που περιορίζουν την επανάληψη κώδικα διεπαφής σε οθόνες με παρόμοιες ανάγκες. Το FormDialog παρέχει ένα γενικό modal πλαίσιο με τυποποιημένα κουμπιά επιβεβαίωσης και ακύρωσης,

πάνω στο οποίο στηρίζονται όλες οι φόρμες δημιουργίας και επεξεργασίας. Το ConfirmDialog καλύπτει τις ενέργειες που απαιτούν επιβεβαίωση, όπως η διαγραφή ή η απενεργοποίηση μιας εγγραφής, και απενεργοποιεί τα κουμπιά του μετά το πρώτο πάτημα (§7.9). Το QrCodeDialog εμφανίζει τον κωδικό QR ενεργοποίησης μιας νέας συσκευής (§3.3.4, §8.2), ενώ το SnackbarController προσφέρει έναν ενιαίο μηχανισμό για τα προσωρινά μηνύματα. Τα AddressSearch, GuestInfoFields, FoodPlatformControls και ImportProductsDialog (§7.6, §7.7) χρησιμοποιούνται τόσο στις οθόνες του ιδιοκτήτη όσο και στις αντίστοιχες του διαχειριστή, ενώ το PaymentSection (§7.7) χρησιμοποιείται στις οθόνες παραγγελιών του ιδιοκτήτη. Στοιχεία όπως τα ActiveBadge, StatusChip, ErrorBanner, EmptyState και LoadingState τυποποιούν την εμφάνιση της κατάστασης ενεργοποίησης, της κατάστασης των παραγγελιών, των μηνυμάτων σφάλματος, των κενών λιστών και της φόρτωσης, ώστε όλες οι οθόνες να έχουν ενιαία οπτική εμφάνιση.

Κεφάλαιο 8. Εφαρμογή Guest Kiosk

Το κεφάλαιο περιγράφει την εφαρμογή kiosk που χρησιμοποιεί ο φιλοξενούμενος: τη θέση της στην αρχιτεκτονική (§8.1), την ενεργοποίηση της συσκευής (§8.2), τη λειτουργία kiosk (§8.3), τη ροή της παραγγελίας και της πληρωμής (§8.4, §8.5), την παρακολούθηση των παραγγελιών (§8.6), τις επιλογές σχεδίασης για μια κοινόχρηστη συσκευή (§8.7), την τοπική αποθήκευση (§8.8) και την ανεξαρτησία της εφαρμογής από τον φορέα πληρωμών (§8.9).

8.1 Ρόλος και Θέση στην Αρχιτεκτονική

Το module :app υλοποιεί την εφαρμογή που εκτελείται στη συσκευή Android κάθε καταλύματος και είναι η μοναδική διεπαφή του φιλοξενούμενου με το σύστημα. Σε αντίθεση με το :owner-web (Κεφάλαιο 7), που μεταγλωττίζεται σε WebAssembly με το Compose Multiplatform, το :app είναι μια συμβατική εφαρμογή Android, γραμμένη με Jetpack Compose για τη διεπαφή και Jetpack Navigation για την πλοήγηση ανάμεσα στις οθόνες. Παρά τη διαφορά αυτή στο επίπεδο παρουσίασης, το module χρησιμοποιεί τα ίδια DTOs των :domain και :data με το backend και το owner-web (§4.5.4).

Η εφαρμογή δεν χρησιμοποιεί framework έγχυσης εξαρτήσεων (dependency injection), όπως το Hilt ή το Koin. Ένα singleton αντικείμενο, ο AppContainer, λειτουργεί ως απλός container που δημιουργείται χειροκίνητα: παρέχει πρόσβαση στα preferences (§8.8), στον AppApiClient που επικοινωνεί με το backend, στον κατάλογο προϊόντων του καταλύματος και στον κατάλογο εστιατορίων (FoodCatalog), στο καλάθι του επισκέπτη (GuestCart) και στον αναγνώστη καρτών (CardTerminal, §8.5). Ο ίδιος container διατηρεί και την κατάσταση της τρέχουσας επίσκεψης: το email και τις σημειώσεις του επισκέπτη, καθώς και τις παραγγελίες που έχουν υποβληθεί από τη στιγμή που ο επισκέπτης άρχισε να χρησιμοποιεί τη συσκευή. Η επίσκεψη τελειώνει όταν η εφαρμογή επιστρέφει στην οθόνη αδράνειας (§8.6), οπότε τα δεδομένα αυτά διαγράφονται, ώστε να μη μεταφέρονται από έναν επισκέπτη στον επόμενο.

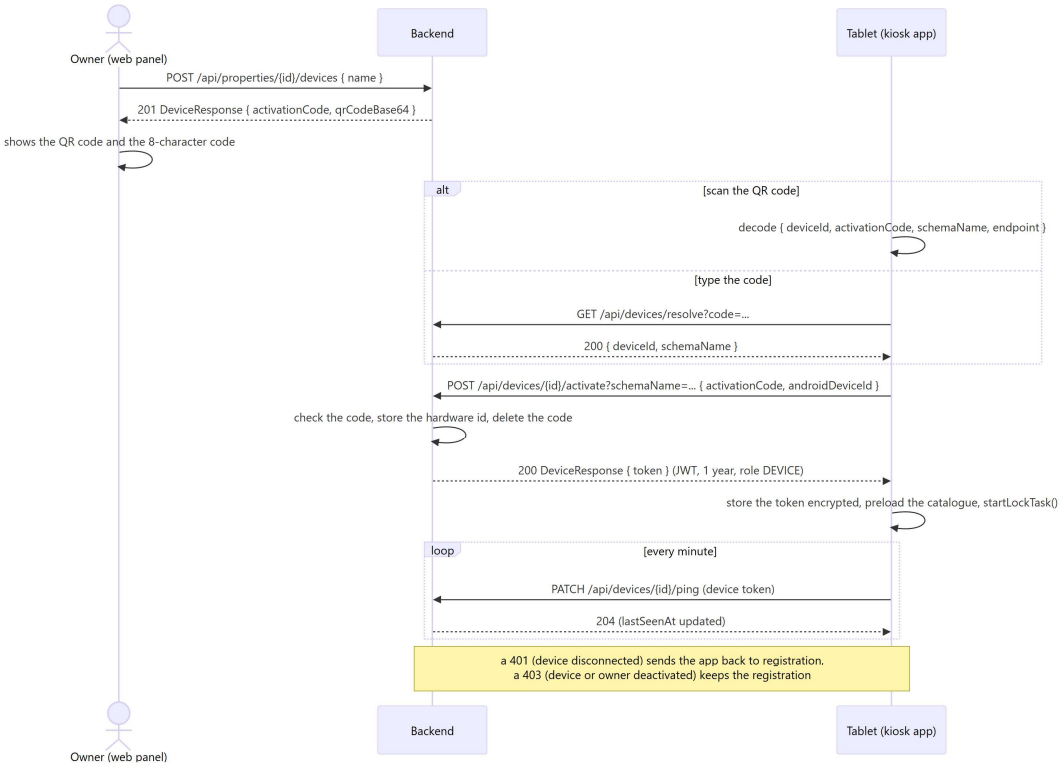
8.2 Ροή Ενεργοποίησης Συσκευής

Κατά την εκκίνηση, το MainActivity διαβάζει την αποθηκευμένη κατάσταση εγγραφής της συσκευής (§8.8): αν η συσκευή δεν έχει ακόμη καταχωρηθεί, το navigation graph ξεκινά από την οθόνη Registration αντί για την οθόνη αδράνειας. Η οθόνη αυτή (Εικόνα 8.2) και το RegistrationViewModel υποστηρίζουν δύο εναλλακτικούς τρόπους ενεργοποίησης (Εικόνα 8.1).

Στον πρώτο τρόπο, η εφαρμογή σαρώνει, με τη βιβλιοθήκη ZXing [35], τον κωδικό QR που εμφανίζει ο ιδιοκτήτης στο web panel (§7.3). Το αποτέλεσμα της σάρωσης μεταβιβάζεται από το MainActivity

σε ένα κοινό MutableStateFlow του AppContainer, που παρακολουθεί το ViewModel. Το περιεχόμενο του κωδικού QR είναι ένα αντικείμενο JSON με τα πεδία deviceId, activationCode, schemaName και endpoint· η εφαρμογή το μετατρέπει με το kotlinx.serialization σε μια τοπική κλάση που αντιστοιχεί στη δομή που παράγει το backend και, αν τα υποχρεωτικά πεδία δεν είναι κενά, προχωρά απευθείας στην ενεργοποίηση, χρησιμοποιώντας το endpoint ως διεύθυνση του server.

Στον δεύτερο τρόπο, ο χρήστης πληκτρολογεί τη διεύθυνση του server και τον κωδικό ενεργοποίησης. Επειδή τότε δεν είναι γνωστό το deviceId, η εφαρμογή καλεί πρώτα το GET /api/devices/resolve, που μετατρέπει τον κωδικό σε deviceId και schemaName, και μετά την ίδια την ενεργοποίηση (§6.2).

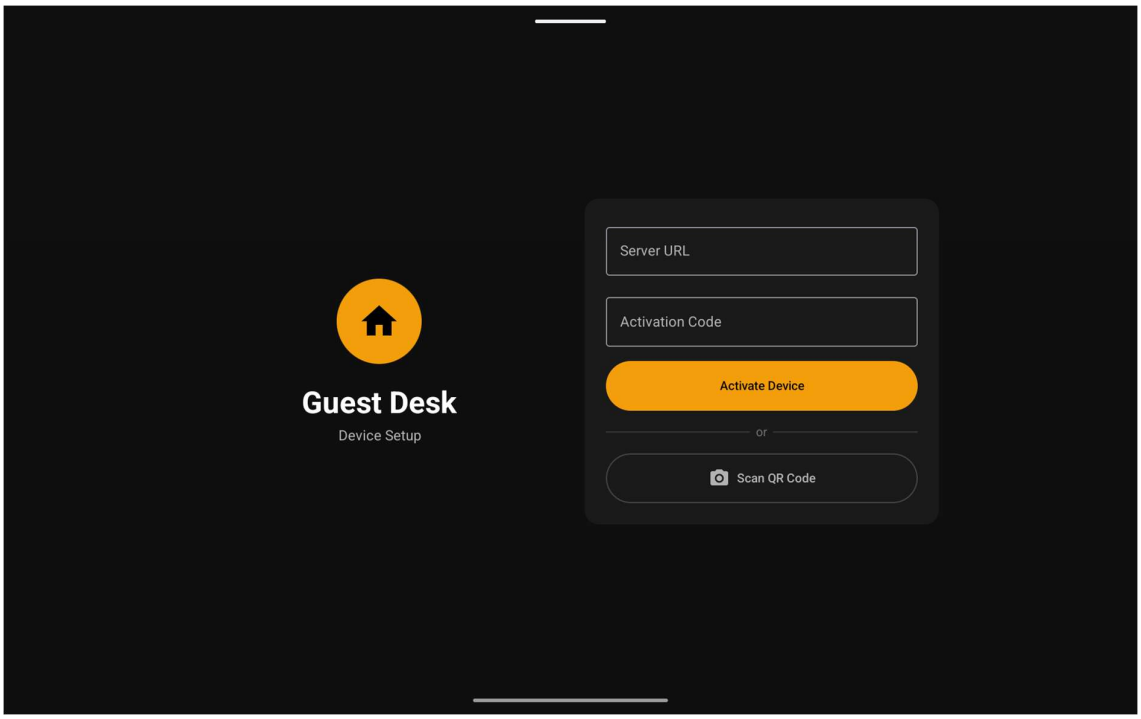


Εικόνα 8.1: Ροή ενεργοποίησης συσκευής — σάρωση QR ή χειροκίνητη εισαγωγή, resolve, activate, έκδοση και αποθήκευση token.

Και οι δύο τρόποι καταλήγουν στο ίδιο σημείο: κλήση του endpoint ενεργοποίησης, με αναγνωριστικό της συσκευής το Settings.Secure.ANDROID_ID. Το backend επαληθεύει τον κωδικό, συνδέει μόνιμα τη συσκευή με αυτό το αναγνωριστικό και εκδίδει το token της συσκευής (§6.2). Αν η ενεργοποίηση απορριφθεί, η εφαρμογή εμφανίζει την αιτία: άγνωστος κωδικός (404), λάθος κωδικός (403) ή, για το 409, η αιτία που επιστρέφει το backend (συσκευή ήδη ενεργοποιημένη ή απενεργοποιημένο κατάλυμα ή ιδιοκτήτης). Μόλις παραλάβει την απόκριση, η εφαρμογή αποθηκεύει τα στοιχεία εγγραφής, με το token σε κρυπτογραφημένη μορφή (§8.8), και προσπαθεί να φορτώσει εκ των προτέρων τον κατάλογο προϊόντων και τα στοιχεία του καταλύματος, ώστε η πρώτη οθόνη μετά την ενεργοποίηση να μην

εμφανιστεί άδεια. Η προφόρτωση δεν είναι κρίσιμη: αν αποτύχει, η εφαρμογή την επαναλαμβάνει σε κάθε επίσκεψη στην αρχική οθόνη.

Το token συνοδεύει, ως κεφαλίδα Authorization: Bearer, κάθε επόμενο αίτημα της εφαρμογής. Αν το backend απαντήσει με 401, για παράδειγμα επειδή ο ιδιοκτήτης αποσύνδεσε τη συσκευή, η εφαρμογή διαγράφει τα τοπικά στοιχεία εγγραφής και το navigation graph επιστρέφει αυτόματα στην οθόνη Registration, αντί να μείνει σε μια οθόνη όπου κάθε κλήση αποτυγχάνει σιωπηλά. Ένα 403 (§4.5.7) αφήνει τα στοιχεία εγγραφής ανέπαφα, οπότε το kiosk συνεχίζει μόλις επανενεργοποιηθεί η συσκευή ή ο ιδιοκτήτης. Το ίδιο token χρησιμοποιεί και το heartbeat: όσο εκτελείται η εφαρμογή, ο AppContainer στέλνει κάθε λεπτό ένα αίτημα PATCH /devices/{id}/ping, που ενημερώνει την ένδειξη τελευταίας επικοινωνίας που βλέπει ο ιδιοκτήτης (§5.3.2). Επειδή το heartbeat συνεχίζεται και όταν κανείς δεν χρησιμοποιεί τη συσκευή, η αποσύνδεση από τον ιδιοκτήτη γίνεται αντιληπτή μέσα σε ένα λεπτό και η συσκευή επιστρέφει στην οθόνη εγγραφής χωρίς να περιμένει τον επόμενο επισκέπτη. Ένα heartbeat που αποτυγχάνει λόγω δικτύου απλώς επαναλαμβάνεται το επόμενο λεπτό.



Εικόνα 8.2: Η οθόνη ρύθμισης της εφαρμογής kiosk.

8.3 Kiosk Mode μέσω Android LockTask API

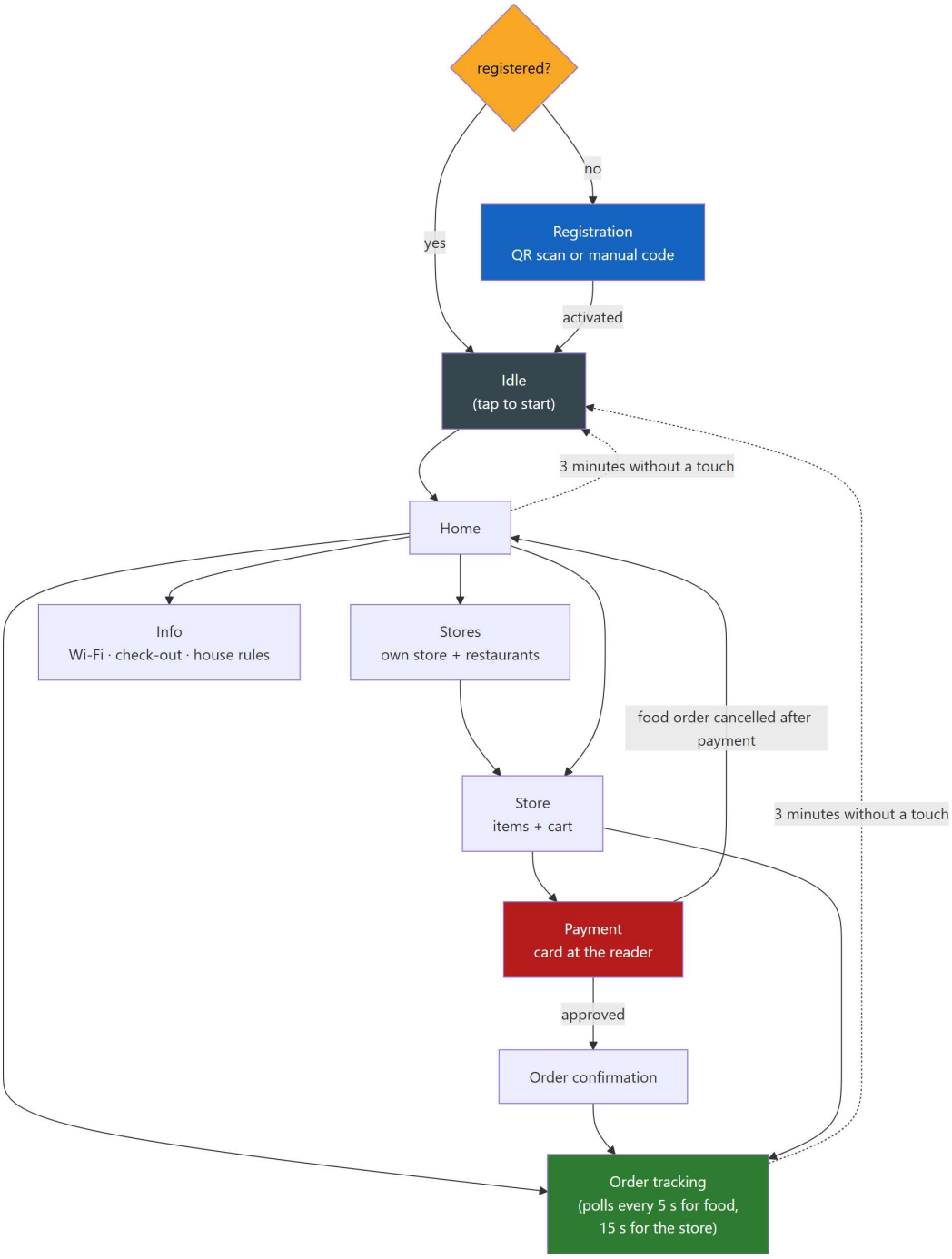
Η απαίτηση της §3.2.5, να μην μπορεί ο φιλοξενούμενος να εξέλθει από την εφαρμογή προς άλλες λειτουργίες της συσκευής, υλοποιείται από τον KioskManager, ένα ελαφρύ περίβλημα γύρω από το Android LockTask API [23]. Όταν η συσκευή είναι καταχωρημένη, το MainActivity καλεί το kioskManager.startKiosk(), που καλεί με τη σειρά του το Activity.startLockTask() του λειτουργικού συστήματος και κλειδώνει τη συσκευή στην τρέχουσα εφαρμογή.

Το πλήρες κλείδωμα προϋποθέτει ότι τη συσκευή διαχειρίζεται μια εφαρμογή διαχείρισης συσκευών (Device Policy Controller, DPC), η οποία έχει οριστεί ως Device Owner και έχει εγκρίνει την εφαρμογή kiosk για λειτουργία LockTask. Η διαμόρφωση αυτή γίνεται μία φορά κατά την αρχική εγκατάσταση της συσκευής και είναι ανεξάρτητη από τον κώδικα της εφαρμογής. Σε μια συσκευή χωρίς τέτοια πολιτική, όπως μια συσκευή ανάπτυξης, το Android είτε εφαρμόζει αντί για το πλήρες κλείδωμα το καρφίτσωμα οθόνης (screen pinning), το οποίο ο χρήστης επιβεβαιώνει και μπορεί να αναιρέσει, είτε απορρίπτει την κλήση· ο KioskManager αγνοεί σκόπιμα την αποτυχία και η εφαρμογή συνεχίζει κανονικά. Έτσι η εφαρμογή λειτουργεί και δοκιμάζεται και σε συσκευές χωρίς πλήρη διαχείριση, ενώ η πλήρης συμπεριφορά ενεργοποιείται αυτόματα όταν μια πραγματική εγκατάσταση κάνει το αντίστοιχο provisioning.

Για την έξοδο από το kiosk mode, για συντήρηση ή επανεγκατάσταση, η επικεφαλίδα των οθονών της εφαρμογής και η οθόνη αδράνειας περιέχουν μια αόρατη περιοχή: πέντε πατήματα μέσα σε τρία δευτερόλεπτα ανοίγουν διάλογο εισαγωγής PIN διαχείρισης, και με τον σωστό κωδικό καλείται το kioskManager.stopKiosk() και η εφαρμογή κλείνει. Στο πρωτότυπο ο κωδικός αυτός είναι σταθερός, για τις ανάγκες της επίδειξης. Επιπλέον, ένας δέκτης συμβάντων συστήματος (BroadcastReceiver) για το BOOT_COMPLETED επιχειρεί να εκκινήσει την εφαρμογή αυτόματα μετά από επανεκκίνηση της συσκευής, ικανοποιώντας την απαίτηση αυτονομίας της §3.2.5. Από το Android 10 και μετά, το λειτουργικό σύστημα περιορίζει την εκκίνηση εφαρμογών από το παρασκήνιο, οπότε η συμπεριφορά αυτή εξαρτάται από την έκδοση και τη διαμόρφωση της συσκευής.

8.4 Ροή Παραγγελίας και Πλοήγηση

Η πλοήγηση υλοποιείται με Jetpack Navigation Compose, με έναν γράφο πλοήγησης (Εικόνα 8.3) και μια sealed class Screen που ορίζει τα διαθέσιμα routes (Registration, Idle, Home, Stores, Store, Payment, OrderConfirmation, OrderTracking, Info). Κάθε οθόνη έχει το δικό της ViewModel, και όλα μοιράζονται κοινή κατάσταση μέσω του AppContainer.



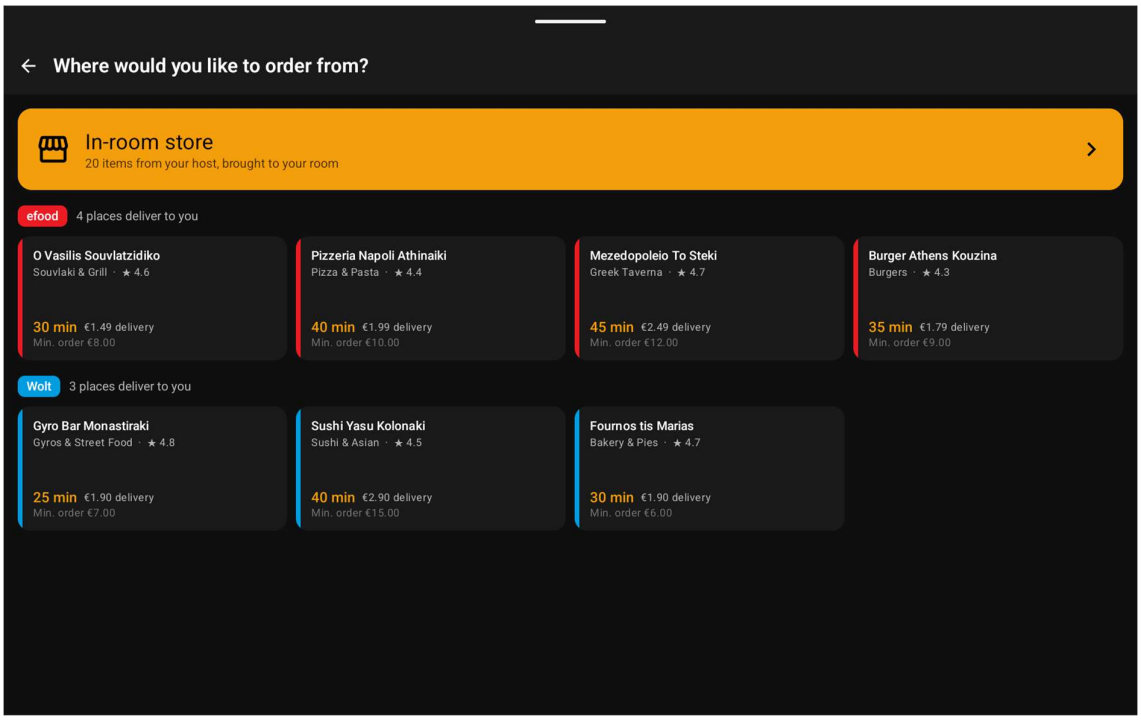
Εικόνα 8.3: Ροή οθονών του επισκέπτη — από την αρχική οθόνη έως την πληρωμή και την παρακολούθηση της παραγγελίας.

Η αρχική οθόνη (Home) προσφέρει στον επισκέπτη τρεις επιλογές: αγορές, πληροφορίες διαμονής και παρακολούθηση των παραγγελιών του. Η επιλογή των αγορών οδηγεί στην οθόνη Stores (Εικόνα 8.4), που δείχνει πρώτα το κατάστημα του καταλύματος, με τον αριθμό των διαθέσιμων προϊόντων, και μετά

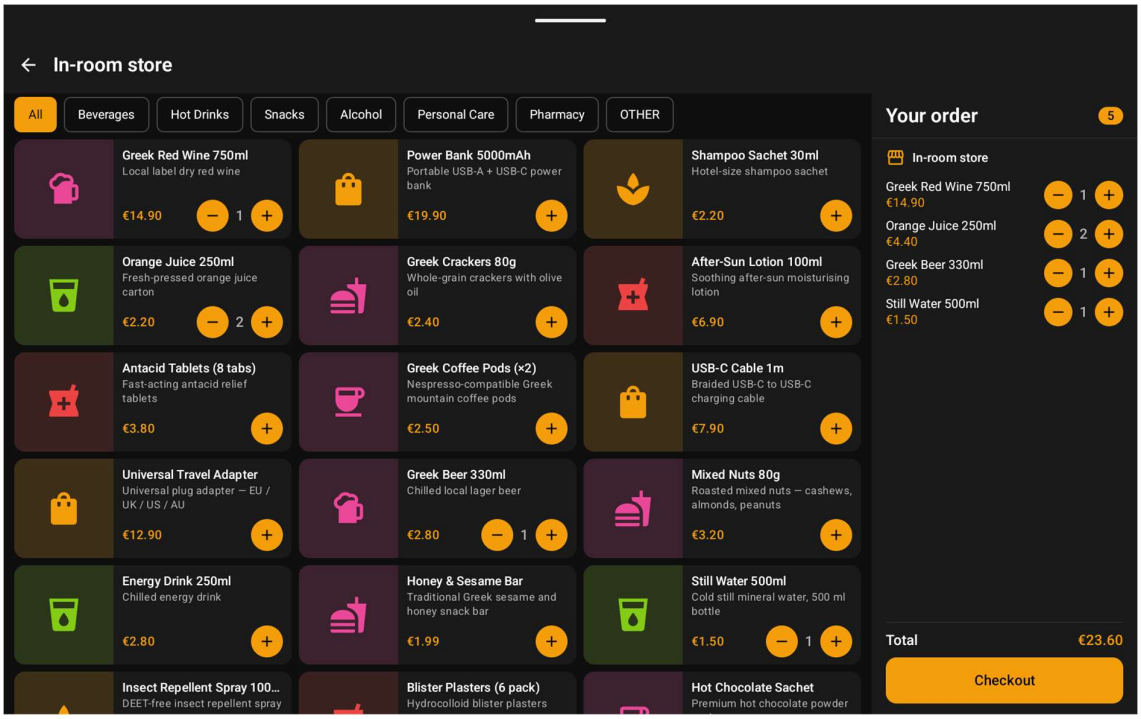
τα εστιατόρια των πλατφορμών διανομής που προσφέρονται στο κατάλυμα, ομαδοποιημένα ανά πλατφόρμα, με το είδος κουζίνας, τη βαθμολογία, το κόστος αποστολής, τον εκτιμώμενο χρόνο παράδοσης και την ελάχιστη παραγγελία κάθε εστιατορίου. Πλατφόρμα χωρίς εστιατόρια για το κατάλυμα δεν εμφανίζεται. Τα εστιατόρια και τα μενού ανακτώνται από το backend (§6.7) και διατηρούνται προσωρινά στη μνήμη, ώστε η επιστροφή σε ένα εστιατόριο να μη χρειάζεται νέα κλήση.

Η οθόνη Store είναι κοινή για το κατάστημα του καταλύματος και για ένα εστιατόριο (Εικόνες 8.5 και 8.6): δείχνει τα είδη ομαδοποιημένα σε κατηγορίες και, δίπλα τους, το καλάθι (CartPanel) με το μερικό σύνολο, το κόστος αποστολής και το τελικό ποσό. Για μια παραγγελία φαγητού, το καλάθι ενημερώνει τον επισκέπτη για το ποσό που υπολείπεται της ελάχιστης παραγγελίας και δεν επιτρέπει την ολοκλήρωση μέχρι να καλυφθεί. Το GuestCart περιέχει είδη ενός μόνο καταστήματος κάθε φορά· αν ο επισκέπτης προσθέσει είδος από άλλο κατάστημα, η εφαρμογή ζητά επιβεβαίωση και, μόνο αν τη δώσει, αδειάζει το καλάθι. Η απλούστευση αυτή οφείλεται στο ότι κάθε παραγγελία πληρώνεται και διεκπεραιώνεται από ένα μόνο μέρος (τον ιδιοκτήτη ή μία πλατφόρμα). Η ποσότητα κάθε είδους περιορίζεται σε 99, όπως και στο backend.

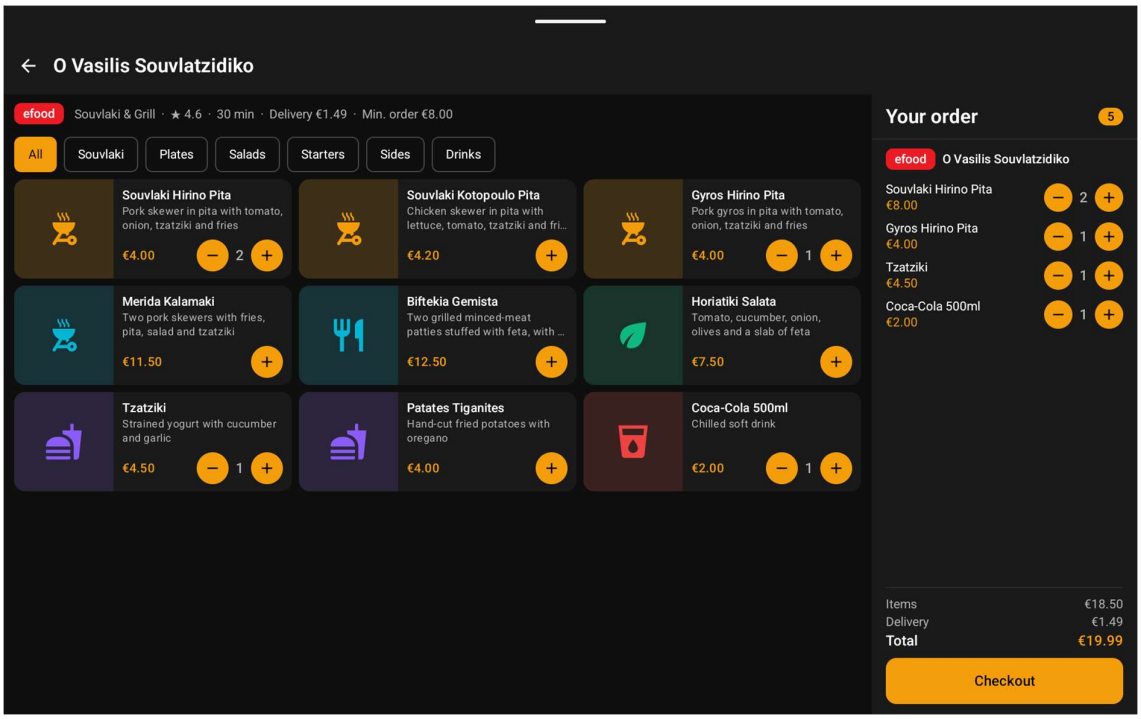
Όταν ολοκληρώσει την επιλογή των ειδών, ο επισκέπτης μεταβαίνει στην οθόνη πληρωμής (§8.5) και, μετά την εγκεκριμένη πληρωμή, στην οθόνη επιβεβαίωσης (Εικόνα 8.9), που συνοψίζει την παραγγελία και οδηγεί στην παρακολούθηση.



Εικόνα 8.4: Επιλογή καταστήματος.



Εικόνα 8.5: Το κατάστημα του καταλύματος με καλάθι.



Εικόνα 8.6: Μενού εστιατορίου με το κόστος αποστολής στο καλάθι.

8.5 Ανάγνωση Κάρτας και Πληρωμή

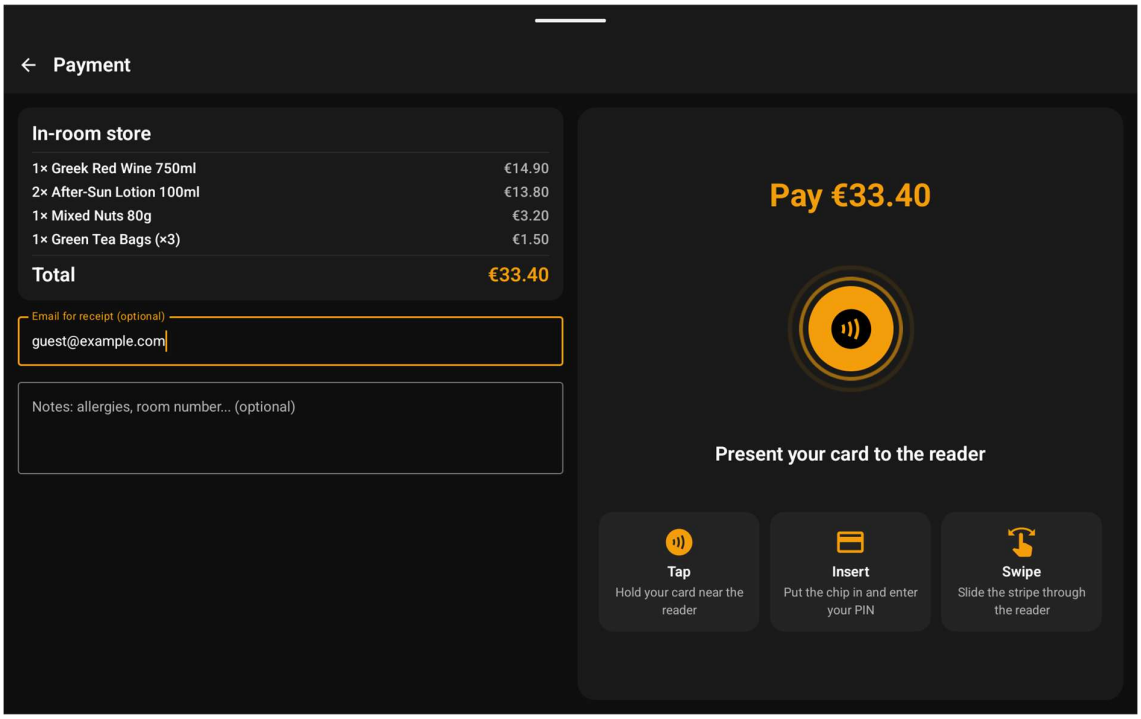
Η οθόνη πληρωμής (Εικόνα 8.7) δεν προσφέρει επιλογή μεθόδου: η πληρωμή ξεκινά όταν ο επισκέπτης χρησιμοποιήσει την κάρτα. Το PaymentViewModel περιμένει πρώτα μια κάρτα από τον αναγνώστη της συσκευής. Με το πρώτο συμβάν κάρτας δημιουργεί την παραγγελία (POST /orders, §6.5), που επιστρέφεται σε κατάσταση PENDING με το ποσό που υπολόγισε ο server, και αμέσως μετά στέλνει το αίτημα πληρωμής. Το αναγνωριστικό της παραγγελίας διατηρείται, ώστε μια νέα απόπειρα μετά από απόρριψη να αφορά την ίδια παραγγελία και να μη δημιουργείται διπλότυπη.

Η πρόσβαση στον αναγνώστη καρτών κρύβεται πίσω από το interface CardTerminal, με την ίδια λογική port/adapter που ακολουθεί το backend. Η μέθοδός του awaitCard() αναστέλλει την εκτέλεση μέχρι ο επισκέπτης να ακουμπήσει, να εισαγάγει ή να περάσει μια κάρτα, και επιστρέφει ένα συμβάν της sealed interface CardEvent: Tapped (ανέπαφη ανάγνωση), Inserted (εισαγωγή chip), Swiped (μαγνητική ταινία, με τα τελευταία τέσσερα ψηφία της κάρτας όταν μπορούν να διαβαστούν), ReadFailed (η κάρτα δεν διαβάστηκε) ή Unavailable (ο αναγνώστης δεν είναι διαθέσιμος). Η υλοποίηση του interface επικοινωνεί, μέσω του μηχανισμού διαδιεργασιακής επικοινωνίας AIDL του Android, με την υπηρεσία αναγνώστη καρτών που παρέχει ο κατασκευαστής της συσκευής. Αν ακυρωθεί η αναμονή, για παράδειγμα επειδή ο επισκέπτης μετέβη σε άλλη οθόνη, ακυρώνεται και το αίτημα προς τον αναγνώστη, ώστε να μη μένει ενεργό στο παρασκήνιο.

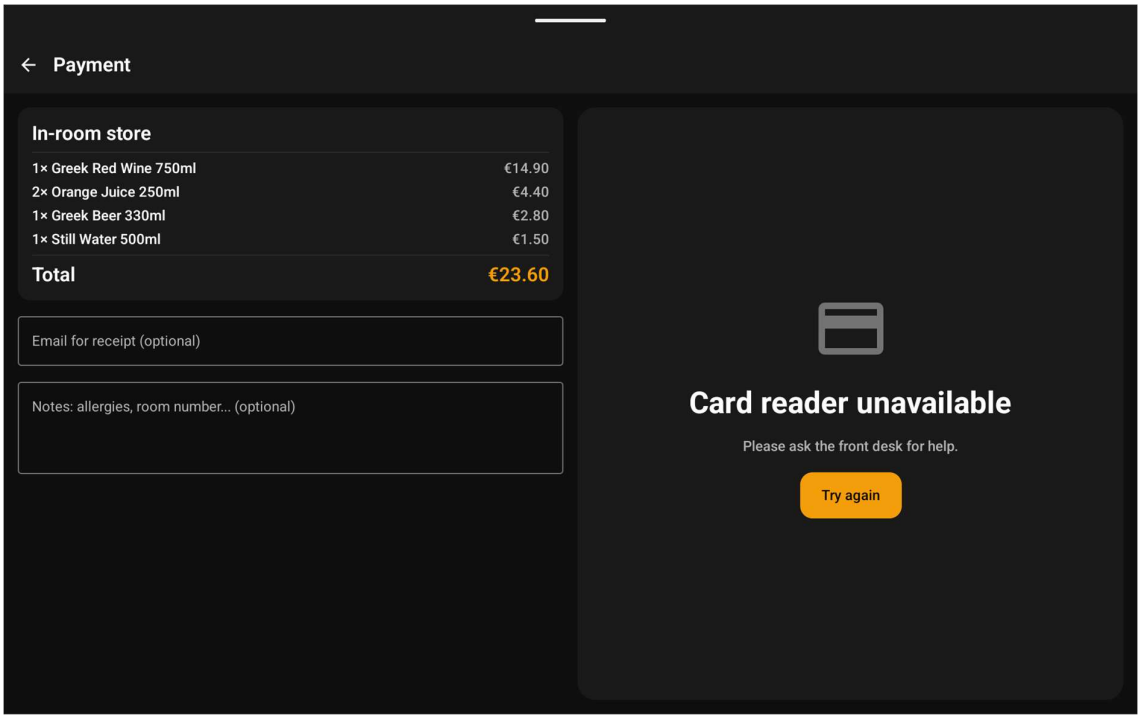
Το ViewModel μετατρέπει κάθε συμβάν σε αίτημα πληρωμής (POST /payments). Για την ανέπαφη ανάγνωση και τη μαγνητική ταινία, το αίτημα αποστέλλεται αμέσως. Η εισαγωγή chip απαιτεί επιπλέον επαλήθευση του κατόχου με PIN, η οποία κανονικά γίνεται στον ασφαλή αναγνώστη με κλειδιά του φορέα πληρωμών· στο πρωτότυπο προσομοιώνεται στην εφαρμογή (§9.4). Το PIN δεν αποθηκεύεται, δεν καταγράφεται και δεν στέλνεται· στο backend φτάνει μόνο το αποτέλεσμα της επαλήθευσης (§3.2.2).

Η απάντηση του backend καθορίζει τη συνέχεια. Η έγκριση ολοκληρώνει την αγορά: το καλάθι αδειάζει, η παραγγελία προστίθεται στις παραγγελίες της τρέχουσας επίσκεψης και η εφαρμογή περνά στην επιβεβαίωση. Η απόρριψη εμφανίζει για λίγα δευτερόλεπτα την αιτία της και η αναμονή κάρτας συνεχίζεται, ώστε ο επισκέπτης να δοκιμάσει ξανά. Μια παραγγελία φαγητού που χρεώθηκε αλλά δεν έγινε δεκτή από την πλατφόρμα επιστρέφεται ως ακυρωμένη, με μήνυμα επιστροφής χρημάτων (§6.6), που η εφαρμογή δείχνει στον επισκέπτη. Ένα τεχνικό σφάλμα (δίκτυο, server) διακρίνεται από την απόρριψη της κάρτας. Επειδή ένα τέτοιο σφάλμα μπορεί να συμβεί αφού η κάρτα έχει ήδη χρεωθεί, για παράδειγμα όταν χάνεται η απάντηση στο δίκτυο ή όταν σε μια επανάληψη του αιτήματος ο server απαντά «ήδη πληρωμένη», το ViewModel ελέγχει πρώτα στον server για την κατάσταση της παραγγελίας: αν είναι πλέον επιβεβαιωμένη, η πληρωμή εμφανίζεται ως εγκεκριμένη και η ροή συνεχίζεται κανονικά, αν έχει ακυρωθεί εμφανίζεται το αντίστοιχο μήνυμα, και μόνο αν παραμένει

απλήρωτη ο επισκέπτης καλείται να δοκιμάσει ξανά. Έτσι ένας επισκέπτης που έχει ήδη χρεωθεί δεν βλέπει μήνυμα αποτυχίας.



Εικόνα 8.7: Η οθόνη πληρωμής σε αναμονή κάρτας.



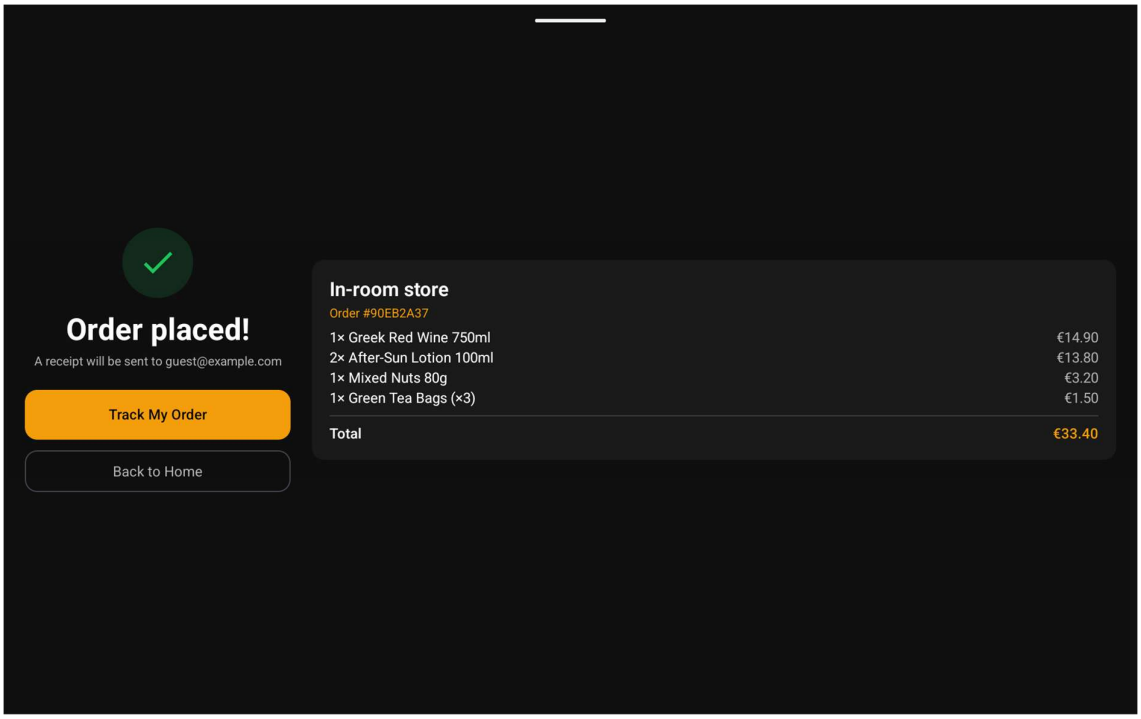
Εικόνα 8.8: Η οθόνη πληρωμής σε συσκευή χωρίς την υπηρεσία αναγνώστη καρτών.

8.6 Παρακολούθηση Κατάστασης Παραγγελίας και Αδράνεια

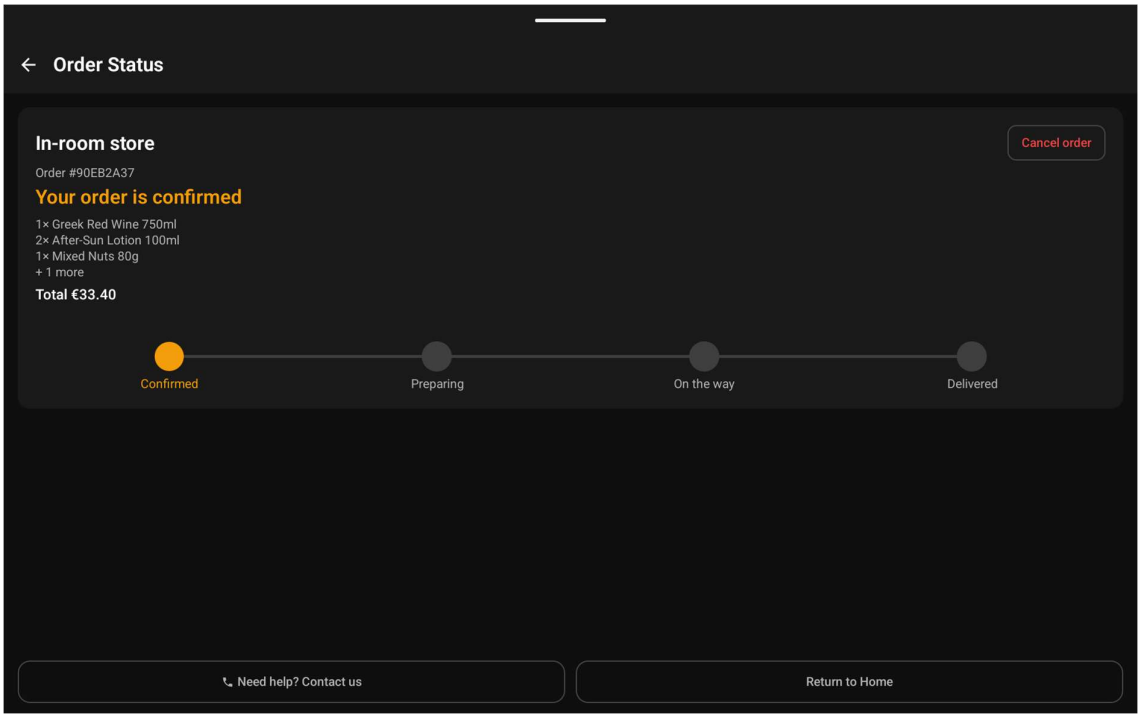
Η οθόνη παρακολούθησης (Εικόνες 8.10 και 8.11) δείχνει όλες τις παραγγελίες της τρέχουσας επίσκεψης ως κάρτες, καθεμία με την κατάστασή της. Το αντίστοιχο ViewModel ανακτά τις παραγγελίες μέσω του GET /device/orders/{id}, κάθε 5 δευτερόλεπτα για τις παραγγελίες φαγητού και κάθε 15 δευτερόλεπτα για τις παραγγελίες του καταστήματος του καταλύματος, μέχρι όλες οι παραγγελίες να φτάσουν σε τελική κατάσταση. Η επιλογή του polling αντί για μηχανισμό push (όπως WebSocket ή Server-Sent Events) ταιριάζει με την απλότητα που επιδιώκεται σε αυτό το στάδιο: σε κάθε έλεγχο η εφαρμογή ανακτά ολόκληρη την παραγγελία και εμφανίζει την τρέχουσα κατάστασή της. Για τις παραγγελίες φαγητού, η κάρτα δείχνει επιπλέον αντίστροφη μέτρηση μέχρι την εκτιμώμενη ώρα άφιξης που ανακοίνωσε η πλατφόρμα (§6.7). Όσο η προετοιμασία μιας παραγγελίας δεν έχει αρχίσει (κατάσταση PENDING ή CONFIRMED), η κάρτα της έχει κουμπί ακύρωσης. Επειδή μια επιβεβαιωμένη παραγγελία είναι ήδη πληρωμένη, η ακύρωση ζητά πρώτα επιβεβαίωση, με διάλογο που αναφέρει το ποσό που θα επιστραφεί, και μετά καλεί το PATCH /device/orders/{id}/cancel· το backend εφαρμόζει τον ίδιο κανόνα (§6.5) και ζητά αυτόματα από τον φορέα πληρωμών την επιστροφή του ποσού (§6.6). Αν η ακύρωση αποτύχει, για παράδειγμα επειδή η προετοιμασία ξεκίνησε στο μεταξύ ή επειδή την αρνήθηκε η πλατφόρμα, ο επισκέπτης ενημερώνεται με μήνυμα.

Ανεξάρτητα από τη ροή της παραγγελίας, ένα περιτύλιγμα αδράνειας (InactivityWrapper) περιβάλλει τις κύριες οθόνες του επισκέπτη (αρχική οθόνη, καταστήματα, κατάλογος, πληρωμή, παρακολούθηση παραγγελίας και πληροφορίες καταλύματος) και παρακολουθεί κάθε άγγιγμα. Αν δεν υπάρξει καμία αλληλεπίδραση για τρία λεπτά, η εφαρμογή επιστρέφει στην οθόνη αδράνειας, που σηματοδοτεί το τέλος της επίσκεψης και σβήνει από τη μνήμη τα δεδομένα του επισκέπτη (§8.1). Πρόκειται για απαίτηση ευχρηστίας και ιδιωτικότητας που προκύπτει από την απρόσκοπτη εμπειρία (frictionless) του φιλοξενούμενου (§3.2.3): μια συσκευή kiosk δεν πρέπει να μένει σε ενδιάμεση οθόνη ούτε να δείχνει τα δεδομένα ενός επισκέπτη στον επόμενο.

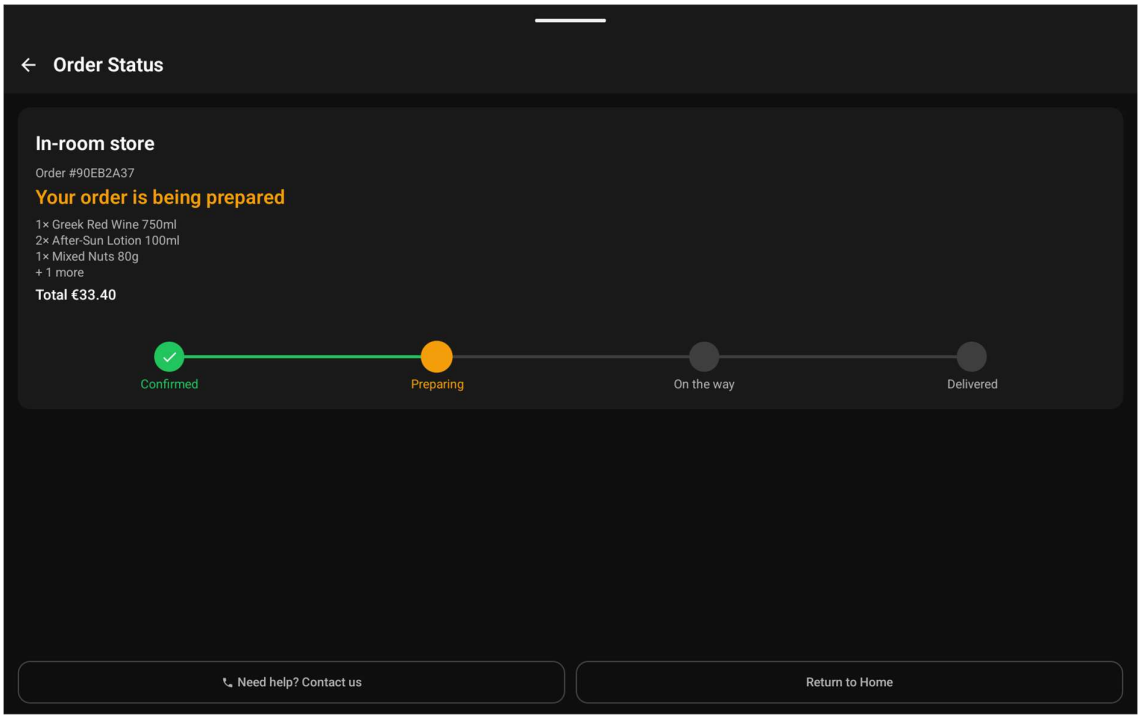
Η οθόνη πληροφοριών (Εικόνα 8.12) δείχνει τα στοιχεία του καταλύματος όπως τα όρισε ο ιδιοκτήτης (§7.6): τη διεύθυνση, το δίκτυο και τον κωδικό Wi-Fi, την ώρα αναχώρησης, τους κανόνες του καταλύματος, το τηλέφωνο του ιδιοκτήτη και τους αριθμούς έκτακτης ανάγκης. Τα στοιχεία ανανεώνονται σε κάθε επίσκεψη στην αρχική οθόνη, ώστε μια αλλαγή του ιδιοκτήτη να εμφανίζεται χωρίς επανεκκίνηση της εφαρμογής. Αν ο κατάλογος δεν μπορεί να φορτωθεί, η αρχική οθόνη ενημερώνει ότι η συσκευή είναι εκτός σύνδεσης και εμφανίζει το τηλέφωνο του ιδιοκτήτη.



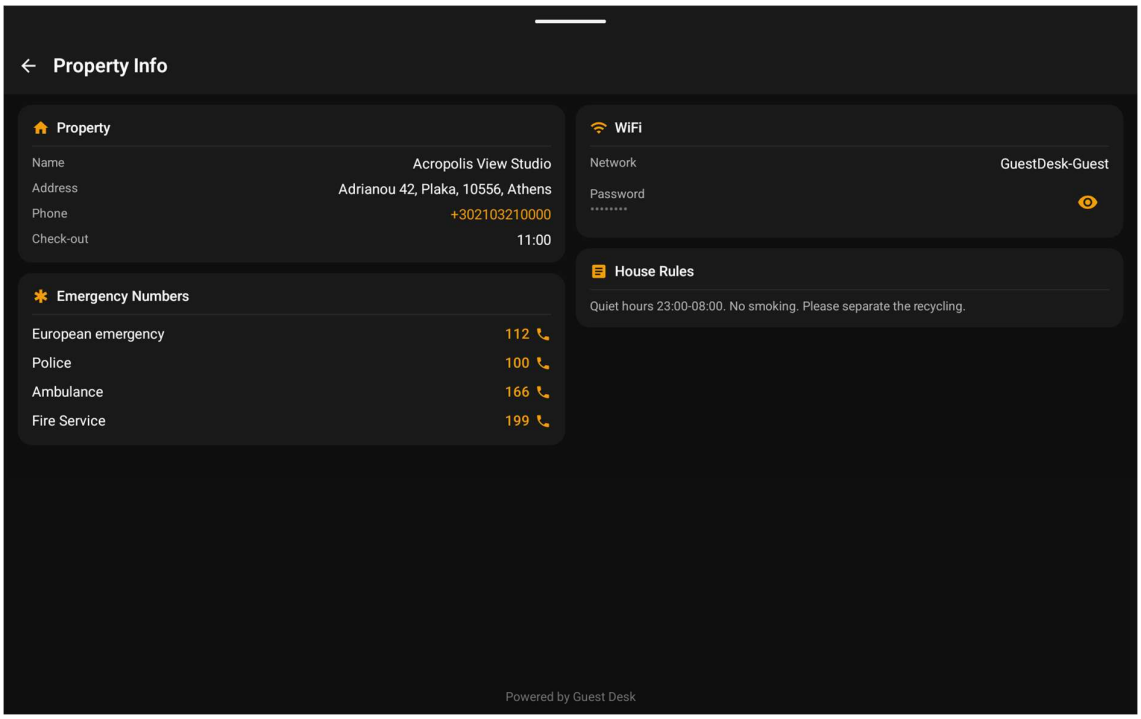
Εικόνα 8.9: Επιβεβαίωση παραγγελίας.



Εικόνα 8.10: Παρακολούθηση παραγγελίας, με το κουμπί ακύρωσης πριν ξεκινήσει η προετοιμασία.



Εικόνα 8.11: Παρακολούθηση παραγγελίας αφού ο ιδιοκτήτης ξεκίνησε την προετοιμασία.



Εικόνα 8.12: Η οθόνη πληροφοριών του καταλύματος.

8.7 Σχεδίαση για Λειτουργία Kiosk

Μια εφαρμογή που τρέχει μόνιμα σε κοινόχρηστη συσκευή και χρησιμοποιείται από ανθρώπους που τη βλέπουν για πρώτη φορά έχει διαφορετικές απαιτήσεις από μια προσωπική εφαρμογή κινητού. Οι απαιτήσεις αυτές καθόρισαν μια σειρά επιλογών στο επίπεδο της πλατφόρμας Android και της διεπαφής.

Σε επίπεδο πλατφόρμας, η εφαρμογή δηλώνεται στο manifest όχι μόνο ως εφαρμογή εκκίνησης αλλά και ως αρχική οθόνη της συσκευής (κατηγορία HOME). Αν οριστεί ως προεπιλεγμένη αρχική οθόνη, το πλήκτρο αρχικής οθόνης επιστρέφει στην ίδια την εφαρμογή αντί για την οθόνη του Android, κάτι που συμπληρώνει το LockTask (§8.3) σε συσκευές χωρίς πλήρη διαχείριση. Η οθόνη είναι κλειδωμένη σε οριζόντιο προσανατολισμό, και οι γραμμές συστήματος (κατάστασης και πλοήγησης) κρύβονται κάθε φορά που η εφαρμογή αποκτά την εστίαση, ώστε ο επισκέπτης να βλέπει μόνο την εφαρμογή· εμφανίζονται προσωρινά μόνο αν ο επισκέπτης σύρει το δάχτυλο από την άκρη της οθόνης. Η εφαρμογή ζητά μόνο τα δικαιώματα που χρειάζεται: πρόσβαση στο δίκτυο, στην κάμερα για τη σάρωση του κωδικού QR κατά την ενεργοποίηση, και λήψη του συμβάντος εκκίνησης της συσκευής.

Στη διεπαφή, τα προϊόντα του καταλύματος και τα πιάτα των εστιατορίων εμφανίζονται με το ίδιο στοιχείο, χάρη σε ένα ενιαίο μοντέλο (BrowseItem) που εκφράζει «ένα είδος που μπορεί να παραγγελθεί», ανεξάρτητα από την προέλευσή του. Έτσι η οθόνη καταστήματος, η κάρτα του είδους και το καλάθι γράφτηκαν μία φορά για δύο πολύ διαφορετικές πηγές δεδομένων (τον κατάλογο του ιδιοκτήτη και το μενού μιας πλατφόρμας). Τα είδη εμφανίζονται σε πλέγμα με προσαρμοζόμενο αριθμό στηλών, ώστε η διάταξη να γεμίζει την οθόνη ανεξάρτητα από το μέγεθος του tablet. Οι εικόνες των προϊόντων φορτώνονται ασύγχρονα από τη διεύθυνσή τους με τη βιβλιοθήκη Coil, και όπου δεν υπάρχει εικόνα ή η φόρτωση αποτύχει εμφανίζεται ένα εικονίδιο της κατηγορίας, ώστε το πλέγμα να μην έχει κενά.

Ορισμένες λεπτομέρειες προέκυψαν από τη φύση της ίδιας της συσκευής. Τα τηλέφωνα της οθόνης πληροφοριών (του ιδιοκτήτη και των υπηρεσιών έκτακτης ανάγκης) ανοίγουν την εφαρμογή κλήσεων όπου υπάρχει· σε tablet χωρίς δυνατότητα κλήσης, ο αριθμός εμφανίζεται σε διάλογο με μεγάλα γράμματα, ώστε ο επισκέπτης να τον καλέσει από το δικό του κινητό. Η διεπαφή είναι στα αγγλικά, ως κοινή γλώσσα για τους διεθνείς επισκέπτες στους οποίους απευθύνεται η βραχυχρόνια μίσθωση· η υποστήριξη πολλών γλωσσών δεν υλοποιήθηκε (§9.4). Τέλος, ο client δικτύου της εφαρμογής αγνοεί τα πεδία μιας απόκρισης που δεν γνωρίζει, ώστε μια νεότερη έκδοση του backend που προσθέτει πεδία να παραμένει συμβατή με τις συσκευές που έχουν ήδη εγκατασταθεί, και κωδικοποιεί κάθε τιμή που ενσωματώνεται σε διεύθυνση URL, αφού τα αναγνωριστικά και οι κωδικοί προέρχονται από κωδικούς QR ή από πληκτρολόγηση.

8.8 Τοπική Αποθήκευση: DataStore και Κρυπτογραφημένα Preferences

Η εφαρμογή χρησιμοποιεί δύο ανεξάρτητους χώρους αποθήκευσης, έναν για κάθε είδος κατάστασης, με επίπεδο ασφάλειας ανάλογο με την ευαισθησία των δεδομένων.

Ο DevicePreferences αποθηκεύει την κατάσταση εγγραφής της συσκευής (αν είναι καταχωρημένη, τη διεύθυνση του server, το αναγνωριστικό της συσκευής, το tenant schema και το κατάλυμα) σε ένα κανονικό preferencesDataStore, με μία εξαίρεση: το token ταυτοποίησης της συσκευής αποθηκεύεται ξεχωριστά, σε EncryptedSharedPreferences της βιβλιοθήκης AndroidX Security Crypto, κρυπτογραφημένο με κλειδί AES-256 που προστατεύεται από ένα MasterKey. Τα υπόλοιπα πεδία δεν είναι ευαίσθητα και ωφελούνται από το reactive API του DataStore, ενώ το token, ως το μοναδικό διαπιστευτήριο της συσκευής, αποθηκεύεται κρυπτογραφημένο στον δίσκο, ώστε η φυσική πρόσβαση σε ένα χαμένο ή κλεμμένο tablet να μην αρκεί από μόνη της για να το εξαγάγει κάποιος.

Ο CartPreferences είναι ένα ανεξάρτητο DataStore που αποθηκεύει, σε μορφή JSON, τα είδη του καταστήματος του καταλύματος που βρίσκονται στο καλάθι, ώστε να διατηρούνται αν η εφαρμογή τερματιστεί απροσδόκητα. Τα πιάτα των εστιατορίων διατηρούνται μόνο στη μνήμη, επειδή οι τιμές και η διαθεσιμότητά τους εξαρτώνται από την πλατφόρμα και δεν έχει νόημα να διατηρηθούν μετά από επανεκκίνηση. Σε κάθε περίπτωση, το τέλος της επίσκεψης (§8.6) αδειάζει το καλάθι.

8.9 Ανεξαρτησία από τον Φορέα Πληρωμών

Η εφαρμογή kiosk δεν επικοινωνεί απευθείας με κανέναν φορέα πληρωμών: ο αναγνώστης καρτών της δίνει μόνο τα συμβάντα της κάρτας, και η απόφαση έγκρισης λαμβάνεται στο backend μέσω του port PaymentGateway (§6.6). Έτσι η σύνδεση με έναν πραγματικό φορέα θα άλλαζε μόνο τον adapter του backend (§4.4), και μια συσκευή με άλλον αναγνώστη καρτών θα χρειαζόταν μόνο μια νέα υλοποίηση του CardTerminal (§8.5), χωρίς αλλαγή στις οθόνες ή στη ροή πληρωμής. Σε συσκευή χωρίς την υπηρεσία αναγνώστη καρτών, η οθόνη πληρωμής ενημερώνει ότι ο αναγνώστης δεν είναι διαθέσιμος (Εικόνα 8.8), οπότε η πληρωμή, και επομένως η ολοκλήρωση μιας παραγγελίας, δεν είναι δυνατή (§9.4).

Κεφάλαιο 9. Ολοκλήρωση Συστήματος και Επίδειξη

Τα προηγούμενα κεφάλαια περιέγραψαν το σύστημα module προς module: τη βάση δεδομένων και τη στρατηγική multi-tenancy (Κεφάλαιο 5), το backend (Κεφάλαιο 6), το web panel διαχείρισης (Κεφάλαιο 7) και την εφαρμογή kiosk (Κεφάλαιο 8), πάνω στα θεμέλια που έθεσε η αρχιτεκτονική του Κεφαλαίου 4. Το κεφάλαιο αυτό συνδέει τα επιμέρους υποσυστήματα σε μία ενιαία εικόνα λειτουργίας από άκρο σε άκρο. Περιγράφει το περιβάλλον ανάπτυξης και επίδειξης, τον τρόπο που επαληθεύτηκε η συνολική ροή και τους περιορισμούς που ισχύουν ακόμη στην τρέχουσα υλοποίηση, ώστε η αξιολόγηση του Κεφαλαίου 10 να στηριχθεί σε συγκεκριμένα στοιχεία.

9.1 Πλήρης Ροή Λειτουργίας

Η πλήρης διαδρομή του συστήματος, από τη ρύθμιση της πλατφόρμας έως την παράδοση μιας παραγγελίας σε έναν φιλοξενούμενο, περνά και από τα τρία modules εφαρμογής (backend, owner-web, app), γύρω από τον κοινό πυρήνα domain/data. Παρακάτω παρουσιάζεται ένα συγκεκριμένο σενάριο βήμα προς βήμα (Εικόνα 9.1), με τα πραγματικά endpoints και τις οντότητες που εμπλέκονται σε κάθε στάδιο.

Το σενάριο ξεκινά με τον Platform Admin, ο οποίος συνδέεται στο owner-web και ορίζει, από την οθόνη ρυθμίσεων, τον διακομιστή ηλεκτρονικού ταχυδρομείου και το κλειδί πρόσβασης του εταιρικού μητρώου (§7.8). Στη συνέχεια δημιουργεί έναν νέο ιδιοκτήτη (Use Case 2, §3.3.3): πληκτρολογεί τον ΑΦΜ της επιχείρησης, το σύστημα συμπληρώνει τα στοιχεία της από το μητρώο και ο διαχειριστής προσθέτει τον υπεύθυνο επικοινωνίας και υποβάλλει το POST /api/owners. Το backend αποθηκεύει την εγγραφή στο public schema, δημιουργεί ένα αποκλειστικό schema (tenant_{id}) με τους εννέα πίνακες του tenant (§5.1, §5.3) και αποστέλλει στον ιδιοκτήτη μήνυμα καλωσορίσματος με τον προσωρινό κωδικό πρόσβασης, ο οποίος αποθηκεύεται μόνο ως BCrypt hash.

Ο ιδιοκτήτης συνδέεται με τον προσωρινό κωδικό και, από την οθόνη Food delivery, ενεργοποιεί τη σύνδεση με τις πλατφόρμες διανομής που χρησιμοποιεί, εισάγοντας τα κλειδιά πρόσβασής τους (§7.7). Έπειτα δημιουργεί το πρώτο του κατάλυμα μέσω POST /api/properties, συμπληρώνοντας τη διεύθυνση μέσω της αναζήτησης διευθύνσεων, τις πληροφορίες για τον επισκέπτη (Wi-Fi, ώρα αναχώρησης, κανόνες) και τις πλατφόρμες που προσφέρονται στο κατάλυμα (§7.6). Ο κατάλογος προϊόντων του καταλύματος δημιουργείται χειροκίνητα ή με εισαγωγή ειδών από τον κατάλογο μιας πλατφόρμας.

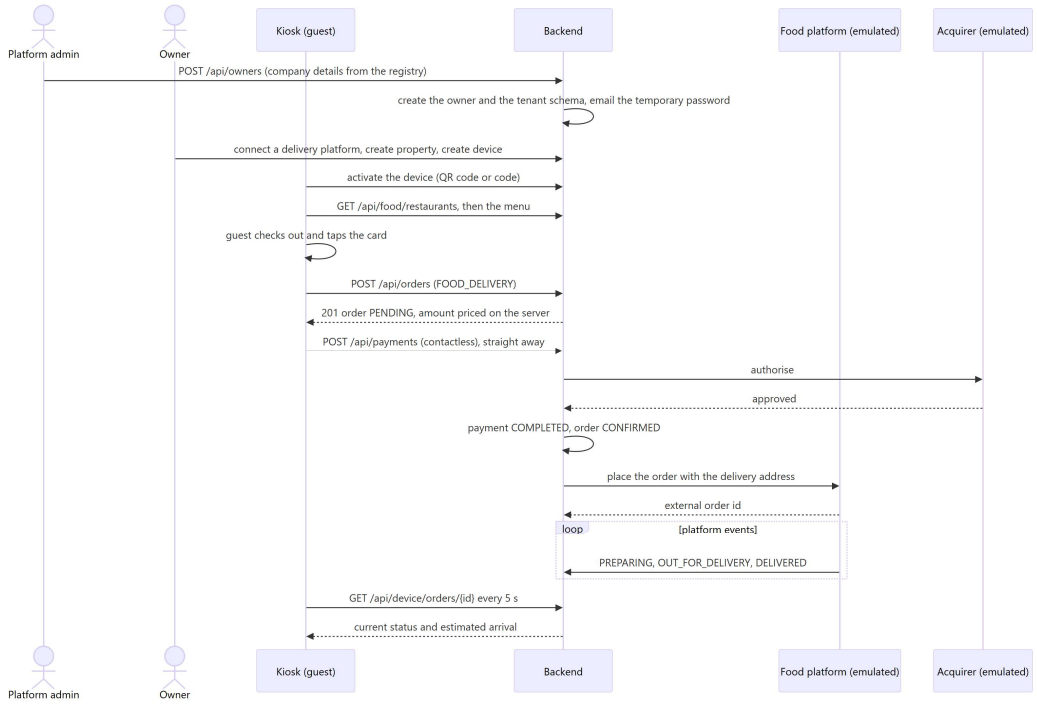
Στη συνέχεια ο ιδιοκτήτης καταχωρεί μια συσκευή kiosk για το κατάλυμα (POST /api/properties/{propertyId}/devices). Το backend παράγει κωδικό ενεργοποίησης και κωδικό QR που περιέχει το αναγνωριστικό της συσκευής, τον κωδικό, το όνομα του tenant schema και τη διεύθυνση του API. Η εφαρμογή kiosk σαρώνει τον κωδικό QR (ή εναλλακτικά καλεί το GET /api/devices/resolve

με τον κωδικό) και καλεί το POST /api/devices/{id}/activate (Use Case 3, §3.3.4, §8.2). Το backend εκδίδει ένα JWT με ρόλο DEVICE (§4.5.7), το οποίο η εφαρμογή αποθηκεύει κρυπτογραφημένο και χρησιμοποιεί σε κάθε επόμενο αίτημα. Η εφαρμογή μεταβαίνει στην οθόνη αδράνειας, έτοιμη να δεχθεί επισκέπτες.

Ο φιλοξενούμενος αγγίζει την οθόνη και ξεκινά τις αγορές του. Η οθόνη καταστημάτων εμφανίζει το κατάστημα του καταλύματος (GET /api/products, φιλτραρισμένο ώστε να εξαιρούνται τα προϊόντα που έχουν απενεργοποιηθεί για το κατάλυμα) και τα εστιατόρια των ενεργοποιημένων πλατφορμών (GET /api/food/restaurants). Ο επισκέπτης διαλέγει ένα εστιατόριο, προσθέτει πιάτα στο καλάθι, ολοκληρώνει την επιλογή των ειδών και ακουμπά την κάρτα του στον αναγνώστη. Η εφαρμογή υποβάλλει τότε το POST /api/orders με τύπο FOOD_DELIVERY και το backend, αφού τιμολογήσει τα πιάτα από το μενού της πλατφόρμας και ελέγξει την ελάχιστη παραγγελία, την αποθηκεύει ως PENDING. Αμέσως μετά η εφαρμογή υποβάλλει το POST /api/payments, ο φορέας πληρωμών εγκρίνει τη συναλλαγή και η παραγγελία γίνεται CONFIRMED. Μόνο τότε το backend υποβάλλει την παραγγελία στην πλατφόρμα, μαζί με τη διεύθυνση του καταλύματος, και αποστέλλει απόδειξη στο email του επισκέπτη, αν δόθηκε (Use Case 5, §3.3.6).

Στη συνέχεια, η πλατφόρμα ενημερώνει το σύστημα για την πρόοδο της παραγγελίας (PREPARING, OUT_FOR_DELIVERY, DELIVERED) μέσω του FoodOrderEventListener (§6.7), και η οθόνη παρακολούθησης της συσκευής εμφανίζει κάθε νέα κατάσταση μαζί με την αντίστροφη μέτρηση έως την εκτιμώμενη ώρα άφιξης. Μια παραγγελία προϊόντων του καταλύματος ακολουθεί την ίδια ροή μέχρι την πληρωμή, αλλά την πρόοδό της την ενημερώνει ο ιδιοκτήτης από το web panel (Use Case 4, §3.3.5). Αν μια πληρωμένη παραγγελία ακυρωθεί, είτε από τον ιδιοκτήτη, είτε από τον επισκέπτη, είτε επειδή η πλατφόρμα δεν την αποδέχθηκε, το σύστημα ζητά αυτόματα από τον φορέα πληρωμών την επιστροφή του ποσού. Οι διαχειριστικές ενέργειες αυτής της αλυσίδας (δημιουργία tenant, σύνδεση πλατφόρμας, δημιουργία συσκευής) και κάθε επιστροφή χρημάτων καταγράφονται στο audit log, ορατό στον Platform Admin (§6.9).

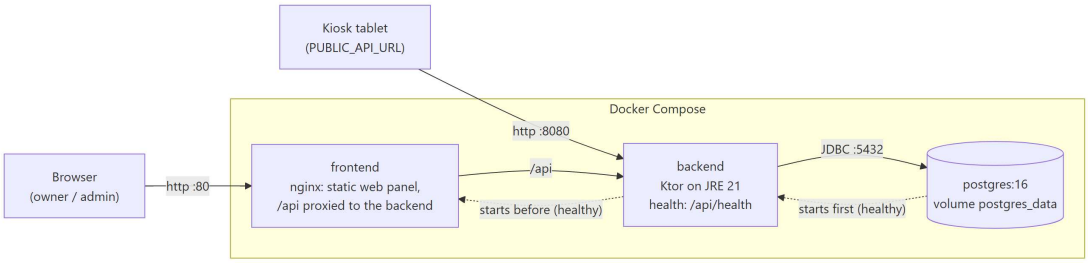
Η αλυσίδα αυτή αποτυπώνει στην πράξη την ιεραρχία πλατφόρμα → ιδιοκτήτης → κατάλυμα → συσκευή → φιλοξενούμενος (§4.3): κάθε επίπεδο εξαρτάται λειτουργικά από το προηγούμενο, και κάθε αίτημα του kiosk καταλήγει αυτόματα στο σωστό tenant schema μέσω του token της συσκευής.



Εικόνα 9.1: Διάγραμμα ακολουθίας (sequence diagram) της πλήρους ροής από άκρο σε άκρο, από τη δημιουργία ιδιοκτήτη έως την παράδοση μιας παραγγελίας φαγητού.

9.2 Περιβάλλον Ανάπτυξης και Επίδειξης

Η ανάπτυξη και η επίδειξη του συστήματος στηρίζονται στο Docker Compose, με τρεις υπηρεσίες (Εικόνα 9.2). Η υπηρεσία postgres (PostgreSQL 16) φιλοξενεί τη βάση δεδομένων, με τα δεδομένα σε μόνιμο volume, και διαθέτει healthcheck βάσει pg_isready· η θύρα της δημοσιεύεται μόνο στη διεπαφή loopback (127.0.0.1:5432), για τα scripts αρχικοποίησης δεδομένων του ίδιου μηχανήματος, και δεν είναι προσβάσιμη από το τοπικό δίκτυο. Η υπηρεσία backend (η διανομή του Ktor server σε JRE 21) εκκινεί μόνο αφού η βάση δεδομένων χαρακτηριστεί υγιής, εκθέτει τη θύρα 8080 και διαθέτει δικό της healthcheck βάσει του endpoint GET /api/health. Η υπηρεσία frontend εξυπηρετεί μέσω nginx το βελτιστοποιημένο bundle παραγωγής (production) wasmJs του owner-web στη θύρα 80, συμπιέζοντας τις αποκρίσεις με gzip, και προωθεί τα αιτήματα /api στο backend (§7.1), εκκινώντας μόνο αφού το backend χαρακτηριστεί υγιές. Οι υπηρεσίες backend και frontend κατασκευάζονται από κοινό multi-stage Dockerfile με διαφορετικό target η καθεμία, χρησιμοποιώντας ως context ολόκληρο το monorepo, άμεση συνέπεια της οργάνωσης σε modules της §4.1. Κατά την πρώτη κατασκευή, το Gradle, το Node.js (για τη μεταγλώττιση σε Wasm) και οι εξαρτήσεις μεταφορτώνονται μέσα στο στάδιο κατασκευής, γι' αυτό η πρώτη εκτέλεση είναι αισθητά πιο αργή από τις επόμενες.



Εικόνα 9.2: Τοπολογία Docker Compose — τρεις υπηρεσίες και οι μεταξύ τους εξαρτήσεις εκκίνησης.

Οι ρυθμίσεις ορίζονται με μεταβλητές περιβάλλοντος. Τα δύο μυστικά του backend, το μυστικό υπογραφής των JWT και η πηγή του κλειδιού κρυπτογράφησης των μυστικών (§4.5.8), είναι υποχρεωτικά: το backend αρνείται να ξεκινήσει χωρίς αυτά, αντί να χρησιμοποιήσει μια προεπιλεγμένη τιμή που θα μπορούσε να μείνει κατά λάθος σε πραγματική εγκατάσταση. Η διαδικασία εγκατάστασης που περιγράφεται παρακάτω τα παράγει στην πρώτη της εκτέλεση ως κρυπτογραφικά τυχαίες τιμές, ώστε τιμή επίδειξης να παραμένει μόνο ο κωδικός της βάσης δεδομένων. Άλλες μεταβλητές ορίζουν τη διεύθυνση του backend όπως είναι προσβάσιμη από τη συσκευή kiosk στο τοπικό δίκτυο, η οποία ενσωματώνεται στους κωδικούς QR, τη διεύθυνση σύνδεσης που περιέχεται στο μήνυμα καλωσορίσματος και το διάστημα μεταξύ των ενημερώσεων των προσομοιωμένων πλατφορμών (§6.7). Οι ρυθμίσεις του ηλεκτρονικού ταχυδρομείου και του εταιρικού μητρώου δεν ορίζονται εδώ, αλλά από το web panel (§7.8).

Για τη φόρτωση ρεαλιστικών δεδομένων επίδειξης στη βάση χρησιμοποιούνται δύο scripts αρχικοποίησης σε Python. Το πρώτο εισάγει απευθείας στη βάση δεδομένων τον πρώτο λογαριασμό Platform Admin, με τον κωδικό πρόσβασης σε μορφή BCrypt hash, όπως τον αποθηκεύει το backend (§4.5.7). Η απευθείας πρόσβαση στη βάση είναι εδώ απαραίτητη, αφού δεν υπάρχει ακόμη χρήστης που να μπορεί να δημιουργήσει τον πρώτο διαχειριστή μέσω του API.

Το δεύτερο script λειτουργεί αντίθετα, κυρίως μέσω του δημόσιου REST API: συνδέεται ως διαχειριστής και δημιουργεί πέντε ιδιοκτήτες-επιχειρήσεις σε διαφορετικές ελληνικές πόλεις (μεταξύ άλλων Αθήνα, Θεσσαλονίκη και Ρόδο), δύο έως πέντε καταλύματα ανά ιδιοκτήτη (δεκαοκτώ συνολικά) και, για κάθε ιδιοκτήτη, έναν κατάλογο είκοσι προϊόντων κοινό για τα καταλύματά του, επιλεγμένων από έναν κατάλογο ρεαλιστικών ειδών (ποτά και ζεστά ροφήματα, σνακ, είδη προσωπικής φροντίδας, φαρμακευτικά, αλκοόλ, αξεσουάρ ταξιδιού)· σε κάθε κατάλυμα τέσσερα από αυτά απενεργοποιούνται μέσω της διαθεσιμότητας ανά κατάλυμα (§5.3.4). Επειδή κάθε εγγραφή περνά από το API, το script δοκιμάζει ταυτόχρονα και τους κανόνες επικύρωσης του backend. Δημιουργεί επίσης έναν ξεχωριστό ιδιοκτήτη δοκιμών με σταθερό κωδικό, χρήσιμο για επαναλαμβανόμενες δοκιμές και για γρήγορη σύνδεση στην επίδειξη. Το script είναι idempotent: ένας ιδιοκτήτης του οποίου το email υπάρχει ήδη παραλείπεται, οπότε μπορεί να εκτελεστεί ξανά οποιαδήποτε στιγμή· διαβάζει απευθείας τη βάση μόνο

για τον έλεγχο αυτό και για να ορίσει τον σταθερό κωδικό του ιδιοκτήτη δοκιμών. Τις πλατφόρμες διανομής δεν τις συνδέει, ώστε η σύνδεση να γίνεται ζωντανά από το web panel κατά την επίδειξη.

Η εγκατάσταση μπορεί επίσης να γίνει ολόκληρη σε ένα βήμα, με ένα script που συνοδεύει τον κώδικα σε εκδόσεις για Windows, Linux και macOS. Το script ελέγχει τις εξαρτήσεις (Docker, Docker Compose και Python· στα Windows προσφέρεται, αφού ρωτήσει, να εγκαταστήσει ό,τι λείπει), δημιουργεί το αρχείο ρυθμίσεων με τα παραγόμενα μυστικά, χτίζει και εκκινεί τα τρία containers, περιμένει έως ότου το backend και το web panel απαντήσουν στους ελέγχους υγείας τους και τέλος εκτελεί τα scripts αρχικοποίησης. Η πρώτη εκτέλεση διαρκεί περίπου τρία έως έξι λεπτά, ανάλογα με το μηχάνημα και τη σύνδεση, κυρίως λόγω της κατασκευής, ενώ οι επόμενες εκτελέσεις, που αξιοποιούν τις κρυφές μνήμες (caches), διαρκούν λιγότερο από ένα λεπτό. Επειδή οι υπάρχουσες ρυθμίσεις διατηρούνται και τα scripts αρχικοποίησης παραλείπουν ό,τι υπάρχει ήδη, η εγκατάσταση μπορεί να επαναληφθεί με ασφάλεια. Οι αναλυτικές οδηγίες εγκατάστασης βρίσκονται στο αποθετήριο του κώδικα.

9.3 Επαλήθευση της Ολοκληρωμένης Ροής

Η λειτουργία του συνόλου επαληθεύτηκε σε ένα πλήρες, απομονωμένο αντίγραφο του περιβάλλοντος Docker Compose, με πραγματική βάση PostgreSQL και δεδομένα από τα scripts της §9.2. Ένα script που καλεί αποκλειστικά το δημόσιο REST API εκτέλεσε τη ροή της §9.1 από την πλευρά της συσκευής, του ιδιοκτήτη και του διαχειριστή, ελέγχοντας σε κάθε βήμα τον κωδικό κατάστασης HTTP και το περιεχόμενο της απάντησης του server. Το script συνοδεύει τον κώδικα, ώστε η επαλήθευση να μπορεί να επαναληφθεί σε νέα εγκατάσταση. Οι 53 έλεγχοι ομαδοποιούνται στον Πίνακα 9.1 και όλοι ολοκληρώθηκαν με το αναμενόμενο αποτέλεσμα. Ιδιαίτερη σημασία έχουν οι αρνητικοί έλεγχοι, που επιβεβαιώνουν ότι μια ενέργεια που δεν πρέπει να επιτραπεί απορρίπτεται με τον σωστό κωδικό, αφού έτσι επαληθεύονται οι κανόνες ασφαλείας και οι επιχειρησιακοί κανόνες των Κεφαλαίων 4 έως 6.

Πίνακας 9.1: Ομάδες ελέγχων της επαλήθευσης της ολοκληρωμένης ροής

Ομάδα	**Τι επαληθεύτηκε**	**Έλεγχοι**
Ταυτοποίηση	Σύνδεση ιδιοκτήτη και διαχειριστή, ανάγνωση των καταλυμάτων του ιδιοκτήτη	3
Ενεργοποίηση και κύκλος ζωής συσκευής	Δημιουργία συσκευής, επίλυση κωδικού, ενεργοποίηση με έκδοση token, ανάγνωση καταλόγου και καταλύματος, heartbeat με ενημέρωση της τελευταίας επικοινωνίας, απόρριψη heartbeat χωρίς token (401) ή με token ιδιοκτήτη (403), απόρριψη του token μετά την απενεργοποίηση (403)	11
Παραγγελία και πληρωμή	Ποσό που υπολογίζει ο server, έγκριση πληρωμής και μετάβαση σε CONFIRMED, απόρριψη δεύτερης πληρωμής (409), απενεργοποίηση	9

Ομάδα	Τι επαληθεύτηκε	Έλεγχοι
	προϊόντος σε κατάλυμα και απόρριψη παραγγελίας που το περιέχει (409), απόρριψη παραγγελίας από λογαριασμό ιδιοκτήτη (403)	
Ακυρώσεις και επιστροφές	Προώθηση σε προετοιμασία από τον ιδιοκτήτη, απόρριψη ακύρωσης από τη συσκευή κατά την προετοιμασία (409), ακύρωση από τον ιδιοκτήτη με επιστροφή χρημάτων, ακύρωση από τη συσκευή πριν από την προετοιμασία με επιστροφή χρημάτων	6
Πλατφόρμες διανομής	Σύνδεση πλατφόρμας, ενεργοποίηση σε κατάλυμα, εστιατόρια και μενού από τη συσκευή, παραγγελία φαγητού και πληρωμή, αυτόνομη πρόοδος έως DELIVERED	8
Φίλτρα λίστας παραγγελιών	Το φίλτρο κατάστασης μαζί με κατάλυμα επιστρέφει μόνο τις παραγγελίες του καταλύματος, άγνωστο πεδίο JSON απορρίπτεται (400)	4
Λογαριασμοί και κωδικοί	Αλλαγή κωδικού με λανθασμένο τρέχοντα κωδικό (400), επιτυχής αλλαγή με νέα tokens, απόρριψη της ανανέωσης των άλλων συνεδριών, σύνδεση με νέο κωδικό, επαναφορά από τον διαχειριστή και σύνδεση με τον προσωρινό κωδικό, απενεργοποίηση ιδιοκτήτη και απόρριψη του access token του στο αμέσως επόμενο αίτημα	9
Ασφάλεια εισόδου	Απόρριψη κακόβουλου ονόματος schema (400) και άκυρου κωδικού ενεργοποίησης, rate limiting στις απόπειρες σύνδεσης (429)	3

Η επαλήθευση αυτή αφορά το backend και το συμβόλαιο του API. Οι διεπαφές του web panel και της εφαρμογής kiosk δεν καλύπτονται από αυτοματοποιημένους ελέγχους, και το σύστημα δεν έχει δοκιμές μονάδας (unit tests)· η συστηματική κάλυψη με αυτοματοποιημένες δοκιμές περιλαμβάνεται στις προτάσεις του Κεφαλαίου 10.

9.4 Γνωστοί Περιορισμοί

Η τρέχουσα υλοποίηση έχει, πέρα από τα όρια εύρους που τέθηκαν εξαρχής στην ανάλυση απαιτήσεων (§3.4), και ορισμένους περιορισμούς, που καταγράφονται παρακάτω.

Ο φορέας πληρωμών και οι πλατφόρμες διανομής φαγητού προσομοιώνονται (§3.4.1): καμία συναλλαγή δεν χρεώνει πραγματικά μια κάρτα και καμία παραγγελία δεν φτάνει σε πραγματικό εστιατόριο. Ο έλεγχος του PIN προσομοιώνεται στην εφαρμογή. Τα συμβάντα της κάρτας προέρχονται από πραγματικό αναγνώστη καρτών, όμως η υλοποίηση του CardTerminal χρειάζεται συσκευή Android με την αντίστοιχη υπηρεσία του κατασκευαστή· σε εξομοιωτή ή σε κοινό tablet ο αναγνώστης δηλώνεται μη διαθέσιμος και η παραγγελία σταματά στην πληρωμή (§8.9).

Το πλήρες κλείδωμα της συσκευής προϋποθέτει τη διαχείρισή της από έναν Device Policy Controller που έχει οριστεί ως Device Owner, διαμόρφωση που δεν έγινε στο πλαίσιο της εργασίας· χωρίς αυτήν η εφαρμογή λειτουργεί κανονικά, αλλά με το πολύ το καρφίτσωμα οθόνης αντί για το πραγματικό κλείδωμα του λειτουργικού συστήματος (§8.3). Ο κωδικός εξόδου από το kiosk mode είναι σταθερός για τις ανάγκες της επίδειξης.

Η εγκατάσταση επίδειξης δεν έχει ρυθμιστεί για πραγματική χρήση. Η επικοινωνία συσκευής, browser και backend γίνεται με απλό HTTP, χωρίς TLS· αυτό είναι αποδεκτό σε ένα κλειστό τοπικό δίκτυο επίδειξης, όχι όμως σε πραγματική εγκατάσταση. Τα μυστικά του backend παράγονται τυχαία κατά την εγκατάσταση, όμως ο κωδικός της βάσης, όπως και οι λογαριασμοί επίδειξης που δημιουργούν τα scripts αρχικοποίησης, είναι τιμή επίδειξης που πρέπει να αντικατασταθεί. Τα tokens του web panel αποθηκεύονται στο localStorage του browser (§7.5). Το backend συνδέεται στη βάση με έναν λογαριασμό για όλους τους tenants, στην εγκατάσταση επίδειξης τον διαχειριστή της βάσης· η απομόνωση των tenants στηρίζεται επομένως στην επιλογή του schema από την εφαρμογή (§4.3) και όχι σε δικαιώματα της βάσης, τα οποία θα πρόσθετε ένας ξεχωριστός ρόλος ανά tenant. Το rate limit των δημόσιων endpoints βασίζεται στη διεύθυνση IP που αναφέρει η κεφαλίδα X-Forwarded-For, την οποία το nginx αντικαθιστά με τη διεύθυνση της σύνδεσης, ώστε ένας client να μην μπορεί να την πλαστογραφήσει, αλλά η οποία είναι αξιόπιστη μόνο όταν το backend είναι προσβάσιμο αποκλειστικά μέσω του proxy· σε μια πραγματική εγκατάσταση η θύρα του backend δεν θα εκτίθεται απευθείας.

Τέλος, ορισμένες επιλογές έγιναν για να μείνει απλή η υλοποίηση: η ενημέρωση της συσκευής και του web panel για την κατάσταση των παραγγελιών γίνεται με polling και όχι με ειδοποιήσεις push, η ενημέρωση της δομής της βάσης γίνεται με idempotent εντολές κατά την εκκίνηση, χωρίς πίνακα εκδόσεων του schema (§5.1), και οι διεπαφές του web panel και της εφαρμογής kiosk υπάρχουν μόνο στα αγγλικά, χωρίς μηχανισμό μετάφρασης.

Οι περιορισμοί αυτοί δεν επηρεάζουν τη βασική ροή της §9.1: η ταυτοποίηση της συσκευής, η δρομολόγηση στο σωστό tenant schema, η τιμολόγηση στον server, η διασύνδεση με τον φορέα πληρωμών, με τους μηχανισμούς μοναδικότητας και επιστροφής, η ενσωμάτωση των πλατφορμών διανομής και η παρακολούθηση των παραγγελιών λειτούργησαν από άκρο σε άκρο, με πραγματικό backend και πραγματική βάση δεδομένων, όπως επαληθεύτηκε στην §9.3.

9.5 Τεχνολογίες, Εκδόσεις και Μέγεθος του Κώδικα

Ο Πίνακας 9.2 συγκεντρώνει τις εκδόσεις των βασικών τεχνολογιών και βιβλιοθηκών με τις οποίες υλοποιήθηκε και επαληθεύτηκε το σύστημα. Οι εκδόσεις δηλώνονται όλες σε έναν κοινό κατάλογο του Gradle (version catalog), ώστε κάθε module να χρησιμοποιεί την ίδια έκδοση κάθε βιβλιοθήκης.

Πίνακας 9.2: Βασικές τεχνολογίες, βιβλιοθήκες και εκδόσεις

Επίπεδο	**Τεχνολογία**	**Έκδοση**
Γλώσσα και κοινός κώδικας	Kotlin, Kotlin Multiplatform	2.4.21
Σειριοποίηση και ασύγχρονη εκτέλεση	kotlinx.serialization, kotlinx.coroutines	1.11.0
Σύστημα δόμησης	Gradle (Kotlin DSL), Android Gradle Plugin	9.8.1, 9.1.0
Περιβάλλον εκτέλεσης backend	JDK 17 (ανάπτυξη), JRE 21 (Docker)	—
Web framework	Ktor (server και client)	3.6.0
Πρόσβαση σε βάση δεδομένων	JetBrains Exposed, HikariCP, PostgreSQL JDBC driver	1.5.1, 7.1.0, 42.7.14
Βάση δεδομένων	PostgreSQL	16
Ασφάλεια	Ktor Auth JWT, BCrypt (at.favre), AES-GCM του JDK	3.6.0, 0.10.2, —
Ηλεκτρονικό ταχυδρομείο	Jakarta Mail	2.0.2
Παραγωγή και σάρωση QR	ZXing, ZXing Android Embedded	3.5.4, 4.3.0
Web panel	Compose Multiplatform (wasmJs)	1.12.0
Εφαρμογή Android	Jetpack Compose (BOM), Navigation Compose, DataStore, Security Crypto, Coil	2026.06.01, 2.9.8, 1.2.1, 1.1.0, 2.7.0
Πλατφόρμα Android	minSdk / compileSdk	28 / 36
Καταγραφή	Logback	1.6.5
Ανάπτυξη και επίδειξη	Docker Compose, nginx	—

Ο Πίνακας 9.3 δείχνει το μέγεθος του πηγαίου κώδικα ανά module, σε αρχεία και γραμμές Kotlin. Συνολικά το σύστημα αποτελείται από 268 αρχεία και περίπου 22.500 γραμμές κώδικα, και το backend εκθέτει 65 endpoints.

Πίνακας 9.3: Μέγεθος του πηγαίου κώδικα ανά module

Module	**Αρχεία Kotlin**	**Γραμμές κώδικα**	**Περιεχόμενο**
:domain	42	523	Οντότητες, enums, repository και port interfaces

Module	**Αρχεία Kotlin**	**Γραμμές κώδικα**	**Περιεχόμενο**
:data	46	637	DTOs αιτημάτων και αποκρίσεων
:backend	79	5.695	Routes, services, repositories, adapters, ασφάλεια
:owner-web	46	9.450	Οθόνες και στοιχεία του web panel
:app	55	6.186	Οθόνες, ViewModels, πληρωμή, kiosk, αποθήκευση
Σύνολο	**268**	**22.491**	

Δύο παρατηρήσεις προκύπτουν από τα μεγέθη αυτά. Πρώτον, ο κοινός κώδικας (:domain και :data) είναι μικρός, περίπου 1.160 γραμμές ή το 5% του συνόλου, και περιέχει μόνο τους ορισμούς που μοιράζονται και τα τρία περιβάλλοντα εκτέλεσης· η αξία του είναι ότι οι ορισμοί αυτοί γράφονται μία φορά. Δεύτερον, ο κώδικας της διεπαφής χρήστη (web panel και εφαρμογή Android) ξεπερνά τα δύο τρίτα του συνόλου, ενώ το backend, που περιέχει το κύριο μέρος της επιχειρησιακής λογικής, αντιστοιχεί περίπου στο ένα τέταρτο. Η αναλογία αυτή είναι αναμενόμενη σε ένα σύστημα με πολλές διαχειριστικές φόρμες και δείχνει πού θα απέδιδε περισσότερο μια ακόμη μεγαλύτερη κοινή χρήση κώδικα (§10.6).

9.6 Διαδικασία Ανάπτυξης

Η ανάπτυξη ξεκίνησε τον Μάρτιο του 2026 και προχώρησε επαναληπτικά, με σύντομους κύκλους τους πρώτους μήνες. Το σύστημα υλοποιήθηκε με την εξής σειρά: έρευνα και επιλογή τεχνολογιών, ανάλυση απαιτήσεων και μοντελοποίηση, επίπεδο βάσης δεδομένων και ταυτοποίηση, επίπεδο υπηρεσιών και REST API, και στη συνέχεια οι οθόνες του web panel για τον ιδιοκτήτη και τον διαχειριστή. Η σειρά αυτή ακολουθεί τη φορά των εξαρτήσεων της §4.1: κάθε module υλοποιήθηκε αφού είχε σταθεροποιηθεί αυτό από το οποίο εξαρτάται. Η εφαρμογή kiosk, οι πλατφόρμες διανομής, η πληρωμή στη συσκευή και οι πραγματικές εξωτερικές υπηρεσίες προστέθηκαν αργότερα, πάνω στον σταθερό πυρήνα.

Το εργαλείο που βοήθησε περισσότερο στην ανάπτυξη του backend, πριν υλοποιηθούν οι διεπαφές χρήστη, ήταν μια συλλογή αιτημάτων Postman με 79 αιτήματα σε δώδεκα ομάδες (από την ταυτοποίηση έως τις πλατφόρμες διανομής, τις ρυθμίσεις και τις αναζητήσεις εταιρειών και

διευθύνσεων). Κάθε αίτημα αποθηκεύει τα στοιχεία της απάντησης (tokens, αναγνωριστικά, όνομα schema) σε μεταβλητές της συλλογής, ώστε τα επόμενα αιτήματα να εκτελούνται με τη σειρά, χωρίς χειροκίνητη αντιγραφή τιμών. Η συλλογή συνοδεύει τον κώδικα και λειτουργεί ταυτόχρονα ως εκτελέσιμη τεκμηρίωση του API.

Παράλληλα με την ανάπτυξη έγιναν συστηματικοί έλεγχοι ποιότητας του κώδικα, για την ασφάλεια, την ορθότητα των συναλλαγών και τη σωστή χρήση των coroutines. Οι έλεγχοι αυτοί εντόπισαν προβλήματα όπως η ευπάθεια στην επιλογή του tenant schema (§6.10), η έλλειψη ατομικότητας στη δημιουργία παραγγελίας και στην ένταξη ιδιοκτήτη, η δυνατότητα δύο ταυτόχρονων πληρωμών για την ίδια παραγγελία και η εσφαλμένη διαχείριση της ακύρωσης των coroutines στο web panel (§7.9), τα οποία διορθώθηκαν και επαληθεύτηκαν πριν ολοκληρωθεί η εργασία. Τα διδάγματα από αυτά συζητούνται στο Κεφάλαιο 10.

Κεφάλαιο 10. Συμπεράσματα και Μελλοντική Εργασία

Το κεφάλαιο αποτιμά την εργασία: αποτιμά την επίτευξη των στόχων της §1.2 (§10.1), συγκρίνει τις απαιτήσεις με τα αποτελέσματα (§10.2), συνοψίζει τα συμπεράσματα για το Kotlin Multiplatform και τα διδάγματα της υλοποίησης (§10.3, §10.4), καταγράφει τα όρια της αξιολόγησης (§10.5) και προτείνει επόμενα βήματα (§10.6).

10.1 Αποτίμηση Ακαδημαϊκών Στόχων

Το ακαδημαϊκό κίνητρο της εργασίας (§1.2.1) στηρίχθηκε σε δύο αρχιτεκτονικές επιλογές: στη χρήση της Kotlin σε όλο το σύστημα μέσω του Kotlin Multiplatform, ώστε ένα backend σε JVM, μια εφαρμογή Android και μια εφαρμογή browser να μοιράζονται κοινό κώδικα, και στη σχεδίαση κατά τις αρχές της Clean Architecture και του προτύπου Port/Adapter, ώστε η ενσωμάτωση και η εναλλαγή εξωτερικών παρόχων να μην επηρεάζουν τον πυρήνα.

Η χρήση της Kotlin σε όλο το σύστημα λειτούργησε όπως σχεδιάστηκε. Τα modules :domain και :data μεταγλωττίζονται από τον ίδιο κώδικα για τους στόχους JVM (backend και εφαρμογή Android) και wasmJs (§4.5.4)· το owner-web δείχνει ότι το Compose Multiplatform για browser είναι στην πράξη επαρκές για μια πλήρη διαχειριστική εφαρμογή (Κεφάλαιο 7)· και η εφαρμογή kiosk χρησιμοποιεί τα ίδια DTOs σε κάθε αίτημα προς το backend (§8.1). Η αλυσίδα backend–owner-web–kiosk λειτουργεί ως ενιαίο σύνολο, με τον ίδιο κώδικα domain και data και στα τρία περιβάλλοντα εκτέλεσης, και η πλήρης ροή επαληθεύτηκε μέσω του API σε ολοκληρωμένο περιβάλλον (§9.3).

Και η σχεδίαση κατά Port/Adapter λειτούργησε όπως αναμενόταν. Τα έξι port interfaces του :domain (§4.4) υλοποιούνται τόσο από adapters που συνδέονται με πραγματικές εξωτερικές υπηρεσίες (ηλεκτρονικό ταχυδρομείο, εταιρικό μητρώο, γεωκωδικοποίηση) όσο και από προσομοιωμένους (§6.6–§6.8). Οι επιχειρησιακές υπηρεσίες δεν γνωρίζουν ποιος adapter βρίσκεται πίσω από κάθε διεπαφή, και η αντικατάσταση ενός adapter αφορά μόνο το σημείο όπου συναρμολογούνται οι εξαρτήσεις (§6.4), χωρίς αλλαγή σε route, service ή domain entity. Το ίδιο πρότυπο εφαρμόστηκε και στην εφαρμογή Android, όπου ο αναγνώστης καρτών κρύβεται πίσω από το interface CardTerminal (§8.5).

Η εκπλήρωση των πρακτικών στόχων ανά ρόλο (§1.2.2) αποτιμάται αναλυτικά στην §10.2.

10.2 Απαιτήσεις έναντι Αποτελεσμάτων ανά Ρόλο

Η σύγκριση των λειτουργικών απαιτήσεων της §3.1 με την τελική υλοποίηση επιτρέπει πιο συγκεκριμένη αποτίμηση από μια γενική δήλωση επιτυχίας. Ο Πίνακας 10.1 συνοψίζει το αποτέλεσμα ανά ρόλο και παραπέμπει στο κεφάλαιο που τεκμηριώνει κάθε ισχυρισμό.

Πίνακας 10.1: Λειτουργικές απαιτήσεις έναντι αποτελεσμάτων ανά ρόλο

Ρόλος	**Απαίτηση**	**Έκβαση**	**Τεκμηρίωση**
Platform Admin	Αυτοματοποιημένη δημιουργία tenant schema κατά την ένταξη ιδιοκτήτη	Ικανοποιημένη	§5.1, §6.4
Platform Admin	Συμπλήρωση εταιρικών στοιχείων από το εταιρικό μητρώο με βάση τον ΑΦΜ	Ικανοποιημένη (πραγματική υπηρεσία)	§6.8, §7.8
Platform Admin	Αποστολή στοιχείων πρόσβασης με email	Ικανοποιημένη (πραγματικό SMTP)	§6.8
Platform Admin	Επαναφορά κωδικού ιδιοκτήτη· αλλαγή κωδικού από τον ίδιο τον χρήστη	Ικανοποιημένη	§6.4, §7.8
Platform Admin	Κεντρικό audit log και εποπτεία στατιστικών όλων των tenants	Ικανοποιημένη	§5.2.4, §7.3
Owner	Διαχείριση καταλυμάτων, συσκευών και καταλόγου προϊόντων από ενιαίο web panel	Ικανοποιημένη	Κεφάλαιο 7
Owner	Εισαγωγή διεύθυνσης μέσω αναζήτησης διευθύνσεων	Ικανοποιημένη (πραγματική υπηρεσία)	§6.8, §7.6
Owner	Ενεργοποίηση συσκευής χωρίς χειροκίνητη παραμετροποίηση (QR ή κωδικός)	Ικανοποιημένη	§8.2
Owner	Παρακολούθηση της τελευταίας επικοινωνίας κάθε συσκευής	Ικανοποιημένη (heartbeat ανά λεπτό)	§5.3.2, §8.2
Owner	Σύνδεση πλατφορμών διανομής και επιλογή ανά κατάλυμα	Ικανοποιημένη (προσομοιωμένες πλατφόρμες)	§6.7, §7.7
Owner	Παρακολούθηση παραγγελιών με μηχανή καταστάσεων (state machine) και ακύρωση με επιστροφή χρημάτων	Ικανοποιημένη	§6.5, §6.6

Ρόλος	**Απαίτηση**	**Έκβαση**	**Τεκμηρίωση**
Guest	Περιήγηση καταλόγου και παραγγελία χωρίς λογαριασμό	Ικανοποιημένη	§8.4
Guest	Παραγγελία φαγητού από εστιατόρια πλατφορμών διανομής	Ικανοποιημένη (προσομοιωμένες πλατφόρμες)	§6.7, §8.4
Guest	Πληρωμή στη συσκευή, χωρίς ανακατεύθυνση	Μερικώς ικανοποιημένη: συμβάντα από πραγματικό αναγνώστη καρτών, προσομοιωμένος φορέας πληρωμών	§8.5, §9.4
Guest	Παρακολούθηση κατάστασης παραγγελίας	Ικανοποιημένη μέσω polling (κάθε 5 δευτ. για το φαγητό, 15 δευτ. για το κατάστημα) αντί για push	§8.6

Οι γραμμές με μερική ή προσομοιωμένη έκβαση αφορούν μόνο τον φορέα πληρωμών και τις πλατφόρμες διανομής, των οποίων η πραγματική σύνδεση απαιτεί εμπορική συμφωνία (§3.4.1). Για τις υπόλοιπες απαιτήσεις, η τεκμηρίωση παραπέμπει στην υλοποιημένη λειτουργικότητα.

10.3 Συμπεράσματα για το Kotlin Multiplatform σε Πρακτική Εφαρμογή

Από την εμπειρία της υλοποίησης προκύπτουν τρία συγκεκριμένα συμπεράσματα για τη χρήση του KMP.

Πρώτον, ο κοινός κώδικας ωφελεί περισσότερο εκεί όπου συνήθως προκύπτουν τα σφάλματα ενσωμάτωσης: στη συνέπεια των DTOs ανάμεσα σε client και server. Το OrderResponse είναι ακριβώς η ίδια κλάση στο backend, στο owner-web και στην εφαρμογή kiosk (§7.1, §8.1), και έτσι αποφεύγεται μια ολόκληρη κατηγορία σφαλμάτων ασυμφωνίας τύπων, που σε ένα σύστημα με διαφορετικές γλώσσες προγραμματισμού θα εμφανιζόταν μόνο κατά την εκτέλεση. Το όφελος έγινε ιδιαίτερα αισθητό κατά την προσθήκη νέων λειτουργιών, όπως οι παραγγελίες φαγητού και η πληρωμή στη συσκευή: κάθε νέο πεδίο σε ένα κοινό DTO το έλεγχε ο μεταγλωττιστής ταυτόχρονα σε όλα τα σημεία όπου χρησιμοποιείται.

Δεύτερον, η ωριμότητα του KMP δεν είναι ίδια για κάθε στόχο μεταγλώττισης. Ο συνδυασμός JVM (backend) και Android (kiosk) είναι πολύ σταθερός, αφού και οι δύο στηρίζονται στην ίδια οικογένεια εργαλείων. Ο στόχος wasmJs, στον οποίο μεταγλωττίζεται το owner-web, είναι αισθητά νεότερος:

αρκούσε για τις ανάγκες της εργασίας, όμως το οικοσύστημα γύρω του (βιβλιοθήκες τρίτων συμβατές με Wasm, εργαλεία αποσφαλμάτωσης στον browser) είναι ακόμη μικρότερο από αυτό ενός καθιερωμένου JavaScript framework, και ορισμένες δυνατότητες του browser, όπως το Geolocation API, χρειάζονται άμεση διασύνδεση με JavaScript. Το κόστος αυτό ήταν γνωστό από νωρίς και διαχειρίσιμο.

Τρίτον, η αρχιτεκτονική πειθαρχία της δομής monorepo (§4.1, §4.2) φάνηκε ιδιαίτερα χρήσιμη, επειδή την επιβάλλει το ίδιο το build system: το :domain δεν μπορεί να αποκτήσει εξάρτηση από κάποιο εξωτερικό framework χωρίς σφάλμα μεταγλώττισης. Η πειθαρχία αυτή δεν αποτρέπει κάθε σφάλμα σχεδίασης, αποτρέπει όμως τη διαρροή αποφάσεων υποδομής στο :domain. Η επιβολή αυτή ισχύει πάντως μόνο στα όρια των modules: οι υπηρεσίες του backend βρίσκονται στο ίδιο module με την υποδομή και χρησιμοποιούν απευθείας τον μηχανισμό συναλλαγών (§4.2), κάτι που ο μεταγλωττιστής δεν εμποδίζει.

10.4 Διδάγματα από την Υλοποίηση

Εκτός από τα συμπεράσματα για την τεχνολογία, από την υλοποίηση προέκυψαν και διδάγματα που αφορούν γενικότερα ένα multi-tenant σύστημα με πολλούς clients. Τα περισσότερα προήλθαν από συγκεκριμένα σφάλματα που εντοπίστηκαν και διορθώθηκαν κατά την ανάπτυξη (§9.6).

Τα σφάλματα ταυτοχρονισμού δεν φαίνονται στις δοκιμές ενός χρήστη. Η αρχική ροή πληρωμής έλεγχε αν μια παραγγελία είχε ήδη πληρωθεί και στη συνέχεια κατέγραφε την πληρωμή σε ξεχωριστή συναλλαγή· όταν στελνόταν ένα αίτημα τη φορά λειτουργούσε σωστά, όμως δύο σχεδόν ταυτόχρονα αιτήματα, όπως μια επανάληψη του kiosk μετά από καθυστέρηση του δικτύου, μπορούσαν να περάσουν και τα δύο τον έλεγχο. Η λύση ήταν να κλειδώνεται η γραμμή της παραγγελίας στην αρχή της συναλλαγής πληρωμής, με έναν μερικό μοναδικό δείκτη της βάσης ως τελευταία δικλίδα, ενώ οι αλλαγές κατάστασης της παραγγελίας προστατεύονται με compare-and-set (§5.6, §6.6). Το συμπέρασμα είναι ότι κάθε ενέργεια που καλεί εξωτερική υπηρεσία ή αλλάζει κατάσταση πρέπει να σχεδιάζεται εξαρχής με την παραδοχή ότι μπορεί να εκτελεστεί ταυτόχρονα με τον εαυτό της.

Ιδιαίτερη προσοχή χρειάζεται και η κατάσταση που διατηρούν οι κοινόχρηστοι πόροι. Μια σύνδεση του connection pool διατηρεί τη ρύθμιση search_path που της δόθηκε και επιστρέφει στο pool, για να εξυπηρετήσει αίτημα άλλου tenant. Μια ρύθμιση που ισχύει για ολόκληρη τη σύνδεση, και όχι μόνο για τη συναλλαγή, μπορεί έτσι να στείλει σιωπηλά ένα ερώτημα στο schema άλλου tenant. Η χρήση του SET LOCAL και η εκτέλεση όλων των εντολών δημιουργίας ενός schema σε μία συναλλαγή (§5.1) περιορίζουν σημαντικά αυτή την κατηγορία σφαλμάτων, και δείχνουν ότι η απομόνωση του schema-per-tenant είναι τόσο ισχυρή όσο ο μηχανισμός που επιλέγει το schema.

Εξίσου σημαντικό είναι να ορίζονται με σαφήνεια τα όρια εμπιστοσύνης. Η ευπάθεια στην επιλογή του schema (§6.10) προήλθε από μια τιμή που ο κώδικας θεωρούσε εσωτερική, το όνομα του schema, ενώ σε ένα δημόσιο endpoint την έστελνε ο client. Η διόρθωση εφαρμόστηκε σε ένα μοναδικό σημείο ελέγχου, από το οποίο περνά υποχρεωτικά κάθε πρόσβαση σε tenant schema, ώστε η ασφάλεια να μη βασίζεται στη σωστή συμπεριφορά κάθε route. Η ίδια λογική εφαρμόστηκε στις τιμές και στην ταυτότητα της συσκευής: ό,τι μπορεί να υπολογιστεί ή να επαληθευτεί στον server δεν γίνεται δεκτό από τον client.

Η προσομοίωση μιας εξωτερικής υπηρεσίας είναι πιο χρήσιμη όταν μιμείται ολόκληρο το συμβόλαιό της, μαζί με τον τρόπο που ειδοποιεί το σύστημα. Οι προσομοιωμένες πλατφόρμες διανομής ενημερώνουν το σύστημα μέσω του ίδιου port (FoodOrderEventListener) που θα χρησιμοποιούσε μια πραγματική πλατφόρμα, οπότε ο πυρήνας χειρίζεται ήδη τα συμβάντα ως ασύγχρονα, πιθανώς καθυστερημένα ή διπλά. Από την άλλη πλευρά, οι πραγματικές υπηρεσίες θέτουν όρους που δεν υπάρχουν σε μια προσομοίωση, όπως το όριο ενός αιτήματος ανά δευτερόλεπτο της υπηρεσίας γεωκωδικοποίησης ή τα όρια αιτημάτων του εταιρικού μητρώου, και η τήρησή τους επηρέασε τόσο τον adapter όσο και τη διεπαφή (§6.8, §7.6).

Τέλος, σφάλματα μπορεί να κρύβονται και στις λεπτομέρειες ενός framework. Ένα από τα σφάλματα που κόστισαν περισσότερο χρόνο αφορούσε τη δομή των routes: όταν τα δημόσια και τα ταυτοποιημένα routes των συσκευών δηλώθηκαν σε δύο ξεχωριστά δέντρα, το Ktor αντιστοίχιζε ένα δημόσιο αίτημα σε ταυτοποιημένο route και απαντούσε 401 (§6.1). Η ρητή δήλωση των routes στο Ktor (§4.5.2) βοήθησε να βρεθεί το σφάλμα, αφού ολόκληρη η δομή φαίνεται στον κώδικα, αλλά δεν το απέτρεψε.

10.5 Όρια της Αξιολόγησης

Η παραπάνω αποτίμηση αφορά ένα σύστημα που αναπτύχθηκε από έναν μόνο προγραμματιστή, σε περιβάλλον επίδειξης με δοκιμαστικά δεδομένα (seed data) και όχι με πραγματική παραγωγική κίνηση, και πρέπει να διαβάζεται μαζί με τους περιορισμούς της υλοποίησης που καταγράφει η §9.4. Δύο παράγοντες περιορίζουν επιπλέον την εμβέλεια των συμπερασμάτων.

Πρώτον, η αρχιτεκτονική Port/Adapter επιτρέπει την αντικατάσταση των προσομοιωμένων υπηρεσιών (§3.4.1) χωρίς αλλαγή στον πυρήνα, δεν αποκλείει όμως ζητήματα ενσωμάτωσης ειδικά για κάθε πάροχο (διαφορετικά πρωτόκολλα, μοντέλα σφαλμάτων, απαιτήσεις πιστοποίησης), τα οποία δεν μπορούν να αξιολογηθούν χωρίς πραγματική σύνδεση.

Δεύτερον, η επαλήθευση της ολοκληρωμένης ροής έγινε μέσω του API (§9.3) και όχι με αυτοματοποιημένες δοκιμές μονάδας ή διεπαφής. Η κλίμακα, για παράδειγμα η ταυτόχρονη λειτουργία εκατοντάδων tenants ή χιλιάδων συσκευών kiosk, δεν δοκιμάστηκε στην πράξη, αν και το schema-per-tenant (§4.5.3) και τα stateless JWT (§4.5.7) θεωρητικά επιτρέπουν οριζόντια κλιμάκωση. Ορισμένες

λειτουργίες έχουν γνωστό γραμμικό κόστος ως προς τον αριθμό των tenants, όπως η επίλυση ενός κωδικού ενεργοποίησης (§6.2) και η ενημέρωση των schemas κατά την εκκίνηση (§5.1).

10.6 Προτάσεις Μελλοντικής Επέκτασης

Εκτός από τις επεκτάσεις που έμειναν εκτός εύρους από την αρχή της εργασίας (§3.4.4) και την επέκταση σε άλλες χώρες (§3.4.3), η εμπειρία της υλοποίησης δείχνει τις ακόλουθες κατευθύνσεις.

Η σύνδεση ενός πραγματικού φορέα πληρωμών, στον οποίο ανήκουν η χρέωση, η έγκριση, η επαλήθευση του PIN και οι επιστροφές, θα ολοκλήρωνε τη ροή πληρωμής (§8.5), χωρίς αλλαγή στον πυρήνα του συστήματος (§4.4).

Με τον ίδιο τρόπο θα μπορούσαν να συνδεθούν οι πραγματικές διεπαφές των πλατφορμών διανομής, όπου αυτές διατίθενται σε συνεργάτες. Οι ειδοποιήσεις κατάστασης θα έφταναν τότε μέσω webhooks στο port FoodOrderEventListener, που έχει ήδη σχεδιαστεί γι' αυτόν τον σκοπό (§6.7).

Ένας μηχανισμός push, όπως WebSocket ή Server-Sent Events, θα μπορούσε να αντικαταστήσει το περιοδικό polling στην ενημέρωση της κατάστασης των παραγγελιών στην εφαρμογή kiosk και στο web panel· έτσι θα μειωνόταν η καθυστέρηση χωρίς αλλαγή στη μηχανή καταστάσεων (state machine, §6.5), που είναι ήδη ανεξάρτητη από τον μηχανισμό μεταφοράς.

Η εγκατάσταση για παραγωγική χρήση χρειάζεται επιπλέον προετοιμασία: επικοινωνία μέσω TLS, έκθεση του backend μόνο μέσω του reverse proxy, ξεχωριστός ρόλος της βάσης δεδομένων ανά tenant, ώστε η απομόνωση να επιβάλλεται και από τα δικαιώματα της βάσης (§9.4), αποθήκευση της συνεδρίας του web panel με μηχανισμό ασφαλέστερο από το localStorage, πίνακας εκδόσεων για την ενημέρωση της δομής της βάσης και συστηματική κάλυψη με δοκιμές μονάδας και ολοκλήρωσης.

Η υποστήριξη πολλών γλωσσών στις διεπαφές θα ωφελούσε ιδιαίτερα την εφαρμογή του επισκέπτη, που απευθύνεται σε διεθνές κοινό.

Η κοινή χρήση κώδικα θα μπορούσε να επεκταθεί και στη διεπαφή: επειδή ο κώδικας της διεπαφής χρήστη είναι το μεγαλύτερο μέρος του συστήματος (§9.5), η μεταφορά της εφαρμογής kiosk σε Compose Multiplatform θα επέτρεπε να μοιράζονται το web panel και η εφαρμογή Android στοιχεία διεπαφής, όπως οι κάρτες προϊόντων και οι ενδείξεις κατάστασης.

10.7 Επίλογος

Η εργασία παρουσίασε τη σχεδίαση και την υλοποίηση ενός ολοκληρωμένου multi-tenant συστήματος ψηφιακών παραγγελιών και υπηρεσιών για καταλύματα βραχυχρόνιας μίσθωσης, που καλύπτει την

ιεραρχία από τον διαχειριστή της πλατφόρμας έως τον φιλοξενούμενο. Το πρωτότυπο δείχνει ότι μία γλώσσα, η Kotlin, μπορεί μέσω του Kotlin Multiplatform να στηρίξει backend, εφαρμογή Android και web panel με κοινό κώδικα, και ότι η διάκριση ανάμεσα στον πυρήνα και στην υποδομή επιτρέπει να συνδέονται πραγματικές και προσομοιωμένες υπηρεσίες πίσω από τις ίδιες διεπαφές. Με τη σύνδεση πραγματικών παρόχων πληρωμών και διανομής και την προετοιμασία της εγκατάστασης για παραγωγική χρήση, το σύστημα θα μπορούσε να αποτελέσει τη βάση μιας υπηρεσίας για πραγματικά καταλύματα.

Βιβλιογραφία

[1] Airbnb, Inc., "Q4 2023 Shareholder Letter," Exhibit 99.1 to Form 8-K, U.S. Securities and Exchange Commission, Feb. 13, 2024. [Διαδικτυακά]. Διαθέσιμο: https://www.sec.gov/Archives/edgar/data/1559720/000119312524033706/d646462dex991.htm

[2] D. Guttentag, "Airbnb: disruptive innovation and the rise of an informal tourism accommodation sector," Current Issues in Tourism, vol. 18, no. 12, pp. 1192–1217, Dec. 2015, doi: 10.1080/13683500.2013.827159.

[3] B. Aldebert, R. J. Dang, and C. Longhi, "Innovation in the tourism industry: The case of Tourism@," Tourism Management, vol. 32, no. 5, pp. 1204–1213, Oct. 2011, doi: 10.1016/j.tourman.2010.08.010.

[4] JetBrains, "The basics of Kotlin Multiplatform project structure," "Expected and actual declarations," and "Kotlin/Wasm," Kotlin Documentation. [Διαδικτυακά]. Διαθέσιμο: https://kotlinlang.org/docs/multiplatform/multiplatform-discover-project.html, https://kotlinlang.org/docs/multiplatform/multiplatform-expect-actual.html και https://kotlinlang.org/docs/wasm-overview.html

[5] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA, USA: Prentice Hall, 2018, ISBN: 978-0-13-449416-6.

[6] R. Krebs, C. Momm, and S. Kounev, "Architectural concerns in multi-tenant SaaS applications," in Proc. 2nd Int. Conf. on Cloud Computing and Services Science (CLOSER), Porto, Portugal, 2012, pp. 426–431, doi: 10.5220/0003957604260431.

[7] A. Cockburn, "Hexagonal architecture," Alistair Cockburn (Blog), 2005. [Διαδικτυακά]. Διαθέσιμο: https://alistair.cockburn.us/hexagonal-architecture/

[8] J. Klensin, "Simple Mail Transfer Protocol," RFC 5321, Oct. 2008, doi: 10.17487/RFC5321.

[9] Κεντρική Υπηρεσία ΓΕΜΗ, Κεντρική Ένωση Επιμελητηρίων Ελλάδος, "ΓΕΜΗ Open Data," ver. 1.0.0. [Διαδικτυακά]. Διαθέσιμο: https://opendata-api.businessportal.gr/opendata/docs/

[10] Nominatim Developers, "Nominatim API Overview," Nominatim 5.3.3 Manual. [Διαδικτυακά]. Διαθέσιμο: https://nominatim.org/release-docs/latest/api/Overview/

[11] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," IEEE Pervasive Computing, vol. 7, no. 4, pp. 12–18, Oct.–Dec. 2008, doi: 10.1109/MPRV.2008.80.

[12] D. Guttentag, S. Smith, L. Potwarka, and M. Havitz, "Why tourists choose Airbnb: A motivation-based segmentation study," Journal of Travel Research, vol. 57, no. 3, pp. 342–359, Mar. 2018, doi: 10.1177/0047287517696980.

[13] F. Chong, G. Carraro, and R. Wolter, "Multi-tenant data architecture," Microsoft Corporation, MSDN Library, Jun. 2006. [Διαδικτυακά]. Διαθέσιμο (αρχειοθετημένο αντίγραφο): https://docs.citusdata.com/en/v8.0/_static/mt-data-arch.pdf

[14] P. Nawrocki, K. Wrona, M. Marczak, and B. Sniezynski, "A comparison of native and cross-platform frameworks for mobile applications," Computer, vol. 54, no. 3, pp. 18–27, Mar. 2021, doi: 10.1109/MC.2020.2983893.

[15] A. Punia, A. Singh, A. Goyal, and A. Arya, "Evaluating Kotlin Multiplatform: Superior cross-platform development," SSRN preprint, 2024, doi: 10.2139/ssrn.4836587.

[16] E. Petrova, "Kotlin Multiplatform Is Stable and Production-Ready," The Kotlin Blog, Nov. 1, 2023. [Διαδικτυακά]. Διαθέσιμο: https://blog.jetbrains.com/kotlin/2023/11/kotlin-multiplatform-stable/

[17] A. Haas et al., "Bringing the web up to speed with WebAssembly," in Proc. 38th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI), Barcelona, Spain, Jun. 2017, pp. 185–200, doi: 10.1145/3062341.3062363.

[18] W. Zhu and C. Morosan, "An empirical examination of guests' adoption of interactive mobile technologies in hotels: Revisiting cognitive absorption, playfulness, and security," Journal of Hospitality and Tourism Technology, vol. 5, no. 1, pp. 78–94, 2014, doi: 10.1108/JHTT-09-2013-0029.

[19] M. Kim and H. Qu, "Travelers' behavioral intention toward hotel self-service kiosks usage," International Journal of Contemporary Hospitality Management, vol. 26, no. 2, pp. 225–245, 2014, doi: 10.1108/IJCHM-09-2012-0165.

[20] N. Provos and D. Mazières, "A future-adaptable password scheme," in Proc. FREENIX Track: 1999 USENIX Annual Technical Conf., Monterey, CA, USA, Jun. 1999, pp. 81–92. [Διαδικτυακά]. Διαθέσιμο: https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf

[21] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015, doi: 10.17487/RFC7519.

[22] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," RFC 9700, Jan. 2025, doi: 10.17487/RFC9700.

[23] Google, "Lock task mode," Android Developers. [Διαδικτυακά]. Διαθέσιμο: https://developer.android.com/work/dpc/dedicated-devices/lock-task-mode

[24] OpenStreetMap Foundation, "Nominatim Usage Policy (aka Geocoding Policy)," OSMF Operations. [Διαδικτυακά]. Διαθέσιμο: https://operations.osmfoundation.org/policies/nominatim/

[25] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, CA, USA, 2000. [Διαδικτυακά]. Διαθέσιμο: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[26] Gradle Inc., "Multi-Project Builds," Gradle Documentation. [Διαδικτυακά]. Διαθέσιμο: https://docs.gradle.org/current/userguide/multi_project_builds.html

[27] Google, "Android's Kotlin-first approach," Android Developers. [Διαδικτυακά]. Διαθέσιμο: https://developer.android.com/kotlin/first

[28] C. Haase, "Google I/O 2019: Empowering developers to build the best experiences on Android + Play," Android Developers Blog, May 7, 2019. [Διαδικτυακά]. Διαθέσιμο: https://android-developers.googleblog.com/2019/05/google-io-2019-empowering-developers-to-build-experiences-on-Android-Play.html

[29] Spring, "Auto-configuration," Spring Boot Reference Documentation. [Διαδικτυακά]. Διαθέσιμο: https://docs.spring.io/spring-boot/reference/using/auto-configuration.html

[30] JetBrains, "Ktor: Build Asynchronous Servers and Clients in Kotlin," Ktor Framework. [Διαδικτυακά]. Διαθέσιμο: https://ktor.io/

[31] JetBrains, "Compose Multiplatform – Beautiful UIs Everywhere," kotlinlang.org. [Διαδικτυακά]. Διαθέσιμο: https://kotlinlang.org/compose-multiplatform/

[32] JetBrains, "Exposed: the SQL library for Kotlin," jetbrains.com. [Διαδικτυακά]. Διαθέσιμο: https://www.jetbrains.com/exposed/

[33] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," Federal Information Processing Standards Publication (FIPS) 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

[34] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, National Institute of Standards and Technology, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.

[35] ZXing Authors, "ZXing ('Zebra Crossing') barcode scanning library for Java, Android," GitHub repository. [Διαδικτυακά]. Διαθέσιμο: https://github.com/zxing/zxing

Παράρτημα Α: Αναφορά API

Το backend εκθέτει ένα REST API σε JSON κάτω από το πρόθεμα `/api`: 65 endpoints, από τα οποία τα 5 είναι δημόσια. Κάθε endpoint ανοίγει με κλικ και δείχνει το σώμα του αιτήματος, την απάντηση και τα σφάλματά του· τα πεδία κάθε τύπου προκύπτουν από τα κοινά DTOs του κώδικα.

Σύμβαση	**Τιμή**
Βασική διεύθυνση	`/api`, π.χ. `http://localhost:8080/api`
Ταυτοποίηση	JWT στην κεφαλίδα `Authorization: Bearer <token>`
Διάρκεια tokens	access token 15 λεπτά, refresh token 30 ημέρες, token συσκευής kiosk 1 έτος
Επιλογή tenant	από το token· ο διαχειριστής της πλατφόρμας, του οποίου το token δεν φέρει schema, δίνει την παράμετρο `schemaName` στα endpoints των tenants
Ποσά και χρόνος	ποσά σε ακέραια λεπτά του ευρώ· χρονοσφραγίδες σε epoch milliseconds
Αυστηρό JSON	άγνωστο πεδίο ή υποχρεωτικό πεδίο που λείπει: 400 Bad Request
Σφάλματα	`{"code": "string", "message": "string"}`· JWT που λείπει ή δεν είναι έγκυρο: σκέτο 401 με κεφαλίδα `WWW-Authenticate`
Token συσκευής	401 όταν η συσκευή είναι άγνωστη ή αποσυνδεδεμένη· 403 όταν η συσκευή ή ο ιδιοκτήτης της είναι απενεργοποιημένοι
Rate limit	login, refresh, επίλυση κωδικού και ενεργοποίηση συσκευής: 20 αιτήματα ανά λεπτό ανά IP· αναζήτηση διευθύνσεων: 30 ανά λεπτό ανά χρήστη

Α.1 Ταυτοποίηση Χρηστών (/auth)

POST /auth/login: Ταυτοποίηση χρήστη πλατφόρμας.

Ρόλοι: Δημόσιο (χωρίς JWT)· με rate limiting

Σώμα αιτήματος: LoginRequest

Απάντηση: 200 OK, TokenResponse

Το προαιρετικό πεδίο `schemaName` δεν χρησιμοποιείται: το schema του ιδιοκτήτη προκύπτει από τον λογαριασμό του.

POST /auth/refresh: Ανανέωση συνεδρίας με εναλλαγή του refresh token (Refresh Token Rotation).

Ρόλοι: Δημόσιο (χωρίς JWT)· με rate limiting

Σώμα αιτήματος: RefreshTokenRequest

Απάντηση: 200 OK, TokenResponse

POST /auth/change-password: Αλλαγή του κωδικού του συνδεδεμένου διαχειριστή ή ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: ChangePasswordRequest

Απάντηση: 200 OK, TokenResponse

Επιστρέφει νέο ζεύγος tokens. Ο νέος κωδικός πρέπει να έχει τουλάχιστον 8 χαρακτήρες και να διαφέρει από τον τρέχοντα.

Σφάλμα 400 Bad Request: Λάθος τρέχων κωδικός (όχι 401).

Σφάλμα 403 Forbidden: Αίτημα με token συσκευής.

Α.2 Διαχείριση Πλατφόρμας και Ιδιοκτητών (/owners, /admin, /company-lookup, /settings)

GET /owners: Ανάκτηση όλων των καταχωρημένων ιδιοκτητών πλατφόρμας.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από OwnerResponse

POST /owners: Εγγραφή νέου ιδιοκτήτη και αυτόματη δημιουργία αποκλειστικού schema βάσης δεδομένων (Tenant Schema Provisioning).

Ρόλοι: PLATFORM_ADMIN

Σώμα αιτήματος: CreateOwnerRequest

Απάντηση: 201 Created, OwnerResponse

Η απάντηση φέρει το `temporaryPassword` του νέου λογαριασμού μόνο σε αυτή την απάντηση.

GET /owners/{id}: Ανάκτηση στοιχείων συγκεκριμένου ιδιοκτήτη βάσει αναγνωριστικού.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, OwnerResponse

PUT /owners/{id}: Ενημέρωση των στοιχείων ιδιοκτήτη (τα ίδια πεδία με το αίτημα δημιουργίας, χωρίς `vatId` και `contactEmail`).

Ρόλοι: PLATFORM_ADMIN

Σώμα αιτήματος: UpdateOwnerRequest

Απάντηση: 200 OK, OwnerResponse

PATCH /owners/{id}/deactivate: Προσωρινή αναστολή του λογαριασμού ιδιοκτήτη (Soft Delete).

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 204 No Content

PATCH /owners/{id}/activate: Επανενεργοποίηση ανασταλμένου λογαριασμού ιδιοκτήτη.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 204 No Content

PATCH /owners/{id}/reset-password: Δημιουργία νέου προσωρινού κωδικού για έναν ιδιοκτήτη.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, OwnerResponse

Η απάντηση φέρει το πεδίο `temporaryPassword`.

GET /admin/owners/stats: Συγκεντρωτικά στατιστικά της πλατφόρμας ανά ιδιοκτήτη.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από OwnerWithStatsResponse

GET /admin/orders/summary: Το πλήθος των ενεργών παραγγελιών και η κατανομή όλων των παραγγελιών της πλατφόρμας ανά κατάσταση.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, AdminOrderSummaryResponse

GET /company-lookup?vatId={vatId}: Αναζήτηση εταιρείας στο εταιρικό μητρώο με εννεαψήφιο ΑΦΜ, για την προσυμπλήρωση της φόρμας ιδιοκτήτη.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, CompanyLookupResponse

Σφάλμα 404 Not Found: Το μητρώο δεν έχει τέτοια εταιρεία.

GET /settings/smtp: Ανάγνωση των ρυθμίσεων εξερχόμενης αλληλογραφίας.

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, SmtpSettingsResponse

Ο κωδικός δεν επιστρέφεται ποτέ, μόνο το `passwordSet`.

PUT /settings/smtp: Αποθήκευση των ρυθμίσεων εξερχόμενης αλληλογραφίας.

Ρόλοι: PLATFORM_ADMIN

Σώμα αιτήματος: UpdateSmtpSettingsRequest

Απάντηση: 200 OK, SmtpSettingsResponse

Αν το `password` λείπει, διατηρείται ο αποθηκευμένος κωδικός.

POST /settings/smtp/test: Αποστολή δοκιμαστικού email με τις αποθηκευμένες ρυθμίσεις.

Ρόλοι: PLATFORM_ADMIN

Σώμα αιτήματος: SendTestEmailRequest

Απάντηση: 204 No Content

GET /settings/gemh: Έλεγχος ύπαρξης αποθηκευμένου κλειδιού API του εταιρικού μητρώου (το ίδιο το κλειδί δεν επιστρέφεται ποτέ).

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, GemhSettingsResponse

PUT /settings/gemh: Αποθήκευση του κλειδιού API του εταιρικού μητρώου (κρυπτογραφημένο).

Ρόλοι: PLATFORM_ADMIN

Σώμα αιτήματος: UpdateGemhSettingsRequest

Απάντηση: 200 OK, GemhSettingsResponse

A.3 Διαχείριση Καταλυμάτων (/properties, /geocode)

GET /properties: Ανάκτηση όλων των καταχωρημένων καταλυμάτων του συνδεδεμένου ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από PropertyResponse

POST /properties: Εγγραφή νέου καταλύματος στο schema του ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: CreatePropertyRequest

Απάντηση: 201 Created, PropertyResponse

Το `checkOutTime` έχει τη μορφή "HH:mm".

GET /properties/{id}: Ανάκτηση στοιχείων συγκεκριμένου καταλύματος.

Ρόλοι: OWNER, PLATFORM_ADMIN, DEVICE

Απάντηση: 200 OK, PropertyResponse

Μια συσκευή διαβάζει μόνο το δικό της κατάλυμα.

PUT /properties/{id}: Ενημέρωση των στοιχείων καταλύματος· το σώμα έχει τα ίδια πεδία με το αίτημα δημιουργίας.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: UpdatePropertyRequest

Απάντηση: 200 OK, PropertyResponse

PATCH /properties/{id}/deactivate: Αναστολή λειτουργίας καταλύματος.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

PATCH /properties/{id}/activate: Επανενεργοποίηση καταλύματος.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

DELETE /properties/{id}: Οριστική διαγραφή καταλύματος.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

Σφάλμα 409 Conflict: Το κατάλυμα έχει παραγγελίες ή συσκευές.

GET /geocode/search?q={text}: Αναζήτηση διεύθυνσης μέσω της δημόσιας υπηρεσίας γεωκωδικοποίησης.

Ρόλοι: OWNER, PLATFORM_ADMIN· με rate limiting

Απάντηση: 200 OK, λίστα από GeocodedAddressResponse

GET /geocode/reverse?lat={lat}&lon={lon}: Εύρεση της διεύθυνσης μιας θέσης στον χάρτη.

Ρόλοι: OWNER, PLATFORM_ADMIN· με rate limiting

Απάντηση: 200 OK, GeocodedAddressResponse

A.4 Διαχείριση Συσκευών Kiosk (/properties/{propertyId}/devices, /devices)

GET /properties/{propertyId}/devices: Λίστα των συσκευών kiosk που είναι συνδεδεμένες στο συγκεκριμένο κατάλυμα.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από DeviceResponse

POST /properties/{propertyId}/devices: Εγγραφή νέας συσκευής και δημιουργία κωδικού ενεργοποίησης (activation code) με τον αντίστοιχο κωδικό QR.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: CreateDeviceRequest

Απάντηση: 201 Created, DeviceResponse

Η απάντηση φέρει τα πεδία `activationCode` και `qrCodeBase64`.

GET /devices/{id}: Ανάκτηση στοιχείων συσκευής βάσει αναγνωριστικού.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, DeviceResponse

DELETE /devices/{id}: Διαγραφή συσκευής kiosk από το κατάλυμα.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

Σφάλμα 409 Conflict: Η συσκευή έχει παραγγελίες.

PATCH /devices/{id}/deactivate: Απενεργοποίηση συσκευής.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

PATCH /devices/{id}/reactivate: Επανενεργοποίηση απενεργοποιημένης συσκευής.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

POST /devices/{id}/unregister: Αποσύνδεση ενεργοποιημένου tablet από τη συσκευή kiosk, ώστε να μπορεί να ενεργοποιηθεί ξανά.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, DeviceResponse

GET /devices/{id}/orders: Ανάκτηση του ιστορικού παραγγελιών που υποβλήθηκαν από τη συγκεκριμένη συσκευή.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από OrderResponse

GET /devices/resolve?code={code}: Αναζήτηση της συσκευής και του tenant schema από κωδικό ενεργοποίησης που πληκτρολογήθηκε (pre-activation lookup).

Ρόλοι: Δημόσιο (χωρίς JWT)· με rate limiting

Απάντηση: 200 OK, DeviceResolveResponse

POST /devices/{id}/activate?schemaName={schemaName}: Ενεργοποίηση tablet kiosk και έκδοση JWT μακράς διάρκειας, με ισχύ ενός έτους.

Ρόλοι: Δημόσιο (χωρίς JWT)· με rate limiting

Σώμα αιτήματος: ActivateDeviceRequest

Απάντηση: 200 OK, DeviceResponse

Η απάντηση φέρει το πεδίο `token`.

Σφάλμα 400 Bad Request: Το `schemaName` λείπει ή δεν είναι έγκυρο.

Σφάλμα 403 Forbidden: Λάθος κωδικός ενεργοποίησης.

Σφάλμα 404 Not Found: Άγνωστη συσκευή.

Σφάλμα 409 Conflict: Η συσκευή είναι ήδη ενεργοποιημένη (και για τη δεύτερη από δύο ταυτόχρονες ενεργοποιήσεις) ή το κατάλυμα ή ο ιδιοκτήτης της είναι απενεργοποιημένοι.

PATCH /devices/{id}/ping: Περιοδικός έλεγχος λειτουργίας (heartbeat) και καταγραφή του `lastSeenAt`.

Ρόλοι: DEVICE

Απάντηση: 204 No Content

Μια συσκευή μπορεί να αναφερθεί μόνο για τον εαυτό της.

Σφάλμα 401 Unauthorized: Η συσκευή έχει αποσυνδεθεί.

Σφάλμα 403 Forbidden: Η συσκευή είναι απενεργοποιημένη.

Α.5 Κατάλογος Προϊόντων και Διαθεσιμότητα (/products, /food-platforms)

GET /products: Ανάκτηση του καταλόγου προϊόντων.

Ρόλοι: OWNER, PLATFORM_ADMIN, DEVICE

Απάντηση: 200 OK, λίστα από ProductResponse

Οι συσκευές kiosk λαμβάνουν αυτόματα μόνο τα προϊόντα που είναι διαθέσιμα στο κατάλυμά τους.

POST /products: Δημιουργία νέου είδους στον κατάλογο του ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: CreateProductRequest

Απάντηση: 201 Created, ProductResponse

GET /products/{id}: Εμφάνιση των στοιχείων συγκεκριμένου προϊόντος.

Ρόλοι: OWNER, PLATFORM_ADMIN, DEVICE

Απάντηση: 200 OK, ProductResponse

PUT /products/{id}: Ενημέρωση των στοιχείων προϊόντος.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: UpdateProductRequest

Απάντηση: 200 OK, ProductResponse

PATCH /products/{id}/availability: Καθολική ενεργοποίηση ή απενεργοποίηση προϊόντος για όλα τα καταλύματα του ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: UpdateProductRequest

Απάντηση: 200 OK, ProductResponse

Το σώμα είναι ίδιο με εκείνο της ενημέρωσης και εφαρμόζεται μόνο το `isAvailable`.

PATCH /products/{id}/properties/{propertyId}/availability: Ειδική ρύθμιση διαθεσιμότητας προϊόντος σε συγκεκριμένο κατάλυμα (Product Property Override).

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: SetProductAvailabilityRequest

Απάντηση: 200 OK, ProductResponse

Το πεδίο `disabledPropertyIds` της απάντησης απαριθμεί τα καταλύματα όπου το προϊόν είναι απενεργοποιημένο.

DELETE /products/{id}: Διαγραφή προϊόντος από τον κατάλογο.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

POST /products/import: Αντιγραφή προϊόντων από τον κατάλογο μιας συνδεδεμένης πλατφόρμας διανομής φαγητού στον κατάλογο του ιδιοκτήτη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: ImportProductsRequest

Απάντηση: 200 OK, ImportProductsResponse

GET /food-platforms: Λίστα των τριών υποστηριζόμενων πλατφορμών διανομής και αν η καθεμία είναι συνδεδεμένη.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από FoodPlatformResponse

PUT /food-platforms/{platform}: Σύνδεση πλατφόρμας με το κλειδί API του ιδιοκτήτη (το κλειδί αποθηκεύεται κρυπτογραφημένο και δεν επιστρέφεται ποτέ).

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: ConnectFoodPlatformRequest

Απάντηση: 200 OK, FoodPlatformResponse

Σφάλμα 400 Bad Request: Η πλατφόρμα απέρριψε το κλειδί.

DELETE /food-platforms/{platform}: Αποσύνδεση πλατφόρμας.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 204 No Content

Σφάλμα 409 Conflict: Η πλατφόρμα έχει παραγγελίες σε εξέλιξη.

GET /food-platforms/{platform}/catalog: Ανάγνωση του καταλόγου προϊόντων μιας συνδεδεμένης πλατφόρμας.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από CatalogProductResponse

GET /properties/{id}/food-platforms: Λίστα των πλατφορμών που είναι ενεργοποιημένες για ένα κατάλυμα.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, PropertyFoodPlatformsResponse

PUT /properties/{id}/food-platforms: Ορισμός των πλατφορμών που είναι ενεργοποιημένες για ένα κατάλυμα.

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: SetPropertyFoodPlatformsRequest

Απάντηση: 200 OK, PropertyFoodPlatformsResponse

Α.6 Διαχείριση Παραγγελιών (/orders, /device/orders, /food)

POST /orders?propertyId={propertyId}: Υποβολή νέας παραγγελίας από το kiosk· το κατάλυμα πρέπει να είναι της ίδιας της συσκευής.

Ρόλοι: DEVICE

Σώμα αιτήματος: CreateOrderRequest

Απάντηση: 201 Created, OrderResponse

Η παραγγελία ξεκινά ως PENDING και επιβεβαιώνεται όταν εγκριθεί η πληρωμή της.

Για παραγγελίες FOOD_DELIVERY τα `platform` και `restaurantId` είναι υποχρεωτικά και το `productId` κάθε είδους είναι το είδος του μενού της πλατφόρμας.

Κάθε ποσότητα πρέπει να είναι από 1 έως 99.

Σφάλμα 400 Bad Request: Παραγγελία φαγητού κάτω από την ελάχιστη παραγγελία του εστιατορίου· `guestEmail` που δεν είναι έγκυρη διεύθυνση ή ξεπερνά τους 255 χαρακτήρες· `notes` άνω των 1.024 χαρακτήρων.

Σφάλμα 403 Forbidden: Η πλατφόρμα δεν είναι ενεργοποιημένη στο κατάλυμα.

Σφάλμα 409 Conflict: Προϊόν μη διαθέσιμο συνολικά ή στο κατάλυμα.

GET /orders?propertyId={propertyId}: Ανάκτηση των παραγγελιών ενός καταλύματος.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από OrderResponse

Προαιρετικό φιλτράρισμα με την παράμετρο `status` (PENDING, CONFIRMED, PREPARING, OUT_FOR_DELIVERY, DELIVERED, CANCELLED).

GET /orders/{id}: Εμφάνιση των αναλυτικών στοιχείων μιας παραγγελίας.

Ρόλοι: OWNER, PLATFORM_ADMIN

Απάντηση: 200 OK, OrderResponse

PATCH /orders/{id}/status: Ενημέρωση της κατάστασης παραγγελίας από τον ιδιοκτήτη, ένα βήμα τη φορά (CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED).

Ρόλοι: OWNER, PLATFORM_ADMIN

Σώμα αιτήματος: UpdateOrderStatusRequest

Απάντηση: 200 OK, OrderResponse

Η ακύρωση (CANCELLED) επιτρέπεται από κάθε μη τελική κατάσταση· μια παραγγελία PENDING μπορεί μόνο να ακυρωθεί, αφού την επιβεβαιώνει η πληρωμή της, και μια παραγγελία φαγητού μόνο να ακυρωθεί, αφού την πρόοδό της την ενημερώνει η πλατφόρμα.

Η ακύρωση μιας παραγγελίας φαγητού απαιτεί να την αποδεχθεί η πλατφόρμα.

Σφάλμα 409 Conflict: Κάθε άλλη μετάβαση· άρνηση της ακύρωσης από την πλατφόρμα (η κατάσταση μένει αμετάβλητη)· πληρωμένη παραγγελία φαγητού που υποβάλλεται ακόμη στην πλατφόρμα της (χωρίς ακόμη αναγνωριστικό πλατφόρμας), για έως 5 λεπτά από την τελευταία ενημέρωσή της: «The order is being placed with the delivery platform, try again in a moment».

GET /device/orders/{id}: Έλεγχος της κατάστασης μιας παραγγελίας από τη συσκευή kiosk που την υπέβαλε.

Ρόλοι: DEVICE

Απάντηση: 200 OK, OrderResponse

PATCH /device/orders/{id}/cancel: Ακύρωση παραγγελίας από τον επισκέπτη μέσω της συσκευής kiosk, μόνο όσο είναι PENDING ή CONFIRMED.

Ρόλοι: DEVICE

Απάντηση: 200 OK, OrderResponse

Σφάλμα 403 Forbidden: Παραγγελία άλλης συσκευής.

Σφάλμα 409 Conflict: Η παραγγελία δεν είναι PENDING ή CONFIRMED· πληρωμένη παραγγελία φαγητού που υποβάλλεται ακόμη στην πλατφόρμα της, όπως στο `PATCH /orders/{id}/status`.

GET /food/restaurants: Λίστα των εστιατορίων των πλατφορμών που είναι ενεργοποιημένες για το κατάλυμα της συσκευής.

Ρόλοι: DEVICE

Απάντηση: 200 OK, λίστα από FoodRestaurantResponse

GET /food/restaurants/{platform}/{restaurantId}/menu: Ανάγνωση του μενού ενός εστιατορίου.

Ρόλοι: DEVICE

Απάντηση: 200 OK, λίστα από FoodMenuItemResponse

Α.7 Πληρωμές και Βοηθητικές Υπηρεσίες (/payments, /dashboard, /audit-logs, /health)

POST /payments: Πληρωμή μιας παραγγελίας που υποβλήθηκε από την καλούσα συσκευή: το backend ζητά έγκριση από τον φορέα πληρωμών και καταγράφει το αποτέλεσμα.

Ρόλοι: DEVICE

Σώμα αιτήματος: InitiatePaymentRequest

Απάντηση: 201 Created, PaymentResponse

Ο αριθμός της κάρτας και το PIN δεν αποστέλλονται· το `pinVerified` αποστέλλεται μόνο για κάρτες chip και το `cardLast4` μόνο όταν το εκθέτει ο αναγνώστης.

Μια πληρωμή που απορρίφθηκε επιστρέφεται επίσης με 201 και φέρει `status` FAILED και `declineReason`.

Για παραγγελία φαγητού που η πλατφόρμα της δεν μπόρεσε να δεχτεί μετά την πληρωμή, η απάντηση φέρει `orderStatus` CANCELLED και `message`, και ζητείται από τον φορέα πληρωμών η επιστροφή του ποσού.

Σφάλμα 409 Conflict: Η παραγγελία έχει ήδη πληρωθεί ή δεν είναι πλέον PENDING.

GET /orders/{orderId}/payments: Ανάκτηση των αποπειρών πληρωμής μιας συγκεκριμένης παραγγελίας.

Ρόλοι: OWNER

Απάντηση: 200 OK, λίστα από PaymentResponse

GET /dashboard/summary: Σύνοψη της δραστηριότητας του ιδιοκτήτη.

Ρόλοι: OWNER

Απάντηση: 200 OK, DashboardSummaryResponse

GET /audit-logs?limit={limit}&offset={offset}: Ανάκτηση του ιστορικού ενεργειών του συστήματος (Audit Trail).

Ρόλοι: PLATFORM_ADMIN

Απάντηση: 200 OK, λίστα από AuditLogResponse

Από προεπιλογή επιστρέφονται οι τελευταίες 50.

GET /health: Έλεγχος υγείας της υπηρεσίας (Health Check).

Ρόλοι: Δημόσιο (χωρίς JWT)

Απάντηση: 200 OK, {"status": "ok"}

Παράρτημα Β: Γλωσσάρι Ξενικών Όρων και Ακρωνυμίων

Ο Πίνακας Β.1 συγκεντρώνει τους ξενικούς όρους που χρησιμοποιούνται στο κείμενο, με την ελληνική τους απόδοση και μια σύντομη επεξήγηση. Ο Πίνακας Β.2 συγκεντρώνει τα ακρωνύμια, με την πλήρη μορφή τους και μια σύντομη επεξήγηση.

Β.1 Ξενικοί όροι

Πίνακας Β.1: Ξενικοί όροι, ελληνική απόδοση και επεξήγηση

Ξενικός Όρος	Ελληνική Απόδοση	Επεξήγηση
Access token	Διακριτικό πρόσβασης	Βραχύβιο token που συνοδεύει κάθε αίτημα και αποδεικνύει την ταυτότητα του χρήστη.
Acquirer	Φορέας πληρωμών (αποδέκτης)	Ο φορέας που επεξεργάζεται και εγκρίνει τις πληρωμές με κάρτα για λογαριασμό του εμπόρου.
Adapter	Προσαρμογέας	Υλοποίηση ενός port για μια συγκεκριμένη τεχνολογία ή εξωτερική υπηρεσία.
Audit log	Αρχείο καταγραφής ενεργειών	Χρονολογική καταγραφή των ενεργειών των χρηστών, για έλεγχο και λογοδοσία.
Backend	Σύστημα παρασκηνίου	Η λογική διακομιστή, οι βάσεις δεδομένων και η επεξεργασία επιχειρησιακών κανόνων.
Browser	Πρόγραμμα περιήγησης	Η εφαρμογή με την οποία ο χρήστης ανοίγει ιστοσελίδες, εδώ και το web panel.
Brute force	Εξαντλητική δοκιμή	Επίθεση που δοκιμάζει διαδοχικά πολλές τιμές, π.χ. κωδικούς, μέχρι να βρει τη σωστή.
Build	Δόμηση	Η διαδικασία που παράγει εκτελέσιμα αρχεία από τον πηγαίο κώδικα.
Bytecode	Ενδιάμεσος κώδικας	Κώδικας για εικονική μηχανή (π.χ. JVM) αντί για τον επεξεργαστή.
Clean Architecture	Καθαρή Αρχιτεκτονική	Αρχιτεκτονικό πρότυπο ομόκεντρων επιπέδων με κανόνα εξάρτησης προς το κέντρο.
Client	Πελάτης (λογισμικό)	Πρόγραμμα που στέλνει αιτήματα σε έναν server, εδώ το web panel και η εφαρμογή kiosk.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Compare-and-set	Σύγκριση και ενημέρωση	Ενημέρωση που εφαρμόζεται μόνο αν η τρέχουσα τιμή είναι η αναμενόμενη, ώστε δύο ταυτόχρονες αλλαγές να μην αλληλοεπικαλύπτονται.
Compiler	Μεταγλωττιστής	Πρόγραμμα που μετατρέπει τον πηγαίο κώδικα σε εκτελέσιμη μορφή και ελέγχει τους τύπους.
Composable	Συνάρτηση διεπαφής (Compose)	Συνάρτηση που περιγράφει ένα τμήμα της διεπαφής στο Jetpack Compose και στο Compose Multiplatform.
Connection pool	Δεξαμενή συνδέσεων	Σύνολο ανοιχτών συνδέσεων με τη βάση που επαναχρησιμοποιούνται από τα αιτήματα.
Container	Περιέκτης	Απομονωμένο περιβάλλον εκτέλεσης μιας εφαρμογής μαζί με τις εξαρτήσεις της (Docker).
Coroutine	Συρρουτίνα	Ελαφριά μονάδα ασύγχρονης εκτέλεσης που αναστέλλεται χωρίς να δεσμεύει νήμα.
Cross-platform	Διαπλατφορμικός	Λογισμικό ή κώδικας που λειτουργεί σε περισσότερες από μία πλατφόρμες.
Dashboard	Πίνακας ελέγχου	Οθόνη που συγκεντρώνει συνοπτικά τα βασικά στοιχεία και μεγέθη.
Declarative UI	Δηλωτική διεπαφή	Η διεπαφή περιγράφεται ως συνάρτηση της κατάστασης, και το framework ενημερώνει την οθόνη όταν αλλάζει η κατάσταση.
Dependency injection	Έγχυση εξαρτήσεων	Οι εξαρτήσεις ενός αντικειμένου παρέχονται εξωτερικά αντί να δημιουργούνται από το ίδιο.
Device Owner	Ιδιοκτήτης συσκευής (Android)	Ειδικό δικαίωμα διαχείρισης του Android που επιτρέπει πλήρη έλεγχο της συσκευής, π.χ. για λειτουργία kiosk.
Domain	Πεδίο εφαρμογής	Οι έννοιες και οι κανόνες του προβλήματος που λύνει το σύστημα, ανεξάρτητα από την τεχνολογία.
Emulated adapter	Προσομοιωμένος προσαρμογέας	Υλοποίηση εξωτερικής υπηρεσίας που αναπαράγει τη λειτουργικότητα τοπικά.
Emulator	Εξομοιωτής	Λογισμικό που αναπαράγει μια συσκευή, π.χ. ένα tablet Android, στον υπολογιστή.
Endpoint	Σημείο πρόσβασης	Ο συνδυασμός μεθόδου HTTP και διαδρομής στον οποίο απαντά το API.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Enum	Απαριθμητός τύπος	Τύπος με προκαθορισμένο σύνολο τιμών, π.χ. οι καταστάσεις μιας παραγγελίας.
Epoch milliseconds	Χιλιοστά δευτερολέπτου από την εποχή Unix	Χρονοσφραγίδα ως αριθμός χιλιοστών του δευτερολέπτου από την 1η Ιανουαρίου 1970 (UTC).
Firewall	Τείχος προστασίας	Λογισμικό ή συσκευή που επιτρέπει ή απορρίπτει συνδέσεις δικτύου βάσει κανόνων.
Foreign key	Ξένο κλειδί	Πεδίο πίνακα που παραπέμπει σε εγγραφή άλλου πίνακα και διατηρεί τη συνέπεια των δεδομένων.
Framework	Πλαίσιο ανάπτυξης	Έτοιμη υποδομή λογισμικού πάνω στην οποία αναπτύσσεται μια εφαρμογή.
Frontend	Σύστημα προσκηνίου	Το τμήμα της εφαρμογής με το οποίο αλληλεπιδρά άμεσα ο τελικός χρήστης.
Garbage collection	Συλλογή απορριμμάτων	Αυτόματη αποδέσμευση της μνήμης που δεν χρησιμοποιείται πλέον.
Hash	Σύνοψη κατακερματισμού	Μονόδρομος μετασχηματισμός δεδομένων σε τιμή σταθερού μήκους· χρησιμοποιείται για την αποθήκευση κωδικών και tokens.
Health check	Έλεγχος λειτουργίας	Endpoint ή εντολή που επιβεβαιώνει ότι μια υπηρεσία λειτουργεί.
Heartbeat (Ping)	Παλμός λειτουργίας	Περιοδικό σήμα ελέγχου διαθεσιμότητας και ενεργής σύνδεσης συσκευής.
Hexagonal Architecture	Εξαγωνική Αρχιτεκτονική	Πρότυπο Ports and Adapters για την απομόνωση του πυρήνα από εξωτερικές εξαρτήσεις.
Idempotent	Ταυτοδύναμος	Λειτουργία που, αν εκτελεστεί ξανά, δεν αλλάζει το αποτέλεσμα.
Kiosk	Σημείο αυτοεξυπηρέτησης	Συσκευή με οθόνη αφής, κλειδωμένη σε μία εφαρμογή, για χρήση από το κοινό.
Kiosk mode	Λειτουργία περιπτέρου	Κλείδωμα συσκευής σε συγκεκριμένη εφαρμογή πλήρους οθόνης χωρίς πρόσβαση στο OS.
LockTask mode	Λειτουργία κλειδώματος εργασίας	Εγγενής μηχανισμός του Android για το κλείδωμα της συσκευής σε μία εφαρμογή.
Log	Αρχείο καταγραφής	Τα μηνύματα που καταγράφει μια εφαρμογή κατά τη λειτουργία της, για διάγνωση προβλημάτων.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Loopback	Διεπαφή επιστροφής	Η διεύθυνση δικτύου του ίδιου του υπολογιστή (localhost), μη προσβάσιμη από άλλες συσκευές.
Middleware	Ενδιάμεσο λογισμικό	Στάδιο επεξεργασίας από το οποίο περνά κάθε αίτημα πριν φτάσει στον κώδικα που το εξυπηρετεί.
Migration	Μετάπτωση σχήματος	Ελεγχόμενη αλλαγή της δομής της βάσης από μία έκδοση στην επόμενη.
Module	Λειτουργική μονάδα	Αυτοτελές τμήμα του έργου με δικό του κώδικα και εξαρτήσεις.
Monorepo	Ενιαίο αποθετήριο	Στρατηγική αποθήκευσης πολλαπλών σχετιζόμενων υποέργων σε ένα κοινό αποθετήριο.
Multi-tenancy	Πολυμίσθωση	Αρχιτεκτονική εξυπηρέτησης πολλαπλών ανεξάρτητων πελατών από μία ενιαία υποδομή.
Nullable	Με δυνατότητα κενής τιμής	Πεδίο ή τύπος που επιτρέπει την τιμή null, δηλαδή την απουσία τιμής.
Onboarding	Ένταξη	Η εγγραφή και η αρχική ρύθμιση ενός νέου ιδιοκτήτη στην πλατφόρμα.
Plaintext	Μη κρυπτογραφημένο κείμενο	Δεδομένα που μεταδίδονται ή αποθηκεύονται χωρίς κρυπτογράφηση.
Plugin	Πρόσθετο	Αυτοτελές τμήμα που προσθέτει μια λειτουργία σε ένα framework ή εργαλείο.
Polling	Περιοδική ερώτηση	Ο client αποστέλλει αιτήματα στον server σε τακτά διαστήματα για να ελέγξει αν υπάρχουν αλλαγές.
Port	Θύρα	Στην αρχιτεκτονική: interface του domain που ορίζει τι χρειάζεται ο πυρήνας από τον έξω κόσμο. Στα δίκτυα: ο αριθμός στον οποίο ακούει μια υπηρεσία.
Race condition	Συνθήκη ανταγωνισμού	Σφάλμα που εμφανίζεται όταν το αποτέλεσμα εξαρτάται από τη σειρά ταυτόχρονων λειτουργιών.
Rate limit (rate limiting)	Όριο ρυθμού αιτημάτων	Όριο στον αριθμό των αιτημάτων που δέχεται ο server ανά χρονικό διάστημα, π.χ. 20 ανά λεπτό· η υπέρβασή του απαντάται με 429 Too Many Requests.
Reflection	Ανάκλαση	Επιθεώρηση και χρήση κλάσεων και μεθόδων κατά την εκτέλεση του προγράμματος.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Refresh token	Διακριτικό ανανέωσης	Μακρόβιο token που χρησιμοποιείται μόνο για την έκδοση νέου access token.
Refresh token rotation	Κυκλική εναλλαγή διακριτικών	Μηχανισμός ασφαλείας όπου κάθε ανανέωση παράγει νέο μοναδικό refresh token.
Replay attack	Επίθεση επανάληψης	Επίθεση που επαναχρησιμοποιεί ένα έγκυρο μήνυμα ή token που υποκλάπηκε.
Repository pattern	Πρότυπο αποθετηρίου	Μοτίβο σχεδίασης που διαμεσολαβεί μεταξύ επιχειρησιακού επιπέδου και πηγών δεδομένων.
Reverse proxy	Αντίστροφος διαμεσολαβητής	Server που δέχεται τα αιτήματα των clients και τα προωθεί σε εσωτερικές υπηρεσίες (εδώ το nginx).
Route	Διαδρομή	Η διεύθυνση (path) του API στην οποία αντιστοιχεί ένα τμήμα κώδικα.
Runtime	Περιβάλλον εκτέλεσης	Το περιβάλλον και η χρονική φάση κατά την οποία εκτελείται ένα πρόγραμμα.
Schema	Σχήμα βάσης	Χώρος ονομάτων μέσα σε μια βάση PostgreSQL με τους δικούς του πίνακες.
Schema-per-tenant	Σχήμα ανά μισθωτή	Μοντέλο απομόνωσης όπου κάθε πελάτης διαθέτει ξεχωριστό schema στην ίδια βάση.
Screen pinning	Καρφίτσωμα οθόνης	Λειτουργία του Android που διατηρεί μία εφαρμογή στην οθόνη μέχρι να την αποδεσμεύσει ο χρήστης.
Script	Σενάριο εντολών	Αρχείο με εντολές που εκτελούνται αυτόματα με τη σειρά.
Seed data	Αρχικά δεδομένα	Δεδομένα επίδειξης που φορτώνονται στη βάση για δοκιμή του συστήματος.
Self check-in	Αυτόνομη άφιξη	Ο επισκέπτης εισέρχεται στο κατάλυμα χωρίς να συναντήσει τον ιδιοκτήτη.
Server	Διακομιστής	Υπολογιστής ή πρόγραμμα που δέχεται και εξυπηρετεί αιτήματα από clients.
Server-Sent Events	Συμβάντα από τον διακομιστή	Μηχανισμός με τον οποίο ο server στέλνει ενημερώσεις στον browser μέσω μιας ανοιχτής σύνδεσης HTTP.
Session	Συνεδρία	Το χρονικό διάστημα κατά το οποίο ένας χρήστης παραμένει συνδεδεμένος.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Shared schema	Κοινό σχήμα	Μοντέλο όπου όλοι οι tenants μοιράζονται τους ίδιους πίνακες και διακρίνονται με μια στήλη αναγνωριστικού.
Soft delete	Λογική διαγραφή	Σημείωση εγγραφής ως ανενεργής χωρίς φυσική διαγραφή της από τον πίνακα.
Source set	Σύνολο πηγαίου κώδικα	Φάκελος κώδικα ενός module για συγκεκριμένη πλατφόρμα, π.χ. commonMain ή jvmMain.
State machine	Μηχανή καταστάσεων	Υπολογιστικό μοντέλο που ελέγχει τις επιτρεπόμενες μεταβάσεις καταστάσεων παραγγελίας.
Stateless	Χωρίς κατάσταση	Αρχιτεκτονική όπου κάθε αίτημα περιέχει όλες τις απαραίτητες πληροφορίες εκτέλεσης.
Tablet	Ταμπλέτα	Φορητή συσκευή με οθόνη αφής· εδώ η συσκευή kiosk μέσα στο κατάλυμα.
Tech stack	Τεχνολογίες του έργου	Το σύνολο των γλωσσών, βιβλιοθηκών και εργαλείων που χρησιμοποιεί ένα έργο.
Tenant	Μισθωτής	Πελάτης της πλατφόρμας με δικά του, απομονωμένα δεδομένα· εδώ κάθε ιδιοκτήτης.
Token	Διακριτικό ασφαλείας	Κρυπτογραφικό σύμβολο που πιστοποιεί την ταυτότητα και τα δικαιώματα ενός χρήστη.
Trade-off	Αντιστάθμισμα	Το κόστος ή ο περιορισμός που συνοδεύει μια σχεδιαστική επιλογή.
Transaction	Συναλλαγή (βάσης δεδομένων)	Σύνολο λειτουργιών στη βάση που εφαρμόζονται είτε όλες είτε καμία.
Type safety	Ασφάλεια τύπων	Ο compiler εντοπίζει ασυμβατότητες τύπων πριν από την εκτέλεση.
Use case	Περίπτωση χρήσης	Περιγραφή μιας αλληλεπίδρασης ενός χρήστη με το σύστημα για την επίτευξη ενός στόχου.
ViewModel	Μοντέλο προβολής	Κλάση που διατηρεί την κατάσταση μιας οθόνης ανεξάρτητα από τη διεπαφή.
Volume	Τόμος δεδομένων (Docker)	Μόνιμος χώρος αποθήκευσης που διατηρείται όταν διαγράφεται το container.
Web panel	Διαδικτυακός πίνακας διαχείρισης	Η εφαρμογή web με την οποία διαχειριστές και ιδιοκτήτες διαχειρίζονται το σύστημα.

Γλωσσάρι Ξενικών Όρων

Ξενικός Όρος	**Ελληνική Απόδοση**	**Επεξήγηση**
Webhook	Ειδοποίηση μέσω HTTP	Αίτημα HTTP που στέλνει μια εξωτερική υπηρεσία όταν συμβεί ένα γεγονός.
WebSocket	Αμφίδρομη σύνδεση web	Πρωτόκολλο για μόνιμη, αμφίδρομη σύνδεση μεταξύ browser και server.

Β.2 Ακρωνύμια

Πίνακας Β.2: Ακρωνύμια, πλήρης μορφή και επεξήγηση

Ακρωνύμιο	Πλήρης Μορφή	Επεξήγηση
AES	Advanced Encryption Standard	Πρότυπο συμμετρικής κρυπτογράφησης· εδώ για τα αποθηκευμένα μυστικά.
AI	Artificial Intelligence	Τεχνητή νοημοσύνη.
AIDL	Android Interface Definition Language	Γλώσσα ορισμού διεπαφών για επικοινωνία μεταξύ εφαρμογών και υπηρεσιών στο Android.
API	Application Programming Interface	Διεπαφή προγραμματισμού εφαρμογών· το σύνολο των endpoints που εκθέτει το backend.
APK	Android Package	Το αρχείο εγκατάστασης μιας εφαρμογής Android.
BOM	Bill of Materials	Κατάλογος συμβατών εκδόσεων για μια ομάδα βιβλιοθηκών.
CLI	Command-Line Interface	Διεπαφή γραμμής εντολών.
CORS	Cross-Origin Resource Sharing	Μηχανισμός του browser που ορίζει ποιοι ιστότοποι μπορούν να καλούν ένα API άλλης προέλευσης (origin).
CSS	Cascading Style Sheets	Γλώσσα μορφοποίησης ιστοσελίδων.
DDL	Data Definition Language	Εντολές SQL που ορίζουν τη δομή της βάσης (CREATE, ALTER).
DEX	Dalvik Executable	Η μορφή bytecode που εκτελεί το Android.
DOM	Document Object Model	Η δενδρική αναπαράσταση μιας ιστοσελίδας στον browser.
DPC	Device Policy Controller	Εφαρμογή διαχείρισης που εφαρμόζει πολιτικές σε μια συσκευή Android, π.χ. τη λειτουργία kiosk.
DSL	Domain-Specific Language	Γλώσσα ειδικού σκοπού, εδώ ενσωματωμένη στην Kotlin (π.χ. Gradle Kotlin DSL, Exposed).
DTO	Data Transfer Object	Αντικείμενο που μεταφέρει δεδομένα μεταξύ client και server.
GCM	Galois/Counter Mode	Τρόπος λειτουργίας του AES που εξασφαλίζει εμπιστευτικότητα και ακεραιότητα.
HTML	HyperText Markup Language	Η γλώσσα σήμανσης των ιστοσελίδων.

Συντομογραφίες & Ακρωνύμια

Ακρωνύμιο	**Πλήρης Μορφή**	**Επεξήγηση**
HTTP / HTTPS	HyperText Transfer Protocol (Secure)	Το πρωτόκολλο επικοινωνίας του web· το HTTPS είναι η κρυπτογραφημένη εκδοχή του μέσω TLS.
I/O	Input/Output	Είσοδος/έξοδος, π.χ. κλήσεις δικτύου και βάσης.
IDE	Integrated Development Environment	Ολοκληρωμένο περιβάλλον ανάπτυξης, π.χ. Android Studio.
IoT	Internet of Things	Διαδίκτυο των πραγμάτων· συνδεδεμένες συσκευές.
IP	Internet Protocol	Το πρωτόκολλο διευθυνσιοδότησης του διαδικτύου· η διεύθυνση IP ταυτοποιεί μια συσκευή στο δίκτυο.
IV	Initialization Vector	Τυχαία τιμή που χρησιμοποιείται σε κάθε κρυπτογράφηση ώστε ίδια δεδομένα να μη δίνουν ίδιο αποτέλεσμα.
JDBC	Java Database Connectivity	Το πρότυπο API της Java για σύνδεση με σχεσιακές βάσεις.
JDK / JRE	Java Development Kit / Java Runtime Environment	Τα εργαλεία ανάπτυξης και το περιβάλλον εκτέλεσης της Java.
JPA	Jakarta Persistence API	Πρότυπο της Java για αντιστοίχιση αντικειμένων σε πίνακες (π.χ. Hibernate).
JSON	JavaScript Object Notation	Μορφή κειμένου για την ανταλλαγή δομημένων δεδομένων.
JVM	Java Virtual Machine	Η εικονική μηχανή που εκτελεί το bytecode της Java και της Kotlin.
JWT	JSON Web Token	Υπογεγραμμένο token που μεταφέρει την ταυτότητα και τον ρόλο του χρήστη.
KMP	Kotlin Multiplatform	Τεχνολογία της Kotlin για κοινό κώδικα σε server, web και Android.
LAN	Local Area Network	Τοπικό δίκτυο, π.χ. το Wi-Fi του καταλύματος.
LF	Line Feed	Ο χαρακτήρας αλλαγής γραμμής των Linux και macOS (τα Windows χρησιμοποιούν CRLF).
ORM	Object-Relational Mapping	Αντιστοίχιση αντικειμένων του κώδικα σε πίνακες της βάσης.
OS	Operating System	Λειτουργικό σύστημα.
PIN	Personal Identification Number	Προσωπικός αριθμός αναγνώρισης, π.χ. της κάρτας ή της εξόδου από το kiosk.

Συντομογραφίες & Ακρωνύμια

Ακρωνύμιο	**Πλήρης Μορφή**	**Επεξήγηση**
PMS	Property Management System	Σύστημα διαχείρισης καταλυμάτων (κρατήσεις, διαθεσιμότητα).
PNG	Portable Network Graphics	Μορφή αρχείου εικόνας.
QR	Quick Response (code)	Δισδιάστατος κωδικός που διαβάζεται με κάμερα· εδώ για την ενεργοποίηση της συσκευής.
R2DBC	Reactive Relational Database Connectivity	Μη ανασταλτικό (reactive) API πρόσβασης σε σχεσιακές βάσεις.
RBAC	Role-Based Access Control	Έλεγχος πρόσβασης βάσει ρόλων.
REST	Representational State Transfer	Αρχιτεκτονικό στυλ για APIs πάνω από HTTP, με πόρους και τυποποιημένες μεθόδους.
SaaS	Software as a Service	Λογισμικό ως υπηρεσία, που διατίθεται μέσω διαδικτύου με συνδρομή.
SDK	Software Development Kit	Σύνολο εργαλείων και βιβλιοθηκών για ανάπτυξη σε μια πλατφόρμα.
SHA-256	Secure Hash Algorithm, 256 bits	Αλγόριθμος κατακερματισμού· εδώ για την αποθήκευση των refresh tokens.
SLF4J	Simple Logging Facade for Java	Κοινή διεπαφή καταγραφής (logging) για βιβλιοθήκες της Java.
SMTP	Simple Mail Transfer Protocol	Πρωτόκολλο αποστολής ηλεκτρονικού ταχυδρομείου.
SQL	Structured Query Language	Γλώσσα ερωτημάτων των σχεσιακών βάσεων δεδομένων.
TLS / SSL	Transport Layer Security / Secure Sockets Layer	Πρωτόκολλα κρυπτογράφησης των συνδέσεων δικτύου· το STARTTLS αναβαθμίζει μια σύνδεση σε κρυπτογραφημένη.
UI	User Interface	Διεπαφή χρήστη.
URL	Uniform Resource Locator	Διεύθυνση ενός πόρου στο διαδίκτυο.
UTC	Coordinated Universal Time	Συντονισμένη παγκόσμια ώρα, η βάση των χρονοσφραγίδων.
UUID	Universally Unique Identifier	Αναγνωριστικό 128 bit, πρακτικά μοναδικό.
Wasm	WebAssembly	Δυαδική μορφή κώδικα που εκτελείται στον browser· το WasmGC προσθέτει συλλογή απορριμμάτων.

Συντομογραφίες & Ακρωνύμια

Ακρωνύμιο	**Πλήρης Μορφή**	**Επεξήγηση**
ΑΦΜ	Αριθμός Φορολογικού Μητρώου (VAT ID)	Ο φορολογικός αριθμός μιας επιχείρησης· εδώ για την αναζήτηση στο εταιρικό μητρώο.
ΤΠΕ	Τεχνολογίες Πληροφορίας και Επικοινωνιών (ICT)	Οι τεχνολογίες πληροφορικής και επικοινωνιών.